

数学建模与 Python 应用

系统性笔记与实战指南

LZX

2026 年 8 月 12 日

前言

关于这本书

数学建模是连接数学理论与现实问题的桥梁，它将复杂的实际场景抽象为可求解的数学模型，借助计算机工具完成定量分析与决策。在各类学科竞赛、工程应用与科研工作中，数学建模都展现出极强的实用性与生命力。

本书是笔者在数学建模学习与实践过程中整理的系统性笔记，面向高等院校理工科学生、数学建模竞赛参与者以及相关领域工程技术人员，旨在将数学原理与 Python 编程实践深度融合，帮助读者快速掌握数学建模的核心方法与实现技能。

全书内容兼顾理论严谨性与工程实用性，覆盖数学建模竞赛中最核心的知识模块：从建模基础方法与论文写作规范，到 Python 科学计算生态概述，再到线性代数模型、线性整数规划、非线性多目标规划以及图论网络模型等经典内容。所有知识点均配有清晰的数学原理推导、完整的实战代码实现和场景化应用说明，真正做到“理论可理解、代码可运行、结果可复现”。

本书主要参考由司守奎、孙玺菁主编的《数学建模算法与应用》一书，同时舍弃了重复的习题，主要突出数学原理、具体例子、代码实现、优缺点分析、方法选择建议以及应用场景，在比较中体现出各种算法的优缺点，希望能为各类需要数学建模的科研场景或科技竞赛参与者提供有价值的参考。

使用说明

- 全书所有代码均基于 Python 3 环境开发，依赖库包括 NumPy、Pandas、Matplotlib、SciPy、SymPy、CVXPY、NetworkX 等常用科学计算库，读者可以通过 pip 直接安装。
- 书中所有示例都经过实际运行验证，你可以直接复制代码到本地环境运行，结合注释理解实现逻辑。
- 笔记中特意加入了大量“方法选择建议”和“应用场景总结”，帮助你在实际问题中快速定位合适的建模方案。
- 对于竞赛备考读者，建模算法浩如烟海，一本如同字典一样的参考书具有重要意义，它可以使选手们在比赛时方便的快速获取希望使用的算法，也可以避免将大量时间耗费在无意义的编程上，可以有更多精力与队友讨论，因此极大的提高了建模效率。

LZX

2026 年 8 月 12 日

目录

1	数学建模概论	1
1.1	数学模型的基本方法与步骤	1
1.1.1	数学模型的要求	1
1.1.2	数学建模的一般步骤	1
1.2	建模论文写作	2
2	Python 使用概述	4
2.1	基本数据类型	4
2.1.1	number (数字)	4
2.1.2	list (列表)	5
2.1.3	dictionary (字典)	6
2.2	函数	6
2.2.1	自定义函数	6
2.2.2	模块的导入与使用	7
2.2.3	Python 常用内置函数用法	7
2.3	NumPy 简介	9
2.3.1	NumPy 基本使用	9
2.3.2	矩阵合并与分割	11
2.3.3	矩阵的简单运算	12
2.3.4	线性代数运算	13
2.3.5	Pandas 库介绍	15
2.4	Pandas 库介绍	16
2.4.1	DataFrame 基本操作	16
2.4.2	数据预处理	17
2.4.3	常用方法速查	19
2.5	文件操作	19
2.5.1	文件操作基本知识	20
2.5.2	文本文件操作	21
2.5.3	常用方法速查	22

2.6	SciPy 库	22
2.6.1	SciPy 简介	22
2.6.2	SciPy 基本操作	23
2.6.3	常用函数速查	26
2.7	SymPy 库	26
2.7.1	SymPy 简介	26
2.7.2	符号变量与符号函数的定义	26
2.7.3	符号方程求解	27
2.7.4	符号矩阵运算	28
2.7.5	常用函数速查	28
2.8	Matplotlib 库介绍	29
2.8.1	二维绘图	29
2.8.2	三维绘图	32
2.8.3	常用函数速查	34
3	线性代数模型	35
3.1	特征值与特征向量	35
3.1.1	差分方程与斐波那契数列	35
3.1.2	莱斯利 (Leslie) 种群模型	38
3.1.3	PageRank 算法	46
3.1.4	奇异值分解应用	51
4	线性规划和整数规划模型	59
4.1	线性规划模型	59
4.1.1	线性规划模型及概念	59
4.2	整数规划	64
4.2.1	整数线性规划模型	64
4.2.2	投资的收益与风险	71
4.2.3	比赛项目排序问题	75
5	非线性规划和多目标规划模型	80
5.1	非线性规划概念和理论	80
5.1.1	非线性规划问题的数学模型	80
5.2	一个简单非线性规划模型	85
5.3	二次规划模型	90

5.3.1	二次规划模型的标准形式	91
5.4	非线性规划的求解及应用	95
5.5	多目标规划	102
5.5.1	多目标规划问题的基本理论	102
5.5.2	求有效解的常用方法	103
5.6	飞行管理问题	106
6	图论模型	112
6.1	图与网络的基础理论	112
6.1.1	图与网络的基本概念	112
6.1.2	图的矩阵表示	113
6.2	NetworkX 简介	114
6.2.1	导入 NetworkX	114
6.2.2	图的生成	114
6.2.3	顶点与边的添加	115
6.2.4	图的绘制	115
6.2.5	数据导出	116
6.2.6	常用算法	116
6.3	最短路算法	118
6.3.1	最短路径问题的 0-1 整数规划模型	124
6.4	最小生成树	127
6.4.1	基本概念和算法	127
6.5	着色问题	133
6.6	最大流与最小费用流问题	137
6.6.1	最大流问题	137
6.6.2	最小费用流问题	139
6.7	关键路径	143
6.7.1	计划网络图	143
7	插值与拟合	151
7.1	插值方法	151
7.1.1	一维插值	151
7.1.2	二维插值	157
7.1.3	用 Python 求解插值问题	161

7.2	曲线拟合	169
7.2.1	线性最小二乘法	169
7.3	函数逼近	175
7.3.1	函数逼近的基本概念	175
7.4	黄河小浪底调水调沙问题	178
8	常微分方程与差分方程	183
8.1	常微分方程问题的数学模型	183
8.2	传染病预测问题	187
8.3	常微分方程的求解	192
8.3.1	常微分方程的符号解	192
8.3.2	常微分方程的数值解	195
8.4	常微分方程建模实例	200
8.4.1	Malthus 模型	200
8.4.2	Logistic 模型	200
8.4.3	种群竞争模型	201
8.4.4	捕食者-被捕食者模型	202
8.5	差分方程建模方法	207
8.5.1	差分方程建模实例	207
8.5.2	差分方程的基本概念与理论	208
8.6	应用案例：最优捕鱼策略	211
9	数据的描述性统计方法	216
9.1	scipy.stats 模块简介	216
9.1.1	连续型随机变量及分布	216
9.1.2	离散型随机变量及分布	217
9.2	统计的基本概念和统计图	221
9.2.1	统计的基本概念	221
9.2.2	统计图	223
9.3	参数估计和假设检验	228
9.3.1	参数估计	228
9.3.2	参数假设检验	229
9.3.3	非参数假设检验	230
9.4	方差分析	233

9.4.1	单因素方差分析	234
9.4.2	双因素方差分析	235
10	回归分析	239
10.1	一元线性回归模型	239
10.1.1	一元线性回归分析	239
10.2	多元线性回归	243
10.2.1	多元线性回归理论	243
10.2.2	多元线性回归应用	246
10.3	多项式回归	250
10.4	逐步回归	254
10.5	广义线性回归模型	257
10.5.1	广义线性模型的基本概念	257
10.5.2	Logistic 回归模型	258
10.5.3	赔率与赔率比	259
10.5.4	Probit 回归模型	259
11	聚类分析与判别分析	263
11.1	聚类分析	263
11.1.1	数据变换	263
11.1.2	亲疏程度的测度	264
11.1.3	系统聚类法	264
11.1.4	动态聚类法——K 均值聚类	265
11.2	判别分析	269
11.2.1	距离判别法	269
11.2.2	Fisher 判别	270
11.2.3	判别准则的评价	271
12	主成分分析与因子分析	275
12.1	主成分分析	275
12.1.1	主成分分析的基本原理和步骤	275
12.2	因子分析	280
12.2.1	因子分析的数学理论	280
13	偏最小二乘回归分析	285

13.1 偏最小二乘回归分析	285
14 综合评价方法	291
14.1 综合评价指标体系	291
14.1.1 指标体系的概念	291
14.1.2 指标体系的设置原则	291
14.2 综合评价数据处理	291
14.2.1 定量指标的一致化处理	291
14.2.2 定量指标的无量纲化处理	292
14.2.3 定性指标的定量化	293
14.3 常用的综合评价数学模型	295
14.3.1 线性加权综合评价法	295
14.3.2 TOPSIS 法	295
14.3.3 灰色关联度分析	296
14.3.4 熵值法	296
14.3.5 秩和比法	297
14.4 模糊数学方法	299
14.4.1 模糊数学基本概念	299
14.4.2 常见的模糊分布	300
14.4.3 模糊贴近度	300
14.4.4 模糊综合评价	301
14.5 数据包络分析	304
14.5.1 DEA 的 C^2R 模型	304
14.6 招聘公务员问题	307
14.6.1 问题提出	307
14.6.2 模型假设与符号说明	307
14.6.3 模型准备	308
14.6.4 问题 (1) 模型: 用人单位视角	308
14.6.5 问题 (2) 模型: 双向满意度	309
15 预测方法	315
15.1 灰色预测模型	315
15.1.1 GM(1,1) 预测模型	315
15.1.2 GM(2,1) 模型	317

15.1.3 DGM(2,1) 模型	317
15.1.4 灰色 Verhulst 模型	318
15.2 马尔可夫预测	322
15.2.1 马尔可夫链的定义	322
15.2.2 转移概率矩阵及柯尔莫哥洛夫定理	322
15.2.3 转移概率的渐近性质——极限概率分布	323
15.3 神经网络	327
15.3.1 人工神经网络概述	327
15.3.2 感知器	328
15.3.3 BP 神经网络	328
15.3.4 RBF 神经网络	329
15.3.5 数据预处理	329
16 博弈论	334
16.1 基本概念	334
16.1.1 博弈论与优化理论的异同	334
16.1.2 博弈论中的经典案例	334
16.1.3 博弈的一般概念	335
16.2 零和博弈	338
16.2.1 基本概念	338
16.3 双矩阵博弈模型	343
16.3.1 非合作双矩阵博弈的纯策略解	343
17 偏微分方程	348
17.1 3 类偏微分方程的定解问题	348
17.1.1 椭圆型偏微分方程	348
17.1.2 抛物型偏微分方程	349
17.1.3 双曲型偏微分方程	349
17.2 简单偏微分方程的符号解	349
17.3 偏微分方程的差分解法	352
17.3.1 椭圆型方程第一边值问题的差分解法	352
17.3.2 抛物型方程的差分解法	353
17.3.3 双曲型方程的差分解法	353
17.4 Python 求偏微分方程数值解举例	357

第 1 章

数学建模概论

1.1 数学模型的基本方法与步骤

1.1.1 数学模型的要求

对数学模型来说一般有以下要求：

1. 要有足够的精确度，要反映本质的性质和关系，可忽略非本质的东西而不至于影响反应现实本质的真实程度。
2. 模型既要精确，又要简单。因为过于复杂的模型难以求解，当一个相对简单的模型已经可以在很大程度上真实反映现实时，我们没必要构造过于复杂的模型进行求解。
3. 要借鉴已有的标准模型。
4. 构造模型的依据要充分。

数学建模的方法按大类一般可以分为：

1. 机理分析法。即根据人们对现实现象的了解和已有知识，分析研究对象中各因素之间的因果关系，找出反应其内部机理的规律的方法，使用此方法的前提是我们对研究对象的机理有一定了解。
2. 测试分析法。当我们对研究对象的机理不了解的时候，可以将研究对象视为一个“黑箱”系统，对系统的输入和输出进行预测，并以这些实测数据为基础进行统计分析。
3. 综合分析法。即结合以上两种方法综合分析。

1.1.2 数学建模的一般步骤

数学建模的步骤常常取决于具体的问题，下面介绍机理分析建模的一般步骤：

1. 模型准备：通过了解模型所研究的问题及其希望达到的目的，我们进行调查研究并初步确定它所属的模型。

2. 模型假设：分清主要影响因素和次要影响因素，抓住主要因素，忽略次要因素，对实际问题做一些必要的假设，使得在简化模型的同时保持较高的精确度。
3. 模型构成：根据所作出的假设，分析研究对象之间的因果关系，用数学语言加以概括，得到对于所研究的问题的数学描述，即构成研究问题的数学模型，通常它是主要因素之间的一系列关系式。在初步构成数学关系式之后，可对关系式进行适当化简，使得其便于求解。
4. 模型求解：选择合适的数学方法求解已建立的模型。在多数情况下，由于现实问题中影响因素很多，且条件限制不够理想，我们几乎不可能获得准确的解析解，而只能通过各种数值方法（包括但不限于各类数值优化方法，线性和非线性方程组的数值方法，偏微分方程的数值解法以及各种决策、概率、统计方法）。若现有的数学方法不足以很好的求解模型，我们需考虑对现有方法进行改进或提出新的便于求解的模型。
5. 模型分析：对求解结果进行各种数学上的分析以论证其是否可靠，例如误差分析、统计分析、灵敏度分析等。
6. 模型检验：将求解得到的结果代回现实问题，与现实进行比较，验证合理性。
7. 模型应用：将模型反复应用与实际系统并同时根据结果进行改进。

1.2 建模论文写作

建模论文写作是体现建模思路和方法的重要手段，下面介绍建模论文的一般结构：

- 题目：论文的题目应涵盖论文的主要的研究内容和研究方法，应简短精炼。
- 摘要：摘要是整篇论文的关键部分，它应当保证读者仅通过阅读摘要便可大致判断研究方法及成果水平，即从摘要中便可了解问题分析是否透彻，做了哪些关键假设，合理程度如何，建模方法及模型结构是否可信，模型求解算法设计是否可信，结果是否可靠等问题。摘要中应简要叙述论文的研究内容及研究背景，研究的目的，从何处获取了怎样的初始研究数据，做了哪些假设，建立了怎样的模型，是怎样进行求解的，结果是什么，做了哪些检验。
- 关键词：3~5 个反应主要研究内容和研究方法的词。
- 问题提出或重述：介绍所研究的问题、研究背景、研究目的、已知信息或条件。
- 问题分析：查阅资料，结合已有研究成果找出与问题关联的所有可能因素，分析他们之间的因果关系。确定主要因素和次要因素，确定模型的基本结构和建模方法。
- 基本假设：依据分析结果，给出假设并论证其合理性。
- 模型建立：依据分析及假设，给出所研究问题的数学描述。注意应详细说明模型中每个记号的具体含义，包括其明确的含义和量纲等。同时记号和变量应保持通篇一致，模型表达式也尽量具有较高的一般性和可推广性。

-
- 参数识别和模型求解：根据相关数学理论以及数值方法，给出详细的求解过程。包括解析解的公式推导过程、数值解的算法步骤和计算机编程计算结果等。若模型中含有未知参数或条件，应根据具体信息给出模型参数的识别方法和识别结果，在此基础上给出模型的最终求解结果。注意论文正文不要出现用于模型求解的计算机程序。
 - 结果分析和检验：对计算结果进行一些数学上的分析验证，如误差分析、统计假设检验、模型中参数稳定性及灵敏度分析等，验证模型的正确性以及结果的可靠性。分析模型的优缺点，以及可能的改进方向。
 - 参考文献：所有在研究中参考过的文献资料，尤其是在论文中直接提及或引用的，都应在文中相应位置标明出处，并在参考文献中逐一列出，并注明作者姓名、出版物名称、出版日期、出版单位、参考页码。
 - 附录：对正文的补充，一些比较重要，却又不便放入正文的内容。

第 2 章

Python 使用概述

2.1 基本数据类型

Python 中有 6 种基本数据类型，分别为 number（数字）、string（字符串）、list（列表）、tuple（元组）、set（集合）、dictionary（字典）。本节将分别简要介绍几种数据类型。

2.1.1 number（数字）

Python 支持下面 4 种数值类型：

1. 整型（**int**）：也称整数，包含正整数和负整数，不带小数点。Python 整型无大小限制。
2. 浮点型（**float**）：浮点型由整数部分和小数部分组成，也可以用科学计数法表示。
3. 复数型（**complex**）：复数由实数部分和虚数部分组成，可以用 $a + bj$ ，或者 `complex(a, b)` 表示，实数部分和虚数部分都是浮点型。
4. 布尔型（**bool**）：Python 中，把 True 和 False 定义为关键字，但它们的值还是 1 和 0。

有时需要对数据类型进行转换，只需将数据类型作为函数名即可。例如：

```
int(x)          # 将 x 转换为一个整数
float(x)        # 将 x 转换为一个浮点数
complex(x,y)    # 将 x 转换为一个复数，实数部分是 x，虚数部分是 y
```

Python 中字符串用单引号（'）或双引号（"）括起来。创建字符串只要为变量分配一个值即可，例如 `str1 = "Hello World!"`。

Python 访问字符串，可以使用方括号（[]）来截取字符串，基本语法如下：

变量 [头下标：尾下标]

其中下标最小索引值以 0 为开始值，-1 为从末尾的开始位置。Python 中的字符串有两种索引方式，从左往右以 0 开始，从右往左以 -1 开始。

字符串操作实例：

```
1      str1 = "Hello World!"
2      print(str1)           # 输出字符串
3      print(str1[0:-1])     # 输出第一个到倒数第二个的所有字符
4      print(str1[-1])       # 输出字符串的最后一个字符
5      print(str1[2:5])      # 输出第三个到第五个字符
6      print(str1[2:])       # 输出从第三个开始的所有字符
7      print(str1 * 2)       # 输出字符串两遍
```

除了可以使用内置函数和运算符对字符串进行操作，Python 字符串对象自身还提供了大量方法用于字符串的检测、替换和排版等操作。需要注意，字符串对象是不可变的，所以字符串对象提供的涉及字符串修改的方法都是返回修改后的字符串，并不是对原始字符串进行任何修改。

1. find()、rfind()、index()、rindex()、count()

find() 和 rfind() 方法分别用来查找一个字符串在另一个字符串的指定范围中首次出现和最后一次出现的位置，若不存在则返回 -1；index() 和 rindex() 方法用来返回一个字符串在另一个字符串指定范围中首次和最后一次出现的位置，若不存在则抛出异常；count() 方法用来返回一个字符串在另一个字符串中出现的次数，若不存在则返回 0。

2. split()、rsplit()

split() 和 rsplit() 方法分别用来以指定字符为分隔符，从字符串左端和右端开始将其分割成多个字符串，并返回包含分割结果的列表。若不指定分隔符，则字符串中的任何空白字符（包括空格、换行符、制表符等）的连续出现都将被认为是分隔符，并且自动删除字符串两侧的空白字符，返回包含最终分隔结果的列表。

3. join()

join() 方法用来将列表中多个字符串进行连接，并在相邻两个字符串之间插入指定字符，返回新字符串。

4. strip()、rstrip()、lstrip()

这几个方法分别用来删除两端、右端或左端连续的空白字符或指定字符。

5. startswith()、endswith()

这两个方法用来判断字符串是否以指定字符串开始或结束，可以接收两个整数参数来限定字符串的检测范围。这两个方法还可以接收一个字符串元组作为参数来表示前缀或后缀。

2.1.2 list（列表）

列表可以完成大多数集合类的数据结构的实现。列表中的元素类型可以是不同的，它支持数字、字符串，甚至可以包含其他列表（嵌套）。

列表是写在方括号（[]）里、用逗号分隔开的元素列表。和字符串一样，列表同样可以被索引和截取，列表被截取后返回一个包含所需元素的新列表。

列表截取的语法格式如下：

变量 [头下标：尾下标]

其中下标最小的索引值以 0 为开始值，以 -1 为从末尾的开始位置。与字符串不同的是，列表中的元素是可以改变的。

列表操作实例：

```
1 L = ['abc', 12, 3.45, 'Python', 2.789]
2 print(L)           # 输出完整列表
3 print(L[0])        # 输出列表的第一个元素
4 L[0] = 'a'         # 修改列表的第一个元素
5 L[1:3] = ['b', 'Hello'] # 修改列表的第二、三个元素
6 L[2:4] = []         # 删除列表的第三、四个元素
```

2.1.3 dictionary（字典）

字典是无序的对象集合，与列表的区别是字典的元素是通过键来存取的，而不是通过索引值。字典是一种映射类型，字典用 {} 标识，它是一个无序的键（key）：值（value）对集合。在同一个字典中，键（key）必须是唯一的，但值不必唯一，值可以取任何数据类型，但键必须是不可变的，如字符串、数字或元组。

字典操作实例：

```
1 dict1 = {'Alice': '123', 'Beth': '456', 'Cecil': 'abc'}
2 print(dict1['Alice']) # 输出 123
3 dict1['new'] = 'Hello' # 增加新的键值对
4 dict1['Alice'] = '1234' # 修改已有键值对
```

字典中 get() 方法用来返回指定“键”对应的“值”，并允许指定该键不存在时返回特定的“值”。

可以对字典对象进行迭代或遍历，默认是遍历字典的“键”，如果需要遍历字典的元素必须使用字典对象的 items() 方法进行说明，如果需要遍历字典的“值”则必须使用字典对象的 values() 方法说明。

2.2 函数

2.2.1 自定义函数

1. 函数定义及调用

Python 中定义函数的语法如下：

```
1 def functionName(formalParameters):
2     functionBody
```

1. functionName 是函数名，可以是任何有效的 Python 标识符。

2. `formalParameters` 是形参列表，在调用该函数时通过给形参赋值来传递调用值，形参可以由多个、一个或零个参数组成，当有多个参数时各个参数由逗号分割；圆括号是必不可少的，即使没有参数也不能没有它，括号外面的冒号也是必不可少的。
3. `functionBody` 是函数体，是函数每次被调用时执行的一组语句，可以由一个语句或多个语句组成。整个函数体要注意缩进。

2. 匿名函数

匿名函数是指不以 `def` 语句定义的没有名称的函数，它在使用时临时声明、立刻执行，其特点是效率高。

Python 常用 `lambda` 来创建匿名函数，它是一个可以接收任意多个参数并返回单个表达式值的函数，其语法格式为：

```
lambda arg1[, arg2, ..., argn]: expression
```

2.2.2 模块的导入与使用

随着程序的变大及代码的增多，为了更好的维护程序，一般会把代码进行分类，分别放在不同的文件中。公共类、函数等都可以放在独立的文件中，这样其他多个程序都可以使用，而不必把这些公共的类、函数等在每个程序中复制一份，这样独立的文件就称为模块。

1. 标准库与扩展库中对象的导入与使用

Python 标准库和扩展库中的对象必须先导入才能使用，导入方法如下：

1. `import 模块名 [as 别名]`

使用这种方式导入后，使用时需要将对象之前加上模块名作为前缀，必须以“模块名. 对象名”的形式访问。如果模块名字很长，可以为导入的模块设置一个别名，然后使用“别名. 对象名”的方式使用。

2. `from 模块名 import 对象名 [as 别名]`

使用这种方式仅导入明确指定的对象，并且可以为导入的对象起一个别名。这种导入方式可以减少查询次数，提高访问速度，同时也可以减少代码量，不需要使用模块名作为前缀。

3. `from 模块名 import *`

使用这种方式可以一次导入模块中所有对象，可以直接使用模块中所有对象而不需要模块名作为前缀，但一般不推荐这样使用。

2.2.3 Python 常用内置函数用法

内置函数不需要额外导入任何模块即可直接使用，具有非常快的运行速度，推荐优先使用。使用 `dir(__builtins__)` 可以查看所有内置函数和内置对象，使用 `help(函数名)` 可以查看某个函数的用法。

常用内置函数及其功能的简要说明如下表：

函数	说明
<code>abs(x)</code>	返回 x 的绝对值
<code>all(iterable)</code>	判断可迭代对象中所有元素是否都为 True
<code>any(iterable)</code>	判断可迭代对象中是否存在元素为 True
<code>bin(x)</code>	将整数 x 转换为二进制字符串
<code>bool(x)</code>	将 x 转换为布尔值
<code>chr(i)</code>	将整数 i 转换为对应的 Unicode 字符
<code>dict()</code>	创建字典
<code>dir(obj)</code>	返回对象的属性列表
<code>divmod(a, b)</code>	返回 $(a//b, a\%b)$ 的元组
<code>enumerate(iterable)</code>	返回索引和元素组成的枚举对象
<code>eval(str)</code>	计算字符串中的表达式并返回结果
<code>float(x)</code>	将 x 转换为浮点数
<code>format(x, fmt)</code>	按格式 <code>fmt</code> 格式化 x
<code>frozenset(iterable)</code>	创建不可变集合
<code>help(obj)</code>	查看对象的帮助信息
<code>hex(x)</code>	将整数 x 转换为十六进制字符串
<code>int(x)</code>	将 x 转换为整数
<code>isinstance(obj, class)</code>	判断 <code>obj</code> 是否为 <code>class</code> 的实例
<code>len(obj)</code>	返回对象的长度（元素个数）
<code>list(iterable)</code>	将可迭代对象转换为列表
<code>max(iterable)</code>	返回可迭代对象中的最大值
<code>min(iterable)</code>	返回可迭代对象中的最小值
<code>oct(x)</code>	将整数 x 转换为八进制字符串
<code>ord(c)</code>	返回字符 c 的 Unicode 编码
<code>pow(x, y)</code>	返回 x 的 y 次方
<code>print(obj)</code>	输出对象
<code>range(start, stop, step)</code>	生成等差数列
<code>round(x, n)</code>	将 x 四舍五入保留 n 位小数
<code>set(iterable)</code>	创建集合
<code>sorted(iterable)</code>	返回排序后的列表
<code>str(obj)</code>	将对象转换为字符串
<code>sum(iterable)</code>	对可迭代对象求和
<code>tuple(iterable)</code>	将可迭代对象转换为元组
<code>type(obj)</code>	返回对象的类型
<code>zip(*iterables)</code>	将多个可迭代对象打包成元组序列

下面给出几个内置函数的应用举例：

1. 排序

`sorted()` 可以对列表、元组、字典、集合或其他可迭代对象进行排序并返回新列表，支持使用 `key` 参数指定排序规则。

2. 枚举

`enumerate()` 函数用来枚举对象中的元素，返回可迭代的 `enumerate` 对象，利用该函数可同时获得索引和值。在使用时，既可以把 `enumerate` 对象转换为列表、元组、集合，也可以使用 `for` 循环直接遍历其中的元素。

3. `map()` 函数

函数 `map(func, *iterables)` 把一个函数 `func` 依次映射到一个可迭代对象 `iterables` 的每个元素上，并返回一个可迭代的 `map` 作为结果，`map` 对象中每个元素是 `iterables` 中元素经过函数 `func` 处理后的结果。

4. `filter()` 函数

内置函数 `filter()` 将一个单参数函数作用到一个序列上，返回该序列中使得该函数取值为 `True` 的那些元素组成的 `filter` 对象，可以把 `filter` 对象转换为列表、元组、集合，也可以直接使用 `for` 循环遍历其中的元素。

5. `zip()` 函数

`zip()` 函数用来把多个可迭代对象中对应位置上的元素压缩在一起，返回一个可迭代的 `zip` 对象，其中每个元素都是包含原来多个可迭代对象对应位置上元素的元组，最终结果中包含的元素个数取决于所有参数序列或可迭代对象中最短的那一个。

2.3 NumPy 简介

2.3.1 NumPy 基本使用

NumPy 是 Python 科学计算的基础库，其核心优势在于：

- **ndarray**：多维数组对象，所有元素类型相同，因此内存连续、访问高效，远优于 Python 原生列表（列表存储的是对象的指针）。
- **ufunc**：通用函数对象，能对数组进行逐元素快速运算，避免显式循环。

导入方式

```
1 import numpy as np
```

建议使用别名 `np`，这是社区通用惯例。

数组创建方法

从列表或元组创建：`array`

`np.array()` 将 Python 序列（列表或元组）转换为 `ndarray`。可通过 `dtype` 参数指定数据类型，若不指定则自动推断。

```
1 a1 = np.array([1, 2, 3, 4])           # 默认 int 类型
2 a2 = np.array([1, 2, 3, 4], dtype=float) # 强制转为 float
3 b = np.array([[1, 2, 3], [4, 5, 6]])  # 创建二维数组
    (矩阵)
```

等差数列: arange

`np.arange()` 类似于 Python 内置的 `range()`，但返回的是 `ndarray`。区间为左闭右开 `[start, stop)`，支持步长可为浮点数。

```
1 np.arange(start, stop, step, dtype)
2 # 示例:
3 c = np.arange(1, 5)          # 生成 [1, 2, 3, 4]
4 d = np.arange(0, 1, 0.1)    # 生成 [0, 0.1, ..., 0.9]
```

等间隔采样: linspace

`np.linspace()` 在指定区间内生成固定数量的等间隔点，区间默认是闭区间 `[start, stop]`，可通过 `endpoint=False` 改为左闭右开。

```
1 np.linspace(start, stop, num=50, endpoint=True)
2 # 示例: 生成 [1, 2, 3, 4]，共4个点，间隔相等
3 d = np.linspace(1, 4, 4)
```

等比数列: logspace

`np.logspace()` 生成等比数列，默认以 10 为底，实际值为 $\text{base}^{\text{start}}$ 到 $\text{base}^{\text{stop}}$ 之间的等比序列。

```
1 np.logspace(start, stop, num=50, endpoint=True, base=10.0)
2 # 示例: 生成 [2, 4, 8]，即  $2^1, 2^2, 2^3$ 
3 e = np.logspace(1, 3, 3, base=2)
```

特殊矩阵生成函数

这些函数用于快速生成特定结构的数组，非常实用：

- `ones(shape)`：生成全 1 数组。
- `zeros(shape)`：生成全 0 数组。
- `empty(shape)`：分配空间但不初始化，值随机（速度最快）。
- `eye(N, k=0)`：生成 N 阶单位阵， k 控制对角线偏移（正数向右上偏移）。
- `ones_like(a)`、`zeros_like(a)`：生成与 `a` 形状相同的全 1 或全 0 数组。

```
1 a = np.ones(4, dtype=int)          # [1, 1, 1, 1]
2 b = np.ones((4,), dtype=int)       # 同上，形状用元组表示
3 c = np.ones((4, 1))                # 4行1列的全1列向量
4 d = np.zeros(4)                    # [0, 0, 0, 0]
5 e = np.empty(3)                    # 长度为3的未初始化数组
6 f = np.eye(3)                      # 3阶单位阵
7 g = np.eye(3, k=1)                 # 第1条对角线（右上）为1
8 h = np.zeros_like(a)               # 与a形状相同的全0数组
```

数组索引与切片

基本规则

- 切片语法 `arr[start:end:step]` 与列表一致，不包含 `end`。
- 二维数组推荐使用 `a[i, j]` 索引，比 `a[i][j]` 更高效。
- `ndarray` 所有元素类型相同，而 Python 列表可混合类型。

示例：基本索引与切片

```

1      a = np.arange(16).reshape(4, 4)      # 生成 0~15 的 4x4 矩阵
2      b = a[1][2]                          # 第 2 行第 3 列 -> 6
3      c = a[1, 2]                          # 更推荐的写法，结果同上
4      d = a[1:2, 2:3]                      # 切片得到 [[6]] (保留二
      维结构)

```

布尔索引

布尔索引通过布尔数组筛选元素，非常灵活。例如 `a[x==1]` 会选出所有对应位置为 True 的行。

```

1      x = np.array([0, 1, 2, 1])
2      print(a[x == 1])    # 输出 a 的第 2 行和第 4 行 (索引为 1 的行)

```

花式索引

花式索引指用整数数组作为索引，可以一次取出任意位置的数据。

```

1      rows = [0, 2]                      # 取第 1 行和第 3 行
2      cols = [1, 3]                      # 取第 2 列和第 4 列
3      print(a[rows, :])                  # 输出指定行
4      print(a[:, cols])                  # 输出指定列
5      print(a[rows, cols])              # 输出 (0,1) 和 (2,3) 两个位置的值

```

2.3.2 矩阵合并与分割

矩阵合并

在实际应用中，经常需要合并矩阵，可以使用 `vstack()` 和 `hstack()` 实现不同轴上的合并。

- `np.vstack([A, B])`: 将矩阵 A 和 B 按垂直方向（行方向）上下合并，要求列数相同。
- `np.hstack([A, B])`: 将矩阵 A 和 B 按水平方向（列方向）左右合并，要求行数相同。

```

1      import numpy as np
2      a = np.arange(16).reshape(4, 4)    # 生成 4 行 4 列的数组

```

```

3      b = np.floor(5 * np.random.random((2, 4))) # 生成 2 行 4 列的
      随机数组
4      c = np.ceil(6 * np.random.random((4, 2))) # 生成 4 行 2 列的
      随机数组
5
6      d = np.vstack([a, b]) # 上下合并矩阵，结果形状为 (6, 4)
7      e = np.hstack([a, c]) # 左右合并矩阵，结果形状为 (4, 6)

```

矩阵分割

- `np.vsplit(a, m)`: 将矩阵 `a` 沿垂直方向平均分割成 `m` 个行数组，要求行数能被 `m` 整除。
- `np.hsplit(a, n)`: 将矩阵 `a` 沿水平方向平均分割成 `n` 个列数组，要求列数能被 `n` 整除。

```

1      import numpy as np
2      a = np.arange(16).reshape(4, 4) # 生成 4 行 4 列的数组
3
4      b = np.vsplit(a, 2) # 行分割，得到 2 个形状为 (2, 4) 的数组
5      c = np.hsplit(a, 4) # 列分割，得到 4 个形状为 (4, 1) 的数组

```

2.3.3 矩阵的简单运算

求和

`sum()` 函数用于计算数组元素的和，可以通过 `axis` 参数指定求和方向。

- `a.sum()`: 求矩阵所有元素的和。
- `np.sum(a, axis=0)`: 按列求和（沿行方向压缩），返回形状为 `(n,)` 的一维数组。
- `np.sum(a, axis=1)`: 按行求和（沿列方向压缩），返回形状为 `(m,)` 的一维数组。
- `np.sum(a, axis=0, keepdims=True)`: 按列求和并保持矩阵的二维特性，返回形状为 `(1, n)` 的二维数组。

```

1      import numpy as np
2      a = np.array([[0, 3, 4], [1, 6, 4]])
3
4      b = a.sum() # 所有元素求和，标量
5      c1 = sum(a) # Python 内置函数，逐列求和
6      c2 = np.sum(a, axis=0) # 逐列求和，返回 (3,) 数组
7      c3 = np.sum(a, axis=0, keepdims=True) # 逐列求和，返回 (1, 3) 数组

```


逐个元素运算

对于 ndarray 数组，+、-、*、/ 和 ** 都是对应的逐个元素运算。当两个数组形状不同时，NumPy 会自动进行广播（broadcast）使其形状兼容。

```

1  import numpy as np
2  a = np.array([[0, 3, 4], [1, 6, 4]])
3  b = np.array([[1, 2, 3], [2, 1, 4]])
4
5  c = a / b          # 两个矩阵对应元素相除
6  d = np.array([[2, 3, 2]]) # 形状为 (1, 3)
7  e = a * d          # 广播后逐个元素相乘
8  f = np.array([[3], [2]]) # 形状为 (2, 1)
9  g = a * f          # 广播后逐个元素相乘
10 h = a ** (1/2)      # 矩阵逐个元素开平方

```

矩阵乘法

NumPy 中使用 @ 运算符或 np.matmul() 进行矩阵乘法。

- a @ b: 矩阵乘法，要求左矩阵的列数等于右矩阵的行数。
- 一维数组与二维数组相乘时，一维数组会被视为行向量或列向量，具体取决于其在乘法中的位置。

```

1  import numpy as np
2  a = np.ones(4)          # 形状为 (4,)
3  b = np.arange(2, 10, 2) # 形状为 (4,)
4  c = a @ b              # a 作为行向量, b 作为列向
                          # 量, 结果为标量
5
6  d = np.arange(16).reshape(4, 4) # 形状为 (4, 4)
7  f = a @ d                # a 作为行向量, 形状为 (4,)
8  g = d @ a                # a 作为列向量, 形状为 (4,)

```

2.3.4 线性代数运算

Python 中的线性代数运算主要使用 numpy.linalg 模块，常用函数如下表所示：

函数	说明
norm	求向量或矩阵的范数
inv	求矩阵的逆矩阵
pinv	求矩阵的广义逆矩阵（Moore-Penrose 伪逆）
solve	求解线性方程组 $Ax = b$ （要求 A 为方阵且非奇异）

续下页

函数	说明
det	求矩阵的行列式
lstsq	最小二乘法求解超定线性方程组
eig	求矩阵的特征值和特征向量
eigvals	求矩阵的特征值
svd	矩阵的奇异值分解
qr	矩阵的 QR 分解

范数计算

`np.linalg.norm()` 用于计算向量或矩阵的范数，其调用格式为：

```
np.linalg.norm(x, ord=None, axis=None, keepdims=False)
```

参数说明：

- `x`：要度量的向量或矩阵。
- `ord`：范数的种类，如 1 范数、2 范数、 ∞ 范数等。`ord=None` 时默认为 2 范数。
- `axis`：指定计算方向。
 - `axis=1`：按行向量处理，计算每个行向量的范数。
 - `axis=0`：按列向量处理，计算每个列向量的范数。
 - `axis=None`：计算整个矩阵的范数。
- `keepdims`：是否保持矩阵的二维特性。`True` 保持二维结构，`False` 则返回一维数组。

```

1  import numpy as np
2  a = np.array([[0, 3, 4], [1, 6, 4]])
3
4  b = np.linalg.norm(a, axis=1)    # 求每个行向量的 2 范数
5  c = np.linalg.norm(a, axis=0)    # 求每个列向量的 2 范数
6  d = np.linalg.norm(a)            # 求整个矩阵的 2 范数

```

求解线性方程组

对于形如 $Ax = b$ 的线性方程组，当 A 为方阵且非奇异时，可以使用以下两种方法求解：

1. `np.linalg.inv(A) @ b`：先求逆矩阵，再与 b 做矩阵乘法。
2. `np.linalg.solve(A, b)`：直接求解，数值稳定性更好，推荐使用。

```
1 import numpy as np
2 a = np.array([[3, 1], [1, 2]]) # 系数矩阵
3 b = np.array([9, 8])          # 常数向量
4
5 x1 = np.linalg.inv(a) @ b     # 第一种解法：求逆后相乘
6 x2 = np.linalg.solve(a, b)    # 第二种解法：直接求解（推荐）
```

超定线性方程组的最小二乘解

当方程个数多于未知数个数时，方程组为超定方程组，通常不存在精确解，要求其最小二乘解。可以使用 `np.linalg.pinv()` 求广义逆矩阵后计算：

```
1 import numpy as np
2 a = np.array([[3, 1], [1, 2], [1, 1]]) # 3 个方程，2 个未知数
3 b = np.array([9, 8, 6])
4
5 x = np.linalg.pinv(a) @ b # 求广义逆后相乘，得到最小二乘解
```

特征值与特征向量

`np.linalg.eig()` 用于计算方阵的特征值和特征向量：

```
1 import numpy as np
2 a = np.eye(4) # 生成 4 阶单位阵
3 b = np.rot90(a) # 将矩阵逆时针旋转 90 度
4
5 c, d = np.linalg.eig(b) # c 为特征值数组，d 为特征向量矩阵
```

返回值中：

- `c`：特征值数组，每个特征值对应一列特征向量。
- `d`：特征向量矩阵，每一列是对应于 `c` 中相应位置特征值的特征向量。

2.3.5 Pandas 库介绍

Pandas 库是在 NumPy 库基础上开发的一种数据分析工具，主要提供了 3 种数据结构：

- **Series**：带标签的一维数组。
- **DataFrame**：带标签且大小可变的二维表格结构。
- **Panel**：带标签且大小可变的三维数组。

2.4 Pandas 库介绍

Pandas 库是在 NumPy 库基础上开发的一种数据分析工具，主要提供了 3 种数据结构：

- **Series**：带标签的一维数组。
- **DataFrame**：带标签且大小可变的二维表格结构。
- **Panel**：带标签且大小可变的三维数组。

这里主要介绍 Pandas 的 DataFrame 数据结构。

2.4.1 DataFrame 基本操作

创建 DataFrame

可以使用 `pd.DataFrame()` 从 NumPy 数组或字典创建 DataFrame。

- `DataFrame(data, index, columns)`：data 为数据，index 为行索引，columns 为列名。
- `pd.date_range(start, end, freq)`：生成日期索引，freq='D' 表示按天。

```
1  import pandas as pd
2  import numpy as np
3
4  # 生成日期索引
5  dates = pd.date_range(start='20191101', end='20191124', freq='D
6      ')
7
8  # 创建 24x4 的随机数组，指定行索引和列名
9  a1 = pd.DataFrame(np.random.randn(24, 4), index=dates, columns=
10     list('ABCD'))
11
12 # 不指定索引和列名，默认从 0 开始
13 a2 = pd.DataFrame(np.random.randn(24, 4))
```

读写文件

Pandas 支持从 Excel 和 CSV 文件读写数据。

写入文件：

- `df.to_excel(' 文件名.xlsx')`：将 DataFrame 写入 Excel 文件。
- `df.to_csv(' 文件名.csv')`：将 DataFrame 写入 CSV 文件。

- index=False: 不写入行索引。
- 使用 pd.ExcelWriter() 可在一个 Excel 文件中写入多个表单。

```
1  import pandas as pd
2  import numpy as np
3
4  dates = pd.date_range(start='20191101', end='20191124', freq='D
5      ')
6  a1 = pd.DataFrame(np.random.randn(24, 4), index=dates, columns=
7      list('ABCD'))
8  a2 = pd.DataFrame(np.random.randn(24, 4))
9
10 # 写入 Excel 和 CSV
11 a1.to_excel('data.xlsx')          # 写入 Excel
12 a2.to_csv('data.csv')             # 写入 CSV
13
14 # 不包含行索引
15 a1.to_excel('data.xlsx', index=False)
16 a2.to_csv('data.csv', index=False)
17
18 # 一个 Excel 文件写入多个表单
19 f = pd.ExcelWriter('data.xlsx')
20 a1.to_excel(f, 'Sheet1')
21 a2.to_excel(f, 'Sheet2')
22 f.save()
```

读取文件:

- pd.read_csv(' 文件名.csv', usecols=range(1,5)):从 CSV 文件读取数据,usecols 指定读取的列范围。
- pd.read_excel(' 文件名.xlsx', 'Sheet1', usecols=range(1,5)):从 Excel 文件的指定表单读取数据。

```
1  import pandas as pd
2
3  # 从 CSV 读取数据, 读取第 1 到第 4 列
4  a = pd.read_csv('data.csv', usecols=range(1, 5))
5
6  # 从 Excel 的 Sheet2 读取数据, 读取第 1 到第 4 列
7  b = pd.read_excel('data.xlsx', 'Sheet2', usecols=range(1, 5))
```

2.4.2 数据预处理

拆分、合并与分组计算

- 使用切片操作可以实现数据拆分，如 `df[:4]` 获取前 4 行。
- 使用 `pd.concat([df1, df2])` 可以将多个 DataFrame 合并。
- 使用 `df.groupby('列名').mean()` 按指定列分组后求均值。
- 使用 `df.groupby('列名').sum()` 按指定列分组后求和。
- 使用 `df.groupby('列名').apply(函数)` 按指定列分组后应用自定义函数。

```
1 import pandas as pd
2 import numpy as np
3
4 d = pd.DataFrame(np.random.randint(1, 6, (10, 4)), columns=list
5                  ("ABCD"))
6
7 d1 = d[:4]                # 获取前 4 行
8 d2 = d[4:]               # 获取第 5 行以后的数据
9 dd = pd.concat([d1, d2]) # 数据行合并
10
11 s1 = d.groupby('A').mean() # 按 A 列分组求均值
12 s2 = d.groupby('A').apply(sum) # 按 A 列分组求和
```

数据选取

对 DataFrame 进行选取，可以从 3 个层次考虑：

- `df[列名]`：使用方括号选取列。
- `df.loc[行标签, 列名]`：使用行和列的标签进行定位选取。
- `df.iloc[行索引, 列索引]`：使用整型索引（绝对位置索引）进行定位选取。

缺失值处理

- `df.reindex([行标签列表])`：重新索引，新增的行默认为 NaN。
- `df.dropna()`：删除包含缺失值（NaN）的行。
- 可以使用 `np.nan` 为指定位置赋值缺失值。

```
1 import pandas as pd
2 import numpy as np
3
4 a = pd.DataFrame(
5     np.random.randint(1, 6, (5, 3)),
```

```

6     index=['a', 'b', 'c', 'd', 'e'],
7     columns=['one', 'two', 'three']
8 )
9
10    # 修改指定位置的值
11    a.loc['a', 'one'] = np.nan           # 修改第 1 行第 1 列为缺失
        值
12
13    # 标签定位选取
14    b = a.loc['a', 'one']               # 选取指定位置的值
15    c = a.loc['a':'c', 'one':'three']   # 选取指定区域
16
17    # 整型索引定位选取
18    d = a.iloc[1:3, 0:2]                # 提取第 2,3 行, 第 1,2 列
        数据
19
20    # 新增列
21    a['four'] = 'bar'                   # 增加第 4 列, 所有行值为
        'bar'
22
23    # 重新索引与缺失值处理
24    a2 = a.reindex(['a', 'b', 'c', 'd', 'e', 'f']) # 新增行 'f',
        值为 NaN
25    a3 = a2.dropna()                   # 删除包含缺失值的行

```

2.4.3 常用方法速查

方法	说明
pd.DataFrame()	创建 DataFrame
pd.date_range()	生成日期索引
pd.read_csv()	读取 CSV 文件
pd.read_excel()	读取 Excel 文件
df.to_csv()	写入 CSV 文件
df.to_excel()	写入 Excel 文件
pd.ExcelWriter()	创建 Excel 写入对象, 支持多表单
pd.concat()	合并多个 DataFrame
df.groupby()	按指定列分组
df.loc[]	使用标签选取数据
df.iloc[]	使用整数索引选取数据
df.reindex()	重新索引
df.dropna()	删除包含缺失值的行

2.5 文件操作

2.5.1 文件操作基本知识

无论是文本文件还是二进制文件，其操作流程基本一致：首先打开文件并创建文件对象，然后通过该文件对象对文件进行读取、写入等操作，最后关闭并保存文件内容。

内置函数 open()

Python 内置函数 open() 可以按指定模式打开文件并创建文件对象，其完整用法为：

```
1 open(file, mode='r', buffering=-1, encoding=None, errors=None,
2      newline=None, closedf=True, opener=None)
```

主要参数说明：

- file: 指定要打开或创建的文件名称，如果文件不在当前目录中，可以使用相对路径或绝对路径。
- mode: 指定打开文件的处理方式，取值范围见下表。
- encoding: 指定文本的编码和解码方式，只适用于文本模式，可使用 GBK、UTF-8、CP936 等格式。

模式	说明
r	读模式（默认模式，可省略），如果文件不存在则抛出异常
w	写模式，如果文件已存在，先清空原有内容
x	写模式，创建新文件，如果文件已存在则抛出异常
a	追加模式，不覆盖文件中原有内容
b	二进制模式（可与其他模式组合使用），使用时不支持指定 encoding 参数
t	文本模式（默认模式，可省略）
+	读写模式（可与其他模式组合使用）

文件对象常用方法

如果执行正常，open() 函数返回一个可迭代的文件对象，通过该对象可以对文件进行读写操作。常用方法如下：

方法	功能说明
close()	把缓冲区的内容写入文件，同时关闭文件，并释放文件对象
read([size])	从文本文件中读取 size 个字符，或从二进制文件中读取指定数量的字节
readline()	从文本文件中读取一行内容并返回
readlines()	将文本文件中的每行文本作为一个字符串存入列表中并返回；对于大文件，返回的列表可能非常大
seek(offset[, whence])	将文件指针移动到指定位置，offset 表示相对于 whence 的偏移量。whence 可以是 0（从文件开头）、1（从文件当前位置）或 2（从文件末尾）
tell()	返回文件指针的当前位置
write(s)	将字符串 s 的内容写入文件

方法	功能说明
writelines(s)	将字符串列表写入文本文件，不自动添加换行符

上下文管理语句 with

在实际应用中，读写文件应优先使用上下文管理语句 with，它可以自动管理资源，确保无论使用过程中是否发生异常都会执行必要的“清理”操作并释放资源，例如文件使用后自动关闭，无需显式调用 close()。

```

1 with open(filename, mode, encoding) as fp:
2     # 通过文件对象 fp 读写文件内容
3     # 代码块结束后自动关闭文件

```

2.5.2 文本文件操作

遍历文件内容

文件对象是可迭代的，可以直接使用 for 循环逐行遍历。每行末尾包含换行符，可使用 strip() 去除。

```

1 with open('data.txt') as fp:
2     L1 = []           # 存储带换行符的每行长度
3     L2 = []           # 存储去掉换行符后的每行长度
4     for line in fp:
5         L1.append(len(line))
6         L2.append(len(line.strip())) # strip() 去除首尾空白字符（含换行符）

```

writelines() 方法用于将字符串列表写入文件，不会自动添加换行符，需要手动添加：

```

1 # 将数据转换为字符串列表，并在每个元素后添加制表符
2 data = [str(num) + '\t' for num in L2]
3
4 with open('output.txt', 'w') as fp2:
5     fp2.writelines(data) # 写入文件

```

使用 NumPy 读写文本文件

NumPy 提供了 savetxt() 和 loadtxt() 函数，方便地读写数值矩阵。

写入文件：

- np.savetxt(' 文件名.txt', 数组)：将数组保存为空格分隔的文本文件。
- np.savetxt(' 文件名.csv', 数组, delimiter=',')：指定分隔符为逗号，生成 CSV 格式文件。

读取文件：

- `np.loadtxt(' 文件名.txt')`：加载空格分隔的文本文件。
- `np.loadtxt(' 文件名.csv', delimiter=',')`：加载指定分隔符的 CSV 文件。

```

1  import numpy as np
2
3  # 生成 6x8 的 [0,1] 上均匀分布的随机数矩阵
4  a = np.random.rand(6, 8)
5
6  # 写入文件
7  np.savetxt("data.txt", a)                # 空格分隔
8  np.savetxt("data.csv", a, delimiter=',') # 逗号分隔
9
10 # 读取文件
11 b = np.loadtxt("data.txt")                # 加载空格分隔的文本文件
12 c = np.loadtxt("data.csv", delimiter=',') # 加载 CSV 文件

```

2.5.3 常用方法速查

函数/方法	说明
<code>open()</code>	打开文件并创建文件对象
<code>with open() as fp</code>	上下文管理语句，自动关闭文件
<code>fp.read()</code>	读取文件全部内容
<code>fp.readline()</code>	读取一行
<code>fp.readlines()</code>	读取所有行，返回列表
<code>fp.write()</code>	写入字符串
<code>fp.writelines()</code>	写入字符串列表（不自动换行）
<code>fp.close()</code>	关闭文件
<code>fp.seek()</code>	移动文件指针
<code>fp.tell()</code>	获取文件指针当前位置
<code>np.savetxt()</code>	将数组保存为文本文件
<code>np.loadtxt()</code>	从文本文件加载数组

2.6 SciPy 库

2.6.1 SciPy 简介

SciPy 是在 NumPy 库基础上增加了众多数学、科学以及工程计算中常用函数的库。SciPy 依赖于 NumPy，提供了便捷且快速的 n 维数组操作，涵盖了统计、优化、线性代数、傅里叶变换、信号处理、图像处理和常微分方程求解等众多功能。

SciPy 被组织成覆盖不同科学计算领域的模块，主要模块如下：

模块	功能
scipy.cluster	聚类分析
scipy.constants	物理和数学常数
scipy.fftpack	傅里叶变换
scipy.integrate	积分
scipy.interpolate	插值
scipy.io	数据输入和输出
scipy.linalg	线性代数
scipy.ndimage	n 维图像处理
scipy.odr	正交距离回归
scipy.optimize	优化与方程求解
scipy.signal	信号处理
scipy.sparse	稀疏矩阵
scipy.spatial	空间数据结构和算法
scipy.special	特殊函数
scipy.stats	统计

2.6.2 SciPy 基本操作

求解非线性方程（组）

scipy.optimize 模块的 fsolve 和 root 函数用于求解非线性方程和非线性方程组。

- fsolve(fun, x0): 求解非线性方程（组），fun 为方程函数，x0 为初始值。
- root(fun, x0): 功能与 fsolve 类似，返回更详细的解信息。

使用 fsolve 或 root 求解非线性方程组时，需将方程组写成标准形式：

$$F(x) = 0$$

其中 x 为向量， $F(x)$ 为向量函数。

求解单个方程：

```

1  from scipy.optimize import fsolve, root
2
3  # 定义方程函数
4  fx = lambda x: x**3 - 5*x + 3
5
6  # 求解
7  x1 = fsolve(fx, 1.5)    # 初值为 1.5
8  x2 = root(fx, 1.5)     # 返回详细解信息

```

求解非线性方程组：

将方程组写成向量形式，x[0]、x[1] 分别表示未知数 x_1 、 x_2 。

```

1  from scipy.optimize import fsolve, root
2
3  # 定义方程组:  $x_1^2 + x_2^2 = 1$ ,  $x_1 = x_2$ 
4  fx = lambda x: [x[0]**2 + x[1]**2 - 1, x[0] - x[1]]
5
6  s1 = fsolve(fx, [1, 1])    # 初值为 [1, 1]
7  s2 = root(fx, [1, 1])     # 返回详细解信息

```

数值积分

scipy.integrate 模块提供多种数值积分函数。

函数	说明
quad(func, a, b, args)	计算一重数值积分 $\int_a^b f(x)dx$
dblquad(func, a, b, gfun, hfun, args)	计算二重数值积分
tplquad(func, a, b, gfun, hfun, qfun, rfun)	计算三重数值积分
nquad(func, ranges, args)	计算多变量积分
trapz(y, x)	使用梯形法则对给定离散点进行积分

quad 函数返回一个元组，第一个元素为积分值，第二个元素为绝对误差。

```

1  from scipy.integrate import quad
2
3  # 定义被积函数, a 和 b 为参数
4  def fun(x, a, b):
5      return a * x**2 + b * x
6
7  # 计算积分: a=2, b=1
8  I1 = quad(fun, 0, 1, args=(2, 1))
9  # 计算积分: a=2, b=10
10 I2 = quad(fun, 0, 1, args=(2, 10))

```

最小二乘解

对于非线性齐次方程组：

$$\begin{cases} f_1(x) = 0, \\ f_2(x) = 0, \\ \vdots \\ f_n(x) = 0. \end{cases}$$

其中 x 为 m 维向量，当 $n > m$ 时（超定方程组），要求最小二乘解，即求下式的极小值：

$$\delta(x) = \sum_{i=1}^n f_i^2(x)$$

`scipy.optimize.least_squares` 用于求解非线性方程组的最小二乘解，调用格式为：

$$\text{least_squares}(\text{fun}, \text{x0})$$

参数说明：

- `fun`：定义向量函数 $[f_1(x), f_2(x), \dots, f_n(x)]^T$ 的函数。
- `x0`：未知数的初始值。

示例：信号定位问题

已知观测站坐标 (x_i, y_i) 和测量距离 d_i ，求未知信号源位置 (x, y) ，即求解：

$$\sqrt{(x_i - x)^2 + (y_i - y)^2} - d_i = 0 \quad (i = 1, 2, \dots, n)$$

```

1  from scipy.optimize import least_squares
2  import numpy as np
3
4  # 观测站数据: x0, y0, d
5  x0 = np.array([245, 164, 192, 232])
6  y0 = np.array([442, 480, 281, 300])
7  d = np.array([126.2204, 120.7509, 90.1854, 101.4021])
8
9  # 定义误差向量函数
10 fx = lambda x: np.sqrt((x0 - x[0])**2 + (y0 - x[1])**2) - d
11
12 # 求解最小二乘解
13 s = least_squares(fx, np.random.rand(2))
14 print(s.x)    # 输出求解结果 (x, y)

```

最大模特征值及特征向量

`scipy.sparse.linalg.eigs` 用于计算稀疏矩阵的若干特征值和特征向量，特别适合求解最大模特征值。

- `eigs(A, k)`：返回矩阵 A 的 k 个模最大的特征值和对应的特征向量。
- `np.linalg.eig(A)`：返回矩阵的全部特征值和特征向量。

```

1  from scipy.sparse.linalg import eigs
2  import numpy as np
3
4  # 必须指定 dtype=float
5  a = np.array([[1, 2, 3], [2, 1, 3], [3, 3, 6]], dtype=float)
6

```

```

7      # 计算全部特征值
8      b, c = np.linalg.eig(a)
9
10     # 计算最大模特征值 (k=1 表示只求 1 个)
11     d, e = eigs(a, 1)
12
13     print('最大模特征值为:', d)
14     print('对应的特征向量为:', e)

```

2.6.3 常用函数速查

函数	说明
<code>fsolve(fun, x0)</code>	求解非线性方程（组）
<code>root(fun, x0)</code>	求解非线性方程（组），返回详细信息
<code>quad(func, a, b, args)</code>	一重数值积分
<code>dblquad(func, a, b, gfun, hfun, args)</code>	二重数值积分
<code>tplquad(func, a, b, gfun, hfun, qfun, rfun)</code>	三重数值积分
<code>least_square(fun, x0)</code>	非线性方程组的最小二乘解
<code>eigs(A, k)</code>	稀疏矩阵的 k 个最大模特征值和特征向量
<code>np.linalg.eig(A)</code>	矩阵的全部特征值和特征向量

2.7 SymPy 库

2.7.1 SymPy 简介

符号运算又称计算机代数，通俗地讲就是用计算机推导数学公式，如对表达式进行因式分解、化简、微分、积分、解代数方程、求解常微分方程等。与数值运算相比，符号计算具有以下特点：

- 运算以推理方式进行，不受截断误差和累积误差的影响，结果精确。
- 符号计算速度通常比数值计算慢。

2.7.2 符号变量与符号函数的定义

在 SymPy 中，使用 `symbols()` 或 `var()` 函数定义符号变量和符号函数。

使用 `symbols()`

```

1      import sympy as sp
2
3      # 定义符号变量
4      x, y, z = sp.symbols('x, y, z')      # 用逗号分隔
5      x, y, z = sp.symbols('x y z')        # 用空格分隔（效果相同）

```

```

6
7     # 定义符号函数
8     f, g = sp.symbols('f, g', cls=sp.Function)    # 定义多个符号函数
9     y = sp.Function('y')                        # 定义单个符号函数

```

使用 `var()`

`var()` 函数使用更简洁，会自动将符号名称作为变量名：

```

1     import sympy as sp
2
3     sp.var('x, y, z')                # 定义符号变量，用逗号分隔
4     sp.var('a b c')                  # 定义符号变量，用空格分隔
5     sp.var('f, g', cls=sp.Function)  # 定义符号函数

```

2.7.3 符号方程求解

SymPy 提供了 `solve()` 函数求解符号代数方程或方程组，其调用格式为：

```
solve(f, *symbols)
```

参数说明：

- `f`：符号方程或方程组，方程组使用列表形式 `[eq1, eq2, ...]`。
- `symbols`：要求解的符号变量。

求解单个方程

```

1     import sympy as sp
2
3     a, b, c, x = sp.symbols('a, b, c, x')
4
5     # 解方程 ax^2 + bx + c = 0, 以 x 为未知数
6     x0 = sp.solve(a * x**2 + b * x + c, x)
7     print(x0)

```

求解方程组

```

1     import sympy as sp
2
3     # 方法一：使用 var 定义变量
4     sp.var('x1, x2')
5     s = sp.solve([x1**2 + x2**2 - 1, x1 - x2], [x1, x2])

```

```

6
7     # 方法二：使用符号数组
8     x = sp.var('x:2')           # 生成 x0, x1
9     s = sp.solve([x[0]**2 + x[1]**2 - 1, x[0] - x[1]], x)

1
2
3     import sympy as sp
4
5     sp.var('x, y')
6     s = sp.solve([x + y - 2, x - y - 1], [x, y])

```

2.7.4 符号矩阵运算

SymPy 提供了 `sp.Matrix()` 创建符号矩阵，支持特征值、特征向量等线性代数运算。

- `sp.Matrix([[...], [...]])`: 创建符号矩阵。
- 矩阵.`eigenvals()`: 返回特征值字典，键为特征值，值为重数。
- 矩阵.`eigenvectors()`: 返回特征值、重数、特征向量的列表。

```

1     import sympy as sp
2
3     # 创建符号矩阵
4     a = sp.Matrix([
5         [0, 0, 0, 1],
6         [0, 0, 1, 0],
7         [0, 1, 0, 0],
8         [1, 0, 0, 0]
9     ])
10
11    # 求特征值和特征向量
12    print('特征值为:', a.eigenvals())
13    print('特征向量为:', a.eigenvectors())

```

2.7.5 常用函数速查

函数	说明
<code>symbols('x, y, z')</code>	定义符号变量
<code>symbols('f, g', cls=Function)</code>	定义符号函数
<code>var('x, y, z')</code>	定义符号变量（自动赋值）
<code>var('f, g', cls=Function)</code>	定义符号函数（自动赋值）
<code>solve(eq, x)</code>	求解符号方程或方程组
<code>Matrix([[...], [...]])</code>	创建符号矩阵

续下页

函数	说明
矩阵.eigenvals()	求符号矩阵的特征值
矩阵.eigenvectors()	求符号矩阵的特征值和特征向量

2.8 Matplotlib 库介绍

Matplotlib 是 Python 的绘图库，依赖于 NumPy 和 Tkinter，可以绘制多种形式的图形，包括折线图、散点图、饼图、柱状图、雷达图以及三维图形等。

Matplotlib 包含 pyplot 绘图模块以及大量用于字体、颜色、图例等图形元素的管理与控制模块。其中 pyplot 模块提供了类似于 MATLAB 的绘图接口，可以使用简洁的代码绘制出优美的图形。

使用 pyplot 绘图的一般过程为：

1. 读入或生成数据。
2. 根据实际需要选择图形类型（折线图、散点图、柱状图、饼图、三维曲面等）。
3. 设置坐标轴和图形属性（标签、标题、图例等）。
4. 显示或保存绘图结果。

2.8.1 二维绘图

折线图

matplotlib.pyplot.plot() 函数用于绘制折线图，其常用调用格式为：

plot(x, y, s)

参数说明：

- x: 数据点的 x 坐标。
- y: 数据点的 y 坐标。
- s: 指定线条颜色、线条样式和数据点形状的格式字符串。

plot() 函数也支持使用关键字参数进行详细设置：

```

1 plot(x, y,
2     linestyle,          # 折线类型：实线、虚线、点画线等
3     linewidth,          # 折线宽度
4     color,              # 线条颜色
5     marker,             # 数据点形状
6     markersize,         # 点的大小
7     markeredgecolor,    # 点的边框色
8     markerfacecolor,    # 点的填充色

```

```

9     markeredgewidth,      # 点的边框宽度
10    label,                # 图例标签
11    alpha)                # 透明度

```

符号参数	类型	含义
b	线条颜色	蓝色 (Blue)
c	线条颜色	青色 (Cyan)
g	线条颜色	绿色 (Green)
k	线条颜色	黑色 (Black)
m	线条颜色	品红 (Magenta)
r	线条颜色	红色 (Red)
w	线条颜色	白色 (White)
y	线条颜色	黄色 (Yellow)
-	线条样式	实线 (Solid Line)
-	线条样式	虚线 (Dashed Line)
-.	线条样式	点划线 (Dash-dot Line)
:	线条样式	点线 (Dotted Line)
.	数据点形状	点 (Point)
o	数据点形状	圆圈 (Circle)
*	数据点形状	星形 (Star)
x	数据点形状	十字架 (Cross)
s	数据点形状	正方形 (Square)
P	数据点形状	五角星 (Pentagon)
D/d	数据点形状	钻石 (Diamond / 小钻石)
h	数据点形状	六角形 (Hexagon)
+	数据点形状	加号
v/^	数据点形状	下三角 / 上三角
</>	数据点形状	左三角 / 右三角

示例：绘制折线图

```

1     import pandas as pd
2     import pylab as plt
3
4     # 设置中文字体
5     plt.rc('font', family='SimHei', size=16)
6
7     # 读取数据
8     a = pd.read_excel("data.xlsx", header=None)
9     x = a.values[0]          # 月份数据
10    y = a.values[1:]         # 销量数据
11
12    # 绘制多条折线
13    plt.plot(x, y[0], '-*b', label='钻石')
14    plt.plot(x, y[1], '-dr', label='铂金')

```

```

15
16     # 设置标签、图例、网格并显示
17     plt.xlabel('月份')
18     plt.ylabel('每月销量')
19     plt.legend(loc='upper left')
20     plt.grid()
21     plt.show()

```

中文字体设置

Matplotlib 默认字体不支持中文，需进行如下设置：

```

1     # 方法一：使用 rcParams
2     plt.rcParams['font.sans-serif'] = ['SimHei']           # 正常显示中
3     # 文标签
4     plt.rcParams['axes.unicode_minus'] = False           # 正常显示负
5     # 号
6     # 方法二：使用 rc（等价写法）
7     plt.rc('font', family='SimHei')                     # 正常显示中
8     # 文标签
9     plt.rc('axes', unicode_minus=False)                  # 正常显示负
10    # 号

```

柱状图

Pandas 的 DataFrame 对象提供了 plot() 方法，可自动调用 Matplotlib 绘图。

```

1     import pandas as pd
2     import pylab as plt
3
4     plt.rc('font', family='SimHei', size=16)
5
6     # 读取数据
7     a = pd.read_excel("data.xlsx", header=None)
8     b = a.T
9
10    # 绘制柱状图
11    b.plot(kind='bar')
12    plt.legend(['钻石', '铂金'])
13    plt.ylabel('数量')
14    plt.show()

```

kind 参数说明：

- 'line': 折线图（默认）

- 'bar': 垂直柱状图
- 'barh': 水平柱状图
- 'pie': 饼图
- 'scatter': 散点图

子图

使用 `plt.subplot()` 可以将一个窗口划分为多个子图区域，其调用格式为：

`subplot(m, n, p)`

表示将窗口划分为 $m \times n$ 个子图区域，并选中第 p 个区域进行绘制（编号从左到右、从上到下）。

```

1  import pylab as plt
2  import numpy as np
3
4  # 生成数据
5  y1 = np.random.randint(2, 5, 6)
6  y1 = y1 / sum(y1)
7  str_labels = ['Apple', 'grape', 'peach', 'pear', 'banana', '
    pineapple']
8
9  # 子图 1: 水平条形图 (2行2列的第1个位置)
10 plt.subplot(2, 2, 1)
11 plt.barh(str_labels, y1)
12
13 # 子图 2: 饼图 (2行2列的第2个位置)
14 plt.subplot(222)           # 等价于 subplot(2, 2, 2)
15 plt.pie(y1, labels=str_labels)
16
17 # 子图 3: 折线图 (2行2列的第3、4个位置, 即下方合并)
18 plt.subplot(212)           # 等价于 subplot(2, 1, 2)
19 x2 = np.linspace(0.01, 10, 100)
20 y2 = np.sin(10 * x2) / x2
21 plt.plot(x2, y2)
22 plt.xlabel('$x$')
23 plt.ylabel('$y$')
24
25 plt.show()

```

2.8.2 三维绘图

绘制三维图形需要创建 3D 坐标轴对象：

```
ax = plt.axes(projection='3d')
```

三维曲线

`ax.plot(x, y, z)` 用于绘制三维曲线。

```
1  import pylab as plt
2  import numpy as np
3
4  # 设置三维图形模式
5  ax = plt.axes(projection='3d')
6
7  # 生成数据
8  z = np.linspace(-50, 50, 1000)
9  x = z**2 * np.sin(z)
10 y = z**2 * np.cos(z)
11
12 # 绘制三维曲线
13 ax.plot(x, y, z, 'k')
14 plt.show()
```

三维曲面图

`ax.plot_surface(X, Y, Z)` 用于绘制三维曲面图。

```
1  import pylab as plt
2  import numpy as np
3
4  # 设置三维图形模式
5  ax = plt.axes(projection='3d')
6
7  # 生成网格数据
8  x = np.linspace(-4, 4, 100)
9  y = np.linspace(-4, 4, 100)
10 X, Y = np.meshgrid(x, y)
11 Z = 50 * np.sin(X + Y)
12
13 # 绘制曲面
14 ax.plot_surface(X, Y, Z, color='y')
15 plt.show()
```

```
1  import pylab as plt
2  import numpy as np
3
4  ax = plt.axes(projection='3d')
5
```

```

6     X = np.arange(-6, 6, 0.25)
7     Y = np.arange(-6, 6, 0.25)
8     X, Y = np.meshgrid(X, Y)
9     Z = np.sin(np.sqrt(X**2 + Y**2))
10
11     # 使用颜色映射绘制曲面，并添加颜色条
12     surf = ax.plot_surface(X, Y, Z, cmap='coolwarm')
13     plt.colorbar(surf)
14     plt.show()

```

2.8.3 常用函数速查

函数	说明
plot(x, y, s)	绘制折线图
scatter(x, y)	绘制散点图
bar(x, height)	绘制垂直柱状图
barh(y, width)	绘制水平柱状图
pie(data, labels)	绘制饼图
hist(data)	绘制直方图
xlabel() / ylabel()	设置 x/y 轴标签
title()	设置图形标题
legend()	显示图例
grid()	显示网格
subplot(m, n, p)	创建子图
axes(projection='3d')	创建 3D 坐标轴
plot_surface(X, Y, Z)	绘制三维曲面图
colorbar()	添加颜色条
show()	显示图形
savefig()	保存图形

第 3 章

线性代数模型

线性代数是处理矩阵和向量空间的数学分支，在很多实际领域都有应用。本章主要结合斐波那契数列、莱斯利模型和 PageRank 算法，介绍特征值和特征向量的应用，最后给出奇异值分解在推荐算法和图像压缩中的应用。

3.1 特征值与特征向量

3.1.1 差分方程与斐波那契数列

问题描述

斐波那契 (Fibonacci) 数列源于 13 世纪提出的兔子繁殖问题：

一对兔子出生一个月后开始繁殖，每个月出生一对新生兔子。假定兔子只繁殖，没有死亡，问第 k 个月初会有多少对兔子？

以对为单位，每个月繁殖兔子对数构成斐波那契数列：

$$1, 1, 2, 3, 5, 8, 13, 21, \dots$$

该数列满足二阶差分方程：

$$F_0 = 1, \quad F_1 = 1, \quad F_{k+2} = F_{k+1} + F_k \quad (k = 0, 1, 2, \dots)$$

目标是求 F_k 的通项公式。

数学原理: 特征值法

将二阶差分方程化为一阶差分方程组。

令 $\alpha_k = \begin{bmatrix} F_k \\ F_{k+1} \end{bmatrix}$ ，则：

$$\alpha_{k+1} = A\alpha_k, \quad A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad \alpha_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

递推得：

$$\alpha_k = A^k \alpha_0$$

求 F_k 转化为求 A^k 。由特征方程：

$$|\lambda E - A| = \begin{vmatrix} \lambda & -1 \\ -1 & \lambda - 1 \end{vmatrix} = \lambda^2 - \lambda - 1 = 0$$

得特征值：

$$\lambda_1 = \frac{1 - \sqrt{5}}{2}, \quad \lambda_2 = \frac{1 + \sqrt{5}}{2}$$

对应特征向量：

$$\xi_1 = \begin{bmatrix} \frac{\sqrt{5}+1}{2} \\ 1 \end{bmatrix}, \quad \xi_2 = \begin{bmatrix} \frac{\sqrt{5}-1}{2} \\ 1 \end{bmatrix}$$

令 $P = [\xi_1, \xi_2]$ ，则 $A = P \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} P^{-1}$ ，从而：

$$A^k = P \begin{bmatrix} \lambda_1^k & 0 \\ 0 & \lambda_2^k \end{bmatrix} P^{-1}$$

最终得到斐波那契数列通项：

$$F_k = \left(\frac{1}{2} - \frac{\sqrt{5}}{10} \right) \left(\frac{1 - \sqrt{5}}{2} \right)^k + \left(\frac{1}{2} + \frac{\sqrt{5}}{10} \right) \left(\frac{1 + \sqrt{5}}{2} \right)^k$$

数值验证

当 $k = 19$ 时， $F_{19} = 6765$ ，即 19 个月后有 6765 对兔子。

代码实现

方法一：特征值对角化法

```

1  # 程序文件 ex3_1_1.py
2  import sympy as sp
3
4  sp.var('k', positive=True, integer=True)
5  a = sp.Matrix([[0, 1], [1, 1]])
6
7  val = a.eigenvals()           # 求特征值
8  vec = a.eigenvectors()        # 求特征向量
9  P, D = a.diagonalize()        # 相似对角化
10
11  ak = P @ (D ** k) @ (P.inv())
12  F = ak @ sp.Matrix([[1, 1]])

```



```

13     s = sp.simplify(F[0])
14
15     print(s)                                # 输出通项公式
16
17     # 验证前 20 项
18     sm = []
19     for i in range(20):
20         sm.append(s.subs(k, i).n())
21     print(sm)

```

方法二：差分方程特征根法

```

1     # 程序文件 ex3_1_2.py
2     import sympy as sp
3
4     sp.var('t, c1, c2')
5
6     # 求解特征方程
7     t0 = sp.solve(t**2 - t - 1, t)
8
9     # 利用初值条件建立方程组
10    eq1 = c1 + c2 - 1
11    eq2 = c1 * t0[0] + c2 * t0[1] - 1
12
13    s = sp.solve([eq1, eq2], [c1, c2])
14    print('c1 =', s[c1])
15    print('c2 =', s[c2])

```

方法三：直接使用 SymPy 求解差分方程

```

1     # 程序文件 ex3_1_3.py
2     import sympy as sp
3
4     sp.var('k')
5     y = sp.Function('y')
6
7     # 定义差分方程:  $y(k+2) - y(k+1) - y(k) = 0$ 
8     f = y(k + 2) - y(k + 1) - y(k)
9
10    s = sp.rsolve(f, y(k), {y(0): 1, y(1): 1})
11    print(s)

```

三种方法对比

方法	核心思想	优点	缺点
特征值对角化法	将二阶差分方程转化为一阶线性方程组，通过特征分解求 A^k	适用于任何线性差分方程组，通用性强	计算量大，需要求特征值和特征向量
特征根法	直接求解差分方程的特征方程，利用初值定常数	计算简单，适合手算推导	只适用于常系数齐次线性差分方程
SymPy 直接求解	调用符号计算库直接求解差分方程	代码最简洁，无需推导数学公式	需要安装 SymPy，无法展示中间过程

方法选择建议

- 理论分析：使用特征根法，形式简洁，便于推导通项。
- 一般数值计算：使用特征值对角化法，适用于任意线性差分方程组。
- 快速验证：使用 SymPy 的 `rsolve()`，一行代码直接求解。

应用场景

斐波那契数列及其差分方程模型可推广到：

- 人口增长模型（Leslie 矩阵）
- 经济时间序列预测
- 算法复杂度分析（如递归算法）
- 自然界中的斐波那契结构（花瓣数、螺线等）

3.1.2 莱斯利（Leslie）种群模型

问题描述

莱斯利模型是研究动物种群数量增长的重要模型，用于分析种群中雌性动物的年龄分布和数量增长规律。

基本假设：

- 仅考虑雌性动物，忽略雄性（假设雄性充足，不影响繁殖）。
- 最大生存年龄为 L ，将其等分为 n 个年龄组，组距为 L/n 。
- 第 i 个年龄组的生育率为 a_i （每只雌性平均生育雌性后代数），存活率为 b_i （存活到下一年龄组的比例）。
- $a_i \geq 0$, $0 < b_i \leq 1$ ，且至少有一个 $a_i > 0$ 。

已知初始时刻各年龄组雌性数量 $x^{(0)} = [x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)}]^T$, 目标是预测任意时刻 t_k 各年龄组的数量分布。

数学原理

设 $x_i^{(k)}$ 为 t_k 时刻第 i 年龄组的雌性数量。

递推关系:

1. 第一年龄组 (新生雌性): 由所有年龄组在上一时间段内繁殖产生

$$x_1^{(k)} = a_1 x_1^{(k-1)} + a_2 x_2^{(k-1)} + \dots + a_n x_n^{(k-1)}$$

2. 其他年龄组: 由上一年龄组存活下来

$$x_{i+1}^{(k)} = b_i x_i^{(k-1)}, \quad i = 1, 2, \dots, n-1$$

写成矩阵形式:

$$x^{(k)} = Lx^{(k-1)}, \quad k = 1, 2, \dots$$

其中 L 为莱斯利矩阵:

$$L = \begin{bmatrix} a_1 & a_2 & \cdots & a_{n-1} & a_n \\ b_1 & 0 & \cdots & 0 & 0 \\ 0 & b_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & b_{n-1} & 0 \end{bmatrix}$$

递推得:

$$x^{(k)} = L^k x^{(0)}$$

长期行为分析:

若 L 可对角化, 设 λ_1 为 L 的唯一正特征值 (主特征值), 且 $\lambda_1 > |\lambda_j|$ ($j \geq 2$), 则:

$$\lim_{k \rightarrow \infty} \frac{1}{\lambda_1^k} x^{(k)} = c\alpha_1$$

其中 α_1 为 λ_1 对应的特征向量, c 为常数。

结论: 当 k 充分大时, 种群年龄分布趋于稳定比例 (即 α_1 的各分量比例), 总量以 λ_1 为公比增长。

具体例子

问题: 某种动物雌性最大生存年龄为 15 年, 以 5 年为一间隔分为 3 个年龄组 $[0, 5)$ 、 $[5, 10)$ 、 $[10, 15]$ 。已知:

$$a_1 = 0, \quad a_2 = 4, \quad a_3 = 3, \quad b_1 = 0.5, \quad b_2 = 0.25$$

初始时刻 3 个年龄组雌性数量分别为：

$$x^{(0)} = [500, 1000, 500]^T$$

求各时刻种群分布及长期趋势。

莱斯利矩阵：

$$L = \begin{bmatrix} 0 & 4 & 3 \\ 0.5 & 0 & 0 \\ 0 & 0.25 & 0 \end{bmatrix}$$

递推计算：

$$x^{(1)} = Lx^{(0)} = \begin{bmatrix} 5500 \\ 250 \\ 250 \end{bmatrix}, \quad x^{(2)} = Lx^{(1)} = \begin{bmatrix} 1750 \\ 2750 \\ 62.5 \end{bmatrix}, \quad x^{(3)} = Lx^{(2)} = \begin{bmatrix} 11187.5 \\ 875 \\ 687.5 \end{bmatrix}$$

特征分析：

特征多项式：

$$|\lambda E - L| = \left(\lambda - \frac{3}{2}\right) \left(\lambda^2 + \frac{3}{2}\lambda + \frac{1}{4}\right)$$

特征值：

$$\lambda_1 = \frac{3}{2}, \quad \lambda_2 = \frac{-3 + \sqrt{5}}{4}, \quad \lambda_3 = \frac{-3 - \sqrt{5}}{4}$$

λ_1 为主特征值，对应的特征向量：

$$\alpha_1 = [18, 6, 1]^T$$

当 k 充分大时：

$$x^{(k)} \approx c \left(\frac{3}{2}\right)^k [18, 6, 1]^T$$

即 3 个年龄组的数量比稳定为 $18 : 6 : 1$ ，种群总量以 $\lambda_1 = 1.5$ 为公比增长。

代码实现

```

1  # 程序文件 ex3_2.py
2  import numpy as np
3  import sympy as sp
4
5  # 初始数据
6  X0 = np.array([500, 1000, 500])
7  L = np.array([[0, 4, 3], [0.5, 0, 0], [0, 0.25, 0]])
8
9  # 数值计算：递推 3 步
10 X1 = L @ X0

```

```

11 X2 = L @ X1
12 X3 = L @ X2
13 print("X1 =", X1)
14 print("X2 =", X2)
15 print("X3 =", X3)
16
17 # 符号计算：特征值与特征向量
18 Ls = sp.Matrix([
19     [0, 4, 3],
20     [sp.Rational(1, 2), 0, 0],
21     [0, sp.Rational(1, 4), 0]
22 ])
23
24 sp.var('lamda')
25
26 # 方法一：特征多项式求根
27 p = Ls.charpoly(lamda)
28 w1 = sp.roots(p)
29
30 # 方法二：直接求特征值
31 w2 = Ls.eigenvals()
32
33 # 求特征向量
34 v = Ls.eigenvectors()
35
36 print("特征值:", w2)
37 print("特征向量:", v)
38
39 # 相似对角化
40 P, D = Ls.diagonalize()
41 Pinv = P.inv()
42 Pinv = sp.simplify(Pinv)
43
44 # 计算常数 c
45 cc = Pinv @ X0
46 print("P = \n", P)
47 print("c = ", cc[0])

```

输出结果：

```

1 X1 = [5500.  250.  250.]
2 X2 = [1750.  2750.   62.5]
3 X3 = [11187.5  875.   687.5]
4
5 特征值: {3/2: 1, (-3 + sqrt(5))/4: 1, (-3 - sqrt(5))/4: 1}
6 c = 2250/19

```

优缺点分析

方法	优点	缺点
递推法	计算简单直观,易于编程实现	只能得到数值结果,无法分析长期趋势
特征值分析法	可分析长期稳定性和增长率,揭示模型本质	需要计算特征值,高维时计算复杂
符号算法	得到精确解析解,便于理论分析	高维时符号计算速度慢,可能无法求解

方法选择建议

- 仅需数值预测：直接使用递推法 $x^{(k)} = L^k x^{(0)}$ ，简单高效。
- 需要分析长期趋势：计算主特征值 λ_1 和对应特征向量，确定稳定年龄分布和增长率。
- 高维模型：使用数值线性代数方法（如幂法）求主特征值，避免符号计算开销。

应用场景

- 野生动物保护：预测濒危物种种群数量变化，制定保护策略。
- 渔业资源管理：估算鱼类种群年龄结构，确定合理捕捞量。
- 人口预测：分析人口年龄结构，预测未来人口总量和老龄化趋势。
- 害虫防治：预测害虫种群爆发期，制定防治方案。

莱斯利矩阵不可对角化的处理

问题背景 在前面的讨论中，我们假设莱斯利矩阵 L 可相似对角化，即存在可逆矩阵 P 和对角矩阵 Λ ，使得 $L = P\Lambda P^{-1}$ 。然而，当 L 的特征值存在重根且几何重数小于代数重数时， L 不可对角化。此时需要采用更一般的 **Jordan** 标准形理论进行分析。

数学原理：Jordan 标准形 对于任意复方阵 L ，存在可逆矩阵 P ，使得：

$$L = PJP^{-1}$$

其中 J 为 Jordan 标准形，是一个分块对角矩阵：

$$J = \begin{bmatrix} J_1 & & \\ & J_2 & \\ & & \ddots \\ & & & J_m \end{bmatrix}, \quad J_i = \begin{bmatrix} \lambda_i & 1 & & \\ & \lambda_i & 1 & \\ & & \ddots & \ddots \\ & & & \ddots & 1 \\ & & & & \lambda_i \end{bmatrix}_{r_i \times r_i}$$

每个 J_i 称为一个 Jordan 块, r_i 为 Jordan 块的阶数。

Jordan 块的幂次:

设 J_i 为 $r \times r$ 的 Jordan 块 (对应于特征值 λ_i), 则其 k 次幂为:

$$J_i^k = \begin{bmatrix} \lambda_i^k & \binom{k}{1}\lambda_i^{k-1} & \binom{k}{2}\lambda_i^{k-2} & \cdots & \binom{k}{r-1}\lambda_i^{k-r+1} \\ & \lambda_i^k & \binom{k}{1}\lambda_i^{k-1} & \cdots & \binom{k}{r-2}\lambda_i^{k-r+2} \\ & & \ddots & \ddots & \vdots \\ & & & \ddots & \binom{k}{1}\lambda_i^{k-1} \\ & & & & \lambda_i^k \end{bmatrix}$$

其中二项式系数 $\binom{k}{j} = \frac{k(k-1)\cdots(k-j+1)}{j!}$ 是 k 的 j 次多项式。

长期行为分析:

设 λ_1 为主特征值, 满足 $\lambda_1 > |\lambda_j|$ ($j \geq 2$), 对应的 Jordan 块大小为 r 。当 k 充分大时:

$$J_i^k \approx \lambda_i^k \cdot (\text{与 } k \text{ 有关的低阶多项式项})$$

具体地, 若主特征值 λ_1 对应的 Jordan 块大小为 r , 则:

$$L^k = P J^k P^{-1} \approx P \begin{bmatrix} J_1^k & & \\ & \ddots & \\ & & 0 \end{bmatrix} P^{-1}$$

其中 J_1^k 中最大的项为 $\binom{k}{r-1}\lambda_1^{k-r+1} \sim \frac{k^{r-1}}{(r-1)!}\lambda_1^{k-r+1}$ 。

因此种群分布的长期行为分为两种情况:

1. 主特征值对应的 **Jordan** 块大小为 1 (即特征值几何重数等于代数重数):

$$x^{(k)} = c\lambda_1^k\alpha_1 + o(\lambda_1^k)$$

年龄分布比例趋于稳定, 总量以 λ_1^k 纯指数增长。

2. 主特征值对应的 **Jordan** 块大小 $r > 1$ (不可对角化情形):

$$x^{(k)} \approx c \cdot k^{r-1}\lambda_1^{k-r+1}$$

种群总量包含 k 的多项式因子, 出现 多项式 $\times$ 指数增长, 而非纯指数增长。

关键结论 即使莱斯利矩阵不可对角化，长期年龄分布比例仍趋于稳定（由主特征值对应的特征向量决定），但种群总量的增长模式可能发生改变：

$$\boxed{\text{可对角化：总量} \sim C\lambda_1^k} \quad \text{vs} \quad \boxed{\text{不可对角化：总量} \sim C \cdot k^{r-1}\lambda_1^k}$$

代码实现

```

1  # 程序文件 ex3_2_jordan.py
2  import numpy as np
3  import sympy as sp
4
5  def analyze_leslie_jordan(L, X0):
6      """
7      分析莱斯利矩阵，自动处理可对角化和不可对角化情况
8      """
9      Ls = sp.Matrix(L)
10
11     # 1. 计算特征值
12     eigenvals = Ls.eigenvals()
13     print("特征值:", eigenvals)
14
15     # 2. 尝试 Jordan 标准形分解
16     try:
17         J, P = Ls.jordan_form()
18         print("\nJordan 标准形 J = \n", J)
19         print("\n变换矩阵 P = \n", P)
20
21     # 判断是否可对角化
22     is_diag = (J == sp.diag(*J.diagonal()))
23     print("\n是否可对角化:", is_diag)
24
25     # 3. 计算 J^k (符号形式)
26     k = sp.var('k', positive=True, integer=True)
27     Jk = J ** k
28     Lk = P * Jk * P.inv()
29
30     # 4. 预测长期趋势
31     X0_sym = sp.Matrix(X0)
32     Xk = Lk * X0_sym
33     Xk_simplified = sp.simplify(Xk)
34
35     print("\n第 k 时刻种群分布向量:")
36     print(Xk_simplified)
37
38     # 5. 提取主特征值
39     lam_list = list(eigenvals.keys())
40     lam1 = max(lam_list, key=lambda z: abs(float(z)))

```



```

41     print(f"\n主特征值 1 = {lam1}")
42
43     # 6. 获取 Jordan 块信息
44     for val, mult in eigenvals.items():
45         if sp.simplify(val - lam1) == 0:
46             # 计算该特征值对应的 Jordan 块大小
47             nullity = (Ls - val * sp.eye(Ls.shape[0])).nullspace()
48             geom_mult = len(nullity)
49             print(f"1 的代数重数: {mult}, 几何重数: {geom_mult}")
50             if mult > geom_mult:
51                 print("警告: 矩阵不可对角化! 存在 Jordan 块大小 > 1")
52                 break
53
54     return Lk, Xk_simplified
55
56     except Exception as e:
57         print("Jordan 分解失败:", e)
58         return None, None
59
60
61     # ===== 示例 1: 可对角化矩阵 =====
62     print("=" * 60)
63     print("示例 1: 可对角化莱斯利矩阵")
64     print("=" * 60)
65
66     X0 = [500, 1000, 500]
67     L1 = [[0, 4, 3], [0.5, 0, 0], [0, 0.25, 0]]
68     Lk1, Xk1 = analyze_leslie_jordan(L1, X0)
69
70
71     # ===== 示例 2: 不可对角化矩阵 =====
72     print("\n" + "=" * 60)
73     print("示例 2: 不可对角化莱斯利矩阵 (含 Jordan 块)")
74     print("=" * 60)
75
76     # 构造一个含 Jordan 块的矩阵
77     L2 = [[2, 1, 0], [0, 2, 0], [0, 0.5, 0]]
78     X0_2 = [100, 50, 20]
79     Lk2, Xk2 = analyze_leslie_jordan(L2, X0_2)

```

算法评价

方法	优点	缺点
特征值分析法	揭示本质,适用大多数情况	不可对角化时失效
Jordan 标准形法	完全通用,处理所有情况	理论复杂, 计算量大
数值递推法	简单通用,可处理任意矩阵	只能得数值解,无法分析长期趋势
幂法	专门求主特征值,计算高效	只能求主特征值,无法获得完整信息

方法选择建议

- 一般应用（可对角化）：使用特征值分解 $L = P\Lambda P^{-1}$ ，简单直观。
- 矩阵不可对角化或不定时：使用 Jordan 标准形 $L = PJP^{-1}$ ，完全通用。
- 仅需长期增长率：使用幂法求主特征值，无需判断是否可对角化。
- 仅需数值预测：直接递推 $x^{(k)} = L^k x^{(0)}$ ，适用于任意矩阵。

实际应用中的注意事项

1. 数据准确性：莱斯利矩阵中的生育率 a_i 和存活率 b_i 需基于可靠的统计资料，参数微小变化可能导致长期预测出现较大偏差。
2. 模型假设限制：模型假设生育率和存活率为常数，实际中可能随种群密度、环境变化而改变。
3. 高维问题：当年龄组数量 n 较大时，符号计算求解 Jordan 标准形变得困难，建议使用数值方法。
4. 主特征值判断：当存在多个模最大的特征值时（如复数共轭对），种群可能出现周期性波动。

3.1.3 PageRank 算法

Google 拥有多项专利技术，其中 PageRank 算法是关键技术之一，它奠定了 Google 强大的搜索引擎功能的基础。虽然 Google 每天有很多工程师负责全面改进 Google 系统，但是仍把 PageRank 算法作为所有网络搜索引擎的基础结构。

算法概述

PageRank 算法是 Google 搜索引擎对检索结果的一种排序算法。它的基本思想是：搜索引擎在检索结果中，将每个网页的重要性用权值表示，权值越高，表示该网页越重要。Google 在给出页面排序时也有两条标准：

1. 看有多少超链接指向它；

2. 看超链接指向它的那个页面重要不重要。

PageRank 算法利用了互联网独特的超链接结构。具体地讲，就是把所有的网页看作图里面的相应顶点，如果网页 A 有一个指向网页 B 的链接，则认为存在一条从顶点 A 到顶点 B 的有向边，这样就可以利用图论来研究网络的拓扑结构。

数学原理

(1) 基础 PageRank 算法 PageRank 算法的具体实现可以利用网页所对应图的邻接矩阵来表达超链接关系。

步骤 1：构造有向图

设 $V = \{v_1, v_2, \dots, v_N\}$ 为顶点集合，每一个网页是图的一个顶点。定义邻接矩阵 $W = (w_{ij})_{N \times N}$ ，如果从网页 i 到网页 j 有超链接，则 $w_{ij} = 1$ ，否则为 0。

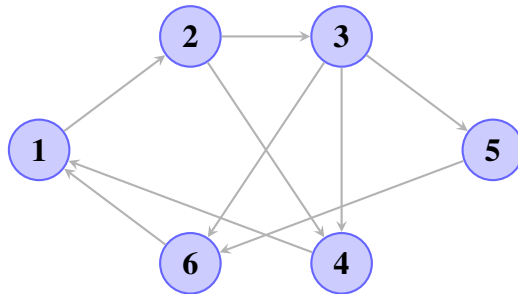


图 3.1 网页链接结构的有向图 (例 3.3)

步骤 2：构造状态转移概率矩阵

记矩阵 W 的行和为 $r_i = \sum_{j=1}^N w_{ij}$ ，它表示页面 i 的链出链接数目。定义矩阵 $P = (p_{ij})_{N \times N}$ ：

$$p_{ij} = \frac{w_{ij}}{r_i}, \quad i, j = 1, 2, \dots, N$$

P 为马尔可夫链的状态转移概率矩阵， p_{ij} 为从页面 i 转移到页面 j 的概率。

步骤 3：求平稳分布

求马尔可夫链的平稳分布 $x = [x_1, x_2, \dots, x_N]^T$ ，它满足：

$$P^T x = x, \quad \sum_{i=1}^N x_i = 1$$

x 表示在极限状态（转移次数趋于无限）下各网页被访问的概率分布，Google 将它定义为各网页的 PageRank 值。

根据马尔可夫链的基本性质，平稳分布（PageRank 值）是状态转移概率矩阵 P 的转置矩阵 P^T 的最大特征值（ $\lambda = 1$ ）所对应的归一化特征向量。

(2) 随机冲浪模型 基础 PageRank 算法要求所有网页形成一个闭合的链接图。但现实网络中：

- 存在没有链出链接的页面（悬挂页面）；
- 存在只有出链接没有入链接的页面；
- 存在链接只在集合内部循环的现象。

为解决这些问题，Google 提出了随机冲浪模型：用户在 85% 的情况下沿着链接前进，在 15% 的情况下随机跳跃到无关页面。

转移概率矩阵为：

$$\tilde{P} = \frac{1-d}{N}ee^T + dP$$

其中 $d = 0.85$ 为阻尼因子（damping factor）， e 为全 1 列向量。

具体例子

例 3.3（基础 PageRank 算法）

已知一个 $N = 6$ 的网络如图所示，求它的 PageRank 取值。

解：

邻接矩阵 W 和转移概率矩阵 P 分别为：

$$W = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad P = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 & 0 \\ 0 & 0 & 0 & 1/3 & 1/3 & 1/3 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

计算得到平稳分布：

$$x = [0.2727, 0.2727, 0.1364, 0.1818, 0.0455, 0.0909]^T$$

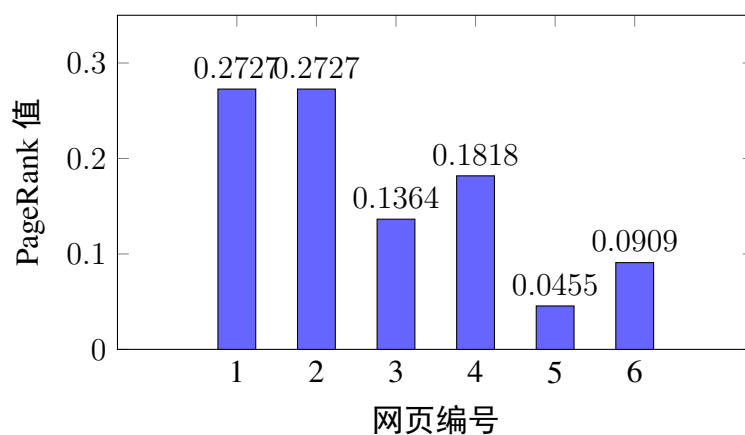


图 3.2 基础 PageRank 值柱状图

网页排序：1 > 2 > 4 > 3 > 6 > 5

例 3.4 (随机冲浪模型)

用随机冲浪模型计算同一网络的 PageRank 值, 取 $d = 0.85$ 。

解:

$$\tilde{P}_{ij} = \frac{1 - 0.85}{6} + 0.85 \frac{w_{ij}}{r_i}$$

计算得 $\tilde{P}^T$ 的最大特征值对应的归一化特征向量:

$$x = [0.2675, 0.2524, 0.1323, 0.1697, 0.0625, 0.1156]^T$$

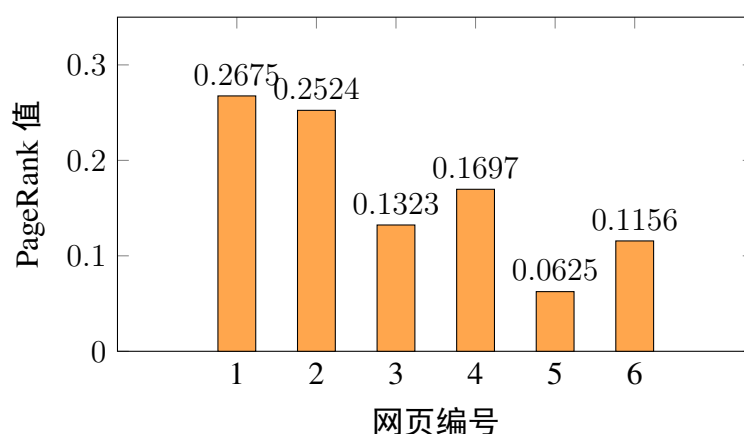


图 3.3 随机冲浪模型的 PageRank 值柱状图

网页排序: $1 > 2 > 4 > 3 > 6 > 5$, 与例 3.3 结果基本一致。

代码实现

基础 PageRank 算法

```

1  # 程序文件 ex3_3.py
2  import numpy as np
3  from scipy.sparse.linalg import eigs
4  import pylab as plt
5
6  # 定义有向边 (from, to)
7  L = [(1,2), (2,3), (2,4), (3,4), (3,5), (3,6), (4,1), (5,6),
8       (6,1)]
9
10 # 构造邻接矩阵
11 w = np.zeros((6, 6))
12 for i in range(len(L)):
13     w[L[i][0] - 1, L[i][1] - 1] = 1
14
15 # 行归一化得到转移概率矩阵
16 r = np.sum(w, axis=1, keepdims=True)

```

```

16     P = w / r
17
18     # 求  $P^T$  的最大特征值对应的特征向量
19     val, vec = eigs(P.T, 1)
20     V = vec.real.flatten()
21     V = V / V.sum()
22
23     print("V =", np.round(V, 4))
24
25     plt.bar(range(1, 7), V, width=0.6, color='b')
26     plt.show()

```

随机冲浪模型

```

1     # 程序文件 ex3_4.py
2     import numpy as np
3     from scipy.sparse.linalg import eigs
4     import pylab as plt
5
6     L = [(1,2), (2,3), (2,4), (3,4), (3,5), (3,6), (4,1), (5,6),
7           (6,1)]
8
9     w = np.zeros((6, 6))
10    for i in range(len(L)):
11        w[L[i][0] - 1, L[i][1] - 1] = 1
12
13    r = np.sum(w, axis=1, keepdims=True)
14
15    d = 0.85 # 阻尼因子
16    P = (1 - d) / w.shape[0] + d * w / r
17
18    val, vec = eigs(P.T, 1)
19    V = vec.real.flatten()
20    V = V / V.sum()
21
22    print("V =", np.round(V, 4))
23
24    plt.bar(range(1, 7), V, width=0.6, color='orange')
25    plt.show()

```

优缺点分析

方法	优点	缺点
基础 PageRank	直接反映链接结构的权威性，计算简单	存在悬挂页面和死胡同问题
随机冲浪模型	解决悬挂页面问题，更符合实际用户行为	阻尼因子取值影响结果，需额外计算
幂法求解	适合大规模稀疏矩阵，每次迭代 $O(N)$	收敛速度依赖谱间隙，可能收敛较慢

方法选择建议

- 理想链接结构：使用基础 PageRank 算法，计算简单直接。
- 实际网络（存在悬挂页面）：使用随机冲浪模型，取 $d = 0.85$ 。
- 大规模网络：使用幂法迭代，配合稀疏矩阵存储，每次迭代 $O(N)$ 。
- 快速近似：设置较少的迭代次数，或用稀疏特征值求解器（如 eigs）。

应用场景

- 搜索引擎排名：Google 等搜索引擎的核心排序算法。
- 学术论文影响力评估：评估学术论文的重要性的影响力。
- 社交网络分析：识别社交网络中的关键意见领袖（KOL）。
- 推荐系统：基于物品的链接关系进行重要性排序。
- 知识图谱：对知识图谱中的实体进行重要性评估。

3.1.4 奇异值分解应用

奇异值分解（Singular Value Decomposition, SVD）是矩阵分解的重要方法，在推荐系统、图像压缩、数据降维等领域有广泛应用。本节通过两个具体例子——推荐系统评分预测和图像压缩，介绍 SVD 的实际应用。

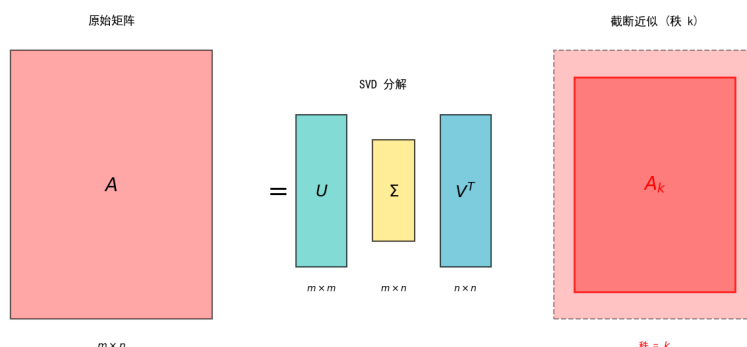
算法概述

SVD 将任意矩阵 $A \in \mathbb{R}^{m \times n}$ 分解为：

$$A = U \Sigma V^T$$

其中 U 和 V 是正交矩阵， Σ 是对角矩阵，对角线元素为奇异值 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ 。

SVD 的核心思想是：通过保留最大的 k 个奇异值及其对应的奇异向量，可以在信息损失最小的条件下实现数据压缩和降维。

图 3.4 SVD 分解示意图: $A = U\Sigma V^T$

数学原理

(1) **SVD 降维与压缩** 对 A 进行奇异值分解, 得到 $A = U\Sigma V^T$, 其中:

$$\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r)$$

选择前 k 个奇异值进行截断, 得到 A 的 k 秩近似:

$$A_k = U_k \Sigma_k V_k^T = \sum_{i=1}^k \sigma_i u_i v_i^T$$

信息累积贡献率定义为:

$$\text{rate} = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}$$

通常选择使累积贡献率达到 90% 或 95% 的 k 值。

压缩比率定义为:

$$p = \left(1 - \frac{k(m+n+1)}{mn}\right) \times 100\%$$

(2) **推荐系统中的协同过滤** 推荐系统的核心是预测用户对未评分物品的评分。基本思路是:

1. 计算物品之间的相似度 (相关系数或余弦相似度)
2. 利用用户已评分的物品分值, 通过相似度加权估计未评分物品的分值

相似度计算 (皮尔逊相关系数):

$$r_{ij} = \frac{\sum_{k=1}^m (a_{ki} - \mu_i)(a_{kj} - \mu_j)}{\sqrt{\sum_{k=1}^m (a_{ki} - \mu_i)^2} \sqrt{\sum_{k=1}^m (a_{kj} - \mu_j)^2}}$$

归一化: $\tilde{r}_{ij} = 0.5 + 0.5r_{ij}$, 值越接近 1 表示相似度越高。

评分估计（加权平均）：

$$s_k = \frac{\sum_{i=1}^{n_k} \tilde{r}_{kj_i} s_{j_i}}{\sum_{i=1}^{n_k} \tilde{r}_{kj_i}}$$

(3) 余弦相似度 对于两个向量 v_1 和 v_2 ，余弦相似度为：

$$\cos \theta = \frac{v_1 \cdot v_2}{\|v_1\| \|v_2\|}$$

归一化： $\text{sim} = 0.5 + 0.5 \cos \theta$ 。

具体例子

例 3.6：推荐系统评分预测

有 18 名用户对 11 道菜品进行打分，分数为 1-5 分，未消费为 0 分。原始评分矩阵 $A \in \mathbb{R}^{18 \times 11}$ 为稀疏矩阵。

模型一：非压缩数据的协同过滤

基于原始评分矩阵，计算菜品间的皮尔逊相关系数，得到相似度矩阵 R 。以用户“高辛”为例，其已评分的菜品包括四川火锅（5 分）、重庆辣子鸡（4 分）等偏好辣味的菜品。

利用相似度加权预测未评分菜品的分值：

表 3.5 高辛未评分菜品的估计评分（模型一）

菜品	叉烧肠粉	大盘鸡拌面	东北饺子	剁椒鱼头	兰州拉面	烤羊排
估计评分	2.3478	3.0337	2.9699	3.1513	2.8950	2.8493

推荐结果：剁椒鱼头得分最高（3.1513），推荐给高辛。逻辑合理：高辛喜欢偏辣菜品（四川火锅 5 分、重庆辣子鸡 4 分），剁椒鱼头符合其口味偏好。

模型二：基于 SVD 压缩数据的协同过滤

先对原始矩阵 A 进行 SVD 降维。计算奇异值的累积贡献率，选择 $k = 6$ 个奇异值（累积贡献率 $\geq 90\%$ ）：

$$B_{6 \times 11} = \text{diag}(\sigma_1, \dots, \sigma_6) U_{6 \times 18}^T A$$

在压缩数据 B 上计算余弦相似度，再进行评分估计：

表 3.6 高辛未评分菜品的估计评分（模型二）

菜品	叉烧肠粉	大盘鸡拌面	东北饺子	剁椒鱼头	兰州拉面	烤羊排
估计评分	2.6347	2.9260	2.9337	2.9657	2.9057	2.9263

两种模型推荐结果一致：剁椒鱼头得分最高。

例 3.7：SVD 图像压缩

以 Lena 图像（ 512×512 像素）为例，对图像矩阵进行 SVD 分解：

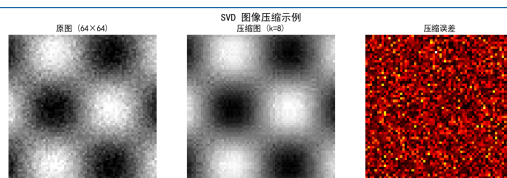


图 3.5 SVD 图像压缩示例：原图（左）与压缩图（中）及误差（右）

图像矩阵的秩为 256，若只保留最大的 21 个奇异值，压缩比率约为：

$$p = \left(1 - \frac{21 \times (512 + 512 + 1)}{512 \times 512} \right) \times 100\% \approx 91.6\%$$

即在仅保留约 8.4% 数据的情况下，仍能保持较好的图像质量。

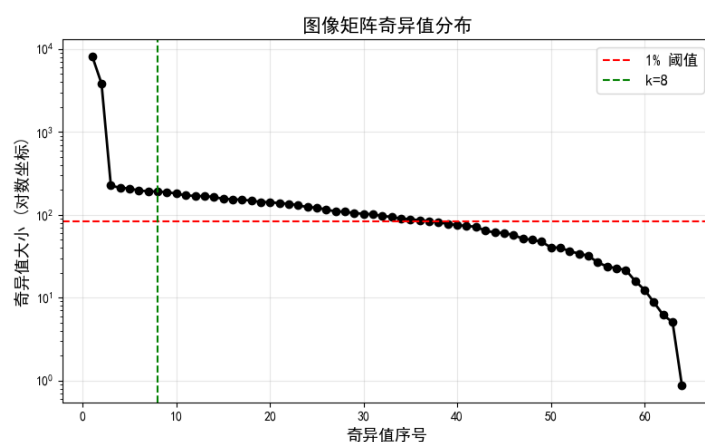


图 3.6 图像矩阵奇异值分布

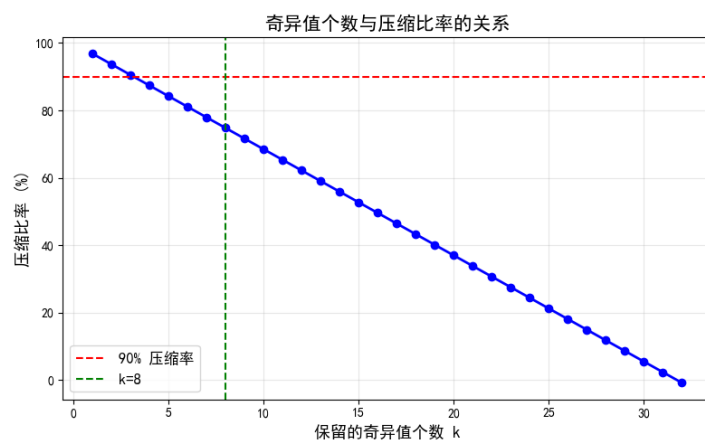


图 3.7 奇异值个数与压缩比率的关系曲线

代码实现

例 3.6 模型一：非压缩数据协同过滤

```

1  # 程序文件 ex3_6_1.py
2  import numpy as np
3  import pandas as pd
4
5  # 加载原始评分矩阵 (18行 x 11列)
6  a = np.loadtxt('data3_6_1.txt')
7
8  # 计算归一化相关系数矩阵 (皮尔逊相关系数)
9  b = 0.5 * np.corrcoef(a.T) + 0.5
10
11 # 保存相似度矩阵
12 c = pd.DataFrame(b)
13 c.to_excel('data3_6_2.xlsx', index=False)
14
15 # 输入用户编号
16 print('请输入人员编号 1-18')
17 user = int(input())
18
19 n = a.shape[1] # 菜品数量
20 no = np.where(a[user-1, :] == 0)[0] # 未评分菜品编号
21 yb = list(set(range(n)) - set(no)) # 已评分菜品编号
22 ys = a[user-1, yb] # 已评分分数
23
24 # 预测未评分菜品的分值
25 sc = np.zeros(len(no))
26 for i in range(len(no)):
27     sim = b[no[i], yb]
28     sc[i] = ys @ sim / sum(sim)
29
30 print('未评分项的编号为:', no + 1)
31 print('未评分项的分数为:', np.round(sc, 4))

```

例 3.6 模型二：基于 SVD 压缩的协同过滤

```

1  # 程序文件 ex3_6_2.py
2  import numpy as np
3  import pandas as pd
4
5  a = np.loadtxt('data3_6_1.txt')
6
7  # SVD 分解
8  u, sigma, vt = np.linalg.svd(a)
9  print(sigma)
10
11 # 计算累积贡献率, 选择 k 值
12 cs = np.cumsum(sigma**2)
13 rate = cs / cs[-1]

```

```

14     ind = np.where(rate >= 0.9)[0][0] + 1
15
16     # 降维：保留前 ind 个奇异值
17     b = np.diag(sigma[:ind]) @ u.T[:ind, :] @ a
18
19     # 计算余弦相似度
20     c = np.linalg.norm(b, axis=0, keepdims=True)
21     d = 0.5 * b.T @ b / (c.T @ c) + 0.5
22
23     # 保存相似度矩阵
24     dd = pd.DataFrame(d)
25     dd.to_excel('data3_6_3.xlsx', index=False)
26
27     # 评分预测（同模型一）
28     print('请输入人员编号 1-18')
29     user = int(input())
30     n = a.shape[1]
31     no = np.where(a[user-1, :] == 0)[0]
32     yb = list(set(range(n)) - set(no))
33     ys = a[user-1, yb]
34
35     sc = np.zeros(len(no))
36     for i in range(len(no)):
37         sim = d[no[i], yb]
38         sc[i] = ys @ sim / sum(sim)
39
40     print('未评分项的编号为:', no + 1)
41     print('未评分项的分数为:', np.round(sc, 4))

```

例 3.7: SVD 图像压缩

```

1     # 程序文件 ex3_7.py
2     import numpy as np
3     from numpy import linalg as LA
4     from PIL import Image
5     import pylab as plt
6
7     # 读取图像并转为灰度图
8     a = Image.open("Lena.bmp")
9     if a.mode != 'L':
10         a = a.convert("L")
11     b = np.array(a).astype(float)
12
13     # SVD 分解
14     p, d, q = LA.svd(b)
15     m, n = b.shape
16     R = LA.matrix_rank(b)

```

```
17
18     # 不同 k 值下的压缩效果
19     CR = []
20     for K in range(1, int(R / 4), 10):
21         # 重构图像
22         I = p[:, :K+1] @ np.diag(d[:K+1]) @ q[:K+1, :]
23
24         # 计算压缩比率
25         src = m * n
26         compress = K * (m + n + 1)
27         ratio = (1 - compress / src) * 100
28         CR.append(ratio)
29         print("K = %d: 压缩比率 = %5.2f%%" % (K, ratio))
30
31     # 注: PIL 安装命令: pip install pillow
```

优缺点分析

方法	优点	缺点
皮尔逊相关系数	消除量纲影响, 反映线性相关程度	对稀疏矩阵敏感, 需要共同评分项
余弦相似度	计算简单高效, 适合高维稀疏数据	对数值大小不敏感, 仅关注方向
SVD 降维 + 相似度	降低计算复杂度, 去除噪声	需要选择合适 k 值, k 选择影响结果
直接协同过滤	直观易懂, 无需训练	稀疏矩阵下相似度计算不准确

方法选择建议

- 数据密集且维度低: 使用皮尔逊相关系数计算相似度, 准确度较高。
- 数据稀疏且维度高: 使用余弦相似度, 计算效率更高。
- 原始矩阵稀疏度大: 先进行 SVD 降维再计算相似度, 可减轻稀疏性问题。
- k 值选择: 通过累积贡献率 (通常 90% 或 95%) 确定, 平衡压缩比与精度。
- 图像压缩: 根据质量要求选择 k 值, 一般 k 在 20-50 之间可取得较好平衡。

应用场景

- 推荐系统: Netflix、Amazon、美团等平台的协同过滤推荐。
- 图像压缩: JPEG 等图像格式的压缩算法基础。

-
- 数据降维：高维数据的特征提取和可视化。
 - 信号处理：噪声过滤和信号重构。
 - 自然语言处理：潜在语义分析（LSA）的基础。
 - 金融风控：高维用户特征的降维处理。

第 4 章

线性规划和整数规划模型

在工程技术、经济管理、科学研究、军事作战训练及日常生活等众多领域中，人们常常会遇到各种优化问题。例如，在生产经营中，我们总是希望制定最优的生产计划，充分利用已有的人力、物力资源，获得最大的经济效益；在运输问题中，我们总是希望设计最优的运输方案，在完成运输任务的前提下，力求运输成本最小等。针对优化问题的数学建模问题也是数学建模竞赛中一类比较常见的问题，这样的问题常常可以使用数学规划模型进行研究。

数学规划是运筹学的一个重要分支，而线性规划又是数学规划中的一部分主要内容。很多实际问题都可以归结为“线性规划”问题。线性规划（Linear Programming, LP）有比较完善的理论基础和有效的求解方法，在实际问题中有极其广泛的应用。

4.1 线性规划模型

4.1.1 线性规划模型及概念

问题描述

引例：某企业利用两种原材料 A 和 B 生产 3 种产品 P_1, P_2 和 P_3 。已知每生产 1kg 的产品所消耗的原材料数量和花费的加工时间，以及每 kg 产品销售后所带来的利润和每天可用的资源数量如表 4.1 所列，求使利润最大的生产计划。

表 4.1 企业生产数据表

资源	P_1	P_2	P_3	可用数量
原材料 A	2	4	3	150
原材料 B	3	1	5	160
加工时间 C	7	3	5	200
产品利润	70	50	60	

数学原理

（1）线性规划的三要素 规划问题的数学模型由 3 个要素组成：

1. 决策变量：问题中要确定的未知量，可由决策者决定和控制。

2. 目标函数：决策变量的函数，优化目标通常是求最大值或最小值。
3. 约束条件：决策变量的取值所受到的约束和限制条件，通常用含有决策变量的等式或不等式表示。

(2) 线性规划模型的一般形式

$$\max(\text{或 } \min) z = c_1x_1 + c_2x_2 + \cdots + c_nx_n \quad (4.1)$$

s.t.

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \leq (\text{或 } =, \geq) b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \leq (\text{或 } =, \geq) b_2, \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \leq (\text{或 } =, \geq) b_m, \\ x_1, x_2, \cdots, x_n \geq 0. \end{cases} \quad (4.2)$$

(3) 矩阵形式

$$\max(\text{或 } \min) z = c^T x \quad (4.3)$$

s.t.

$$\begin{cases} Ax \leq (\text{或 } =, \geq) b, \\ x \geq 0. \end{cases} \quad (4.4)$$

其中： $c = [c_1, c_2, \cdots, c_n]^T$ 为目标函数的系数向量（价值向量）； $x = [x_1, x_2, \cdots, x_n]^T$ 为决策向量； $A = (a_{ij})_{m \times n}$ 为约束方程组的系数矩阵； $b = [b_1, b_2, \cdots, b_m]^T$ 为约束方程组的常数向量。

(4) 线性规划问题的解的概念 标准型：

$$\max z = \sum_{j=1}^n c_j x_j \quad (4.5)$$

s.t.

$$\begin{cases} \sum_{j=1}^n a_{ij}x_j = b_i, & i = 1, 2, \cdots, m, \\ x_j \geq 0, & j = 1, 2, \cdots, n. \end{cases} \quad (4.6)$$

其中 $b_i \geq 0$ 。

- 可行解：满足约束条件的解 $x = [x_1, \cdots, x_n]^T$ 。
- 最优解：使目标函数达到最大值的可行解。
- 可行域：所有可行解构成的集合，记为 Ω 。

(5) 灵敏度分析 灵敏度分析是指对系统因周围条件变化显示出来的敏感程度的分析。在线性规划问题中, a_{ij} 、 b_i 、 c_j 常常是估计值或预测值, 可能有变动。需分析:

1. 如果某个参数发生变化, 现行最优方案会有什么变化?
2. 参数的变化限制在什么范围内, 原最优解仍是最优的?

在实际应用中, 给定参数一个步长使其重复求解线性规划问题, 以观察最优解的变化情况, 是一种可用的数值方法。

具体例子

例 4.1 (引例) 的数学模型:

设 x_1, x_2, x_3 分别表示产品 P_1, P_2, P_3 的日产量 (kg), 建立线性规划模型:

$$\max z = 70x_1 + 50x_2 + 60x_3 \quad (4.7)$$

s.t.

$$\begin{cases} 2x_1 + 4x_2 + 3x_3 \leq 150, \\ 3x_1 + x_2 + 5x_3 \leq 160, \\ 7x_1 + 3x_2 + 5x_3 \leq 200, \\ x_1, x_2, x_3 \geq 0. \end{cases} \quad (4.8)$$

求得最优解为 $x_1 = 15.9091$, $x_2 = 29.9455$, $x_3 = 0$, 最优值 $z = 2590.9091$ 。

例 4.2 (投资问题):

某部门在今后 5 年内考虑给四个项目投资。项目 A 从第一年到第四年每年年初需要投资, 并于次年未回收本利 115

设 x_{ij} 表示第 i 年年初给项目 j 的投资额, 建立线性规划模型求得最优投资方案, 最大本利总额为 143750 元。

例 4.3 (仓库租借问题):

1-4 月每月所需仓库面积分别为 15、10、20、12 (单位: 100m^2), 租借期限 1-4 月的租费分别为 2800、4500、6000、7300 元。求使租借费用最小的租借方案。

设 x_{ij} 表示第 i 个月初签订的租借期为 j 个月的仓库面积, 求得最优解 $x_{11} = 3$, $x_{31} = 8$, $x_{14} = 12$, 其他为 0, 最小租费 118400 元。

例 4.4 (运输问题):

6 个产地 8 个销地的运输问题, 总产量大于总销量, 求使总运费最小的调运方案, 最小运费为 664。

代码实现

一般线性规划模型求解框架:

```

1  # 使用 cvxpy 库求解线性规划
2  import cvxpy as cp
3  import numpy as np
4
5  # 定义数据
6  c = np.array([70, 50, 60])           # 目标函数系数
7  A = np.array([[2, 4, 3],            # 约束矩阵
8               [3, 1, 5],
9               [7, 3, 5]])
10 b = np.array([150, 160, 200])        # 约束右端项
11
12 # 定义决策变量
13 x = cp.Variable(3, pos=True)         # pos=True 表示  $x \geq 0$ 
14
15 # 构造目标函数和约束条件
16 obj = cp.Maximize(c @ x)             # 最大化问题
17 cons = [A @ x <= b]                 # 不等式约束
18
19 # 求解
20 prob = cp.Problem(obj, cons)
21 prob.solve(solver='GLPK_MI')
22
23 print('最优解:', x.value)
24 print('最优值:', prob.value)

```

投资问题代码:

```

1  # 程序文件 ex4_3.py
2  import cvxpy as cp
3
4  x = cp.Variable((5, 4), pos=True)
5
6  # 目标函数
7  obj = cp.Maximize(1.15 * x[3, 0] + 1.40 * x[1, 2] +
8                  1.25 * x[2, 1] + 1.06 * x[4, 3])
9
10 # 约束条件
11 cons = [
12 x[0, 0] + x[0, 3] == 100000,
13 x[1, 0] + x[1, 2] + x[1, 3] == 1.06 * x[0, 3],
14 x[2, 0] + x[2, 1] + x[2, 3] == 1.15 * x[0, 0] + 1.06 * x[1, 3],
15 x[3, 0] + x[3, 3] == 1.15 * x[1, 0] + 1.06 * x[2, 3],
16 x[4, 3] == 1.15 * x[2, 0] + 1.06 * x[3, 3],
17 x[2, 1] <= 40000,
18 x[1, 2] <= 30000
19 ]
20

```

```
21 prob = cp.Problem(obj, cons)
22 prob.solve(solver='GLPK_MI')
23
24 print("最优值:", prob.value)
25 print("最优解:\n", x.value)
```

运输问题代码（从文件读取数据）：

```
1 # 程序文件 ex4_5_1.py
2 import numpy as np
3 import cvxpy as cp
4 import pandas as pd
5
6 # 从文本文件读取数据
7 c = np.genfromtxt("data4_5_1.txt", dtype=float, max_rows=6,
8 usecols=range(8))
9 e = np.genfromtxt("data4_5_1.txt", dtype=float, max_rows=6,
10 usecols=8)
11 d = np.genfromtxt("data4_5_1.txt", dtype=float, skip_header=6)
12
13 # 定义决策变量
14 x = cp.Variable((6, 8), pos=True)
15
16 # 目标函数和约束条件
17 obj = cp.Minimize(cp.sum(cp.multiply(c, x)))
18 con = [
19 cp.sum(x, axis=0) == d, # 销量约束
20 cp.sum(x, axis=1) <= e # 产量约束
21 ]
22
23 prob = cp.Problem(obj, con)
24 prob.solve(solver='GLPK_MI')
25
26 print("最优值:", prob.value)
27 print("最优解:\n", x.value)
28
29 # 保存结果到 Excel
30 xd = pd.DataFrame(x.value)
31 xd.to_excel("data4_5_2.xlsx")
```

优缺点分析

方法	优点	缺点
图解法	直观易懂, 便于理解几何意义	仅适用于二维或三维问题
单纯形法	算法成熟, 理论完善, 计算效率高	最坏情况下指数复杂度
内点法	多项式时间复杂度, 适合大规模问题	实现复杂, 需要矩阵求逆
cvxpy 求解	代码简洁, 自动选择求解器	需要安装额外库, 大规模问题求解慢

方法选择建议

- 小规模问题 (变量数 ≤ 3): 使用图解法, 直观方便。
- 一般规模问题: 使用 cvxpy 库, 代码简洁, 自动调用合适求解器。
- 大规模问题: 使用专用优化软件(如 Gurobi、CPLEX)或 Python 的 `scipy.optimize.linprog`。
- 运输问题等特殊结构: 使用专门算法 (如表上作业法) 效率更高。
- 需要做灵敏度分析: 使用 cvxpy 配合参数循环求解, 观察最优解变化。

应用场景

- 生产计划: 有限资源下最大化利润或最小化成本。
- 投资组合优化: 资金分配使收益最大化或风险最小化。
- 运输调度: 最小化运输成本或最大化运输效率。
- 仓库租借: 满足存储需求下最小化租借费用。
- 广告投放: 有限预算下最大化广告覆盖效果。
- 人员排班: 满足人力需求下最小化人力成本。
- 供应链管理: 优化库存、采购和配送决策。

4.2 整数规划

在人们的生产实践中, 经常会遇到以下类似的问题: 汽车企业在制订生产计划时, 要求所生产的不同类型的汽车数量必须为整数; 用人单位在招聘员工时, 要求所招聘的不同技术水平的员工数量必须为整数等。我们把要求一部分或全部决策变量必须取整数值的规划问题称为整数规划 (Integer Programming, IP)。

4.2.1 整数线性规划模型

问题描述

从决策变量的取值范围来看，整数规划通常可以分为以下几种类型：

1. 纯整数规划：全部决策变量都必须取整数值的整数规划模型。
2. 混合整数规划：决策变量中有一部分必须取整数值，另一部分可以不取整数值的整数规划模型。
3. 0-1 整数规划：决策变量只能取 0 或 1 的整数规划。

特别地，如果一个线性规划模型中的部分或全部决策变量取整数值，则称该线性规划模型为整数线性规划模型。

数学原理

(1) 整数线性规划模型的一般形式

$$\max(\text{或 min}) z = \sum_{j=1}^n c_j x_j \quad (4.9)$$

s.t.

$$\begin{cases} \sum_{j=1}^n a_{ij} x_j \leq (\text{或 } =, \geq) b_i, & i = 1, 2, \dots, m, \\ x_j \geq 0, & j = 1, 2, \dots, n, \\ x_1, x_2, \dots, x_n \text{ 中部分或全部取整数.} \end{cases} \quad (4.10)$$

(2) 0-1 整数规划 0-1 整数规划是整数规划的特殊情形，决策变量 x_i 只能取 0 或 1：

$$x_i = \begin{cases} 1, & \text{选择该方案,} \\ 0, & \text{不选择该方案.} \end{cases} \quad (4.11)$$

0-1 变量通常用于表示“是/否”、“选择/不选择”等二元决策。

(3) 典型问题结构

- 背包问题：在重量限制下选择物品使价值最大化。
- 指派问题：将 n 项任务分配给 n 个人，使总费用最小。
- 旅行商问题（TSP）：访问所有城市一次并返回起点，使总路程最短。

具体例子

例 4.6（背包问题）：

有 n 件物品，每件物品价值 c_j 、重量 a_j ，背包容量为 b kg。每件物品要么带要么不带，求使总价值最大的携带方案。

引入 0-1 变量 $x_i = 1$ 表示携带第 i 种物品, 建立模型:

$$\max z = \sum_{i=1}^n c_i x_i, \quad \text{s.t.} \quad \sum_{i=1}^n a_i x_i \leq b, \quad x_i = 0 \text{ 或 } 1. \quad (4.12)$$

例 4.7 (指派问题):

n 个人完成 n 项任务, c_{ij} 表示第 i 个人完成第 j 项任务的费用, 求使总费用最小的指派方案。

引入 0-1 变量 $x_{ij} = 1$ 表示指派第 i 个人完成第 j 项任务, 建立模型:

$$\min z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (4.13)$$

s.t.

$$\sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n, \quad \sum_{i=1}^n x_{ij} = 1, \quad j = 1, 2, \dots, n, \quad x_{ij} = 0 \text{ 或 } 1. \quad (4.14)$$

例 4.8 (旅行商问题 TSP):

推销员从城市 v_1 出发, 访问城市 $v_2, v_3, \dots, v_n$ 各一次后返回 v_1 , c_{ij} 为从 v_i 到 v_j 的旅费, 求总旅费最小的访问次序。

引入 0-1 变量 $x_{ij} = 1$ 表示从 v_i 直接进入 v_j , 并引入附加变量 u_i 防止子回路:

$$\min z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (4.15)$$

s.t.

$$\sum_{i=1}^n x_{ij} = 1, \quad j = 1, 2, \dots, n, \quad \sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n, \quad (4.16)$$

$$u_i - u_j + nx_{ij} \leq n - 1, \quad i = 1, \dots, n, \quad j = 2, \dots, n, \quad (4.17)$$

$$u_1 = 0, \quad 1 \leq u_i \leq n - 1 \quad (i = 2, \dots, n), \quad x_{ij} = 0 \text{ 或 } 1. \quad (4.18)$$

其中子回路消除约束 $u_i - u_j + nx_{ij} \leq n - 1$ 的证明思路:

- 若存在不含起点 v_1 的子回路, 将约束相加会产生矛盾;
- 对于整体巡回, 适当选取 u_i (按访问顺序编号) 可满足约束。

例 4.9 (工人排班问题):

工厂需要 24h 不间断运转, 各班次所需工人数如表 4.5 所列, 每名工人连续工作 8h, 只能在班次初始时刻报到, 求最少需要多少工人。

设 x_i 为第 i 个班次报到的工人数, 建立整数规划模型。求得最优解为 $x_1 = 35, x_2 = 5, x_3 = 45, x_4 = 25, x_5 = 30, x_6 = 0$, 最少需要 140 人。

例 4.10 (非标准指派问题):

4 家装修公司对 5 个门店的装修费用报价如表 4.6 所列, 每家公司最多承担 2 个门店, 每个门店由 1 家公司承担, 求最小费用分配方案。

引入 0-1 变量 x_{ij} 表示公司 i 是否承担门店 j , 建立模型:

$$\min z = \sum_{i=1}^4 \sum_{j=1}^5 c_{ij} x_{ij} \quad (4.19)$$

s.t.

$$\sum_{i=1}^4 x_{ij} = 1 \ (j = 1, \dots, 5), \quad \sum_{j=1}^5 x_{ij} \leq 2 \ (i = 1, \dots, 4), \quad x_{ij} = 0 \text{ 或 } 1. \quad (4.20)$$

求得最优指派为: A 负责门店 3, B 负责门店 4, C 负责门店 1、2, D 负责门店 5, 总费用 63.8 万元。

代码实现

一般整数规划求解框架:

```

1  # 使用 cvxpy 库求解整数规划
2  import cvxpy as cp
3  import numpy as np
4
5  # 定义决策变量 (integer=True 表示整数)
6  x = cp.Variable(n, integer=True)
7
8  # 或 0-1 变量 (通过约束 0 <= x <= 1 + integer=True)
9  x = cp.Variable(n, integer=True)
10 cons = [0 <= x, x <= 1]
11
12 # 构造目标函数和约束条件
13 obj = cp.Minimize(c @ x) # 或 cp.Maximize()
14 cons = [A @ x <= b, x >= 0]
15
16 # 求解
17 prob = cp.Problem(obj, cons)
18 prob.solve(solver='GLPK_MI')
19
20 print('最优值:', prob.value)
21 print('最优解:', x.value)

```

例 4.9 (工人排班):

```

1  # 程序文件 ex4_9_1.py
2  import cvxpy as cp

```

```

3
4     x = cp.Variable(6, integer=True)
5
6     obj = cp.Minimize(sum(x))
7
8     cons = [
9         x[0] + x[5] >= 35,
10        x[0] + x[1] >= 40,
11        x[1] + x[2] >= 50,
12        x[2] + x[3] >= 45,
13        x[3] + x[4] >= 55,
14        x[4] + x[5] >= 30,
15        x >= 0
16    ]
17
18    prob = cp.Problem(obj, cons)
19    prob.solve(solver='GLPK_MI')
20
21    print("最优值:", prob.value)
22    print("最优解:", x.value)

```

例 4.10 (非标准指派问题):

```

1     # 程序文件 ex4_10.py
2     import cvxpy as cp
3     import numpy as np
4
5     c = np.loadtxt('data4_10.txt')
6
7     x = cp.Variable((4, 5), integer=True)
8
9     obj = cp.Minimize(cp.sum(cp.multiply(c, x)))
10
11    cons = [
12        0 <= x, x <= 1,
13        cp.sum(x, axis=0) == 1,    # 每个门店由一家公司承担
14        cp.sum(x, axis=1) <= 2    # 每家公司最多承担2个门店
15    ]
16
17    prob = cp.Problem(obj, cons)
18    prob.solve(solver='GLPK_MI')
19
20    print("最优值:", prob.value)
21    print("最优解:\n", x.value)

```

例 4.11 (供应站选址):

10 个商业网点, 选择若干位置设置供应站, 供应站覆盖 10km 内的网点, 每个供应

站最多供应 5 个网点，求供应站数目最小的方案。

引入两组 0-1 变量：

- $x_i = 1$ 表示第 i 个网点建立供应站；
- $y_{ij} = 1$ 表示第 j 个网点被第 i 个网点的供应站覆盖。

```

1  # 程序文件 ex4_11.py
2  import cvxpy as cp
3  import numpy as np
4
5  # 读取坐标数据
6  a = np.loadtxt("data4_11.txt")
7
8  # 计算距离矩阵
9  d = np.zeros((10, 10))
10 for i in range(10):
11     for j in range(10):
12         d[i, j] = np.linalg.norm(a[:, i] - a[:, j])
13
14 x = cp.Variable(10, integer=True)
15 y = cp.Variable((10, 10), integer=True)
16
17 obj = cp.Minimize(sum(x))
18
19 cons = [
20     0 <= x, x <= 1,
21     0 <= y, y <= 1,
22     cp.sum(y, axis=0) >= 1,           # 每个网点至少被覆盖
23     cp.sum(y, axis=1) <= 5,         # 每个供应站最多供应 5 个网点
24 ]
25
26 for i in range(10):
27     cons.append(x[i] == y[i, i])    # 供应站本身被自己覆盖
28 for j in range(10):
29     cons.append(d[i, j] * y[i, j] <= 10 * x[i]) # 覆盖距离限制
30     cons.append(x[i] >= y[i, j])    # 只有建站才能覆盖
31
32 prob = cp.Problem(obj, cons)
33 prob.solve(solver='GLPK_MI')
34
35 print("最优值:", prob.value)
36 print("最优解 x:\n", x.value)

```

优缺点分析

方法	优点	缺点
分支定界法	理论上可求全局最优解, 适合中小规模问题	最坏情况下指数复杂度, 大规模问题求解慢
割平面法	理论基础完善, 收敛性有保证	收敛速度可能较慢, 实际应用较少
动态规划	可求解背包等特殊结构问题	状态空间膨胀严重, 仅适用于低维问题
cvxpy + GLPK	代码简洁, 自动选择算法, 适合教学	大规模问题效率较低

方法选择建议

- 背包问题：使用动态规划，效率远优于通用整数规划求解器。
- 指派问题 ($n \leq 100$)：使用匈牙利算法，多项式时间复杂度 $O(n^3)$ 。
- 旅行商问题（小规模 $n \leq 20$ ）：使用整数规划精确求解。
- 旅行商问题（大规模）：使用启发式算法（遗传算法、模拟退火、蚁群算法）求近似解。
- 一般整数规划：使用 Python 的 cvxpy 库，调用 GLPK_MI 求解器。
- 大规模 0-1 规划：使用专用优化软件（Gurobi、CPLEX）或利用问题结构设计近似算法。

应用场景

- 背包问题：旅行行李选择、项目投资组合、资源分配。
- 指派问题：工作任务分配、人员排班、机器调度。
- 旅行商问题：物流配送路线规划、城市交通网络设计、电路板钻孔路径优化。
- 设施选址：仓库/配送中心选址、供应站设置、基站布局。
- 排班问题：工厂工人排班、医院护士排班、机场地勤调度。
- 生产计划：汽车生产数量确定、批次生产决策。

整数规划与线性规划的区别

特征	线性规划	整数规划
决策变量	连续变量	部分或全部取整数
可行域	凸集	离散点集
最优解位置	可行域顶点	可行域内整数格点
求解难度	多项式时间（内点法）	NP 难

建模工具	cvxpy(无需 integer=True)	cvxpy (需 integer=True)
------	------------------------	------------------------

4.2.2 投资的收益与风险

问题描述

市场上有 n 种资产 $s_i (i = 1, 2, \dots, n)$ 供投资者选择, 某公司有数额为 M 的一笔相当大的资金可用作一个时期的投资。已知:

- 资产 s_i 的平均收益率为 r_i , 风险损失率为 q_i ;
- 购买 s_i 的交易费费率为 p_i , 购买额不超过 u_i 时按 u_i 计算交易费;
- 同期银行存款利率 $r_0 = 5\%$, 无交易费、无风险;
- 总体风险用所投资的 s_i 中最大的一个风险来度量。

已知 $n = 4$ 时的数据如表 4.8 所列, 试设计投资组合方案, 使净收益尽可能大、总体风险尽可能小。

数学原理

(1) 符号说明

- s_i : 第 i 种资产, $i = 0, 1, \dots, n$, 其中 s_0 表示银行存款;
- x_i : 投资到 s_i 的资金数量, $i = 0, 1, \dots, n$;
- r_i : s_i 的平均收益率, $r_0 = 0$;
- q_i : s_i 的风险损失率, $q_0 = 0$;
- p_i : s_i 的交易费费率, $p_0 = 0$;
- u_i : s_i 的投资阈值。

(2) 模型假设

1. 可供投资的资金数额 M 相当大;
2. 投资越分散, 总的风险越小, 总体风险用 $\max\{q_i x_i\}$ 度量;
3. 各资产之间相互独立;
4. 每种资产可购买任意数量;
5. 在当前投资周期内, 各参数固定不变。

(3) 多目标规划模型 总体风险: $\max\{q_i x_i \mid i = 1, 2, \dots, n\}$

交易费 (简化后): 购买 s_i 的净收益为 $(r_i - p_i)x_i$

目标函数:

$$\max \sum_{i=0}^n (r_i - p_i)x_i, \quad \min \left\{ \max_{1 \leq i \leq n} q_i x_i \right\} \quad (4.21)$$

约束条件:

$$\sum_{i=0}^n (1 + p_i)x_i = M, \quad x_i \geq 0, \quad i = 0, 1, \dots, n. \quad (4.22)$$

(4) 三种转化模型 模型一: 固定风险水平, 优化收益

给定风险界限 a , 使 $\frac{q_i x_i}{M} \leq a$:

$$\max \sum_{i=0}^n (r_i - p_i)x_i \quad (4.23)$$

s.t.

$$\frac{q_i x_i}{M} \leq a \quad (i = 1, \dots, n), \quad \sum_{i=0}^n (1 + p_i)x_i = M, \quad x_i \geq 0. \quad (4.24)$$

模型二: 固定盈利水平, 极小化风险

给定收益水平 k :

$$\min \left\{ \max_{1 \leq i \leq n} q_i x_i \right\} \quad (4.25)$$

s.t.

$$\sum_{i=0}^n (r_i - p_i)x_i \geq kM, \quad x_i \geq 0. \quad (4.26)$$

模型三: 加权求和

引入投资偏好系数 $w \in [0, 1]$:

$$\min \left[w \max_{1 \leq i \leq n} q_i x_i - (1 - w) \sum_{i=0}^n (r_i - p_i)x_i \right] \quad (4.27)$$

s.t.

$$\sum_{i=0}^n (1 + p_i)x_i = M, \quad x_i \geq 0. \quad (4.28)$$

线性化: 引入 $x_{n+1} = \max\{q_i x_i\}$, 将模型三转化为:

$$\min \left[w x_{n+1} - (1 - w) \sum_{i=0}^n (r_i - p_i)x_i \right] \quad (4.29)$$

s.t.

$$q_i x_i \leq x_{n+1} \quad (i = 1, \dots, n), \quad \sum_{i=0}^n (1 + p_i) x_i = M, \quad x_i \geq 0. \quad (4.30)$$

具体例子

例 4.12 (投资组合优化):

已知 $M = 10000$ 元, $n = 4$, 相关数据见表 4.8。

模型一求解结果:

风险 a 与收益 Q 的关系如图 4.2 所示。在 $a = 0.006$ 附近存在转折点:

- 风险度 $a = 0.006$, 收益 $Q = 2019.08$ 元;
- 最优投资方案: $x_0 = 0$, $x_1 = 2400$, $x_2 = 4000$, $x_3 = 1090.91$, $x_4 = 2212.21$ 元。

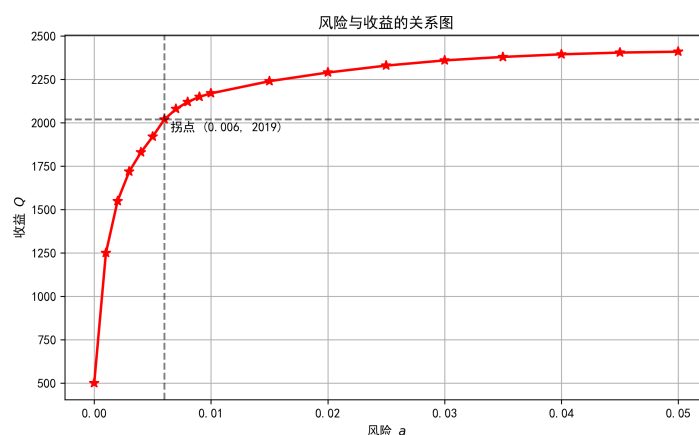


图 4.1 风险与收益的关系图

模型三求解结果:

表 4.5 不同投资偏好系数下的风险与收益

w	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
风险	247.52	247.52	247.52	247.52	247.51	247.52	247.52	247.52	247.53	247.53
收益	2673.27	2673.27	2673.27	2673.27	2673.27	2673.27	2673.27	2674.52	2673.27	2673.27

曲线的拐点坐标约为 (0.006, 2019.08), 对应投资方案: $x_1 = 2375.84$, $x_2 = 3959.73$, $x_3 = 1079.93$, $x_4 = 2284.46$, $x_0 = 0$ 元。

代码实现

模型一: 固定风险水平

```

1 # 程序文件 ex4_12_1.py
2 import cvxpy as cp

```

```

3  import pylab as plt
4  import numpy as np
5
6  b = np.array([0.025, 0.015, 0.055, 0.026])
7  c = np.array([0.05, 0.27, 0.19, 0.185, 0.185])
8  aeq = np.array([1, 1.01, 1.02, 1.045, 1.065])
9
10 x = cp.Variable(5, pos=True)
11 obj = cp.Maximize(c @ x)
12
13 M = 10000
14 a = 0
15 aa = []
16 Q = []
17
18 while a < 0.05:
19     con = [aeq @ x == M, b * x[1:5] <= a * M]
20     prob = cp.Problem(obj, con)
21     prob.solve(solver='GLPK_MI')
22     aa.append(a)
23     Q.append(prob.value)
24     a = a + 0.001
25
26 # 绘制风险-收益曲线
27 plt.rc('text', usetex=True)
28 plt.rc('font', size=15)
29 plt.plot(aa, Q, 'r*')
30 plt.xlabel('$a$')
31 plt.ylabel('$Q$', rotation=0)
32 plt.show()

```

模型三：加权求和

```

1  # 程序文件 ex4_12_2.py
2  import numpy as np
3  import cvxpy as cp
4  import pylab as plt
5
6  plt.rc('font', family='SimHei')
7  plt.rc('font', size=15)
8
9  x = cp.Variable(6, pos=True)
10
11 r = np.array([0.05, 0.28, 0.21, 0.23, 0.25])
12 p = np.array([0, 0.01, 0.02, 0.045, 0.065])
13 q = np.array([0, 0.025, 0.015, 0.055, 0.026])
14

```

```

15     def LP(w):
16         V = []      # 风险
17         Q = []      # 收益
18         X = []      # 最优解
19
20         con = [(1+p) @ x[0:5] == 10000,
21               q[1:5] * x[1:5] <= x[5]]
22
23         for i in range(len(w)):
24             obj = cp.Minimize(w[i] * x[5] - (1-w[i]) * ((r-p) @ x[0:5]))
25             prob = cp.Problem(obj, con)
26             prob.solve(solver='GLPK_MI')
27             xx = x.value
28             V.append(max(q * xx[0:5]))
29             Q.append((r-p) @ xx[0:5])
30             X.append(xx)
31
32         print('w=', w)
33         print('V=', np.round(V, 2))
34         print('Q=', np.round(Q, 2))
35
36         plt.figure()
37         plt.plot(V, Q, '* -')
38         plt.grid('on')
39         plt.xlabel('风险(元)')
40         plt.ylabel('收益(元)')
41         return X
42
43     # 粗粒度搜索
44     w1 = np.arange(0, 1.1, 0.1)
45     LP(w1)
46
47     # 细粒度搜索 (拐点附近)
48     w2 = np.array([0.766, 0.767, 0.810, 0.811, 0.824, 0.825, 0.962,
49                   0.963, 1.0])
49     X = LP(w2)
50     print(X[-3])
51     plt.show()

```

4.2.3 比赛项目排序问题

问题描述

有 14 个比赛项目、40 名运动员参加比赛，报名表用矩阵 $A = (a_{mn})_{40 \times 14}$ 表示， $a_{mn} = 1$ 表示运动员 m 参加项目 n 。要求合理安排比赛项目顺序，使连续参加两项比赛的运动员人次尽可能少。

数学原理

(1) 图论转化 构造赋权图 $G = (V, E, W)$:

- 顶点集 $V = \{1, 2, \dots, 14\}$ 表示比赛项目;
- 边权重 w_{ij} 表示同时参加项目 i 和项目 j 的人数:

$$w_{ij} = \sum_{m=1}^{40} a_{mi}a_{mj}, \quad i \neq j, \quad w_{ii} = \infty. \quad (4.31)$$

问题转化为: 求项目 1 到 14 的一个排列, 使相邻顶点之间的权重之和最小。

(2) TSP 模型 引入虚拟项目 15, 与各项目的连接权重均为 0。扩充图为 $\tilde{G} = (\tilde{V}, \tilde{E}, \tilde{W})$, $\tilde{V} = \{1, 2, \dots, 15\}$, $\tilde{w}_{15,i} = \tilde{w}_{i,15} = 0$ 。

建立 0-1 整数规划模型:

$$\min z = \sum_{i=1}^{15} \sum_{j=1}^{15} \tilde{w}_{ij} x_{ij} \quad (4.32)$$

s.t.

$$\sum_{j=1}^{15} x_{ij} = 1, \quad i = 1, \dots, 15, \quad \sum_{i=1}^{15} x_{ij} = 1, \quad j = 1, \dots, 15, \quad (4.33)$$

$$u_i - u_j + 15x_{ij} \leq 14, \quad i = 1, \dots, 15, \quad j = 2, \dots, 15, \quad (4.34)$$

$$u_1 = 0, \quad 1 \leq u_i \leq 14 \quad (i = 2, \dots, 15), \quad x_{ij} = 0 \text{ 或 } 1. \quad (4.35)$$

在找到的最短回路中去掉虚拟项目 15 关联的两条边, 即得比赛项目排序。

具体例子

例 4.13: 利用上述模型求得 TSP 回路为:

$$15 \rightarrow 4 \rightarrow 9 \rightarrow 8 \rightarrow 1 \rightarrow 5 \rightarrow 11 \rightarrow 7 \rightarrow 3 \rightarrow 6 \rightarrow 2 \rightarrow 14 \rightarrow 12 \rightarrow 10 \rightarrow 13 \rightarrow 15$$

回路总长为 2, 即只有 2 名运动员连续参加比赛。去掉虚拟项目 15, 比赛项目排序为:

$$4 \rightarrow 9 \rightarrow 8 \rightarrow 1 \rightarrow 5 \rightarrow 11 \rightarrow 7 \rightarrow 3 \rightarrow 6 \rightarrow 2 \rightarrow 14 \rightarrow 12 \rightarrow 10 \rightarrow 13$$

代码实现

```
1  # 程序文件 ex4_13.py
2  import numpy as np
3  import pandas as pd
4  import cvxpy as cp
5
6  # 读取报名表 ("#" 替换为 1)
7  a = pd.read_excel('data4_13_1.xlsx', header=None)
8  a = a.values
9  a[np.isnan(a)] = 0
10
11  m, n = a.shape
12
13  # 构造邻接矩阵 (15x15, 含虚拟项目)
14  w = np.ones((n+1, n+1)) * 10000000    # 无穷大初始化
15
16  for i in range(n):
17  for j in range(n):
18  if i != j:
19  w[i, j] = sum(a[:, i] * a[:, j])
20
21  for i in range(n):
22  w[i, n] = 0    # 虚拟项目连接权重为 0
23  w[n, i] = 0
24
25  # 保存邻接矩阵
26  wd = pd.DataFrame(w)
27  wd.to_excel('data4_13_2.xlsx')
28
29  # 定义决策变量
30  x = cp.Variable((n+1, n+1), integer=True)
31  u = cp.Variable(n+1, integer=True)
32
33  # 目标函数
34  obj = cp.Minimize(cp.sum(cp.multiply(w, x)))
35
36  # 约束条件
37  con = [
38  cp.sum(x, axis=0) == 1,
39  cp.sum(x, axis=1) == 1,
40  x >= 0, x <= 1,
41  u[0] == 0,
42  u[1] >= 1, u[1] <= n
43  ]
44
45  # 子回路消除约束
```

```

46     for i in range(n+1):
47         for j in range(1, n+1):
48             con.append(u[i] - u[j] + (n+1) * x[i, j] <= n)
49
50     prob = cp.Problem(obj, con)
51     prob.solve(solver='GLPK_MI')
52
53     print("最优值:", prob.value)
54     print("最优解:\n", x.value)
55
56     # 输出选中的边
57     i, j = np.nonzero(x.value)
58     print("xij = 1 对应的行列位置:")
59     print("行:", i + 1)
60     print("列:", j + 1)

```

优缺点分析

方法	优点	缺点
多目标规划转化	将复杂多目标转化为单目标，易于求解	权重选择具有主观性
风险固定模型	风险可控，适合保守投资者	需预先给定风险水平
收益固定模型	收益有保障，适合进取投资者	需预先给定收益目标
加权求和模型	可调参数灵活，覆盖不同偏好	权重 w 的选择影响结果
TSP 模型	精确求解，保证最优	大规模时计算量大，NP-hard

方法选择建议

- 保守型投资者：使用模型一，给定较小的风险界限 a 。
- 进取型投资者：使用模型二，设定较高的收益目标 k 。
- 无明确偏好：使用模型三，绘制风险-收益曲线，选择拐点。
- 比赛排序问题：使用 TSP 模型，引入虚拟节点处理起点/终点。

应用场景

- 投资组合优化：股票、基金、债券等多资产配置。
- 风险控制：银行信贷组合、保险资金运用。

- 赛事组织：比赛项目排序、赛程安排。
- 生产调度：工序排序、流水线安排。
- 物流配送：车辆路径优化、货物装卸排序。

第 4 章总结

模型类型	核心特征	典型应用
线性规划	连续变量，线性约束	生产计划、运输问题
整数规划	部分/全部变量取整数	排班、选址
0-1 规划	变量只能取 0 或 1	指派、背包、TSP
多目标规划	多个目标函数	投资组合优化

第 5 章

非线性规划和多目标规划模型

在实际应用中，除了线性规划和整数线性规划之外，还大量地存在着另一类优化问题：描述目标函数或约束条件的数学表达式，至少有一个是非线性函数，这样的优化问题通常称为非线性规划。一般来说，解决非线性规划问题要比线性规划问题困难得多，不像线性规划有适用于一般情况的单纯形法，对于非线性规划问题到目前为止还没有一种适用于一般情况的求解方法，现有各种方法都有各自特定的适用范围。

多目标决策问题是管理与日常生活中经常遇到的问题，而这些目标之间常常是相互作用的，如何平衡这些目标，其决策过程十分复杂，决策者通常很难做出最终决策。解决这类问题的建模方法就是多目标决策方法。

5.1 非线性规划概念和理论

5.1.1 非线性规划问题的数学模型

数学原理

记 $x = [x_1, x_2, \dots, x_n]^T$ 是 n 维欧几里得空间 $\mathbb{R}^n$ 中的一个点。 $f(x)$ 、 $g_i(x)$ ($i = 1, 2, \dots, p$) 和 $h_j(x)$ ($j = 1, 2, \dots, q$) 是定义在 $\mathbb{R}^n$ 上的实值函数。

若这些函数中至少有一个是 x 的非线性函数，称如下形式的数学模型：

$$\begin{aligned} \min & f(x), \\ \text{s.t.} & g_i(x) \leq 0, \quad i = 1, 2, \dots, p, \\ & h_j(x) = 0, \quad j = 1, 2, \dots, q. \end{aligned} \tag{5.1}$$

为非线性规划模型的一般形式。

向量表示形式：

$$\begin{aligned} \min & f(x), \\ \text{s.t.} & G(x) \leq 0, \\ & H(x) = 0, \end{aligned} \tag{5.2}$$

其中 $G(x) = [g_1(x), g_2(x), \dots, g_p(x)]^T$ ， $H(x) = [h_1(x), h_2(x), \dots, h_q(x)]^T$ 。

可行域：

$$K = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0, i = 1, \dots, p; h_j(x) = 0, j = 1, \dots, q\} \quad (5.1)$$

(1) 最优解的定义

- 全局最优解： $x^* \in K$ ，且 $\forall x \in K, f(x^*) \leq f(x)$ 。
- 局部最优解： $x^* \in K$ ，且存在 x^* 的邻域 $N_\delta(x^*)$ ， $\forall x \in N_\delta(x^*) \cap K, f(x^*) \leq f(x)$ 。

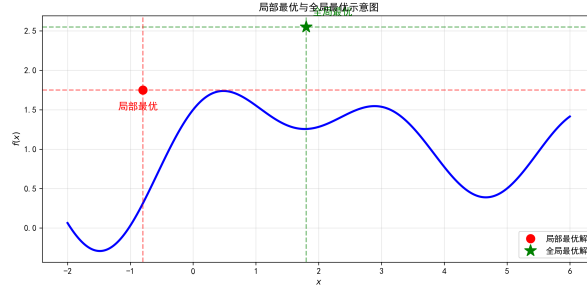


图 5.1 局部最优与全局最优示意图

重要区别：线性规划的最优解（若存在）只能在可行域的边界上达到（特别是在顶点上），且求出的是全局最优解。但非线性规划的最优解可能在可行域的任意一点达到，一般算法给出的通常只是局部最优解，不能保证是全局最优解。

(2) 无约束非线性规划的最优性条件 无约束非线性规划问题：

$$\min f(x), \quad x \in \mathbb{R}^n \quad (5.3)$$

定理 5.1（必要条件）：设 $f(x)$ 具有连续的一阶偏导数，且 x^* 是局部极小点，则 $\nabla f(x^*) = 0$ 。

定义 5.2（Hesse 矩阵）：

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \quad (5.2)$$

定理 5.2（充分条件）：设 $f(x)$ 具有连续的二阶偏导数，点 x^* 满足 $\nabla f(x^*) = 0$ ，且 $\nabla^2 f(x^*)$ 为正定矩阵，则 x^* 为无约束优化问题的局部最优解。

(3) 有约束非线性规划的最优性条件 拉格朗日乘数法（等式约束）：

对只有等式约束的问题：

$$\min f(x), \quad \text{s.t. } h_j(x) = 0, j = 1, 2, \dots, q \quad (5.3)$$

引入拉格朗日函数：

$$L(x, \lambda) = f(x) + \sum_{j=1}^q \lambda_j h_j(x) \quad (5.4)$$

则最优解满足 $\nabla L(x^*, \lambda^*) = 0$ 。

罚函数法（一般约束）：

构造罚函数（外部罚函数法）：

$$T(x, M) = f(x) + M \sum_{i=1}^p [\max\{0, g_i(x)\}]^2 + M \sum_{j=1}^q [h_j(x)]^2 \quad (5.5)$$

将有约束问题转化为无约束问题 $\min T(x, M)$ ，其中 M 是较大的正数。

库恩-塔克条件（K-T 条件）：

引进拉格朗日函数：

$$L(x, \lambda, \mu) = f(x) + \sum_{i=1}^p \lambda_i g_i(x) + \sum_{j=1}^q \mu_j h_j(x) \quad (5.6)$$

K-T 条件：

$$\begin{cases} \nabla f(x^*) + \sum_{i=1}^p \lambda_i^* \nabla g_i(x^*) + \sum_{j=1}^q \mu_j^* \nabla h_j(x^*) = 0, \\ \lambda_i^* g_i(x^*) = 0, \quad i = 1, 2, \dots, p, \\ \lambda_i^* \geq 0, \quad i = 1, 2, \dots, p. \end{cases} \quad (5.7)$$

满足 K-T 条件的点称为 K-T 点。

- 必要条件：局部最优解必为 K-T 点。
- 充分条件：对于凸规划，K-T 点必为全局最优解。

(4) 凸规划

- 凸集：任意两点 $x_1, x_2 \in \Omega$ ，其连线上的所有点 $\alpha x_1 + (1 - \alpha)x_2 \in \Omega$ ($0 \leq \alpha \leq 1$)。
- 凸函数： $f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$ 。
- 凸规划： $f(x)$ 为凸函数， $g_i(x)$ 为凸函数， $h_j(x)$ 为线性函数。

凸规划具有重要性质：局部最优解即为全局最优解。

凸函数的判定：

- 一阶条件： $f(x_1) \geq f(x_2) + \nabla f(x_1)^T (x_2 - x_1)$
- 二阶条件：Hesse 矩阵处处半正定

具体例子

例 5.1：试分析并求解非线性规划

$$\min f(x) = x_1^2 + x_2^2 - 4x_1 + 4 \quad (5.7)$$

s.t.

$$\begin{cases} g_1(x) = -x_1 + x_2 - 2 \leq 0, \\ g_2(x) = x_1^2 - x_2 + 1 \leq 0, \\ x_1, x_2 \geq 0. \end{cases} \quad (5.8)$$

分析：

- $f(x)$ 的 Hesse 矩阵： $\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$ ，正定 $\Rightarrow$ 严格凸函数
- $g_2(x)$ 的 Hesse 矩阵： $\begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$ ，半正定 $\Rightarrow$ 凸函数
- 其他约束为线性函数 $\Rightarrow$ 凸规划

最优解：

$$x^* = [0.5536, 1.3064]^T, \quad f(x^*) = 3.7989 \quad (5.9)$$

代码实现

例 5.1 的求解：

```

1  # 程序文件 ex5_1.py
2  import numpy as np
3  import cvxpy as cp
4
5  x = cp.Variable(2, pos=True)
6
7  # 目标函数
8  obj = cp.Minimize(sum(x**2) - 4*x[0] + 4)
9
10 # 约束条件
11 con = [
12     -x[0] + x[1] - 2 <= 0,
13     x[0]**2 - x[1] + 1 <= 0
14 ]
15
16 prob = cp.Problem(obj, con)
17 prob.solve(solver='CVXOPT')
18
19 print("最优值:", round(prob.value, 4))

```

```
print("最优解:", np.round(x.value, 4))
```

输出结果:

```
1 最优值: 3.7989
2 最优解: [0.5536 1.3064]
```

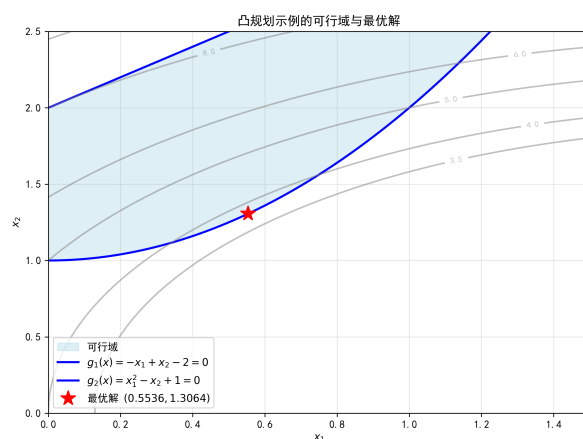


图 5.2 凸规划示例的可行域与最优解

优缺点分析

方法	优点	缺点
拉格朗日乘数法	理论完善, 将等式约束转化为无约束	仅适用于等式约束, 需解非线性方程组
罚函数法	适用于一般约束, 思想简单	计算精度可能较差, M 值选择困难
K-T 条件	确定最优解的必要/充分条件	对非凸问题仅为必要条件
凸规划	局部最优即为全局最优, 理论保证强	要求目标函数和约束函数均为凸函数

方法选择建议

- 等式约束: 使用拉格朗日乘数法, 可直接转化为无约束问题。
- 一般约束: 使用罚函数法将约束问题转化为无约束问题。
- 凸规划: 使用 Python 的 `cvxpy` 库, 保证求得全局最优解。
- 非凸规划: 使用 `scipy.optimize` 的 `minimize` 函数, 尝试不同初始值, 寻找局部最优解。
- 理论分析: 使用 K-T 条件判断某点是否为最优解。

应用场景

- 工程设计：结构优化设计、参数优化。
- 经济管理：成本最小化、利润最大化（非线性成本函数）。
- 机器学习：支持向量机、神经网络训练（凸优化）。
- 信号处理：滤波器的优化设计。
- 控制系统：最优控制问题。

非线性规划与线性规划的区别

特征	线性规划	非线性规划
目标函数	线性函数	至少一个非线性函数
约束条件	线性等式/不等式	至少一个非线性
最优解位置	可行域顶点	可行域任意位置
最优解类型	全局最优解	可能仅为局部最优解
通用算法	单纯形法	无通用算法
凸性要求	总是凸规划	需满足凸性条件才能保证全局最优

5.2 一个简单非线性规划模型

下面通过一个简单的非线性规划模型来说明数学建模的五步方法。

数学建模解决问题的一般过程分为五个步骤，称为五步方法：

1. 提出问题：明确问题背景、目标和已知条件；
2. 选择建模方法：根据问题特点选择合适的数学方法；
3. 推导模型的数学表达式：建立目标函数和约束条件；
4. 求解模型：用数值方法或解析方法求解；
5. 回答问题：将结果用通俗语言表述并给出建议。

问题描述

例 5.2 一家彩电制造商计划推出两种产品：19 英寸液晶平板电视机（售价 339 美元）和 21 英寸液晶平板电视机（售价 399 美元）。成本为 19 英寸每台 195 美元，21 英寸每台 225 美元，固定成本 400000 美元。

市场规律：每多售出一台，平均销售价格下降 1 美分。两种产品之间存在交叉影响：每售出一台 21 英寸彩电，19 英寸彩电的平均售价下降 0.3 美分；每售出一台 19 英寸彩电，21 英寸彩电的平均售价下降 0.4 美分。

数学原理

(1) 第一步：提出问题 定义决策变量和参数：

$$p_1 = 339 - 0.01x_1 - 0.003x_2 \quad (5.10)$$

$$p_2 = 399 - 0.004x_1 - 0.01x_2 \quad (5.11)$$

其中 x_1 、 x_2 分别为 19 英寸和 21 英寸彩电的年销售量， p_1 、 p_2 为对应的平均售价。

成本函数：

$$c = 400000 + 195x_1 + 225x_2 \quad (5.12)$$

收入函数：

$$r = p_1x_1 + p_2x_2 \quad (5.13)$$

利润函数：

$$f = r - c \quad (5.14)$$

约束条件： $x_1 \geq 0, x_2 \geq 0$

(2) 第二步：选择建模方法 该问题为无约束多变量优化问题，目标是在可行域 $S = \{(x_1, x_2) : x_1 \geq 0, x_2 \geq 0\}$ 上最大化利润函数 $f(x_1, x_2)$ 。

(3) 第三步：推导模型 将价格函数代入利润函数：

$$\begin{aligned} f = & (339 - 0.01x_1 - 0.003x_2)x_1 + (399 - 0.004x_1 - 0.01x_2)x_2 \\ & - (400000 + 195x_1 + 225x_2) \end{aligned} \quad (5.15)$$

化简得：

$$f(x_1, x_2) = -0.01x_1^2 - 0.007x_1x_2 - 0.01x_2^2 + 144x_1 + 174x_2 - 400000 \quad (5.9)$$

(4) 第四步：求解模型 目标函数 f 是凹函数（二次项系数为负），最大值点在内部达到。令梯度为零：

$$\begin{cases} \frac{\partial f}{\partial x_1} = 144 - 0.02x_1 - 0.007x_2 = 0, \\ \frac{\partial f}{\partial x_2} = 174 - 0.007x_1 - 0.02x_2 = 0. \end{cases} \quad (5.10)$$

求解得：

$$x_1^* = 4735, \quad x_2^* = 7043, \quad f^* = 553641.025 \quad (5.16)$$

(5) 第五步：回答问题 该公司应生产 4735 台 19 英寸彩电和 7043 台 21 英寸彩电，年净利润为 553641.025 美元。19 英寸彩电平均售价为 270.52 美元，21 英寸彩电平均售价为 309.63 美元。总支出为 2908000 美元，利润率为 19%。

(6) 灵敏度分析 对 19 英寸彩电的价格弹性系数 a (原假设 $a = 0.01$) 进行灵敏度分析。

将 a 作为参数:

$$f(x_1, x_2) = -ax_1^2 - 0.007x_1x_2 - 0.01x_2^2 + 144x_1 + 174x_2 - 400000 \quad (5.11)$$

令梯度为零:

$$\begin{cases} 144 - 2ax_1 - 0.007x_2 = 0, \\ 174 - 0.007x_1 - 0.02x_2 = 0. \end{cases} \quad (5.17)$$

解得:

$$x_1(a) = \frac{1662000}{40000a - 49}, \quad x_2(a) = \frac{48000(7250a - 21)}{40000a - 49} \quad (5.13)$$

灵敏性定义:

$$S(x, a) = \frac{dx}{da} \cdot \frac{a}{x} \quad (5.18)$$

计算得:

$$S(x_1, a) \approx -1.1, \quad S(x_2, a) \approx 0.27, \quad S(y, a) \approx -0.40 \quad (5.19)$$

结论: 若 19 英寸彩电的价格弹性系数提高 10%, 则应:

- 19 英寸彩电生产量缩小 11%
- 21 英寸彩电生产量扩大 2.7%
- 利润下降 4%

模型具有良好稳健性: 即使 a 有 10% 的误差, 利润损失仅约 0.43%。

代码实现

```

1      # 程序文件 ex5_2.py
2      import sympy as sp
3      import pylab as plt
4      import numpy as np
5
6      plt.rc('text', usetex=True)
7      plt.rc('font', size=14)
8
9      # ===== 符号定义与模型建立 =====
10     sp.var('x1, x2')    # 定义符号变量
11
12     # 利润函数
13     y = (339 - 0.01*x1 - 0.003*x2)*x1 + (399 - 0.004*x1 - 0.01*x2)*
        x2 - (400000 + 195*x1 + 225*x2)
14     y = sp.simplify(y)

```

```

15
16     # 求偏导
17     dy1 = y.diff(x1)
18     dy2 = y.diff(x2)
19
20     # 求解最优解
21     s = sp.solve([dy1, dy2], [x1, x2])
22     x10 = round(float(s[x1]))
23     x20 = round(float(s[x2]))
24     y0 = y.subs({x1: x10, x2: x20})
25
26     print(f"最优产量: x1 = {x10}, x2 = {x20}")
27     print(f"最大利润: y = {y0}")
28
29     # ===== 三维曲面图与等高线图 =====
30     f = sp.lambdify((x1, x2), y, 'numpy')    # 符号函数转匿名函数
31     x = np.linspace(0, 10000, 100)
32     X, Y = np.meshgrid(x, x)
33     Z = f(X, Y)
34
35     # 三维曲面
36     ax = plt.subplot(121, projection='3d')
37     ax.plot_surface(X, Y, Z, cmap='viridis')
38     ax.set_xlabel('$x_1$')
39     ax.set_ylabel('$x_2$')
40
41     # 等高线图
42     plt.subplot(122)
43     cont = plt.contour(X, Y, Z, 10)
44     plt.clabel(cont)
45     plt.xlabel('$x_1$')
46     plt.ylabel('$x_2$', rotation=0)
47     plt.show()
48
49     # ===== 灵敏度分析 =====
50     sp.var('a', positive=True)
51
52     # 含参数 a 的利润函数
53     y = (339 - a*x1 - 0.003*x2)*x1 + (399 - 0.004*x1 - 0.01*x2)*x2
54         - (400000 + 195*x1 + 225*x2)
55     y = sp.simplify(y)
56
57     # 求偏导并求解
58     dy1 = y.diff(x1)
59     dy2 = y.diff(x2)
60     s = sp.solve([dy1, dy2], [x1, x2])
61     sx1 = s[x1]

```

```

61     sx2 = s[x2]
62
63     # 转换为数值函数
64     s1 = sp.lambdify(a, sx1, 'numpy')
65     s2 = sp.lambdify(a, sx2, 'numpy')
66
67     # 绘制  $x_1$ ,  $x_2$  关于  $a$  的曲线
68     a0 = np.linspace(0.002, 0.02, 50)
69     plt.figure()
70     plt.subplot(121)
71     plt.plot(a0, s1(a0))
72     plt.xlabel('$a$')
73     plt.ylabel('$x_1$')
74     plt.subplot(122)
75     plt.plot(a0, s2(a0))
76     plt.xlabel('$a$')
77     plt.ylabel('$x_2$')
78     plt.show()
79
80     # 计算灵敏性
81     dx1 = sx1.diff(a)
82     dx10 = dx1.subs(a, 0.01)
83     Sx1a = dx10 * 0.01 / 4735
84
85     dx2 = sx2.diff(a)
86     dx20 = dx2.subs(a, 0.01)
87     Sx2a = dx20 * 0.01 / 7043
88
89     print(f"S(x1, a) = {Sx1a:.4f}")
90     print(f"S(x2, a) = {Sx2a:.4f}")
91
92     # 利润  $y$  关于  $a$  的曲线
93     Y = y.subs({x1: s[x1], x2: s[x2]})
94     Y = sp.simplify(Y)
95     Ya = sp.lambdify(a, Y, 'numpy')
96
97     a0 = np.linspace(0.002, 0.02, 1000)
98     plt.figure()
99     plt.plot(a0, Ya(a0))
100    plt.xlabel('$a$')
101    plt.ylabel('$y$', rotation=0)
102    plt.show()
103
104    # 计算利润的灵敏性
105    Sya = -4735**2 * 0.01 / 553641.025
106    print(f"S(y, a) = {Sya:.4f}")
107

```

```

108     # 稳健性验证
109     y2 = y.subs({x1: 4735, x2: 7043, a: 0.011})    # 使用原模型产量
110     y3 = Y.subs(a, 0.011)                          # 调整产量后的最
        优利润
111     delta = (y3 - y2) / y2
112     print(f"产量不变时利润损失: {delta*100:.4f}%")

```

优缺点分析

方法	优点	缺点
解析求解（梯度为零）	精确解，计算快速	仅适用于目标函数可导且为凸/凹函数
数值优化	适用范围广，不要求可导	可能陷入局部最优
灵敏度分析	评估模型稳健性，指导决策	计算复杂，仅适用于小规模问题
五步建模法	结构清晰，易于理解和应用	依赖于对问题的正确理解

方法选择建议

- 目标函数可导且为凸/凹函数：使用解析法（令梯度为零），得到精确解。
- 目标函数复杂或不可导：使用数值优化方法（如 `scipy.optimize.minimize`）。
- 需要评估模型稳健性：进行灵敏度分析，计算关键参数的灵敏性系数。
- 实际问题应用：建议使用五步建模法，从问题出发，逐步建立、求解和分析模型。

应用场景

- 产品定价与产量决策：确定新产品的最优产量和售价。
- 市场进入策略：评估新产品推出的盈利能力和风险。
- 价格弹性分析：分析价格变化对销量和利润的影响。
- 多产品组合优化：确定多产品线的最优生产组合。
- 商业决策的稳健性检验：评估模型参数误差对决策的影响。

5.3 二次规划模型

如果规划模型的目标函数是决策向量 $x = [x_1, x_2, \dots, x_n]^T$ 的二次函数，约束条件都是线性的，那么这个模型称为二次规划（Quadratic Programming, QP）模型。

5.3.1 二次规划模型的标准形式

数学原理

二次规划模型的一般形式为：

$$\max (\min) \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_i x_j + \sum_{i=1}^n d_i x_i \quad (5.20)$$

s.t.

$$\begin{cases} \sum_{j=1}^n a_{ij} x_j \leq (\text{或 } =, \geq) b_i, & i = 1, 2, \dots, m, \\ x_i \geq 0, & i = 1, 2, \dots, n, \end{cases} \quad (5.21)$$

其中 $c_{ij} = c_{ji}$ ，即系数矩阵 $H = (c_{ij})_{n \times n}$ 为对称矩阵：

$$H = \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} \in \mathbb{R}^{n \times n} \quad (5.22)$$

特别地，当 H 为正定矩阵时，目标函数为凸函数，模型为凸二次规划。凸二次规划的局部最优解即为全局最优解。

典型应用：投资组合问题

具体例子

例 5.3：简单二次规划

求解如下二次规划模型：

$$\max -x_1^2 - 0.3x_1x_2 - 2x_2^2 + 98x_1 + 277x_2 \quad (5.23)$$

s.t.

$$\begin{cases} x_1 + x_2 \leq 100, \\ x_1 - 2x_2 \leq 0, \\ x_1, x_2 \geq 0. \end{cases} \quad (5.24)$$

矩阵形式：

$$\max [x_1, x_2] \begin{bmatrix} -1 & -0.15 \\ -0.15 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + [98, 277] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad (5.25)$$

s.t.

$$\begin{bmatrix} 1 & 1 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} 100 \\ 0 \end{bmatrix}, \quad x_1, x_2 \geq 0. \quad (5.26)$$

最优解：

$$x_1 = 35.365, \quad x_2 = 64.635, \quad f_{\max} = 11077.8703 \quad (5.27)$$

例 5.4：投资组合问题

已知 A, B, C 三种股票在过去 12 年的年收益率数据，试确定最优投资比例。

(1) 问题分析 投资组合问题由 Markowitz 于 1952 年提出，他因此获得 1990 年诺贝尔经济学奖。

核心思想：

- 收益用年收益率的期望度量；
- 风险用年收益率的方差（或标准差）度量；
- 方差越大，风险越大；方差越小，风险越小。

由 12 年数据计算得到：

- 年平均收益率： $\mu = [0.0882, 0.2137, 0.2346]^T$
- 协方差矩阵： $F = \begin{bmatrix} 0.0111 & 0.0128 & 0.0134 \\ 0.0128 & 0.0584 & 0.0554 \\ 0.0134 & 0.0554 & 0.0942 \end{bmatrix}$

投资组合的总收益率： $R = x_1 R_1 + x_2 R_2 + x_3 R_3$

投资组合的期望收益： $E(R) = \mu^T x$

投资组合的方差： $\text{Var}(R) = x^T F x$

(2) 问题 1：固定收益，极小化风险 要求年收益率至少达到 15%，使风险（方差）最小：

$$\min x^T F x \quad (5.28)$$

s.t.

$$x_1 + x_2 + x_3 = 1, \quad \mu^T x \geq 0.15, \quad x_1, x_2, x_3 \geq 0. \quad (5.29)$$

最优解：

$$x_1 = 0.5269, \quad x_2 = 0.3578, \quad x_3 = 0.1153, \quad \text{Var}_{\min} = 0.0228 \quad (5.30)$$

即 A, B, C 的投资比例分别为 52.69%、35.78%、11.53%。

(3) 问题 2：固定风险，最大化收益 要求方差（风险）不超过 0.09，使收益最大化：

$$\max z = \mu^T x \quad (5.31)$$

s.t.

$$x_1 + x_2 + x_3 = 1, \quad x^T F x \leq 0.09, \quad x_1, x_2, x_3 \geq 0. \quad (5.32)$$

最优解:

$$x_1 = 0, \quad x_2 = 0.0562, \quad x_3 = 0.9438, \quad z_{\max} = 0.2334 \quad (5.33)$$

即 A, B, C 的投资比例分别为 0%、5.62%、94.38%，最高收益率为 23.34%。

代码实现

例 5.3 的求解:

```

1  # 程序文件 ex5_3.py
2  import cvxpy as cp
3  import numpy as np
4
5  # 二次项系数矩阵 (对称矩阵)
6  c2 = np.array([[1, -0.15], [-0.15, -2]])
7
8  # 一次项系数向量
9  c1 = np.array([98, 277])
10
11 # 约束矩阵
12 a = np.array([[1, 1], [1, -2]])
13 b = np.array([100, 0])
14
15 x = cp.Variable(2, pos=True)
16
17 # 目标函数:  $x^T c2 x + c1^T x$ 
18 obj = cp.Maximize(cp.quad_form(x, c2) + c1 @ x)
19
20 # 约束条件
21 con = [a @ x <= b]
22
23 prob = cp.Problem(obj, con)
24 prob.solve(solver='CVXOPT')
25
26 print('最优解:', np.round(x.value, 4))
27 print('最优值:', round(prob.value, 4))

```

例 5.4 的求解:

```

1  # 程序文件 ex5_4.py
2  import cvxpy as cp
3  import numpy as np
4
5  # 从文件读取数据
6  a = np.loadtxt('data5_4.txt')

```

```

7
8     # 计算年平均收益
9     mu = a.mean(axis=0)
10
11    # 计算协方差矩阵
12    F = np.cov(a.T)
13
14    x = cp.Variable(3, pos=True)
15
16    # ===== 问题 1: 固定收益, 极小化风险 =====
17    ob1 = cp.Minimize(cp.quad_form(x, F))
18    con1 = [
19        mu @ x >= 0.15,
20        sum(x) == 1
21    ]
22
23    prob1 = cp.Problem(ob1, con1)
24    prob1.solve(solver='CVXOPT')
25
26    print('问题1 最优值:', round(prob1.value, 4))
27    print('问题1 最优解:', np.round(x.value, 4))
28
29    # ===== 问题 2: 固定风险, 最大化收益 =====
30    ob2 = cp.Maximize(mu @ x)
31    con2 = [
32        sum(x) == 1,
33        cp.quad_form(x, F) <= 0.09
34    ]
35
36    prob2 = cp.Problem(ob2, con2)
37    prob2.solve(solver='CVXOPT')
38
39    print('问题2 最优值:', round(prob2.value, 4))
40    print('问题2 最优解:', np.round(x.value, 4))

```

输出结果:

```

1     问题1 最优值: 0.0228
2     问题1 最优解: [0.5269 0.3578 0.1153]
3
4     问题2 最优值: 0.2334
5     问题2 最优解: [0.         0.0562 0.9438]

```

优缺点分析

方法	优点	缺点
二次规划	目标函数可反映收益与风险的权衡关系	仅适用于二次目标函数 + 线性约束
凸二次规划	局部最优即为全局最优, 求解可靠	要求 Hesse 矩阵正定/半正定
Markowitz 模型	理论基础完善, 获得诺贝尔奖	假设收益率服从正态分布, 且方差能完全刻画风险
cvxpy 求解	代码简洁, 自动选择求解器	大规模问题时求解速度较慢

方法选择建议

- 二次目标 + 线性约束：使用二次规划模型，调用 cvxpy 的 `cp.quad_form()`。
- 凸二次规划（**H** 正定）：使用 CVXOPT 求解器，保证全局最优。
- 投资组合优化：采用 Markowitz 均值-方差模型，根据风险偏好选择问题 1 或问题 2。
- 大规模投资组合：使用专用优化软件（如 Gurobi）或采用数值近似方法。

应用场景

- 投资组合优化：确定股票、债券等资产的最优配置比例。
- 风险控制：在给定收益目标下最小化投资风险。
- 资产定价：估计资产的期望收益和风险水平。
- 工程优化：能量最小化、信号处理中的二次优化问题。
- 机器学习：支持向量机（SVM）中的二次规划求解。

5.4 非线性规划的求解及应用

对于一般的非线性规划问题，由于不是凸规划，不能使用 cvxpy 库求解。求解一般的非线性规划问题使用 `scipy.optimize` 模块的 `minimize` 函数。

数学原理

`minimize` 函数求解非线性规划的标准型为：

$$\min f(x)$$

(5.34)

s.t.

$$g_i(x) \geq 0, \quad i = 1, 2, \dots, p, \quad h_j(x) = 0, \quad j = 1, 2, \dots, q.$$

(5.35)

scipy.optimize.minimize 常用参数

参数	说明
fun	目标函数
x0	初始猜测值 (n 维数组)
method	求解方法 (如'SLSQP'、'trust-constr' 等)
constraints	约束条件 (字典列表)
bounds	变量界限 (每个变量的上下界)
options	求解器选项 (如最大迭代次数)

约束条件的定义:

- 不等式约束: $\{ 'type': 'ineq', 'fun': \lambda x: g(x) \}$, 要求 $g(x) \geq 0$
- 等式约束: $\{ 'type': 'eq', 'fun': \lambda x: h(x) \}$, 要求 $h(x) = 0$
- 多个约束: 放入列表中
- 变量界限: $[(x1_min, x1_max), (x2_min, x2_max), \dots]$

具体例子

例 5.5 (非凸二次规划):

$$\min -x_1^2 - 0.3x_1x_2 - 2x_2^2 + 98x_1 + 277x_2 \quad (5.36)$$

s.t.

$$x_1 + x_2 \leq 100, \quad x_1 - 2x_2 \leq 0, \quad x_1, x_2 \geq 0. \quad (5.37)$$

注意: 该二次规划的目标函数不是凸函数, 不能使用 cvxpy 库求解。

最优解: $x_1 = 0, x_2 = 0$, 目标函数最优值为 0。

例 5.6 (一般非线性规划):

$$\min f(x) = x_1^2 + x_2^2 + x_3^2 + 8 \quad (5.38)$$

s.t.

$$\begin{cases} x_1^2 - x_2 + x_3^2 \geq 0, \\ x_1 + x_2^2 + x_3^2 \leq 20, \\ x_1 - x_2^2 + 2 = 0, \\ x_2 + 2x_3^2 = 3, \\ x_1, x_2, x_3 \geq 0. \end{cases} \quad (5.39)$$

最优解: $x_1 = 0.5522, x_2 = 1.2033, x_3 = 0.9478$, 最优值 $f_{\min} = 10.6511$ 。

例 5.7 (绝对值规划问题):

$$\min z = |x_1| + 2|x_2| + 3|x_3| + 4|x_4| \quad (5.40)$$

s.t.

$$\begin{cases} x_1 - x_2 - x_3 + x_4 = 0, \\ x_1 - x_2 + x_3 - 3x_4 = 1, \\ x_1 - x_2 - 2x_3 + 3x_4 = -\frac{1}{2}. \end{cases} \quad (5.41)$$

解法一：直接求解（凸规划）

由于目标函数为绝对值函数（凸函数），约束为线性等式，该问题为凸规划，可使用 cvxpy 求解。

最优解： $x_1 = 0.25$, $x_2 = 0$, $x_3 = 0$, $x_4 = -0.25$, 最优值 $z_{\min} = 1.25$ 。

解法二：线性化变换

做变量变换：

$$u_i = \frac{x_i + |x_i|}{2}, \quad v_i = \frac{|x_i| - x_i}{2} \quad (5.42)$$

则：

$$x = u - v, \quad |x| = u + v \quad (5.43)$$

原问题转化为线性规划：

$$\min c^T(u + v) \quad (5.44)$$

s.t.

$$A(u - v) = b, \quad u, v \geq 0. \quad (5.45)$$

其中 $c = [1, 2, 3, 4]^T$, $b = [0, 1, -1/2]^T$ 。

例 5.8（供应与选址问题）：

有 6 个建筑工地，位置 (a_i, b_i) 和水泥日用量 c_i 如表 5.4 所列。拟建两个料场，日储量各为 20t，确定料场位置和运输方案，使总吨公里数最小。

表 5.6 建筑工地的位置及水泥日用量

参数	1	2	3	4	5	6
a/km	1.25	8.75	0.5	3.75	3	7.25
b/km	1.25	0.75	4.75	5	6.5	7.75
c/t	3	5	4	7	6	11

数学模型：

决策变量：料场位置 (x_j, y_j) ($j = 1, 2$)，运量 z_{ij} ($i = 1, \dots, 6, j = 1, 2$)。

$$\min \sum_{i=1}^6 \sum_{j=1}^2 z_{ij} \sqrt{(x_j - a_i)^2 + (y_j - b_i)^2} \quad (5.46)$$

s.t.

$$\sum_{j=1}^2 z_{ij} = c_i, \quad i = 1, \dots, 6, \quad \sum_{i=1}^6 z_{ij} \leq 20, \quad j = 1, 2, \quad z_{ij} \geq 0. \quad (5.47)$$

最优解:

- 料场 1 位置: (3.2654, 5.1919), 料场 2 位置: (7.25, 7.75)
- 最小吨公里数: 71.9352

表 5.7 两个料场向 6 个工地的运输方案

料场	工地 1	工地 2	工地 3	工地 4	工地 5	工地 6
料场 1	0	5	0	0	0	11
料场 2	3	0	4	7	6	0

代码实现

例 5.5 的求解:

```

1  # 程序文件 ex5_5.py
2  import numpy as np
3  from scipy.optimize import minimize
4
5  c2 = np.array([[1, -0.15], [-0.15, -2]])
6  c1 = np.array([98, 277])
7  a = np.array([[1, 1], [1, -2]])
8  b = np.array([100, 0])
9
10 # 目标函数
11 obj = lambda x: x @ c2 @ x + c1 @ x
12
13 # 约束条件 (不等式约束要求 g(x) >= 0)
14 con = {'type': 'ineq', 'fun': lambda x: b - a @ x}
15
16 # 变量界限
17 bd = [(0, np.inf) for i in range(a.shape[1])]
18
19 res = minimize(obj, np.random.randn(2), constraints=con, bounds
20               =bd)
21 print(res)

```

例 5.6 的求解:

```

1  # 程序文件 ex5_6.py
2  import numpy as np
3  from scipy.optimize import minimize

```

```

4
5     # 目标函数
6     obj = lambda x: sum(x**2) + 8
7
8     # 不等式约束 (返回 >= 0 的数组)
9     def constr1(x):
10         x1, x2, x3 = x
11         return [x1**2 - x2 + x3**2, 20 - x1 - x2**2 - x3**2]
12
13     # 等式约束 (返回 0 的数组)
14     def constr2(x):
15         x1, x2, x3 = x
16         return [-x1 - x2**2 + 2, x2 + 2*x3**2 - 3]
17
18     con1 = {'type': 'ineq', 'fun': constr1}
19     con2 = {'type': 'eq', 'fun': constr2}
20     con = [con1, con2]
21
22     bd = [(0, np.inf) for i in range(3)]
23
24     res = minimize(obj, np.random.randint(3, size=3), constraints=
25         con, bounds=bd)
26     print(res)

```

例 5.7 (解法一: cvxpy 直接求解):

```

1     # 程序文件 ex5_7_1.py
2     import cvxpy as cp
3     import numpy as np
4
5     c = np.arange(1, 5)
6     a = np.array([[1, -1, -1, 1], [1, -1, 1, -3], [1, -1, -2, 3]])
7     b = np.array([0, 1, -1/2])
8
9     x = cp.Variable(4)
10    obj = cp.Minimize(c @ cp.abs(x))
11    con = [a @ x == b]
12
13    prob = cp.Problem(obj, con)
14    prob.solve(solver='GLPK_MI')
15
16    print("最优值:", prob.value)
17    print("最优解:", x.value)

```

例 5.7 (解法二: 线性化变换):

```

1     # 程序文件 ex5_7_2.py
2     import cvxpy as cp

```

```

3     import numpy as np
4
5     c = np.arange(1, 5)
6     a = np.array([[1, -1, -1, 1], [1, -1, 1, -3], [1, -1, -2, 3]])
7     b = np.array([0, 1, -1/2])
8
9     u = cp.Variable(4, pos=True)
10    v = cp.Variable(4, pos=True)
11
12    obj = cp.Minimize(c @ (u + v))
13    con = [a @ (u - v) == b]
14
15    prob = cp.Problem(obj, con)
16    prob.solve(solver='GLPK_MI')
17
18    print("最优值:", prob.value)
19    print("u =", u.value)
20    print("v =", v.value)
21    print("原问题最优解 x = u - v =", u.value - v.value)

```

例 5.8（供应与选址）的求解：

```

1     # 程序文件 ex5_8.py
2     import numpy as np
3     from scipy.optimize import minimize
4
5     # 读取数据：第一行 a，第二行 b，第三行 c
6     d = np.loadtxt('data5_8.txt')
7     a = d[0]; b = d[1]; c = d[2]
8     e = np.array([20, 20])
9
10    def obj(xyz):
11        # xyz = [x1, x2, y1, y2, z11, z12, ..., z61, z62]
12        x = xyz[0:2]
13        y = xyz[2:4]
14        z = xyz[4:].reshape(6, 2)
15        obj_val = 0
16        for i in range(6):
17            for j in range(2):
18                obj_val += z[i, j] * np.sqrt((x[j] - a[i])**2 + (y[j] - b[i])
19                    **2)
20        return obj_val
21
22    # 约束条件
23    con = [
24        {'type': 'eq', 'fun': lambda z: z[4:].reshape(6, 2).sum(axis=1)
25            - c},

```



```

24     {'type': 'ineq', 'fun': lambda z: e - z[4:].reshape(6, 2).sum(
        axis=0)}
25 ]
26
27 bd = [(0, np.inf) for i in range(16)]
28
29 res = minimize(obj, np.random.rand(16), constraints=con, bounds
        =bd)
30
31 s = np.round(res.x, 4)
32 print('目标函数最优值:', round(res.fun, 4))
33 print('料场1坐标 (x1, y1):', s[0], s[2])
34 print('料场2坐标 (x2, y2):', s[1], s[3])
35 print('运输方案:\n', s[4:].reshape(6, 2).T)

```

优缺点分析

方法	优点	缺点
cvxpy (凸规划)	全局最优解, 代码简洁	仅适用于凸规划问题
scipy.minimize	适用于一般非线性规划	只能保证局部最优解
线性化变换	将非线性问题转化为线性问题	需要找到合适的变换方式
罚函数法	可处理一般约束	计算精度受罚因子影响

方法选择建议

- 凸规划问题：优先使用 cvxpy 库，保证全局最优解。
- 非凸规划问题：使用 scipy.optimize.minimize，选择 SLSQP 或 trust-constr 方法。
- 带绝对值的规划问题：先做线性化变换，转化为线性规划求解。
- 规模较大的非线性规划：使用专用优化软件（如 Gurobi、Ipopt）。
- 初始值选择：非凸规划对初始值敏感，建议多尝试几个不同初始值。

应用场景

- 供应链管理：仓库选址与配送路径联合优化。
- 工程设计：结构参数优化、成本最小化。
- 投资组合：非凸风险度量下的资产配置。
- 能源系统：电力调度、能源分配优化。
- 机器学习：非凸损失函数的参数优化。

以下是第 5.5 节多目标规划的完整笔记：

“\latex

5.5 多目标规划

5.5.1 多目标规划问题的基本理论

多目标规划是多目标决策的重要内容之一。在进行多目标决策时，当希望每个目标都尽可能大（或尽可能小）时，就形成了一个多目标规划问题。

数学原理

(1) 多目标规划的一般形式

$$\min f(x) = [f_1(x), f_2(x), \dots, f_m(x)]^T \quad (5.17)$$

s.t.

$$\begin{cases} g_i(x) \leq 0, & i = 1, 2, \dots, p, \\ h_j(x) = 0, & j = 1, 2, \dots, q, \end{cases} \quad (5.18)$$

其中 x 为决策向量； $f_1(x), f_2(x), \dots, f_m(x)$ 为目标函数。

可行域（决策空间）：

$$\Omega = \{x \mid g_i(x) \leq 0, i = 1, \dots, p; h_j(x) = 0, j = 1, \dots, q\} \quad (5.48)$$

像集（目标空间）：

$$f(\Omega) = \{f(x) \mid x \in \Omega\} \quad (5.49)$$

(2) 最优解的概念

- 绝对最优解：设 $x^* \in \Omega$ ，若对任意 $i = 1, \dots, m$ 及任意 $x \in \Omega$ ，均有 $f_i(x^*) \leq f_i(x)$ ，则称 x^* 为绝对最优解。

绝对最优解要求所有目标同时达到最优，通常不存在。

- 有效解（Pareto 最优解）：设 $x^* \in \Omega$ ，若不存在 $x \in \Omega$ ，使得

$$f_i(x) \leq f_i(x^*), \forall i, \quad \text{且至少有一个 } f_i(x) < f_i(x^*), \quad (5.50)$$

则称 x^* 为有效解（Pareto 有效解）。

Pareto 最优解：无法在不损害其他目标的前提下改进任何一个目标。

- 满意解：决策者给出 m 个阈值 α_i ，当 $x \in \Omega$ 满足 $f_i(x) \leq \alpha_i$ 时，称 x 为满意解。

注：多目标规划一般不提出最优解，只提出满意解或有效解。

5.5.2 求有效解的常用方法

多目标规划求解前通常需要进行预处理：

1. 无量纲化处理：消除各目标量纲不同的影响。
2. 数量级归一化处理：避免大数吃小数现象。

(1) 线性加权法 根据目标重要性确定权重 w_j ，以加权平均值为评价函数。

$$\min \sum_{i=1}^m w_i f_i(x), \quad \text{s.t. } x \in \Omega, \quad \sum_{i=1}^m w_i = 1, \quad w_i \geq 0. \quad (5.51)$$

特点：权重反映目标重要程度，由决策者给出，具有主观性。

(2) ε 约束法 选择一个主要目标 $f_k(x)$ ，将其他目标转化为约束条件：

$$\min f_k(x), \quad \text{s.t. } \begin{cases} f_i(x) \leq \varepsilon_i, & i \neq k, \\ x \in \Omega. \end{cases} \quad (5.52)$$

优点：

- 保证主要目标的利益，同时兼顾其他目标；
- 每个 Pareto 有效解都可通过适当选择 ε_i 求得。

(3) 理想点法 以每个单目标最优值为理想值，使各目标与理想值的差的加权平方和最小。

步骤：

1. 求各单目标最优值： $f_i^* = \min_{x \in \Omega} f_i(x)$
2. 构造评价函数： $\sum_{i=1}^m w_i (f_i - f_i^*)^2$
3. 求解： $\min_{x \in \Omega} \sum_{i=1}^m (f_i - f_i^*)^2$

(4) 优先级法（序贯解法） 按目标重要性分级，先优化高级目标，再在保证高级目标最优的前提下优化低级目标。

步骤：

1. 确定优先级；
2. 求第一级最优值： $f_1^* = \min_{x \in \Omega} f_1(x)$ ；
3. 以 $f_1(x) = f_1^*$ 为约束，求第二级最优值；

4. 依次递推求解。

适用场景：各目标重要性差距明显，需首先确保最重要的目标。

具体例子

例 5.9：生产与污染双目标规划

某公司生产两种产品（甲、乙），目标为：

1. 极大化利润： $\max f_1 = 2x_1 + 3x_2$

2. 极小化污染： $\min f_2 = x_1 + 2x_2$

约束条件：

$$\begin{cases} 0.5x_1 + 0.25x_2 \leq 8 & \text{设备工时} \\ 0.2x_1 + 0.2x_2 \leq 4 & \text{工人工时} \\ x_1 + 5x_2 \leq 72 & \text{原材料} \\ x_1 + x_2 \geq 10 & \text{产量约束} \\ x_1, x_2 \geq 0 \end{cases} \quad (5.53)$$

该问题可描述为：

$$\min\{-f_1, f_2\} \quad (5.54)$$

求解结果：

方法	x_1	x_2	利润	污染
线性加权法 ($w_1 = w_2 = 0.5$)	12	8	48	28
理想点法	13.36	5.28	—	—
序贯解法（先利润后污染）	7	13	53	33

代码实现

```

1  # 程序文件 ex5_9.py
2  import numpy as np
3  import cvxpy as cp
4
5  # 数据定义
6  c1 = np.array([-2, -3]) # 利润最大化（取负）
7  c2 = np.array([1, 2])   # 污染最小化
8  a = np.array([[0.5, 0.25], [0.2, 0.2], [1, 5], [-1, -1]])
9  b = np.array([8, 4, 72, -10])
10
11  x = cp.Variable(2, pos=True)
12  con = [a @ x <= b]
13

```

```

14 # ===== 方法一：线性加权法 =====
15 obj1 = cp.Minimize(0.5 * (c1 + c2) @ x)
16 prob1 = cp.Problem(obj1, con)
17 prob1.solve(solver='GLPK_MI')
18 print('线性加权法最优解:', x.value)
19 print('线性加权法最优值:', prob1.value)
20
21 # ===== 方法二：理想点法 =====
22 # 求单目标最优值
23 obj_f1 = cp.Minimize(c1 @ x)
24 prob_f1 = cp.Problem(obj_f1, con)
25 prob_f1.solve(solver='GLPK_MI')
26 v1 = prob_f1.value
27
28 obj_f2 = cp.Minimize(c2 @ x)
29 prob_f2 = cp.Problem(obj_f2, con)
30 prob_f2.solve(solver='GLPK_MI')
31 v2 = prob_f2.value
32
33 print('利润最优值 (负):', v1)
34 print('污染最优值:', v2)
35
36 # 构造理想点评价函数
37 obj3 = cp.Minimize((c1 @ x - v1)**2 + (c2 @ x - v2)**2)
38 prob3 = cp.Problem(obj3, con)
39 prob3.solve(solver='CVXOPT')
40 print('理想点法最优解:', x.value)
41
42 # ===== 方法三：序贯解法 =====
43 # 先保证利润最优
44 con.append(c1 @ x == v1)
45 prob4 = cp.Problem(obj_f2, con)
46 prob4.solve(solver='GLPK_MI')
47 x3 = x.value
48
49 print('序贯解法最优解:', x3)
50 print('利润:', -c1 @ x3)
51 print('污染:', c2 @ x3)

```

输出结果：

```

1  线性加权法最优解: [12.  8.]
2  线性加权法最优值: -10.0
3
4  利润最优值 (负): -53.0
5  污染最优值: 10.0
6

```

7	理想点法最优解：[13.36 5.28]
8	
9	序贯解法最优解：[7. 13.]
10	利润：53.0
11	污染：33.0

优缺点分析

方法	优点	缺点
线性加权法	简单直观，计算效率高	权重确定主观性强
ε 约束法	保证主要目标利益，Pareto 解全覆盖	参数 ε_i 选择困难
理想点法	反映目标与理想值的差距	需求解 $m+1$ 个单目标规划
优先级法	符合实际决策层级思维	要求目标优先级明确

方法选择建议

- 目标重要性明确：使用优先级法，确保重要目标优先满足。
- 目标重要性已知但无层级：使用线性加权法，根据重要性分配权重。
- 有主要关注目标：使用 ε 约束法，以主要目标为优化目标。
- 希望结果接近各目标最优：使用理想点法，使结果尽可能接近各目标理想值。

应用场景

- 生产管理：同时最大化利润和最小化污染排放。
- 投资决策：最大化收益与最小化风险。
- 供应链管理：最小化成本与最大化客户满意度。
- 工程优化：最大化性能与最小化成本。
- 公共政策：平衡经济增长与环境保护。

5.6 飞行管理问题

问题描述

例 5.10（1995 年全国大学生数学建模竞赛 A 题）

在约 10000m 高空的某边长 160km 的正方形区域内，经常有若干架飞机做水平飞行。区域内每架飞机的位置和速度向量均由计算机记录其数据，以便进行飞行管理。当

一架欲进入该区域的飞机到达该区域边缘时,记录其数据后,要立即计算并判断是否会与区域内的飞机发生碰撞。如果会碰撞,则应计算如何调整各架(包括新进入的)飞机飞行的方向角,以避免碰撞。

假设条件:

1. 不碰撞的标准为任意两架飞机的距离大于 8km;
2. 飞机飞行方向角调整的幅度不应超过 30° ;
3. 所有飞机飞行速度均为 800km/h;
4. 进入该区域的飞机在到达区域边缘时,与区域内飞机的距离应在 60km 以上;
5. 最多需考虑 6 架飞机;
6. 不必考虑飞机离开此区域后的状况。

已知数据:区域顶点坐标为 $(0,0)$ 、 $(160,0)$ 、 $(160,160)$ 、 $(0,160)$ 。6 架飞机(含 1 架新进入)的位置和方向角如表 5.7 所列。

表 5.11 飞行记录数据

飞机编号	横坐标 x	纵坐标 y	方向角/ ($^\circ$)
1	150	140	243
2	85	85	236
3	150	155	220.5
4	145	50	159
5	130	150	230
新进入	0	0	52

注:方向角指飞行方向与 x 轴正向的夹角。

数学原理

(1) 符号说明

- $a = 800 \text{ km/h}$: 飞机飞行速度;
- (x_i^0, y_i^0) : 第 i 架飞机的初始位置;
- $(x_i(t), y_i(t))$: 第 i 架飞机在 t 时刻的位置;
- θ_i^0 : 第 i 架飞机的原飞行方向角;
- $\Delta\theta_i$: 第 i 架飞机的方向角调整量, $|\Delta\theta_i| \leq 30^\circ$;
- $\theta_i = \theta_i^0 + \Delta\theta_i$: 调整后的飞行方向角。

(2) 相对运动分析 考虑两架飞机 i 和 j , 将飞机 i 视为不动, 飞机 j 以相对速度运动:

$$v = v_i - v_j = (a \cos \theta_i - a \cos \theta_j, a \sin \theta_i - a \sin \theta_j) \quad (5.23)$$

相对速度大小:

$$v = 2a \left| \sin \left(\frac{\theta_i - \theta_j}{2} \right) \right| \quad (5.24)$$

当 $\theta_i \geq \theta_j$ 时, 相对飞行方向角为:

$$\beta_{ij} = \frac{\pi}{2} + \frac{\theta_i + \theta_j}{2} \quad (5.55)$$

两架飞机的初始距离:

$$r_{ij}(0) = \sqrt{(x_i^0 - x_j^0)^2 + (y_i^0 - y_j^0)^2} \quad (5.25)$$

安全角度阈值:

$$\alpha_{ij}^0 = \arcsin \left(\frac{8}{r_{ij}(0)} \right) \quad (5.26)$$

两架飞机不碰撞的条件 (如图 5.6 所示):

$$|\beta_{ij}^0 + \frac{1}{2}(\Delta\theta_i + \Delta\theta_j)| > \alpha_{ij}^0 \quad (5.28)$$

其中 β_{ij}^0 为调整前相对速度矢量与两架飞机连线矢量的夹角。

图 5.6 相对飞行方向角

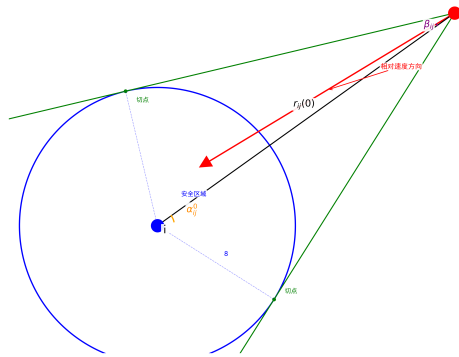


图 5.3 相对飞行方向角示意图

利用复数幅角计算 β_{ij}^0 :

$$\beta_{mn}^0 = \arg \left(\frac{v_m - v_n}{r_m - r_n} \right) \quad (5.56)$$

(3) 优化模型 以所有飞机调整量绝对值之和最小为目标函数:

$$\min \sum_{i=1}^6 |\Delta\theta_i| \quad (5.29)$$

s.t.

$$\begin{cases} |\beta_{ij}^0 + \frac{1}{2}(\Delta\theta_i + \Delta\theta_j)| > \alpha_{ij}^0, & i = 1, \dots, 5, j = i + 1, \dots, 6, \\ |\Delta\theta_i| \leq 30^\circ, & i = 1, 2, \dots, 6. \end{cases} \quad (5.57)$$

计算结果

计算得到 α_{ij}^0 和 β_{ij}^0 的值分别如表 5.8 和表 5.9 所列。

表 5.12 α_{ij}^0 的值 (单位: $^\circ$)

$i \setminus j$	1	2	3	4	5	6
1	0	5.3912	32.2310	5.0918	20.9634	2.2345
2	5.3912	0	4.8040	6.6135	5.8079	3.8159
3	32.2310	4.8040	0	4.3647	22.8337	2.1255
4	5.0918	6.6135	4.3647	0	4.5377	2.9898
5	20.9634	5.8079	22.8337	4.5377	0	2.3098
6	2.2345	3.8159	2.1255	2.9898	2.3098	0

表 5.13 β_{ij}^0 的值 (单位: $^\circ$)

$i \setminus j$	1	2	3	4	5	6
1	0	109.2636	-128.2500	24.1798	173.0651	14.4749
2	109.2636	0	-88.8711	-42.2436	-92.3048	9.0000
3	-128.2500	-88.8711	0	12.4763	-58.7862	0.3108
4	24.1798	-42.2436	12.4763	0	5.9692	-3.5256
5	173.0651	-92.3048	-58.7862	5.9692	0	1.9144
6	14.4749	9.0000	0.3108	-3.5256	1.9144	0

最优解:

$$\Delta\theta_5 = 0.2056^\circ, \quad \Delta\theta_6 = 0.5853^\circ, \quad \text{其他为} 0^\circ \quad (5.58)$$

目标函数最优值 (总调整量):

$$\sum_{i=1}^6 |\Delta\theta_i| = 0.7909^\circ \quad (5.59)$$

代码实现

```

1  # 程序文件 ex5_10.py
2  import numpy as np
3  import pandas as pd
4  from scipy.optimize import minimize
5
6  # 初始数据
7  x0 = np.array([150, 85, 150, 145, 130, 0])
8  y0 = np.array([140, 85, 155, 50, 150, 0])
9  q = np.array([243, 236, 220.5, 159, 230, 52]) # 方向角 (度)
10
11 # 计算距离矩阵
12 d = np.zeros((6, 6))
13 xy0 = np.vstack([x0, y0]).T

```

```

14     for i in range(6):
15         for j in range(6):
16             d[i, j] = np.linalg.norm(xy0[i] - xy0[j])
17             d[d == 0] = np.inf
18
19         # 计算  $\alpha_{ij}$  (安全角度阈值)
20         a0 = np.arcsin(8.0 / d) * 180 / np.pi
21
22         # 计算  $\beta_{ij}$  (相对速度与连线夹角)
23         xy1 = x0 + 1j * y0
24         xy2 = np.exp(1j * q * np.pi / 180) # 单位方向向量
25         b0 = np.zeros((6, 6))
26         for m in range(6):
27             for n in range(6):
28                 if n != m:
29                     b0[m, n] = np.angle((xy2[n] - xy2[m]) / (xy1[m] - xy1[n]))
30             b0 = b0 * 180 / np.pi
31
32         # 保存数据到 Excel
33         f = pd.ExcelWriter('data5_10.xlsx')
34         pd.DataFrame(a0).to_excel(f, 'sheet1', index=None)
35         pd.DataFrame(b0).to_excel(f, 'sheet2', index=None)
36         f.save()
37
38         # 目标函数: 调整量绝对值之和最小
39         obj = lambda x: sum(np.abs(x))
40
41         # 变量界限: 每个调整量不超过 30 度
42         bd = [(-30, 30) for i in range(6)]
43
44         # 初始猜测值
45         x_init = 30 * np.random.rand(6)
46
47         # 约束条件: 每对飞机不碰撞
48         cons = []
49         for i in range(5):
50             for j in range(i + 1, 6):
51                 cons.append({
52                     'type': 'ineq',
53                     'fun': lambda x, i=i, j=j: np.abs(b0[i, j] + (x[i] + x[j])
54                                     / 2) - a0[i, j]
55                 })
56
57         # 求解
58         res = minimize(obj, x_init, constraints=cons, bounds=bd)
59
60         print('目标函数最优值:', round(res.fun, 4))

```

60

```
print('最优解:', np.round(res.x, 4))
```

方法评述

方面	优点	缺点/注意事项
相对运动转化	将两机碰撞问题转化为几何角度判断, 简化计算	需要正确计算相对速度方向
非线性规划模型	以最小调整量为目标, 结果经济合理	非凸问题, 不同初值可能得到不同解
复数角度计算	计算 β_{ij}^0 方便准确	需注意角度范围的规范化

方法选择建议

- 碰撞判断: 使用相对运动方法, 通过 β_{ij}^0 与 α_{ij}^0 的比较判断。
- 优化求解: 使用 `scipy.optimize.minimize` 求解非线性规划, 多尝试几组初值。
- 角度计算: 使用复数幅角 `np.angle` 计算 β_{ij}^0 , 避免繁琐的三角函数推导。

应用场景

- 空中交通管制: 飞机飞行冲突检测与解脱。
- 船舶避碰: 海上船舶航向调整避免碰撞。
- 无人机编队: 多无人机协同飞行避障。
- 机器人路径规划: 多机器人运动冲突解决。

第 6 章

图论模型

图论是近 30 年来发展非常活跃的一个数学分支。大量的最优化问题都可以抽象成网络模型结构来加以解释、描述和求解。它在建模时，具有直观、易理解、适应性强等特点，已广泛应用于管理科学、物理学、化学、计算机科学、信息论、控制论、社会科学以及军事科学等领域。一些实际网络，如运输网、电话网、电力网、计算机网络、交通网络等，都可以用图论加以描述和分析，并借助于计算机算法直接求解。

6.1 图与网络的基础理论

6.1.1 图与网络的基本概念

图，概括地讲就是由一些点和这些点之间的连线组成。

数学原理

(1) 无向图和有向图 定义 6.1 一个无向图 G 是由非空顶点集 V 和边集 E 按一定的对应关系构成的连接结构，记为 $G = (V, E)$ 。其中非空集合 $V = \{v_1, v_2, \dots, v_n\}$ 为 G 的顶点集， V 中的元素称为 G 的顶点；集合 $E = \{e_1, e_2, \dots, e_m\}$ 为 G 的边集， E 中的元素称为 G 的边。用 $|V|$ 表示顶点个数， $|E|$ 表示边的条数。

图 G 的每一条边是由连接 G 中两个顶点而得到的一条线，通常记作 $e_k = (v_i, v_j)$ ，其中顶点 v_i, v_j 称为边 e_k 的两个端点。对无向图来说， (v_i, v_j) 和 (v_j, v_i) 表示同一条边。

带有方向的边称为有向边，又称为弧。

定义 6.2 有向图记为 $D = (V, A)$ ，其中 V 为顶点集， $A = \{a_1, a_2, \dots, a_m\}$ 为弧集合，每一条弧与一个有向的顶点对相对应，弧 $a_k = (v_i, v_j)$ 表示弧的方向自顶点 v_i 指向 v_j ， v_i 称为弧的始端， v_j 称为末端。在有向图中， (v_i, v_j) 与 (v_j, v_i) 表示不同的弧。

(2) 简单图、完全图、赋权图 定义 6.3 如果一条边的两个端点是同一个顶点，则称这条边为环。如果有两条边或多条边的端点是同一对顶点，则称这些边为重边或平行边。称不与任何边相关联的顶点为孤立点。

定义 6.4 无环且无重边的图称为简单图。如不特别声明，一般的图均指简单图。

定义 6.5 任意两顶点均相邻的简单图称为完全图。含 n 个顶点的完全图记为 K_n 。

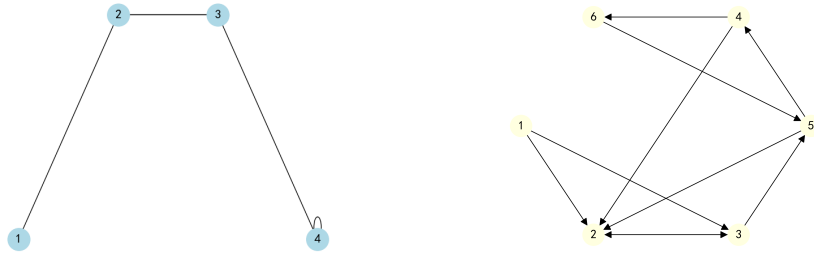


图 6.1 无向图（左）与有向图（右）

定义 6.6 如果图 G 的每条边 e 都附有一个实数 $w(e)$, 则称图 G 为赋权图（网络），实数 $w(e)$ 称为边 e 的权。赋权图一般记作 $G = (V, E, W)$ 。

(3) 顶点的度 定义 6.7

- 在无向图中，与顶点 v 关联的边的数目（环算两次）称为 v 的度，记为 $d(v)$ 。
- 在有向图中，从顶点 v 引出的弧的数目称为 v 的出度，记为 $d^+(v)$ ；从顶点 v 引入的弧的数目称为 v 的入度，记为 $d^-(v)$ ； $d(v) = d^+(v) + d^-(v)$ 称为 v 的度。

度为奇数的顶点称为奇顶点，度为偶数的顶点称为偶顶点。

定理 6.1（握手定理）给定图 $G = (V, E)$ ，所有顶点的度数之和是边数的 2 倍：

$$\sum_{v \in V} d(v) = 2|E| \quad (6.1)$$

推论 6.1 任何图中奇顶点的总数必为偶数。

(4) 子图与图的连通性 定义 6.8 设 $G_1 = (V_1, E_1)$ 与 $G_2 = (V_2, E_2)$ 是两个图，且满足 $V_1 \subseteq V_2$, $E_1 \subseteq E_2$ ，则称 G_1 是 G_2 的子图。如果 $V_1 = V_2$ ，则称 G_1 是 G_2 的生成子图（支撑子图）。

定义 6.9 设 $W = v_0 e_1 v_1 e_2 \cdots e_k v_k$ ，其中 $e_i \in E$, $v_j \in V$ ， e_i 与 v_{i-1} 和 v_i 关联，称 W 是图 G 的一条道路（walk）， k 为路长， v_0 为起点， v_k 为终点；各边相异的道路称为迹（trail）；各顶点相异的道路称为轨道（path），记为 $P(v_0, v_k)$ 。

6.1.2 图的矩阵表示

设图的顶点个数为 n ，边（或弧）的条数为 m 。

(1) 关联矩阵 对于无向图 G ，其关联矩阵 $M = (m_{ij})_{n \times m}$ ，其中

$$m_{ij} = \begin{cases} 1, & \text{顶点 } v_i \text{ 与边 } e_j \text{ 关联,} \\ 0, & \text{顶点 } v_i \text{ 与边 } e_j \text{ 不关联.} \end{cases} \quad (6.2)$$

对于有向图 D ，其关联矩阵 $M = (m_{ij})_{n \times m}$ ，其中

$$m_{ij} = \begin{cases} 1, & \text{顶点 } v_i \text{ 是弧 } a_j \text{ 的始端,} \\ -1, & \text{顶点 } v_i \text{ 是弧 } a_j \text{ 的末端,} \\ 0, & \text{顶点 } v_i \text{ 与弧 } a_j \text{ 不关联.} \end{cases} \quad (6.3)$$

(2) 邻接矩阵 对于无向非赋权图 G ，其邻接矩阵 $W = (w_{ij})_{n \times n}$ ，其中

$$w_{ij} = \begin{cases} 1, & \text{顶点 } v_i \text{ 与 } v_j \text{ 相邻,} \\ 0, & \text{顶点 } v_i \text{ 与 } v_j \text{ 不相邻.} \end{cases} \quad (6.4)$$

对于有向非赋权图 D ，其邻接矩阵 $W = (w_{ij})_{n \times n}$ ，其中

$$w_{ij} = \begin{cases} 1, & \text{弧 } (v_i, v_j) \in A, \\ 0, & \text{弧 } (v_i, v_j) \notin A. \end{cases} \quad (6.5)$$

对于赋权图，邻接矩阵元素为对应边的权值，无边时取 0 或 ∞ 。

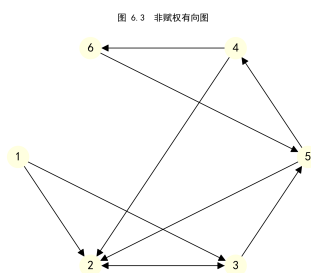
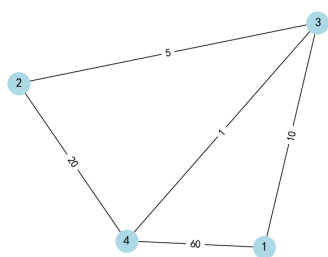


图 6.2 赋权无向图（左）与非赋权有向图（右）

6.2 NetworkX 简介

NetworkX 是 Python 的一个开源库，便于用户对复杂网络进行创建、操作和学习。利用 NetworkX 可以以标准化和非标准化的数据格式存储网络、生成多种随机网络和经典网络、分析网络结构、建立网络模型、设计新的网络算法、进行网络绘制等。

6.2.1 导入 NetworkX

```
1 import networkx as nx
2 import pylab as plt
```

6.2.2 图的生成

NetworkX 中有 4 种基本的图类型：

类型	说明
Graph	无向图
DiGraph	有向图
MultiGraph	多重无向图（允许重边和环）
MultiDiGraph	多重有向图

创建空图对象：

```

1 G = nx.Graph()           # 创建无向图
2 G = nx.DiGraph()        # 创建有向图
3 G = nx.MultiGraph()     # 创建多重无向图
4 G = nx.MultiDiGraph()   # 创建多重有向图

```

由邻接矩阵创建图：

```

1 G = nx.Graph(W)          # 由邻接矩阵 W 创建无向图 G
2 G = nx.DiGraph(W)       # 由邻接矩阵 W 创建有向图 G

```

6.2.3 顶点与边的添加

```

1 G.add_node(1)            # 添加单个顶点
2 G.add_nodes_from(['A', 'B']) # 从列表添加多个顶点
3 G.add_edge('A', 'B')    # 添加边
4 G.add_edge(1, 2, weight=0.5) # 添加带权边
5 G.add_weighted_edges_from([('A', 'B', 0.3), ('B', 'C', 0.9)]) # 添加多条带权边

```

6.2.4 图的绘制

`nx.draw()` 函数的主要参数：

参数	说明
G	要绘制的网络图
pos	顶点位置布局
with_labels	是否显示顶点标签
node_color	顶点颜色
node_shape	顶点形状
edge_color	边的颜色
font_weight	标签字体粗细

布局方式（pos 参数）：

- `circular_layout`：顶点在圆环上均匀分布
- `random_layout`：顶点在单位正方形内随机分布
- `shell_layout`：顶点在多个同心圆上分布

- spring_layout: Fruchterman-Reingold 力导向算法
- spectral_layout: 根据拉普拉斯特征向量排列

6.2.5 数据导出

```

1  W1 = nx.to_numpy_matrix(G)           # 导出邻接矩阵
2  W2 = nx.get_edge_attributes(G, 'weight') # 导出边的权值字典
3  print(G.adj)                         # 显示邻接表（字典）
4  print(list(G.adjacency()))           # 显示邻接表（列表）
5  nx.to_dict_of_lists(G)               # 导出字典格式的邻接表
6  np.savetxt('data.txt', W1, fmt='%d') # 保存邻接矩阵到文件

```

6.2.6 常用算法

NetworkX 提供丰富的图论算法：

函数	说明
dijkstra_path(G, source, target)	求两点间的最短路径
dijkstra_path_length(G, source, target)	求两点间的最短距离
shortest_path(G, source, target)	求最短路径
all_simple_paths(G, source, target)	求所有简单路径
has_path(G, source, target)	判断两点间是否存在路径
connected_components(G)	求连通分量
degree(G)	求各顶点的度
betweenness_centrality(G)	求介数中心性

具体例子

例 6.3：绘制 3 正则图

```

1  # 程序文件 ex6_3.py
2  import networkx as nx
3  import pylab as plt
4
5  G = nx.cubical_graph() # 生成一个 3 正则图（8个顶点，每个顶
   # 点度为 3）
6
7  # 子图 1：默认绘制
8  plt.subplot(121)
9  nx.draw(G, with_labels=True)
10
11 # 子图 2：自定义绘制
12 plt.subplot(122)
13 s = ['v'+str(i) for i in range(1, 9)]
14 s = dict(zip(range(8), s)) # 构造顶点标签字典
15 nx.draw(G, pos=nx.circular_layout(G), labels=s,

```



```

16     node_color='y', node_shape='s', edge_color='b')
17     plt.show()

```

例 6.5: 绘制有向图

```

1     # 程序文件 ex6_5.py
2     import networkx as nx
3     import pylab as plt
4
5     G = nx.DiGraph()
6     List = [(1,2), (1,3), (2,3), (3,2), (3,5), (4,2),
7             (4,6), (5,2), (5,4), (6,5)]
8     G.add_nodes_from(range(1, 7))
9     G.add_edges_from(List)
10
11     pos = nx.shell_layout(G)
12     nx.draw(G, pos, with_labels=True, font_weight='bold',
13             node_color='y')
14
15     W = nx.to_numpy_matrix(G)    # 导出邻接矩阵
16     print(W)
17     plt.show()

```

例 6.6: 赋权图操作

```

1     # 程序文件 ex6_6.py
2     import networkx as nx
3     import pylab as plt
4     import numpy as np
5
6     G = nx.Graph()
7     List = [(1, 3, 10), (1, 4, 60), (2, 3, 5),
8             (2, 4, 20), (3, 4, 1)]
9     G.add_nodes_from(range(1, 5))
10    G.add_weighted_edges_from(List)
11
12    W1 = nx.to_numpy_matrix(G)    # 导出权重邻接矩阵
13    W2 = nx.get_edge_attributes(G, 'weight') # 导出边的权重字典
14
15    pos = nx.spring_layout(G)
16    nx.draw(G, pos, with_labels=True, font_weight='bold')
17    nx.draw_networkx_edge_labels(G, pos, font_size=13, edge_labels=
18        W2)
19
20    print('邻接矩阵: \n', W1)
21    print('邻接表: \n', G.adj)
22    np.savetxt('data6_6.txt', W1, fmt='%d')
23    plt.show()

```

优缺点分析

方面	优点	缺点
图论模型	直观易懂, 适应性强, 便于可视化	大规模图计算复杂度高
NetworkX	功能丰富, API 简洁, 与 NumPy 兼容	大规模图性能不如专用图数据库
邻接矩阵	便于数学计算, 支持快速查询	存储 $O(n^2)$, 稀疏图浪费空间
邻接表	存储 $O(n + m)$, 适合稀疏图	查询两顶点是否相邻需要遍历

方法选择建议

- 简单图: 使用 `nx.Graph()` 或 `nx.DiGraph()`。
- 含重边或环的图: 使用 `nx.MultiGraph()` 或 `nx.MultiDiGraph()`。
- 需要权重: 在添加边时指定 `weight` 参数。
- 大规模稀疏图: 使用邻接表存储, 避免邻接矩阵的高内存占用。
- 可视化: 根据需求选择布局方式, 简单图用 `circular_layout`, 展示结构用 `spring_layout`。

应用场景

- 交通网络: 最短路径、最小生成树、流量优化。
- 社交网络: 影响力分析、社区发现、推荐系统。
- 通信网络: 可靠性分析、路由优化。
- 电力网络: 稳定性分析、故障传播。
- 生物网络: 蛋白质相互作用、基因调控网络。
- 项目管理: 关键路径法 (CPM)、计划评审技术 (PERT)。

6.3 最短路算法

最短路径问题是图论中非常经典的问题之一, 旨在寻找图中两顶点之间的最短路径。实际应用中的许多优化问题, 如管道铺设、线路安排、厂区布局、设备更新等, 都可被归结为最短路径问题来解决。

数学原理

(1) 基本定义 定义 6.11 设图 G 是赋权图, Γ 为 G 中的一条路, 则称 Γ 的各边权之和为路 Γ 的长度。

对于连通的赋权图 G 的两个顶点 u_0 和 v_0 ，从 u_0 到 v_0 的路中长度最小的称为从 u_0 到 v_0 的最短路；最短路的长称为从 u_0 到 v_0 的距离，记为 $d(u_0, v_0)$ 。

(2) **Dijkstra 算法**（单源最短路） Dijkstra 算法适用于边权非负的赋权图，其基本思想是：按距离固定起点从近到远的顺序，依次求得起点到各顶点的最短路径。

数据结构：

- $l(v)$ ：顶点 v 的标号，表示从起点到 v 的当前最短路径长度
- $z(v)$ ：顶点 v 的父顶点标号，用于确定最短路径
- S_i ：具有永久标号的顶点集

算法步骤：

1. 初始化： $l(u_0) = 0$ ，对 $v \neq u_0$ ， $l(v) = \infty$ ， $z(v) = u_0$ ， $S_0 = \{u_0\}$ ， $i = 0$
2. 对每个 $v \notin S_i$ ，更新标号：

$$l(v) = \min_{u \in S_i} \{l(v), l(u) + w(uv)\} \quad (6.6)$$

若通过 u 修改了 $l(v)$ ，则 $z(v) = u$

3. 选择 $v_{i+1} = \arg \min_{v \notin S_i} l(v)$ ，令 $S_{i+1} = S_i \cup \{v_{i+1}\}$
4. 若 $i = |V| - 1$ 或目标点进入 S_i ，终止；否则 $i = i + 1$ ，转第 2 步

时间复杂度： $O(n^2)$ ，使用堆优化可降至 $O(m \log n)$ 。

(3) **Floyd 算法**（所有顶点对最短路） Floyd 算法允许边权为负（但不能有负权圈），通过动态规划求解所有顶点对之间的最短路径。

核心思想：递推产生矩阵序列 $A_0, A_1, \dots, A_n$ ，其中 $A_k(i, j)$ 表示从 v_i 到 v_j 路径上经过的顶点序号不大于 k 的最短路径长度。

迭代公式：

$$A_k(i, j) = \min\{A_{k-1}(i, j), A_{k-1}(i, k) + A_{k-1}(k, j)\} \quad (6.7)$$

路径矩阵：引入 $R_k(i, j)$ 记录最短路径经过的顶点：

$$R_k(i, j) = \begin{cases} k, & A_{k-1}(i, j) > A_{k-1}(i, k) + A_{k-1}(k, j), \\ R_{k-1}(i, j), & \text{否则.} \end{cases} \quad (6.8)$$

算法步骤：

1. 初始化： $A_0(i, j) = w_{ij}$ ， $R_0(i, j) = 0$
2. 对 $k = 1, 2, \dots, n$ ：

$$A_k(i, j) = \min\{A_{k-1}(i, j), A_{k-1}(i, k) + A_{k-1}(k, j)\} \quad (6.9)$$

3. 若 $A_k(i, j)$ 被更新, 则 $R_k(i, j) = k$

时间复杂度: $O(n^3)$, 空间复杂度 $O(n^2)$ 。

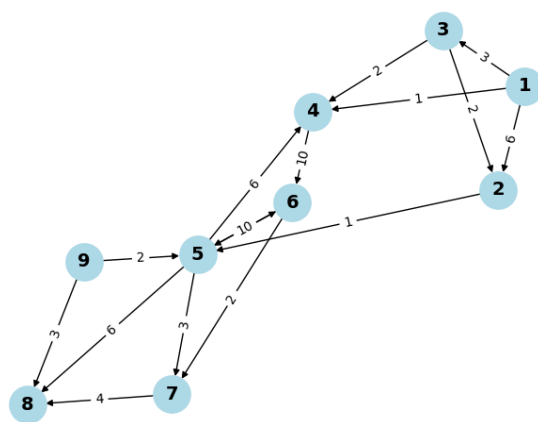


图 6.3 最短路算法示例图

具体例子

例 6.7 (旅行费用问题): 某人要从 v_1 出发到 v_8 , 交通路线如图 6.6 所示, 边上数字为费用, 求最小费用路线。

最短路径:

$$v_1 \rightarrow v_3 \rightarrow v_2 \rightarrow v_5 \rightarrow v_8, \quad \text{最小费用} = 12 \quad (6.10)$$

例 6.8 (所有顶点对最短路): 求图 6.2 所示赋权图所有顶点对之间的最短距离。

最短距离矩阵:

$$D = \begin{bmatrix} 0 & 15 & 10 & 11 \\ 15 & 0 & 5 & 6 \\ 10 & 5 & 0 & 1 \\ 11 & 6 & 1 & 0 \end{bmatrix} \quad (6.11)$$

例 6.9 (设备更新问题): 某设备使用 5 年, 已知各年价格和维修费用, 求最小费用更新计划。

最短路径:

$$v_1 \rightarrow v_3 \rightarrow v_6, \quad \text{最小费用} = 48 \quad (6.12)$$

即第 1 年初买新设备, 使用到第 2 年底; 第 3 年初再买新设备, 使用到第 5 年底。

例 6.10 (选址问题): 6 个销售点, 边权为距离, 求仓库建在哪个点使离仓库最远的销售点距离最近。

结果: 仓库应建在 v_3 , 最远距离为 33。

代码实现

例 6.7: Dijkstra 算法

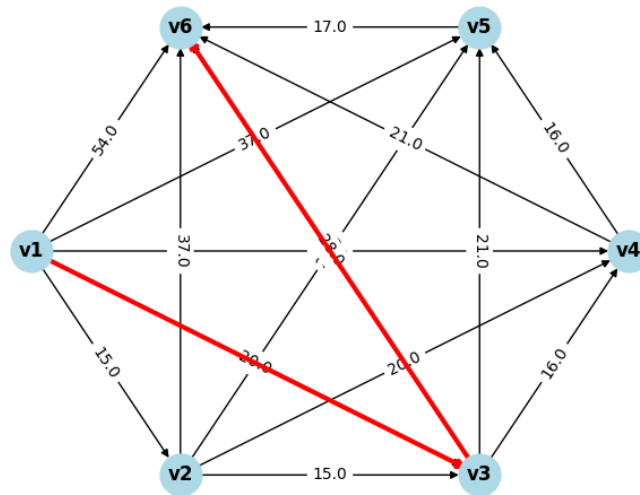


图 6.4 设备更新最小费用路径

```

1  # 程序文件 ex6_7.py
2  import networkx as nx
3
4  G = nx.DiGraph()
5  List = [(1,2,6), (1,3,3), (1,4,1), (2,5,1), (3,2,2), (3,4,2),
6  (4,6,10), (5,4,6), (5,6,4), (5,7,3), (5,8,6), (6,5,10),
7  (6,7,2), (7,8,4), (9,5,2), (9,8,3)]
8  G.add_nodes_from(range(1, 10))
9  G.add_weighted_edges_from(List)
10
11 path = nx.dijkstra_path(G, 1, 8, weight='weight')
12 d = nx.dijkstra_path_length(G, 1, 8, weight='weight')
13 print('最短路径:', path)
14 print('最小费用:', d)

```

例 6.8: Floyd 算法

```

1  # 程序文件 ex6_8.py
2  import networkx as nx
3  import numpy as np
4
5  G = nx.Graph()
6  List = [(1,3,10), (1,4,60), (2,3,5), (2,4,20), (3,4,1)]
7  G.add_nodes_from(range(1, 5))
8  G.add_weighted_edges_from(List)
9
10 d = nx.floyd_warshall_numpy(G)
11 print('最短距离矩阵:\n', d)
12
13 path = nx.shortest_path(G, weight='weight', method='bellman-

```

```

ford')
14 for i in range(1, len(d)):
15 for j in range(i+1, len(d)+1):
16 print(f'顶点{i}到顶点{j}的最短路径:', path[i][j])

```

例 6.9: 设备更新问题

```

1 # 程序文件 ex6_9.py
2 import numpy as np
3 import networkx as nx
4 import pylab as plt
5
6 # 构建邻接矩阵
7 a = np.zeros((6, 6))
8 a[0, 1:] = [15, 20, 27, 37, 54]
9 a[1, 2:] = [15, 20, 27, 37]
10 a[2, 3:] = [16, 21, 28]
11 a[3, 4:] = [16, 21]
12 a[4, 5] = 17
13
14 G = nx.DiGraph(a)
15
16 p = nx.shortest_path(G, 0, 5, weight='weight')
17 d = nx.shortest_path_length(G, 0, 5, weight='weight')
18 print('path=', p)
19 print('d=', d)
20
21 # 绘图
22 plt.rc('font', size=16)
23 pos = nx.shell_layout(G)
24 w = nx.get_edge_attributes(G, 'weight')
25 s = {i: f'v{i+1}' for i in range(6)}
26 nx.draw(G, pos, font_weight='bold', labels=s, node_color='r')
27 nx.draw_networkx_edge_labels(G, pos, edge_labels=w)
28 path_edges = list(zip(p, p[1:]))
29 nx.draw_networkx_edges(G, pos, edgelist=path_edges,
30 edge_color='r', width=3)
31 plt.savefig('fig6_9.png', dpi=500)
32 plt.show()

```

例 6.10: 选址问题

```

1 # 程序文件 ex6_10.py
2 import numpy as np
3 import networkx as nx
4
5 L = [(1,2,20), (1,5,15), (2,3,20), (2,4,60),
6      (2,5,25), (3,4,30), (3,5,18), (5,6,15)]

```

```

7   G = nx.Graph()
8   G.add_weighted_edges_from(L)
9
10  d = nx.floyd_warshall_numpy(G)
11  d = np.array(d)
12  md = np.max(d, axis=1)    # 逐行求最大值
13  mmd = min(md)             # 求最小最大值
14  ind = np.argmin(md) + 1   # 求对应顶点编号
15
16  print(d)
17  print("最小最大距离:", mmd)
18  print("仓库应建在:", ind)

```

优缺点分析

算法	优点	缺点
Dijkstra	单源最短路高效, $O(n^2)$ 可优化	边权不能为负
Floyd	所有顶点间最短路, 允许 负权	$O(n^3)$, 不适合大规模图
NetworkX 实现	API 简洁, 与 NumPy 无 缝集成	大规模图性能不如专用 库

方法选择建议

- 单源最短路、边权非负：使用 `nx.dijkstra_path()`。
- 所有顶点间最短路：使用 `nx.floyd_warshall_numpy()`。
- 含负权边的图：使用 `method='bellman-ford'`。
- 只需要距离、不需要路径：使用 `nx.shortest_path_length()`。
- 大规模图：考虑使用专用图数据库或 C++ 实现。

应用场景

- 交通导航：GPS 导航最短路径规划。
- 通信网络：路由器最短路径转发。
- 设备更新：设备最优更换时机决策。
- 设施选址：仓库、医院、消防站等最优位置选择。
- 供应链管理：最小成本运输路径规划。
- 项目管理：关键路径法（CPM）计算。

6.3.1 最短路径问题的 0-1 整数规划模型

数学原理

对于给定的赋权图 $G = (V, E, W)$, 其中 $V = \{v_1, \dots, v_n\}$ 为顶点集合, E 为边的集合, 邻接矩阵 $W = (w_{ij})_{n \times n}$, 这里

$$w_{ij} = \begin{cases} \text{边}(v_i, v_j) \text{ 的权值,} & \text{当 } v_i \text{ 与 } v_j \text{ 之间有边时,} \\ \infty, & \text{当 } v_i \text{ 与 } v_j \text{ 之间无边时,} \end{cases} \quad w_{ii} = 0. \quad (6.13)$$

现求从 v_s 到 v_t ($s, t \in \{1, \dots, n\}$) 的最短路径。引进 0-1 变量:

$$x_{ij} = \begin{cases} 1, & \text{边}(v_i, v_j) \text{ 位于从 } v_s \text{ 到 } v_t \text{ 的最短路径上,} \\ 0, & \text{否则.} \end{cases} \quad (6.14)$$

于是最短路径问题的数学模型为:

$$\min \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_{ij} \quad (6.1)$$

s.t.

$$\sum_{j=1}^n x_{ij} = \sum_{j=1}^n x_{ji}, \quad i \neq s, t \quad (\text{流量守恒})$$

$$\sum_{j=1}^n x_{sj} = 1 \quad (\text{起点出度为 1})$$

$$\sum_{j=1}^n x_{jt} = 1 \quad (\text{终点入度为 1})$$

$$\sum_{j=1}^n x_{js} = 0 \quad (\text{起点入度为 0})$$

$$\sum_{j=1}^n x_{tj} = 0 \quad (\text{终点出度为 0})$$

$$x_{ij} \in \{0, 1\}, \quad i, j = 1, 2, \dots, n. \quad (6.15)$$

约束条件说明:

- 流量守恒约束: 对于非起点和非终点的中间顶点, 进入的边数等于出去的边数;
- 起点约束: 起点只能发出一条边, 不能进入边;
- 终点约束: 终点只能进入一条边, 不能发出边。

这是一个 0-1 整数规划模型, 可以使用 cvxpy 求解。

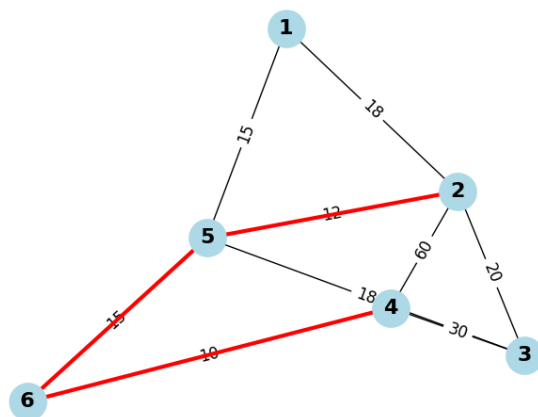


图 6.5 图 6.9 赋权无向图

具体例子

例 6.11 在图 6.9 所示赋权无向图中，求从 v_2 到 v_4 的最短路径和最短距离。

图 6.9 的边集为：

$$(1, 2, 18), (1, 5, 15), (2, 3, 20), (2, 4, 60), (2, 5, 12), (3, 4, 30), (3, 5, 18), (4, 6, 10), (5, 6, 15) \quad (6.16)$$

最短路径：

$$v_2 \rightarrow v_5 \rightarrow v_6 \rightarrow v_4, \quad \text{最短距离} = 12 + 15 + 10 = 37 \quad (6.17)$$

代码实现

```

1  # 程序文件 ex6_11.py
2  import numpy as np
3  import cvxpy as cp
4
5  # 边列表：(起点，终点，权值)
6  L = [(1, 2, 18), (1, 5, 15), (2, 3, 20), (2, 4, 60),
7       (2, 5, 12), (3, 4, 30), (3, 5, 18), (4, 6, 10), (5, 6, 15)]
8
9  # 构建邻接矩阵（无向图，对称）
10 n = 6
11 a = np.ones((n, n)) * 100000 # 用大数表示无边
12 for i in range(len(L)):
13     a[L[i][0]-1, L[i][1]-1] = L[i][2]
14     a[L[i][1]-1, L[i][0]-1] = L[i][2]
15     np.fill_diagonal(a, 0)
16
17 # 定义 0-1 决策变量
18 x = cp.Variable((n, n), integer=True)
19

```

```

20 # 目标函数：最小化路径总权值
21 obj = cp.Minimize(cp.sum(cp.multiply(a, x)))
22
23 # 约束条件
24 s = 1 # 起点 v2 (索引从0开始, 所以是1)
25 t = 3 # 终点 v4 (索引从0开始, 所以是3)
26
27 con = [
28     x >= 0, x <= 1, # 0-1 变量
29     sum(x[s, :]) == 1, # 起点出度为1
30     sum(x[:, s]) == 0, # 起点入度为0
31     sum(x[:, t]) == 1, # 终点入度为1
32     sum(x[t, :]) == 0 # 终点出度为0
33 ]
34
35 # 中间顶点流量守恒
36 for i in set(range(n)) - {s, t}:
37     con.append(sum(x[i, :]) == sum(x[:, i]))
38
39 prob = cp.Problem(obj, con)
40 prob.solve(solver='GLPK_MI')
41
42 print("最优值 (最短距离):", prob.value)
43 print("最优解 (邻接矩阵):\n", x.value)
44
45 # 输出最短路径边
46 i, j = np.nonzero(x.value)
47 print("最短路径边 (起点, 终点):")
48 for k in range(len(i)):
49     print(f"    v{i[k]+1} -> v{j[k]+1}")

```

输出结果:

```

1  最优值 (最短距离): 37.0
2  最短路径边 (起点, 终点):
3  v2 -> v5
4  v5 -> v6
5  v6 -> v4

```

优缺点分析

方法	优点	缺点
0-1 整数规划	建模直观, 约束条件对应图论性质	求解速度慢于专用图算法

续下页

方法	优点	缺点
Dijkstra 算法	单源最短路高效 $O(n^2)$	不能处理负权边
Floyd 算法	所有顶点对最短路, 允许负权	$O(n^3)$, 不适合大规模图
NetworkX	API 简洁, 易于使用	大规模图性能受限

方法选择建议

- 一般情况：优先使用 NetworkX 的 `shortest_path` 或 `dijkstra_path`。
- 需要与其他约束联合优化（如多条路径同时满足容量限制）：使用 0-1 整数规划模型。
- 教学演示：0-1 整数规划模型直观展示路径选择的图论本质。
- 大规模图：避免使用整数规划，改用专门的最短路算法。

应用场景

- 路径规划：GPS 导航中的最短路径选择。
- 网络路由：通信网络中数据包的最优转发路径。
- 物流运输：货物从仓库到客户的最小成本路径。
- 组合优化：路径选择与其他约束（时间窗、容量等）联合优化。

6.4 最小生成树

树 (tree) 是图论中非常重要的一类图，它非常类似于自然界中的树，结构简单，应用广泛，最小生成树问题是其中的经典问题之一。在实际应用中，许多问题的图论模型是最小生成树，如通信网络建设、有线电视铺设、加工设备分组等。

6.4.1 基本概念和算法

数学原理

(1) 树的基本概念 定义 6.12 连通的无圈图称为树。

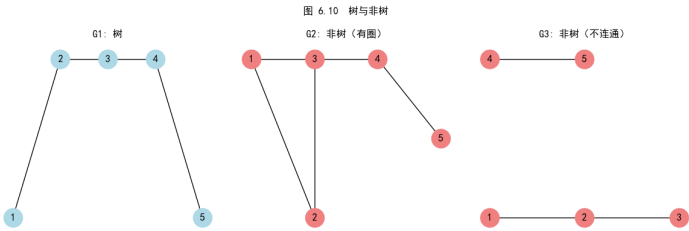


图 6.6 树与非树

定理 6.2 设 G 是具有 n 个顶点 m 条边的图，则以下命题等价：

1. 图 G 是树；
2. 图 G 中任意两个不同顶点之间存在唯一的路径；
3. 图 G 连通，删除任意一条边均不连通；
4. 图 G 连通，且 $n = m + 1$ ；
5. 图 G 无圈，添加任意一条边可得唯一的圈；
6. 图 G 无圈，且 $n = m + 1$ 。

定义 6.13 若图 G 的生成子图 H 是树，则称 H 为 G 的生成树或支撑树。

定理 6.3 连通图的生成树一定存在。

定理 6.4 (破圈法) 给定连通图 G ，若 G 有圈，则删除圈上一条边，重复操作直到无圈，得到的图即为生成树。

定义 6.14 在赋权图 G 中，边权之和最小的生成树称为 G 的最小生成树 (Minimum Spanning Tree, MST)。

n 个顶点的完全图，其不同生成树的个数为 n^{n-2} ，因此不能用枚举法求解。

(2) **Kruskal** 算法 算法思想：每次将一条权最小的边加入子图 T 中，并保证不形成圈。

算法步骤：

1. 选 $e_1 \in E$ ，使得 e_1 是权值最小的边；
2. 若 $e_1, e_2, \dots, e_i$ 已选好，则从剩余边中选取 e_{i+1} ，使得 $e_1, \dots, e_{i+1}$ 中无圈，且 e_{i+1} 是剩余边中权值最小的；
3. 重复直到选得 e_{n-1} 为止。

时间复杂度： $O(m \log m)$ ，适用于稀疏图。

(3) **Prim** 算法 算法思想：从任意顶点出发，每次将连接已选集合与未选集合的最小权边加入树中。

算法步骤：

1. 初始化 $P = \{v_1\}$, $Q = \emptyset$ ；
2. 在 $p \in P$, $v \in V - P$ 的边中选最小权边 pv ；
3. $P = P \cup \{v\}$, $Q = Q \cup \{pv\}$ ；
4. 重复直到 $P = V$ 。

时间复杂度： $O(n^2)$ ，适用于稠密图。

(4) 最小生成树的 0-1 整数规划模型 引入 0-1 变量：

$$x_{ij} = \begin{cases} 1, & \text{边}(v_i, v_j) \text{ 在最小生成树中,} \\ 0, & \text{否则.} \end{cases} \quad (6.18)$$

数学模型：

$$\min z = \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_{ij} \quad (6.3)$$

s.t.

$$\sum_{j=1}^n x_{1j} \geq 1 \quad (\text{根至少有一条边})$$

$$\sum_{i=1}^n x_{ij} = 1, \quad j = 2, 3, \dots, n \quad (\text{每个非根顶点恰有一条边进入})$$

$$u_1 = 0, \quad 1 \leq u_i \leq n-1, \quad i = 2, 3, \dots, n \quad (6.19)$$

$$u_i - u_j + nx_{ij} \leq n-1, \quad i = 1, \dots, n, \quad j = 2, \dots, n \quad (\text{防止子圈})$$

$$x_{ij} \in \{0, 1\}, \quad i, j = 1, 2, \dots, n. \quad (6.20)$$

其中第 4 类约束防止子圈形成。

具体例子

例 6.12 (通信线铺设)：9 个自然村之间道路如图 6.11 所示，边权为距离，求架设通信线的最短总长度。

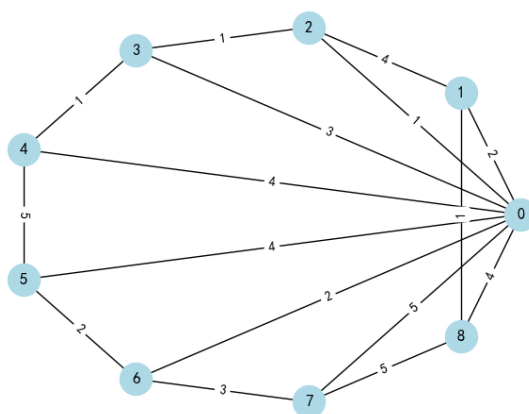


图 6.7 道路示意图

Kruskal 算法求解：

按边权从小到大排序，依次选择不形成圈的边：

$$(v_0, v_2) = 1, (v_2, v_3) = 1, (v_3, v_4) = 1, (v_1, v_8) = 1, (v_0, v_1) = 2, (v_0, v_6) = 2, (v_5, v_6) = 2, (v_6, v_7) = 3 \quad (6.21)$$

图 6.12 生成的最小生成树

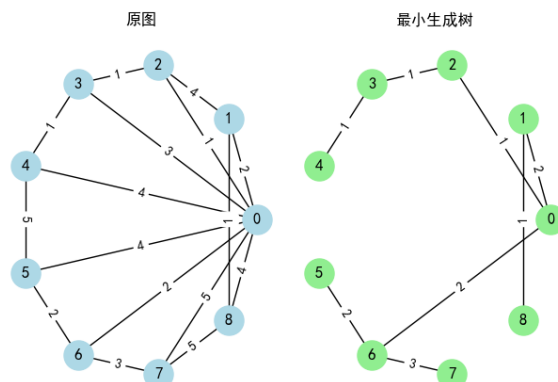


图 6.8 最小生成树结果

排除 (v_0, v_3) (会形成圈), 选择 (v_6, v_7) 。

最小生成树总权值: $W = 13$ 。

选中的边为:

$$(v_0, v_2), (v_2, v_3), (v_3, v_4), (v_1, v_8), (v_0, v_1), (v_0, v_6), (v_5, v_6), (v_6, v_7) \quad (6.22)$$

代码实现

例 6.12: NetworkX 求解

```

1  # 程序文件 ex6_12.py
2  import numpy as np
3  import networkx as nx
4  import pylab as plt
5
6  L = [(0,1,2),(0,2,1),(0,3,3),(0,4,4),(0,5,4),(0,6,2),(0,7,5),
7      (0,8,4),
8      (1,2,4),(1,8,1),(2,3,1),(3,4,1),(4,5,5),(5,6,2),(6,7,3),(7,8,5)
9      ]
10
11  G = nx.Graph()
12  G.add_weighted_edges_from(L)
13
14  # 求最小生成树
15  T = nx.minimum_spanning_tree(G)
16
17  # 导出邻接矩阵
18  c = nx.to_numpy_matrix(T)
19  print("邻接矩阵 c = \n", c)
20
21  # 求总权值
22  w = c.sum() / 2
23  print("最小生成树总权值 W = ", w)

```

```

22
23     # 绘图
24     pos = nx.circular_layout(G)
25     plt.subplot(121)
26     nx.draw(G, pos, with_labels=True, font_size=13)
27     w1 = nx.get_edge_attributes(G, 'weight')
28     nx.draw_networkx_edge_labels(G, pos, edge_labels=w1)
29
30     plt.subplot(122)
31     nx.draw(T, pos, with_labels=True, font_weight='bold')
32     w2 = nx.get_edge_attributes(T, 'weight')
33     nx.draw_networkx_edge_labels(T, pos, edge_labels=w2)
34     plt.show()

```

例 6.13: 0-1 整数规划求解

```

1     # 程序文件 ex6_13.py
2     import cvxpy as cp
3     import numpy as np
4
5     L = [(0,1,2),(0,2,1),(0,3,3),(0,4,4),(0,5,4),(0,6,2),(0,7,5),
6           (0,8,4),
7           (1,2,4),(1,8,1),(2,3,1),(3,4,1),(4,5,5),(5,6,2),(6,7,3),(7,8,5)
8           ]
9
10    n = 9
11    a = np.ones((n, n)) * 10000    # 大数表示无边
12    for i in range(len(L)):
13        a[L[i][0], L[i][1]] = L[i][2]
14        a[L[i][1], L[i][0]] = L[i][2]
15    np.fill_diagonal(a, 0)
16
17    x = cp.Variable((n, n), integer=True)
18    u = cp.Variable(n, pos=True)
19
20    # 目标函数
21    obj = cp.Minimize(cp.sum(cp.multiply(a, x)))
22
23    # 约束条件
24    con = [
25        cp.sum(x[0, :]) >= 1,    # 根至少有一条边
26        u[0] == 0,               # 根 u 值为 0
27        u[1:] >= 1,              # 其他顶点 u >= 1
28        u[1:] <= n-1,            # 其他顶点 u <= n-1
29        x >= 0, x <= 1           # 0-1 变量
30    ]

```

```

30     # 每个非根顶点恰有一条边进入
31     for i in range(1, n):
32         con.append(cp.sum(x[:, i]) == 1)
33
34     # 防止子圈约束
35     for i in range(n):
36         for j in range(1, n):
37             con.append(u[i] - u[j] + n * x[i, j] <= n - 1)
38
39     prob = cp.Problem(obj, con)
40     prob.solve(solver='GLPK_MI')
41
42     print("最优值 (总权值) :", prob.value)
43     print("最优解 x:\n", x.value)
44
45     i, j = np.nonzero(x.value)
46     print("选中的边 (起点, 终点):")
47     for k in range(len(i)):
48         print(f"    v{i[k]} -> v{j[k]}")

```

输出结果:

```

1     最优值 (总权值) : 13.0
2     选中的边:
3     v0 -> v1
4     v0 -> v2
5     v0 -> v6
6     v1 -> v8
7     v2 -> v3
8     v3 -> v4
9     v5 -> v6
10    v6 -> v7

```

优缺点分析

算法	优点	缺点
Kruskal	贪心算法, $O(m \log m)$, 适合稀疏图	需要并查集维护连通性
Prim	$O(n^2)$, 适合稠密图	需要维护 P 和 $V - P$ 的边
0-1 整数规划	可与其他约束联合优化	求解速度慢, 不适合大规模图
NetworkX 实现	API 简洁, 一行代码求解	大规模图性能受限

方法选择建议

- 稀疏图：使用 Kruskal 算法（默认）。
- 稠密图：使用 Prim 算法，设置 `algorithm='prim'`。
- 需要与其他约束联合优化：使用 0-1 整数规划模型。
- 一般情况：直接使用 `nx.minimum_spanning_tree(G)`。

应用场景

- 通信网络：铺设光纤、电话线的最小成本方案。
- 电力网络：电网建设的最小成本布线。
- 交通运输：公路、铁路建设的最小成本连接。
- 供水/供气网络：管道铺设的最小成本设计。
- 聚类分析：单链接层次聚类。
- 近似算法：TSP 问题的近似求解。

6.5 着色问题

已知图 $G = (V, E)$ ，对图 G 的所有顶点进行着色时，要求相邻的两顶点的颜色不一样，问至少需要几种颜色？这就是顶点着色问题。若对图 G 的所有边进行着色时，要求相邻的两条边的颜色不一样，问至少需要几种颜色？这就是边着色问题。

着色模型中的图是无向图。对于顶点着色问题，若是有限图，也可转化为有限简单图。而边着色问题可以转化为顶点着色问题。

数学原理

(1) 基本定义 色数 $\chi(G)$ ：对图 $G = (V, E)$ 的顶点进行着色所需最少的颜色数目，称为图 G 的色数。

定理 6.4 若图 $G = (V, E)$ ， $\Delta = \max\{d(v) \mid v \in V\}$ 为图 G 顶点的最大度数，则

$$\chi(G) \leq \Delta + 1 \quad (6.23)$$

(2) 着色问题的 0-1 整数规划模型 设图 G 有 n 个顶点，最大度为 Δ 。引入 0-1 变量：

$$x_{ik} = \begin{cases} 1, & \text{顶点 } v_i \text{ 着第 } k \text{ 种颜色,} \\ 0, & \text{否则.} \end{cases} \quad (6.24)$$

设颜色总数为 y ，建立如下整数规划模型：

$$\min y \quad (6.25)$$

s.t.

$$\sum_{k=1}^{\Delta+1} x_{ik} = 1, \quad i = 1, 2, \dots, n \quad (\text{每个顶点恰着一种颜色})$$

$$x_{ik} + x_{jk} \leq 1, \quad (v_i, v_j) \in E, \quad k = 1, 2, \dots, \Delta + 1 \quad (\text{相邻顶点颜色不同})$$

$$y \geq \sum_{k=1}^{\Delta+1} k \cdot x_{ik}, \quad i = 1, 2, \dots, n \quad (\text{颜色总数约束})$$

$$x_{ik} \in \{0, 1\}, \quad i = 1, 2, \dots, n, \quad k = 1, 2, \dots, \Delta + 1 \quad (6.26)$$

具体例子

例 6.14 (物资储存问题): n 种化学制品, 不相容的不能放在同一储存室, 求最少储存室数。

构造图 G : 顶点表示化学制品, 两顶点相邻当且仅当对应的化学制品不相容。最少储存室数即为图的色数 $\chi(G)$ 。

例 6.15 (波长分配问题): 5 台设备, 靠得太近的设备不能分配相同波长, 求最少波长数。

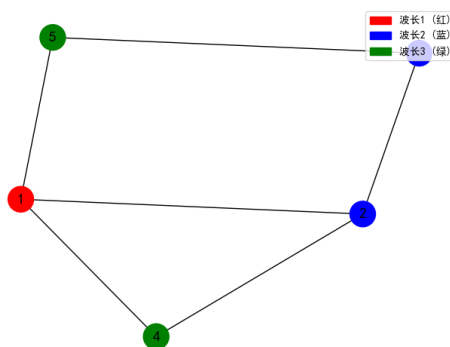


图 6.9 波长分配着色图

最少波长数: 3 个

例 6.16 (会议安排问题): 6 个部门, 有共同成员的部门会议不能安排在同一时段, 求最少时段数。

图 6.14 的着色结果:

- 部门 1 和部门 4: 同一时段
- 部门 2 和部门 5: 同一时段
- 部门 3 和部门 6: 不同时段

最少时段数: 4 个

代码实现

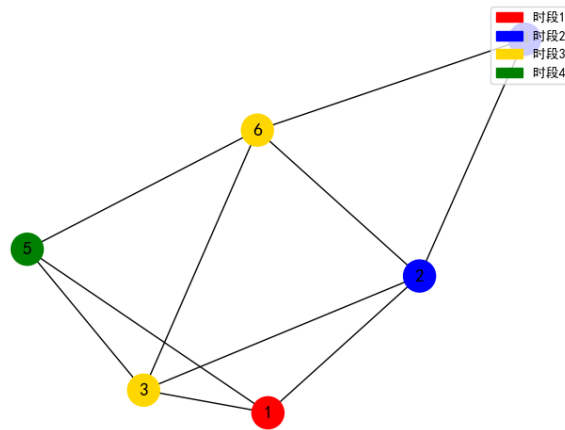


图 6.10 会议安排着色图

```

1  # 程序文件 ex6_16.py
2  import cvxpy as cp
3  import numpy as np
4
5  # 各部门成员列表
6  L = [['张', '李', '王'], ['李', '赵', '刘'], ['张', '刘', '王'],
7       ['赵', '刘', '孙'], ['张', '王', '孙'], ['李', '刘', '王']]
8
9  n = len(L)
10
11 # 构造邻接矩阵 (部门间有共同成员则相邻)
12 w = np.zeros((n, n))
13 for i in range(n-1):
14     for j in range(i+1, n):
15         if len(set(L[i]) & set(L[j])) >= 1: # 有共同成员
16             w[i, j] = 1
17             w[j, i] = 1
18
19 # 计算各顶点度
20 deg = w.sum(axis=1)
21 K = int(max(deg)) # 最大度 Δ
22
23 # 定义决策变量
24 x = cp.Variable((n, K+1), integer=True)
25 y = cp.Variable() # 颜色总数
26
27 # 目标函数
28 obj = cp.Minimize(y)
29
30 # 约束条件
31 con = [

```

```

32     cp.sum(x, axis=1) == 1,    # 每个顶点恰着一种颜色
33     x >= 0, x <= 1           # 0-1 变量
34 ]
35
36 # 颜色总数约束: y 不小于任何顶点使用的颜色编号
37 for i in range(n):
38     con.append(y >= cp.sum([(k+1) * x[i, k] for k in range(K+1)]))
39
40 # 相邻顶点颜色不同
41 for k in range(K+1):
42     for i in range(n):
43         for j in range(i+1, n):
44             if w[i, j] == 1:
45                 con.append(x[i, k] + x[j, k] <= 1)
46
47 prob = cp.Problem(obj, con)
48 prob.solve(solver='GLPK_MI')
49
50 print("最优值 (最少颜色数) :", int(prob.value))
51
52 # 输出顶点和颜色的对应关系
53 i_idx, k_idx = np.nonzero(x.value)
54 print("顶点和颜色的对应关系: ")
55 for idx in range(len(i_idx)):
56     print(f"    顶点 v{i_idx[idx]+1} -> 颜色 {k_idx[idx]+1}")

```

输出结果:

```

1     最优值 (最少颜色数) : 4
2     顶点和颜色的对应关系:
3     顶点 v1 -> 颜色 1
4     顶点 v2 -> 颜色 2
5     顶点 v3 -> 颜色 3
6     顶点 v4 -> 颜色 4
7     顶点 v5 -> 颜色 2
8     顶点 v6 -> 颜色 3

```

优缺点分析

方法	优点	缺点
贪心着色	速度快, 实现简单	不保证最优 (可能用更多颜色)
0-1 整数规划	保证最优解, 可加入额外约束	NP-完全问题, 大规模求解慢

续下页

方法	优点	缺点
回溯法	可求最优解	指数复杂度, 仅适合小规模

方法选择建议

- 小规模图 ($n \leq 20$): 使用 0-1 整数规划保证最优解。
- 大规模图: 使用贪心算法或启发式算法求近似解。
- 需要精确最优解: 使用整数规划, 颜色数上限为 $\Delta + 1$ 。
- 教学演示: 使用 NetworkX 的 `nx.greedy_color()` 快速演示。

应用场景

- 资源分配: 仓库/储存室分配 (不相容物品分开放置)。
- 频率/波长分配: 无线通信中避免信号干扰。
- 时间表安排: 课程/会议排课 (有冲突的不能同时安排)。
- 寄存器分配: 编译优化中变量分配。
- 地图着色: 四色定理的实际应用。

6.6 最大流与最小费用流问题

6.6.1 最大流问题

许多系统包含了流量问题, 如公路系统中有车辆流、物资调配系统中有物资流、金融系统中有现金流等。这些流问题都可归结为网络流问题, 且都存在一个如何安排使流量最大的问题, 即最大流问题。

数学原理

(1) 基本概念 定义 6.15 给定一个有向图 $D = (V, A)$, 在 V 中指定了一点称为发点或源 (记为 v_s), 该点只有发出的弧; 同时指定一个点称为收点或汇 (记为 v_t), 该点只有进入的弧; 其余的点称为中间点。对于每一条弧 $(v_i, v_j) \in A$, 对应有一个容量 $c(v_i, v_j) \geq 0$ 。这样的有向图称为一个网络, 记作 $D = (V, A, C)$ 。

网络上的流, 是指定义在弧集合 A 上的一个函数 $f = \{f_{ij}\}$, 称 f_{ij} 为弧 (v_i, v_j) 上的流量。

定义 6.16 满足下列条件的流 f 称为可行流:

1. 容量限制条件: $0 \leq f_{ij} \leq c_{ij}, \forall (v_i, v_j) \in A$
2. 平衡条件:

- 中间点：流出量 = 流入量
- 发点 v_s ： $\sum_{(v_s, v_j) \in A} f_{sj} = v$ (净输出量)
- 收点 v_t ： $\sum_{(v_k, v_t) \in A} f_{kt} = v$ (净输入量)

最大流问题的线性规划模型：

$$\max v \quad (6.27)$$

s.t.

$$\begin{cases} \sum_{(v_s, v_j) \in A} f_{sj} = v, \\ \sum_{(v_i, v_j) \in A} f_{ij} - \sum_{(v_k, v_i) \in A} f_{ki} = 0, & i \neq s, t, \\ \sum_{(v_k, v_t) \in A} f_{kt} = v, \\ 0 \leq f_{ij} \leq c_{ij}, \end{cases} \quad \forall (v_i, v_j) \in A. \quad (6.28)$$

(2) 基本术语

- 饱和弧： $f_{ij} = c_{ij}$
- 非饱和弧： $f_{ij} < c_{ij}$
- 零流弧： $f_{ij} = 0$
- 非零流弧： $f_{ij} > 0$
- 前向弧 (μ^+)：弧的方向与路的方向一致
- 后向弧 (μ^-)：弧的方向与路的方向相反

定义 6.17 (增广路) 设 f 是一个可行流， μ 是从 v_s 到 v_t 的一条路，若 μ 满足：前向弧是非饱和弧，后向弧是非零流弧，则称 μ 为关于可行流 f 的一条增广路。

(3) Ford-Fulkerson 标号法 标号过程：

1. 给发点 v_s 标号为 $(0, \infty)$;
2. 若 v_i 已标号，对其未标号邻点 v_j ：
 - 若 (v_i, v_j) 为前向弧且 $f_{ij} < c_{ij}$ ，给 v_j 标号 (v_i, δ_j) ， $\delta_j = \min\{c_{ij} - f_{ij}, \delta_i\}$;
 - 若 (v_j, v_i) 为后向弧且 $f_{ji} > 0$ ，给 v_j 标号 $(-v_i, \delta_j)$ ， $\delta_j = \min\{f_{ji}, \delta_i\}$;
3. 重复直到收点 v_t 被标号或无法继续标号。

增流过程：

1. 从 v_t 开始，沿标号反向追踪，获得增广路；
2. 对所有前向弧： $f_{ij} + = \delta_t$ ；对所有后向弧： $f_{ij} - = \delta_t$ ；
3. 清除标号，重复标号过程。

具体例子

例 6.17 求图 6.15 中从顶点 1 到顶点 8 的最大流。

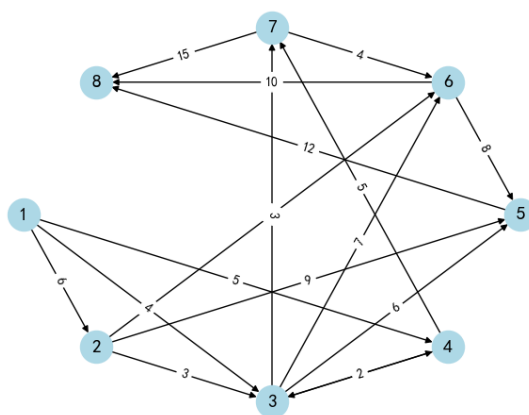


图 6.11 图 6.15 赋权有向图

最大流量：15

例 6.18（多源多汇招聘问题）：4 个公司招聘 5 个专业的毕业生，各专业可供人数已知，各公司招聘需求已知，问是否都能招到。

添加虚拟源点 s 和虚拟汇点 t ，转化为单源单汇最大流问题。

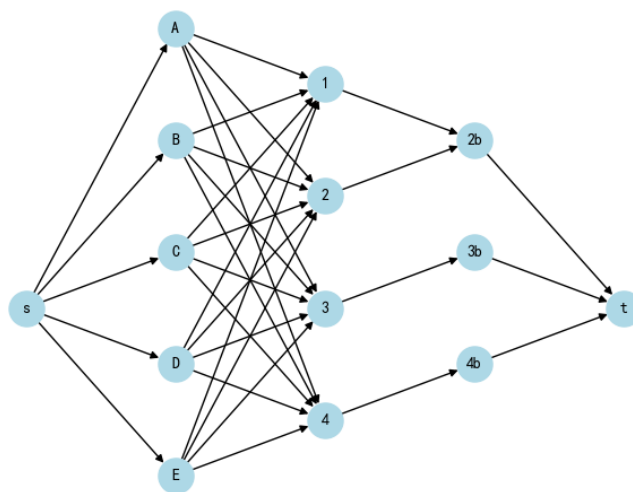


图 6.12 图 6.16 最大流网络

最大流量：16（各公司都能招到所需人才）

6.6.2 最小费用流问题

在实际问题中，往往还要考虑网络上流的费用问题。例如在运输问题中，人们总是希望在完成运输任务的同时，寻求一个使总的运输费用最小的运输方案。

数学原理

设 f_{ij} 为弧 (v_i, v_j) 上的流量, b_{ij} 为弧上的单位费用, c_{ij} 为弧上的容量, 则最小费用流问题的线性规划模型为:

$$\min \sum_{(v_i, v_j) \in A} b_{ij} f_{ij} \quad (6.29)$$

s.t.

$$\begin{cases} \sum_{(v_s, v_j) \in A} f_{sj} = v, \\ \sum_{(v_i, v_j) \in A} f_{ij} - \sum_{(v_k, v_i) \in A} f_{ki} = 0, \quad i \neq s, t, \\ \sum_{(v_k, v_t) \in A} f_{kt} = v, \\ 0 \leq f_{ij} \leq c_{ij}, \end{cases} \quad \forall (v_i, v_j) \in A. \quad (6.30)$$

当 v 取最大流 $v_{\max}$ 时, 本问题为最小费用最大流问题。

Busacker-Gowan 迭代算法:

1. 求从发点到收点的最小费用通路 μ ;
2. 对该通路分配最大可能流量 $\bar{f} = \min_{(v_i, v_j) \in \mu} c_{ij}$, 更新容量;
3. 作通路上所有边的反向边, 容量为 $\bar{f}$, 单位费用为相反数;
4. 重复直到总流量达到 v 。

具体例子

例 6.19 如图 6.17 所示, 弧上标注为 (容量, 单位运费), 求从 v_1 到 v_5 的最小费用最大流。

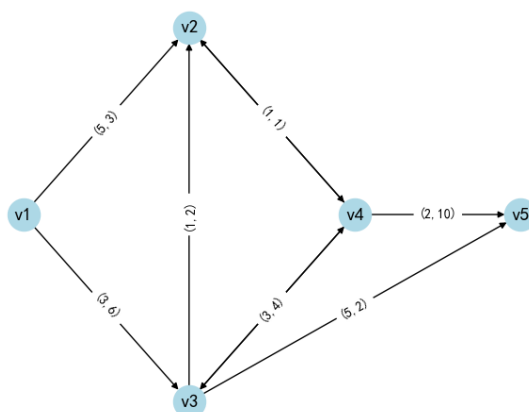


图 6.13 图 6.17 运费网络

最大流量: 5

最小费用: 63

代码实现

例 6.17: 最大流

```

1  # 程序文件 ex6_17.py
2  import networkx as nx
3  import pylab as plt
4
5  L = [(1,2,6),(1,3,4),(1,4,5),(2,3,3),(2,5,9),(2,6,9),
6       (3,4,5),(3,5,6),(3,6,7),(3,7,3),(4,3,2),(4,7,5),
7       (5,8,12),(6,5,8),(6,8,10),(7,6,4),(7,8,15)]
8
9  G = nx.DiGraph()
10 G.add_nodes_from(range(1, 9))
11 G.add_weighted_edges_from(L, weight='capacity')
12
13 value, flow_dict = nx.maximum_flow(G, 1, 8)
14 print("最大流流量:", value)
15 print("最大流:", flow_dict)

```

例 6.18: 多源多汇问题

```

1  # 程序文件 ex6_18_2.py
2  import numpy as np
3  import cvxpy as cp
4
5  # 各专业可供人数: A,B,C,D,E
6  a = np.array([4, 3, 3, 2, 4])
7
8  x = cp.Variable((4, 5), integer=True)
9
10 # 目标: 总招聘人数最大化
11 obj = cp.Maximize(cp.sum(x))
12
13 # 约束条件
14 con = [
15     cp.sum(x, axis=0) <= a,           # 各专业人数限制
16     x[1, 0] + x[1, 1] + x[1, 4] <= 2, # 公司2限制
17     x[2, 0] + x[2, 3] <= 1,           # 公司3限制
18     x[3, 4] <= 2,                     # 公司4限制
19     x >= 0, x <= 1                    # 0-1 变量
20 ]
21
22 prob = cp.Problem(obj, con)
23 prob.solve(solver='GLPK_MI')
24
25 print("最优值 (总招聘人数):", prob.value)
26 print("最优解:", x.value)

```

例 6.19: 最小费用最大流

```

1  # 程序文件 ex6_19.py
2  import numpy as np
3  import networkx as nx
4
5  L = [('v1', 'v2', 5, 3), ('v1', 'v3', 3, 6), ('v2', 'v4', 2, 8),
6      ('v3', 'v2', 1, 2), ('v3', 'v4', 4, 2), ('v4', 'v2', 1, 1),
7      ('v4', 'v3', 3, 4), ('v4', 'v5', 2, 10), ('v3', 'v5', 5, 2)]
8
9  G = nx.DiGraph()
10 for k in range(len(L)):
11     G.add_edge(L[k][0], L[k][1], capacity=L[k][2], weight=L[k][3])
12
13 # 求最小费用最大流
14 maxFlow = nx.max_flow_min_cost(G, 'v1', 'v5')
15 mincost = nx.cost_of_flow(G, maxFlow)
16
17 print("最大流:", sum(maxFlow['v1'].values()))
18 print("最小费用:", mincost)

```

输出结果:

```

1  最大流: 5
2  最小费用: 63

```

优缺点分析

算法	优点	缺点
Ford-Fulkerson	思想直观, 易于理解	可能遇到死循环(若容量为无理数)
Dinic	$O(n^2m)$, 效率高	实现较复杂
NetworkX	API 简洁, 易于使用	大规模图性能受限
最小费用流(线性规划)	可加额外约束	求解速度慢

方法选择建议

- 单源单汇最大流: 使用 `nx.maximum_flow(G, s, t)`。
- 多源多汇问题: 添加虚拟源点和汇点, 转化为单源单汇。
- 最小费用最大流: 使用 `nx.max_flow_min_cost(G, s, t)`。
- 需要额外约束: 使用整数规划模型, 灵活性更高。

应用场景

- 交通流量：城市路网最大通行能力。
- 物流配送：最小成本运输方案。
- 通信网络：最大数据传输带宽。
- 电力网络：最大输电容量。
- 水资源调度：最大供水量。
- 人员招聘：多岗位人才配置。

6.7 关键路径

计划评审方法 (Program Evaluation and Review Technique, PERT) 和关键路线法 (Critical Path Method, CPM) 是网络分析的重要组成部分, 广泛用于系统分析和项目管理。由于 PERT 与 CPM 有着相同的目标应用和很多相同的术语, 这两种方法已合并为一种方法, 在国外称为 PERT/CPM, 在国内称为统筹方法。

6.7.1 计划网络图

数学原理

(1) 基本概念 定义 6.18 将任何消耗时间或资源的行动称为作业。称作业的开始或结束为事件, 事件本身不消耗资源。



图 6.14 计划网络图的基本画法

- 实箭线：表示作业（消耗时间、资源）
- 虚箭线：表示虚作业（工时为零，不消耗资源，仅表示逻辑关系）

定义 6.19 在计划网络图中, 从初始事件到最终事件的由各项作业连贯组成的一条路称为路线。具有累计作业时间最长的路线称为关键路径。

(2) 时间参数 事件时间参数:

- 事件最早时间 $t_E(j)$: 以事件 j 为始点的各作业最早可能开始的时间

$$t_E(1) = 0, \quad t_E(j) = \max_i \{t_E(i) + t(i, j)\} \quad (6.7)$$

- 事件最迟时间 $t_L(i)$: 在不影响任务总工期条件下, 以事件 i 为始点的作业最迟必须开始的时间

$$t_L(n) = t_E(n), \quad t_L(i) = \min_j \{t_L(j) - t(i, j)\} \quad (6.8)$$

作业时间参数:

- 最早开工时间 $t_{ES}(i, j)$ 、最早完工时间 $t_{EF}(i, j)$

$$t_{ES}(i, j) = x_i, \quad t_{EF}(i, j) = x_i + t_{ij} \quad (6.31)$$

- 最迟开工时间 $t_{LS}(i, j)$ 、最迟完工时间 $t_{LF}(i, j)$

$$t_{LF}(i, j) = z_j, \quad t_{LS}(i, j) = z_j - t_{ij} \quad (6.32)$$

- 总时差 $R(i, j)$: 在不影响总工期的条件下, 作业可以延迟开工的最大幅度

$$R(i, j) = t_{LF}(i, j) - t_{EF}(i, j) = t_{LS}(i, j) - t_{ES}(i, j) \quad (6.11)$$

- 单时差 $r(i, j)$: 在不影响紧后作业最早开工时间的条件下, 作业可以延迟开工的最大幅度

$$r(i, j) = t_{ES}(j, k) - t_{EF}(i, j) \quad (6.12)$$

关键路径判定: 总时差 $R(i, j) = 0$ 的作业位于关键路径上。

(3) 计划网络图的数学规划模型 设 x_i 为事件 i 的开始时间, t_{ij} 为作业 (i, j) 的计划时间:

$$\min \sum_{i \in V} x_i \quad (6.13)$$

s.t.

$$x_j \geq x_i + t_{ij}, \quad (i, j) \in A, \quad x_i \geq 0 \quad (6.33)$$

(4) 关键路径的 0-1 整数规划模型 设 $x_{ij} = 1$ 表示作业 (i, j) 位于关键路径上:

$$\max \sum_{(i, j) \in A} t_{ij} x_{ij} \quad (6.34)$$

s.t.

$$\begin{cases} \sum_{(1, j) \in A} x_{1j} = 1, \\ \sum_{(i, j) \in A} x_{ij} - \sum_{(k, i) \in A} x_{ki} = 0, \quad i \neq 1, n, \\ \sum_{(k, n) \in A} x_{kn} = 1, \\ x_{ij} \in \{0, 1\}. \end{cases} \quad (6.35)$$

(5) 计划网络优化模型 当需要压缩工期时, 设 y_{ij} 为作业 (i, j) 缩短的时间, c_{ij} 为缩短一天增加的费用, m_{ij} 为最短完成时间:

$$\min \sum_{(i,j) \in A} c_{ij} y_{ij} \quad (6.36)$$

s.t.

$$\begin{cases} x_j - x_i + y_{ij} \geq t_{ij}, & (i, j) \in A, \\ 0 \leq y_{ij} \leq t_{ij} - m_{ij}, & (i, j) \in A, \\ x_n - x_1 \leq d. \end{cases} \quad (6.37)$$

(6) PERT 概率分析 当作业时间不确定时, 给出三种估计: 最乐观 a 、最可能 m 、最悲观 b :

$$E(t_{ij}) = \frac{a_{ij} + 4m_{ij} + b_{ij}}{6}, \quad Var(t_{ij}) = \frac{(b_{ij} - a_{ij})^2}{36} \quad (6.19)$$

设 T 为总工期, d 为规定工期:

$$P(T \leq d) = \Phi \left(\frac{d - E(T)}{\sqrt{Var(T)}} \right) \quad (6.20)$$

具体例子

例 6.20: 已知作业流程如表 6.4 所列, 求关键路径。

表 6.10 作业流程数据

作业	时间	紧前作业	作业	时间	紧前作业
A	5	—	G	21	B,E
B	10	—	H	35	B,E
C	11	—	I	25	B,E
D	4	B	J	15	F,G,I
E	4	A	K	20	F,G
F	15	C,D			

关键路径: $1 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 8$ (对应作业 $B \rightarrow G \rightarrow K$)

最短工期: 51 天

例 6.23 (计划网络优化): 要求在 49 天内完成, 部分作业可缩短工期, 求最小额外费用。

最优方案: 作业 B 压缩 1 天, 作业 K 压缩 1 天, 额外费用 1200 元。

例 6.25 (PERT 概率分析): 已知各作业三种估计时间, 求 52 天内完成的概率。

结果: 期望工期 51 天, 标准差 3.1623, 52 天内完成概率 62.41%; 若要求 95% 完成概率, 工期至少需要 56.2 天。

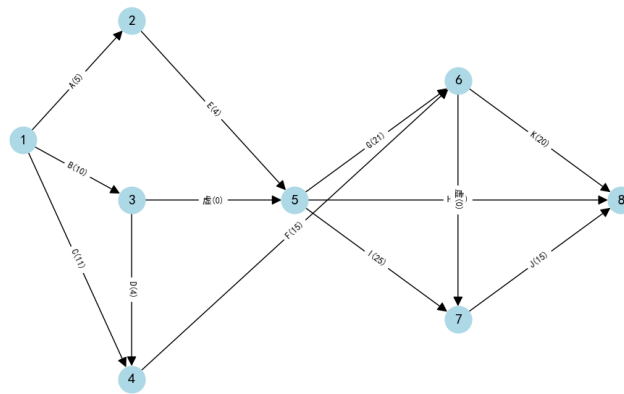


图 6.15 例 6.20 的计划网络图

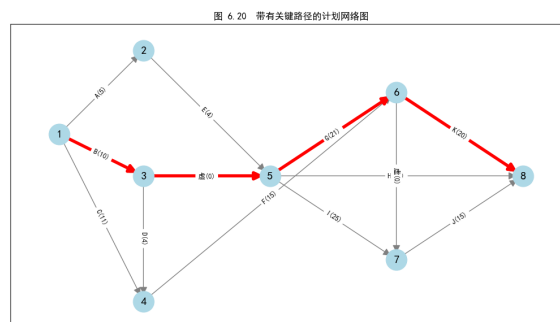


图 6.16 带有关键路径的计划网络图

代码实现

例 6.20: 求关键路径 (数学规划法)

```

1  # 程序文件 ex6_20.py
2  import numpy as np
3  import cvxpy as cp
4
5  x = cp.Variable(8, pos=True)
6  L = [(1,2,5), (1,3,10), (1,4,11), (2,5,4), (3,4,4), (3,5,0),
7       (4,6,15), (5,6,21), (5,7,25), (5,8,35), (6,7,0), (6,8,20),
8       (7,8,15)]
9
10 obj = cp.Minimize(sum(x))
11 con = []
12 for k in range(len(L)):
13     con.append(x[L[k][1]-1] >= x[L[k][0]-1] + L[k][2])
14
15 prob = cp.Problem(obj, con)
16 prob.solve(solver='GLPK_MI')
17 print('最优值 (最短工期):', prob.value)
18 print('各事件最早时间:', x.value)

```

例 6.21: 求所有作业的最早/最迟开工时间

```

1  # 程序文件 ex6_21.py
2  import numpy as np
3  import cvxpy as cp
4
5  n = 8
6  x = cp.Variable(n, pos=True)
7  z = np.zeros(n)
8  L = [(1,2,5), (1,3,10), (1,4,11), (2,5,4), (3,4,4), (3,5,0),
9       (4,6,15), (5,6,21), (5,7,25), (5,8,35), (6,7,0), (6,8,20),
10      (7,8,15)]
11
12 # 求最早时间
13 obj = cp.Minimize(sum(x))
14 con = []
15 for k in range(len(L)):
16     con.append(x[L[k][1]-1] >= x[L[k][0]-1] + L[k][2])
17
18 prob = cp.Problem(obj, con)
19 prob.solve(solver='GLPK_MI')
20 xx = x.value
21
22 # 求最迟时间 (反向递推)
23 z[-1] = xx[-1]

```

```

23     for k in range(n-1, 0, -1):
24         z[k-1] = min([z[a[1]-1] - a[2] for a in L if a[0] == k])
25
26     print('最早时间:', xx)
27     print('最迟时间:', z)
28
29     # 计算各作业的最早/最迟开工时间
30     print('\n作业\t最早开工\t最迟开工\t总时差')
31     for i in range(len(L)):
32         es = xx[L[i][0]-1]
33         ls = z[L[i][1]-1] - L[i][2]
34         slack = ls - es
35         print(f'{L[i][0]}->{L[i][1]}\t{es:.0f}\t\t{ls:.0f}\t\t{slack:.0f}')

```

例 6.23: 计划网络优化

```

1     # 程序文件 ex6_23.py
2     import numpy as np
3     import cvxpy as cp
4
5     # (起点, 终点, 计划时间, 最短时间, 单位压缩费用)
6     L = [(1,2,5,5,0), (1,3,10,8,700), (1,4,11,8,400),
7          (2,5,4,3,450), (3,4,4,4,0), (3,5,0,0,0),
8          (4,6,15,15,0), (5,6,21,16,600), (5,7,25,22,300),
9          (5,8,35,30,500), (6,7,0,0,0), (6,8,20,16,500), (7,8,15,12,400)]
10
11     x = cp.Variable(8, pos=True)
12     y = cp.Variable((8,8), integer=True)
13
14     # 目标: 压缩费用最小
15     fun = 0
16     for i in range(len(L)):
17         fun += y[L[i][0]-1, L[i][1]-1] * L[i][4]
18
19     obj = cp.Minimize(fun)
20     con = [x[7] - x[0] <= 49, y >= 0]
21
22     for i in range(len(L)):
23         con.append(x[L[i][1]-1] - x[L[i][0]-1] + y[L[i][0]-1, L[i][1]-1] >= L[i][2])
24         con.append(y[L[i][0]-1, L[i][1]-1] <= L[i][2] - L[i][3])
25
26     prob = cp.Problem(obj, con)
27     prob.solve(solver='GLPK_MI')
28     print('最小额外费用:', prob.value)

```


例 6.25: PERT 概率分析

```

1  # 程序文件 ex6_25.py
2  import numpy as np
3  import cvxpy as cp
4  from scipy.stats import norm
5
6  # (起点, 终点, a, m, b)
7  L = np.array([[1,2,3,5,7], [1,3,8,9,16], [1,4,8,11,14],
8  [2,5,3,4,5], [3,4,2,4,6], [3,5,0,0,0],
9  [4,6,8,16,18], [5,6,18,20,28], [5,7,18,25,32],
10 [5,8,26,33,52], [6,7,0,0,0], [6,8,11,21,25], [7,8,12,15,18]])
11
12 et = (L[:,2] + 4*L[:,3] + L[:,4]) / 6  # 期望
13 dt = (L[:,4] - L[:,2])**2 / 36        # 方差
14
15 n = 8
16 x = cp.Variable((n, n), integer=True)
17 m = len(L)
18
19 # 求关键路径 (最大期望时间)
20 fun = 0
21 for i in range(m):
22     fun += x[L[i][0]-1, L[i][1]-1] * et[i]
23
24 obj = cp.Maximize(fun)
25 con = [x >= 0, x <= 1]
26
27 # 起点: 出度为 1
28 out = np.where(L[:,0] == 1)[0]
29 con.append(cp.sum(x[0, L[out,1]-1]) == 1)
30
31 # 中间点: 流量守恒
32 for k in range(2, n):
33     out = np.where(L[:,0] == k)[0]
34     ind = np.where(L[:,1] == k)[0]
35     if len(out) > 0 and len(ind) > 0:
36         con.append(cp.sum(x[L[ind,0]-1, k-1]) == cp.sum(x[k-1, L[out
37             ,1]-1]))
38
39 # 终点: 入度为 1
40 ind = np.where(L[:,1] == n)[0]
41 con.append(cp.sum(x[L[ind,0]-1, n-1]) == 1)
42
43 prob = cp.Problem(obj, con)
44 prob.solve(solver='GLPK_MI')
45 print('期望工期:', prob.value)

```

```
45
46     # 计算方差
47     xx = x.value
48     s2 = 0
49     for i in range(m):
50         s2 += xx[L[i][0]-1, L[i][1]-1] * dt[i]
51
52     s = np.sqrt(s2)
53     p = norm.cdf(52, prob.value, s)
54     N = norm.ppf(0.95) * s + prob.value
55
56     print('标准差:', s)
57     print('52天内完成概率:', p)
58     print('95%概率所需工期:', N)
```

优缺点分析

方法	优点	缺点
CPM	明确关键路径, 便于资源调配	作业时间必须确定
PERT	处理不确定时间, 提供概率分析	计算复杂, 假设条件较多
数学规划法	可同时优化工期和费用	大规模问题求解慢
0-1 整数规划	精确求关键路径	仅适用小规模

方法选择建议

- 作业时间确定：使用 CPM，求关键路径和工期。
- 作业时间不确定：使用 PERT，给出三种估计，计算概率。
- 需要压缩工期：使用计划网络优化模型，权衡工期与费用。
- 大规模项目：使用 NetworkX 或专业项目管理软件。

应用场景

- 工程项目管理：建筑施工、设备安装的进度控制。
- 产品研发：新产品开发项目的进度安排。
- 软件开发：敏捷开发中的迭代计划。
- 活动策划：大型活动的组织与协调。
- 军事项目：复杂武器系统的研制管理。

第 7 章

插值与拟合

在数学建模过程中，通常要处理由试验、测量得到的大量数据或一些过于复杂而不便于计算的函数表达式。插值和拟合可以解决这样的问题。

给定一组数据，需要确定满足特定要求的曲线：

- 如果所求曲线通过所给定有限个数据点，这就是插值。
- 如果要求曲线反映对象整体的变化趋势，而不必通过所有数据点，这就是曲线拟合。

7.1 插值方法

7.1.1 一维插值

数学原理

(1) 基本概念 已知未知函数在 $n+1$ 个互不相同的观测点 $x_0 < x_1 < \cdots < x_n$ 处的函数值：

$$y_i = f(x_i), \quad i = 0, 1, \cdots, n. \quad (7.1)$$

寻求近似函数 $\phi(x)$ ，使之满足插值条件：

$$\phi(x_i) = y_i, \quad i = 0, 1, \cdots, n. \quad (7.2)$$

- 插值节点：观测点 x_i
- 被插函数： $f(x)$
- 插值函数： $\phi(x)$
- 插值区间： $[a, b]$ ，其中 $a = \min x_i$ ， $b = \max x_i$
- 内插：插值点 $x \in [a, b]$
- 外推：插值点 $x \notin [a, b]$

(2) 待定系数法与插值多项式的唯一性 n 次多项式有 $n+1$ 个待定系数, $n+1$ 个插值条件可唯一确定一个次数不超过 n 的多项式:

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0. \quad (7.3)$$

系数矩阵为范德蒙德矩阵:

$$\det(A) = \prod_{i=1}^n \prod_{j=0}^{n-1} (x_j - x_i) \neq 0. \quad (7.4)$$

定理 7.1 满足插值条件的次数不超过 n 的多项式存在且唯一。

插值余项:

$$R_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x), \quad \xi \in (a, b), \quad (7.5)$$

其中 $\omega_{n+1}(x) = \prod_{j=0}^n (x - x_j)$ 。

(3) 拉格朗日插值 基函数:

$$l_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}, \quad i = 0, 1, \dots, n. \quad (7.6)$$

满足 $l_i(x_j) = \delta_{ij}$ (Kronecker delta)。

拉格朗日插值多项式:

$$L_n(x) = \sum_{i=0}^n y_i l_i(x). \quad (7.7)$$

特点: 结构对称, 便于理论分析; 但每增加一个节点需要重新计算所有基函数。

(4) 牛顿插值 差商定义:

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}, \quad (7.8)$$

$$f[x_0, x_1, \dots, x_k] = \frac{f[x_1, \dots, x_k] - f[x_0, \dots, x_{k-1}]}{x_k - x_0}. \quad (7.9)$$

差商性质: 差商与节点的顺序无关 (对称性)。

牛顿插值多项式:

$$\begin{aligned} N_n(x) = & f(x_0) + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] \\ & + \cdots + (x - x_0) \cdots (x - x_{n-1})f[x_0, x_1, \dots, x_n]. \end{aligned} \quad (7.10)$$

优点: 每增加一个节点只需增加一项, 便于递推计算, 计算量小于拉格朗日插值。

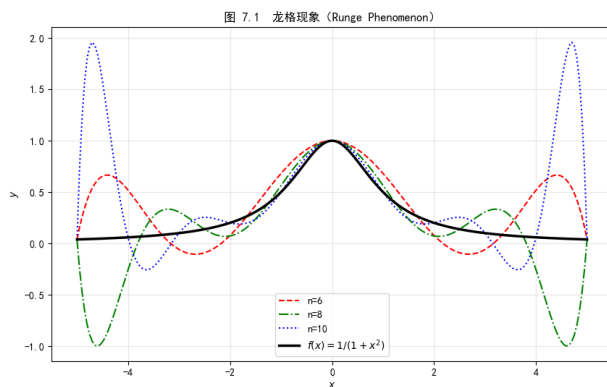


图 7.1 龙格现象： $n = 6, 8, 10$ 时拉格朗日插值与原函数的图形比较

(5) 龙格现象 高次多项式插值在插值区间端点附近可能产生剧烈振荡，这种现象称为龙格现象。

(6) 分段线性插值 将每两个相邻节点用直线连接，形成一条折线。在每个小区间 $[x_i, x_{i+1}]$ 上是线性函数。

- 优点：计算简单，稳定性好，不会产生龙格振荡。
- 缺点：在节点处不可导（不光滑）。

误差估计：

$$|R_n(x)| = |f(x) - I_n(x)| \leq \frac{M_2 h^2}{8}, \quad (7.11)$$

其中 $M_2 = \max |f''(x)|$, $h = \max |x_i - x_{i-1}|$ 。

(7) 三次样条插值 样条函数：具有一定光滑性的分段多项式。

三次样条插值条件：

1. 在每个子区间上是三次多项式；
2. 通过所有数据点 $S(x_i) = y_i$ ；
3. 在节点处一阶、二阶导数连续。

边界条件：

- 自然样条： $S''(x_0) = S''(x_n) = 0$
- 固定边界： $S'(x_0) = f'(x_0)$, $S'(x_n) = f'(x_n)$
- 周期边界： $S'(x_0) = S'(x_n)$, $S''(x_0) = S''(x_n)$

具体例子

例 7.1: 待定系数法插值

已知数据点:

$$(x_i, y_i) = (1, 16), (2, 18), (3, 21), (4, 17), (5, 15), (6, 12). \quad (7.12)$$

求 $x = 1.5$ 和 $x = 2.6$ 处的函数值。

插值多项式:

$$y = -0.2417x^5 + 4.3333x^4 - 28.9583x^3 + 87.6667x^2 - 115.8x + 69.0. \quad (7.13)$$

预测值: $y(1.5) = 14.918$, $y(2.6) = 20.8846$ 。

例 7.3: 龙格现象

函数 $f(x) = 1/(1+x^2)$ 在 $[-5, 5]$ 上取等距节点, 用 $n = 6, 8, 10$ 次拉格朗日插值多项式逼近, 观察高次插值的振荡现象。

代码实现

例 7.1: 待定系数法

```

1  # 程序文件 ex7_1.py
2  import numpy as np
3  import pylab as plt
4
5  x0 = np.arange(1, 7)
6  y0 = np.array([16, 18, 21, 17, 15, 12])
7
8  # 构造范德蒙德矩阵并求解
9  A = np.vander(x0)
10 p = np.linalg.inv(A) @ y0
11 print('从高次幂到低次幂的系数:', np.round(p, 4))
12
13 # 预测
14 yh = np.polyval(p, [1.5, 2.6])
15 print('预测值:', np.round(yh, 4))
16
17 # 绘图
18 plt.plot(x0, y0, 'o', label='已知点')
19 xt = np.linspace(1, 6, 100)
20 plt.plot(xt, np.polyval(p, xt), label='插值曲线')
21 plt.legend()
22 plt.show()

```

例 7.2: 拉格朗日插值

```

1  # 程序文件 ex7_2.py
2  import numpy as np
3  from scipy.interpolate import lagrange
4
5  x0 = np.arange(1, 7)
6  y0 = np.array([16, 18, 21, 17, 15, 12])
7
8  p = lagrange(x0, y0)
9  print('插值多项式系数:', np.round(p, 4))
10
11 yh = np.polyval(p, [1.5, 2.6])
12 print('预测值:', np.round(yh, 4))

```

例 7.3: 龙格现象

```

1  # 程序文件 ex7_3.py
2  import numpy as np
3  import pylab as plt
4  from scipy.interpolate import lagrange
5
6  f = lambda x: 1 / (1 + x**2)
7
8  def fun(n):
9  x = np.linspace(-5, 5, n+1)
10 return lagrange(x, f(x))
11
12 x0 = np.linspace(-5, 5, 100)
13 plt.rc('font', size=12)
14 N = [6, 8, 10]
15 styles = ['--*', '-o', '-.s']
16
17 for k in range(len(N)):
18 p = fun(N[k])
19 plt.plot(x0, np.polyval(p, x0), styles[k], label=f'n={N[k]}')
20
21 plt.plot(x0, f(x0), 'k-', linewidth=2, label='1/(1+x^2)')
22 plt.legend()
23 plt.title('龙格现象')
24 plt.show()

```

三次样条插值

```

1  # 三次样条插值示例
2  from scipy.interpolate import CubicSpline
3
4  x = np.array([1, 2, 3, 4, 5, 6])
5  y = np.array([16, 18, 21, 17, 15, 12])

```

```
6
7  # 自然样条
8  cs = CubicSpline(x, y, bc_type='natural')
9  x_new = np.linspace(1, 6, 100)
10 y_new = cs(x_new)
11
12 plt.plot(x, y, 'o', label='已知点')
13 plt.plot(x_new, y_new, label='三次样条')
14 plt.legend()
15 plt.show()
```

优缺点分析

方法	优点	缺点
待定系数法	理论简单，概念清晰	高次时矩阵病态，计算不稳定
拉格朗日插值	结构对称，便于理论分析	增加节点需重新计算，高次时龙格振荡
牛顿插值	增加节点只需增加一项，计算量小	差商计算稍复杂
分段线性插值	稳定，无龙格振荡，计算简单	在节点处不可导，不够光滑
三次样条插值	光滑性好，二阶导数连续	计算复杂，需要解三对角方程组

方法选择建议

- 数据点少且精度要求高：使用拉格朗日或牛顿插值。
- 节点较多：避免使用高次插值，改用分段线性插值或三次样条。
- 需要光滑曲线：使用三次样条插值。
- 仅需快速估算：使用分段线性插值。
- 外推预测：谨慎使用插值，短期可考虑，中长期不适用。

应用场景

- 实验数据处理：由离散实验数据估计任意点的函数值。
- 图像处理：图像缩放、旋转时的像素插值。
- 数值积分：构造插值多项式近似被积函数。
- 计算机图形学：曲线和曲面的光滑拟合。

- 地理信息系统：数字高程模型的插值。

7.1.2 二维插值

数学原理

二维插值问题描述如下：给定 xOy 平面上 $m \times n$ 个互不相同的插值节点 (x_i, y_j) 处的观测值 z_{ij} ，求一个近似的二元插值曲面函数 $f(x, y)$ ，使其通过全部已知节点：

$$f(x_i, y_j) = z_{ij}, \quad i = 1, 2, \dots, m, j = 1, 2, \dots, n. \quad (7.14)$$

二维插值可分为两种类型：

- 网格节点插值：数据点规则分布在矩形网格上。
- 散乱数据插值：数据点杂乱无章，分布不规则。

(1) 网格节点插值 设 $a = x_1 < \dots < x_m = b$, $c = y_1 < \dots < y_n = d$ ，则 $[a, b] \times [c, d]$ 构成矩形插值区域。

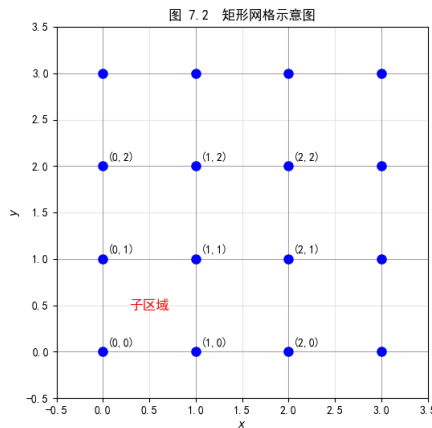


图 7.2 矩形网格示意图

最邻近点插值：取插值点的函数值为其最邻近插值节点的函数值。不连续，但最简单。

分片线性插值：将每个矩形子区域用对角线划分为两个三角形，在每个三角形上进行线性插值。

若插值点 (x, y) 在子区域 $[x_i, x_{i+1}] \times [y_j, y_{j+1}]$ 内：

上三角形区域 ($y >$ 对角线)：

$$f(x, y) = z_{ij} + (z_{i+1, j+1} - z_{i, j+1}) \frac{x - x_i}{x_{i+1} - x_i} + (z_{i, j+1} - z_{ij}) \frac{y - y_j}{y_{j+1} - y_j}. \quad (7.15)$$

下三角形区域 ($y \leq$ 对角线)：

$$f(x, y) = z_{ij} + (z_{i+1, j} - z_{ij}) \frac{x - x_i}{x_{i+1} - x_i} + (z_{i+1, j+1} - z_{i+1, j}) \frac{y - y_j}{y_{j+1} - y_j}. \quad (7.16)$$

双线性插值：在每个子矩形区域上构造双线性函数：

$$f(x, y) = Axy + Bx + Cy + D. \quad (7.17)$$

利用 4 个顶点的函数值确定待定系数 A, B, C, D 。

基函数形式的双线性插值：

$$\begin{aligned} f(x, y) = & z_{ij} \frac{(x - x_{i+1})(y - y_{j+1})}{(x_i - x_{i+1})(y_j - y_{j+1})} + z_{i+1,j} \frac{(x - x_i)(y - y_{j+1})}{(x_{i+1} - x_i)(y_j - y_{j+1})} \\ & + z_{i,j+1} \frac{(x - x_{i+1})(y - y_j)}{(x_i - x_{i+1})(y_{j+1} - y_j)} + z_{i+1,j+1} \frac{(x - x_i)(y - y_j)}{(x_{i+1} - x_i)(y_{j+1} - y_j)}. \end{aligned} \quad (7.18)$$

(2) 散乱数据插值 (Shepard 方法) 已知在 $\Omega = [a, b] \times [c, d]$ 内散乱分布 N 个观测点 (x_k, y_k) 及其观测值 z_k ，要求寻找二元函数 $f(x, y)$ 满足 $f(x_k, y_k) = z_k$ 。

反距离加权平均法 (Shepard 方法)：

计算插值点 (x, y) 到各已知点的欧几里得距离：

$$r_k = \sqrt{(x - x_k)^2 + (y - y_k)^2}, \quad k = 1, 2, \dots, N. \quad (7.19)$$

定义权值 (距离越近, 权值越大)：

$$w_k(x, y) = \frac{1/r_k^2}{\sum_{i=1}^N 1/r_i^2}, \quad k = 1, 2, \dots, N. \quad (7.20)$$

插值函数：

$$f(x, y) = \begin{cases} z_k, & r_k = 0, \\ \sum_{k=1}^N w_k(x, y) z_k, & r_k \neq 0. \end{cases} \quad (7.21)$$

改进的 Shepard 方法：引入可微权函数 $\omega(r)$ 提高光滑性：

$$\omega(r) = \begin{cases} 1/r, & 0 < r \leq R/3, \\ \frac{27}{4R} \left(\frac{r}{R} - 1 \right)^2, & R/3 < r \leq R, \\ 0, & r > R. \end{cases} \quad (7.22)$$

其中 $R > 0$ 为适当常数。

具体例子

例 7.4：二维网格节点插值

已知 3×3 网格节点的函数值：

$y \backslash x$	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

(7.23)

用双线性插值求 $(0.5, 0.5)$ 处的函数值。

解：在子区域 $[0, 1] \times [0, 1]$ 内，4 个顶点为 $(0, 0) = 1$, $(1, 0) = 2$, $(0, 1) = 4$, $(1, 1) = 5$ 。

由双线性插值公式：

$$f(0.5, 0.5) = 1 \times 0.25 + 2 \times 0.25 + 4 \times 0.25 + 5 \times 0.25 = 3.0. \quad (7.24)$$

例 7.5：散乱数据插值

已知 5 个散乱点 $(0, 0, 1)$, $(1, 0, 2)$, $(0, 1, 3)$, $(2, 2, 4)$, $(1, 2, 5)$ ，用 Shepard 方法求 $(0.5, 0.5)$ 处的函数值。

代码实现

网格节点插值

```

1  # 网格节点插值示例
2  import numpy as np
3  from scipy.interpolate import interp2d
4  import pylab as plt
5
6  # 生成网格数据
7  x = np.array([0, 1, 2])
8  y = np.array([0, 1, 2])
9  z = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
10
11 # 双线性插值
12 f = interp2d(x, y, z, kind='linear')
13 print('f(0.5, 0.5) =', f(0.5, 0.5)[0])
14
15 # 三次样条插值
16 f_cubic = interp2d(x, y, z, kind='cubic')
17 print('f_cubic(0.5, 0.5) =', f_cubic(0.5, 0.5)[0])
18
19 # 绘图
20 x_new = np.linspace(0, 2, 50)
21 y_new = np.linspace(0, 2, 50)
22 X, Y = np.meshgrid(x_new, y_new)
23 Z = f(x_new, y_new)
24
25 fig = plt.figure()
26 ax = fig.add_subplot(111, projection='3d')
27 ax.plot_surface(X, Y, Z, cmap='viridis')
28 ax.scatter(x, y, z, color='red', s=50, label='已知点')
29 plt.show()

```

散乱数据插值（Shepard 方法）

```

1  # Shepard 插值
2  import numpy as np
3  from scipy.interpolate import RBFInterpolator
4
5  # 散乱数据点
6  xy = np.array([[0, 0], [1, 0], [0, 1], [2, 2], [1, 2]])
7  z = np.array([1, 2, 3, 4, 5])
8
9  # 使用径向基函数插值 (相当于改进的 Shepard 方法)
10 rbf = RBFInterpolator(xy, z, kernel='inverse_multiquadric')
11 print('f(0.5, 0.5) =', rbf([[0.5, 0.5]])[0])
12
13 # 自定义 Shepard 方法
14 def shepard(xy, z, x_new, y_new, p=2):
15     """反距离加权插值"""
16     result = np.zeros_like(x_new)
17     for i in range(len(x_new)):
18         r = np.sqrt((xy[:, 0] - x_new[i])**2 + (xy[:, 1] - y_new[i])
19                     **2)
20         idx = np.where(r < 1e-10)[0]
21         if len(idx) > 0:
22             result[i] = z[idx[0]]
23         else:
24             w = 1 / (r**p)
25             result[i] = np.sum(w * z) / np.sum(w)
26     return result
27
28 x_new = np.linspace(-0.5, 2.5, 20)
29 y_new = np.linspace(-0.5, 2.5, 20)
30 X, Y = np.meshgrid(x_new, y_new)
31 Z = shepard(xy, z, X.flatten(), Y.flatten()).reshape(20, 20)

```

优缺点分析

方法	优点	缺点
最邻近点插值	计算最快, 实现简单	不连续, 精度低
分片线性插值	连续, 计算简单	在三角形边界上不可导
双线性插值	光滑, 精度较高	仅在矩形网格上适用
Shepard 方法	适用于任意散乱数据	全局计算, 计算量大
改进 Shepard	提高光滑性, 局部影响	需要选择合适参数 R

方法选择建议

- 规则网格数据且要求快速：使用双线性插值。
- 规则网格数据且需要高精度：使用双三次样条插值（'kind='cubic''）。
- 散乱数据点较少：使用 Shepard 反距离加权法。
- 散乱数据点较多：使用径向基函数插值（RBF）。
- 需要全局光滑曲面：使用径向基函数或 Kriging 插值。

应用场景

- 数字高程模型（DEM）：由离散高程点生成地形曲面。
- 气象数据：由气象站观测值生成等温线、等压线图。
- 图像处理：图像几何变换时的像素插值。
- 地质勘探：由钻孔数据生成地层分布图。
- 医学影像：CT、MRI 图像的三维重建。

7.1.3 用 Python 求解插值问题

`scipy.interpolate` 模块提供了一维插值函数 `interp1d`、二维插值函数 `interp2d` 和 multidimensional 插值函数 `interpn`。

一维插值函数 `interp1d`

基本调用格式：

```
interp1d(x, y, kind='linear')
```

其中 `kind` 参数指定插值方法：

- 'nearest': 最邻近插值（0 阶）
- 'linear': 分段线性插值（1 阶，默认）
- 'quadratic': 二次样条插值（2 阶）
- 'cubic': 三次样条插值（3 阶）

具体例子

例 7.4（机床加工）：已知机翼断面下轮廓线的数据点，需要得到 x 坐标每改变 0.1 时的 y 坐标，并求 $x = 0$ 处的曲线斜率和 $13 \leq x \leq 15$ 范围内 y 的最小值。

表 7.3 插值数据点

x	0	3	5	7	9	11	12	13	14	15
y	0	1.2	1.7	2.0	2.1	2.0	1.8	1.2	1.0	1.6

图 7.3 三种插值曲线对比

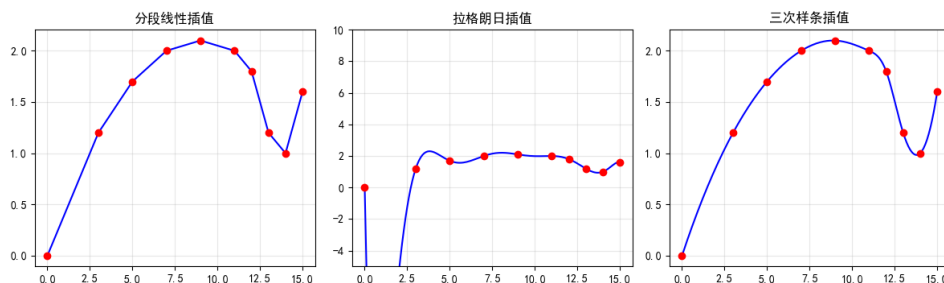


图 7.3 三种插值曲线对比

从图中可以看出，拉格朗日插值结果严重失真，分段线性插值光滑性较差，建议选用三次样条插值。

例 7.5 (速度积分)：已知速度曲线 $v(t)$ 上的 4 个数据点，用三次样条插值求位移 $S = \int_{0.15}^{0.18} v(t) dt$ 。

表 7.4 速度观测值

t	0.15	0.16	0.17	0.18
$v(t)$	3.5	1.5	2.5	2.8

例 7.6 (国土面积与边界长度)：根据地图测量数据，用三次样条插值计算国土面积和边界长度。

面积公式：

$$S = \int_a^b (y_2(x) - y_1(x)) dx \quad (7.25)$$

边界长度公式：

$$L = \int_a^b \sqrt{1 + [y'_1(x)]^2} dx + \int_a^b \sqrt{1 + [y'_2(x)]^2} dx \quad (7.26)$$

计算结果：面积约为 42434 km^2 (准确值 41288 km^2 ，误差 2.78%)，边界长度约为 1163.68 km 。

例 7.7 (二维网格插值)：已知 100×100 网格节点上的高程数据，求区域地表面积，并绘制等高线图和三维表面图。

例 7.8 (最高点搜索)：已知 5×4 网格高程数据，用双三次样条插值求最高点。

最高点位于 $(410, 180)$ ，高程约为 831.05 。

例 7.9 (散乱数据插值)：已知某海域散乱测量点的水深数据，绘制海底地形等高线图。

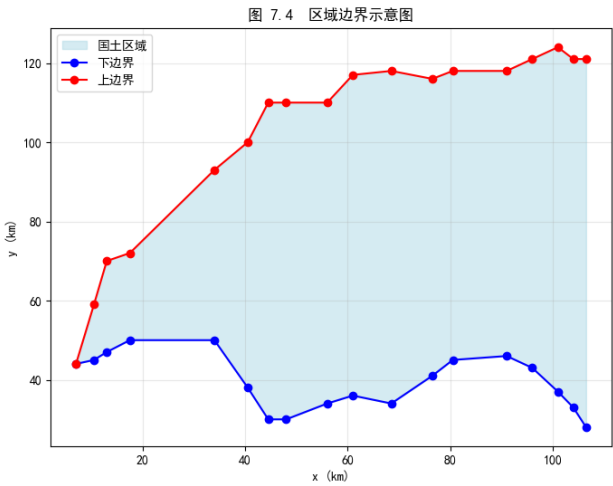


图 7.4 区域边界示意图

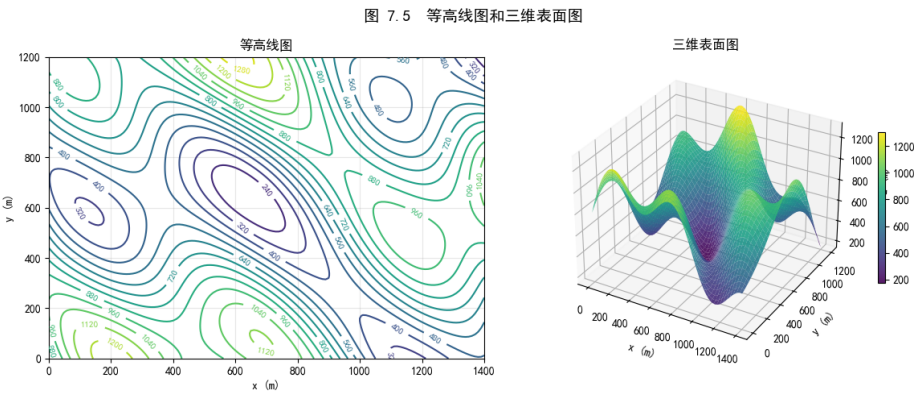


图 7.5 等高线图和三维表面图

表 7.5 高程数据表

$y \backslash x$	100	200	300	400	500
100	636	697	624	478	450
200	698	712	630	800	420
300	680	674	598	412	400
400	662	626	552	334	310

图 7.6 海底地形图及等高线图

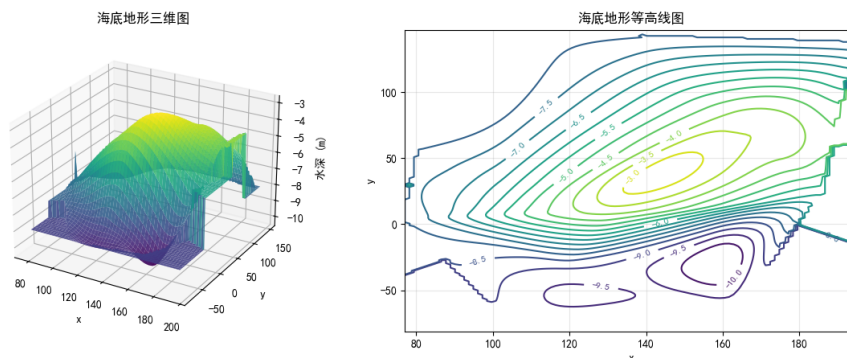


图 7.6 海底地形图及等高线图

代码实现

例 7.4：三种插值方法比较

```

1  # 程序文件 ex7_4.py
2  import numpy as np
3  from scipy.interpolate import interp1d, lagrange
4  import pylab as plt
5
6  a = np.loadtxt('data7_4.txt')
7  x0 = a[0]
8  y0 = a[1]
9  x = np.linspace(0, 15, 151)
10
11 # 分段线性插值
12 y1 = interp1d(x0, y0, 'linear')(x)
13
14 # 拉格朗日插值
15 p2 = lagrange(x0, y0)
16 y2 = np.polyval(p2, x)
17
18 # 三次样条插值
19 y3 = interp1d(x0, y0, 'cubic')(x)
20
21 # 计算  $x=0$  处的斜率
22 dx = np.diff(x)
23 dy = np.diff(y3)
24 dyx0 = (dy / dx)[0]
25
26 # 计算  $13 \leq x \leq 15$  范围内的最小值
27 xt = x[130:]
28 yt = y3[130:]
29 ymin = min(yt)
30 xmin = xt[np.argmin(yt)]
31

```



```

32     print('x=0 处斜率:', dyx0)
33     print('xmin=', xmin, 'ymin=', ymin)
34
35     # 绘图
36     plt.subplots_adjust(wspace=0.5)
37     plt.subplot(131)
38     plt.plot(x, y1)
39     plt.title('分段线性插值')
40     plt.subplot(132)
41     plt.plot(x, y2)
42     plt.title('拉格朗日插值')
43     plt.subplot(133)
44     plt.plot(x, y3)
45     plt.title('三次样条插值')
46     plt.show()

```

例 7.5: 样条积分

```

1     # 程序文件 ex7_5.py
2     from scipy.interpolate import UnivariateSpline, interp1d
3     import numpy as np
4
5     t0 = np.linspace(0.15, 0.18, 4)
6     v0 = np.array([3.5, 1.5, 2.5, 2.8])
7
8     # 方法一: UnivariateSpline
9     sp1 = UnivariateSpline(t0, v0)
10    I1 = sp1.integral(0.15, 0.18)
11    print('方法1积分值:', I1)
12
13    # 方法二: interp1d + 梯形积分
14    sp2 = interp1d(t0, v0, 'cubic')
15    tn = np.linspace(0.15, 0.18, 200)
16    vn = sp2(tn)
17    I2 = np.trapz(vn, tn)
18    print('方法2积分值:', I2)

```

例 7.6: 国土面积与边界长度

```

1     # 程序文件 ex7_6_1.py
2     from scipy.interpolate import UnivariateSpline
3     import numpy as np
4     import pylab as plt
5
6     a = np.loadtxt('data7_6.txt')
7     x0 = a[0, :]
8     y1 = a[1, :]
9     y2 = a[2, :]

```

```

10
11     # 三次样条插值
12     f1 = UnivariateSpline(x0, y1)
13     f2 = UnivariateSpline(x0, y2)
14
15     # 求导
16     d1 = f1.derivative(1)
17     d2 = f2.derivative(1)
18
19     x = np.linspace(x0[0], x0[-1], 1000)
20     d1x = d1(x)
21     d2x = d2(x)
22
23     # 边界长度
24     L = np.trapz(np.sqrt(1 + d1x**2) + np.sqrt(1 + d2x**2), x)
25     L = L / 18 * 40
26     print('周长 L=', round(L, 4))
27
28     # 面积
29     S = np.trapz(f2(x) - f1(x), x)
30     S = S / 18 * 2 * 1600
31     print('面积 S=', round(S, 4))

```

例 7.7：二维网格插值与曲面面积

```

1     # 程序文件 ex7_7.py
2     import numpy as np
3     from scipy.interpolate import interp2d
4     from numpy.linalg import norm
5     import pylab as plt
6
7     # 加载高程数据 (假设已保存在 data7_7.txt)
8     z = np.loadtxt('data7_7.txt')
9     x = np.linspace(0, 1400, z.shape[1])
10    y = np.linspace(0, 1200, z.shape[0])
11
12    # 双线性插值
13    f = interp2d(x, y, z, kind='linear')
14    xn = np.linspace(0, 1400, 141)
15    yn = np.linspace(0, 1200, 121)
16    zn = f(xn, yn)
17
18    # 计算地表面积 (每个小矩形划分成两个三角形)
19    m, n = len(xn), len(yn)
20    s = 0
21    for i in range(m-1):
22    for j in range(n-1):

```

```

23     p1 = np.array([xn[i], yn[j], zn[j, i]])
24     p2 = np.array([xn[i+1], yn[j], zn[j, i]])
25     p3 = np.array([xn[i+1], yn[j+1], zn[j, i+1]])
26     p4 = np.array([xn[i], yn[j+1], zn[j+1, i]])
27
28     # 海伦公式计算三角形面积
29     def tri_area(a, b, c):
30         d1 = norm(a-b); d2 = norm(b-c); d3 = norm(c-a)
31         p = (d1 + d2 + d3) / 2
32         return np.sqrt(p * (p-d1) * (p-d2) * (p-d3))
33
34     s += tri_area(p1, p2, p3) + tri_area(p1, p3, p4)
35
36     print('地表面积:', s)
37
38     # 绘制等高线和三维图
39     plt.subplot(121)
40     contour = plt.contour(xn, yn, zn)
41     plt.clabel(contour)
42     plt.xlabel('x')
43     plt.ylabel('y')
44
45     ax = plt.subplot(122, projection='3d')
46     X, Y = np.meshgrid(xn, yn)
47     ax.plot_surface(X, Y, zn, cmap='viridis')
48     ax.set_xlabel('x')
49     ax.set_ylabel('y')
50     ax.set_zlabel('z')
51     plt.show()

```

例 7.8: 散乱数据插值（网格数据）

```

1     # 程序文件 ex7_8.py
2     import numpy as np
3     from scipy.interpolate import interp2d
4
5     z0 = np.loadtxt('data7_8.txt')
6     x0 = np.linspace(100, 500, 5)
7     y0 = np.linspace(100, 400, 4)
8
9     # 双三次样条插值
10    f = interp2d(x0, y0, z0, kind='cubic')
11    xn = np.linspace(100, 500, 41)
12    yn = np.linspace(100, 400, 31)
13    zn = f(xn, yn)
14
15    max_z = np.max(zn)

```

```

16     idx = np.argmax(zn)
17     print('最大高程:', max_z)

```

例 7.9: 散乱数据插值

```

1     # 程序文件 ex7_9.py
2     import pylab as plt
3     import numpy as np
4     from scipy.interpolate import griddata
5
6     a = np.loadtxt('data7_9.txt')
7     x = a[0]
8     y = a[1]
9     z = -a[2]  # 水深取负
10
11     xy = np.vstack([x, y]).T
12     xn = np.linspace(x.min(), x.max(), 100)
13     yn = np.linspace(y.min(), y.max(), 200)
14     xng, yng = np.meshgrid(xn, yn)
15
16     # 三次样条插值
17     zn1 = griddata(xy, z, (xng, yng), method='cubic')
18     # 最近邻插值填充 NaN
19     zn2 = griddata(xy, z, (xng, yng), method='nearest')
20     zn1[np.isnan(zn1)] = zn2[np.isnan(zn1)]
21
22     # 三维地形图
23     ax = plt.subplot(121, projection='3d')
24     ax.plot_surface(xng, yng, zn1, cmap='viridis')
25     ax.set_xlabel('x')
26     ax.set_ylabel('y')
27     ax.set_zlabel('z')
28
29     # 等高线图
30     plt.subplot(122)
31     plt.contour(xn, yn, zn1)
32     plt.show()

```

优缺点分析

方法	优点	缺点
分段线性插值 拉格朗日插值	稳定、无振荡, 计算简单 结构简单, 便于理论分析	光滑性差, 节点处不可导 高次时龙格振荡, 严重失真

续下页

方法	优点	缺点
三次样条插值	光滑性好, 二阶导数连续	计算量较大, 需要解三对角方程
双线性插值	简单高效, 适合规则网格	仅一阶连续
双三次样条	光滑性好, 精度高	计算量大
Shepard/散乱插值	适用于任意分布数据点	全局计算, 计算量大

方法选择建议

- 数据点少且精度要求高：拉格朗日插值（注意次数不超过 5-6 次）。
- 节点较多、要求稳定：分段线性插值或三次样条插值。
- 需要光滑曲线：三次样条插值（推荐）。
- 二维规则网格数据：双线性插值（`interp2d`）或双三次样条（`kind='cubic'`）。
- 二维散乱数据：`griddata` 方法，配合 `method='cubic'`。
- 需要积分或求导：使用 `UnivariateSpline`，提供 `integral()` 和 `derivative()` 方法。

应用场景

- 数控加工：由离散轮廓点生成加工路径。
- 地图测绘：由边界测量点计算国土面积。
- 地形分析：由高程数据生成等高线和三维地形图。
- 海洋探测：由散乱水深数据绘制海底地形。
- 气象预报：由气象站数据生成等温线、等压线图。
- 图像处理：图像几何变换中的像素插值。

7.2 曲线拟合

7.2.1 线性最小二乘法

数学原理

(1) 线性最小二乘问题 给定观测数据 (x_i, y_i) ($i = 1, \dots, n$)，寻找参数 $a_1, \dots, a_m$ 使拟合函数

$$y = a_1\phi_1(x) + a_2\phi_2(x) + \dots + a_m\phi_m(x) \tag{7.27}$$

在最小二乘意义下逼近观测值：

$$\min \sum_{i=1}^n \left(\sum_{j=1}^m a_j \phi_j(x_i) - y_i \right)^2. \quad (7.28)$$

将观测值代入，得到线性方程组 $RA = Y$ ，其中

$$R_{ij} = \phi_j(x_i), \quad Y_i = y_i. \quad (7.29)$$

拟合参数的最小二乘解为：

$$A = (R^T R)^{-1} R^T Y. \quad (7.30)$$

在 Python 中可用伪逆求解：

$$A = \text{np.linalg.pinv}(R) @ Y$$

(2) 多项式拟合 `np.polyfit(x, y, n)` 拟合 n 次多项式，返回从高次到低次的系数。

(3) 非线性拟合 对于非线性模型 $y = f(x, \theta)$ ，最小二乘问题为：

$$\min_{\theta} \sum_{i=1}^n (f(x_i, \theta) - y_i)^2. \quad (7.31)$$

Python 中可使用 `scipy.optimize.curve_fit` 或 `least_squares` 求解。

(4) 拟合优度检验

- **SSE** (误差平方和): $SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$ ，越小越好。
- **MSE** (均方误差): $MSE = \frac{SSE}{n - m}$ ，消除参数个数影响。
- **RMSE** (剩余标准差): $RMSE = \sqrt{\frac{SSE}{n}}$ 。
- R^2 (判定系数): $R^2 = 1 - \frac{SSE}{SST}$ ，其中 $SST = \sum_{i=1}^n (y_i - \bar{y})^2$ ，越接近 1 越好。
- 调整 R^2 : $\bar{R}^2 = 1 - \frac{SSE/(n - m)}{SST/(n - 1)}$ ，惩罚过多参数。

具体例子

例 7.10: 刀具磨损速度拟合

数据: $t = 0, 1, \dots, 7$, $y = 27.0, 26.8, 26.5, 26.3, 26.1, 25.7, 25.3, 24.8$ 。

拟合直线 $y = at + b$, 求得 $\hat{a} = -0.3036$, $\hat{b} = 27.125$ 。

例 7.11: 小行星轨道拟合

椭圆方程 $a_1x^2 + a_2xy + a_3y^2 + a_4x + a_5y + 1 = 0$, 由 5 个观测点拟合参数:

$$a_1 = 0.0508, a_2 = -0.0702, a_3 = 0.0381, a_4 = -0.4531, a_5 = 0.2643. \quad (7.32)$$

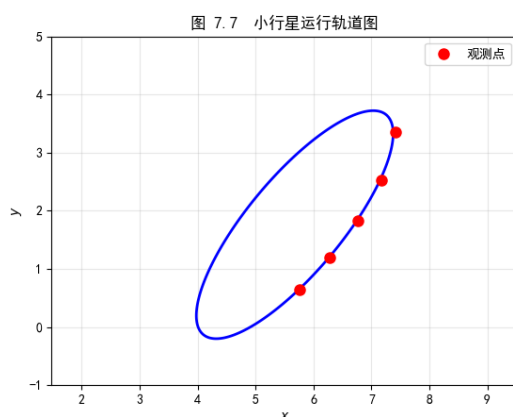


图 7.7 小行星运行轨道图

例 7.13: 化学反应速度拟合

数据如表 7.11, 拟合 $y = ke^{mt}$ 。两边取对数化为线性:

$$\ln y = \ln k + mt. \quad (7.33)$$

求得 $k = 78.5700$, $m = -0.1037$ 。

例 7.14: 非线性拟合

用 `curve_fit` 直接拟合 $y = ke^{mt}$, 求得 $k = 78.4500$, $m = -0.1036$ 。

例 7.17: 分段函数拟合

拟合分段函数:

$$y = \begin{cases} a + bx, & x < k, \\ c + dx, & x \geq k, \end{cases} \quad (7.34)$$

其中 $a = -0.0381$, $b = 1.3538$, $c = 1.2365$, $d = -1.3006$, $k = 0.5340$ 。

代码实现

例 7.10: 线性拟合 (两种方法)

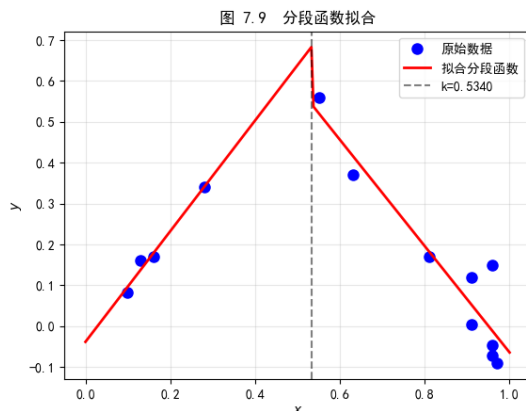


图 7.8 分段函数拟合

```

1  # 程序文件 ex7_10.py
2  import numpy as np
3
4  t = np.arange(8)
5  y = np.array([27.0, 26.8, 26.5, 26.3, 26.1, 25.7, 25.3, 24.8])
6
7  # 方法一：公式法
8  tb = t.mean()
9  yb = y.mean()
10 a1 = np.sum((t - tb) * (y - yb)) / np.sum((t - tb)**2)
11 b1 = yb - a1 * tb
12 print('公式法:', [a1, b1])
13
14 # 方法二：伪逆法
15 A = np.vstack([t, np.ones(len(t))]).T
16 p = np.linalg.pinv(A) @ y
17 print('伪逆法:', p)

```

例 7.11: 椭圆轨道拟合

```

1  # 程序文件 ex7_11.py
2  import numpy as np
3  import pylab as plt
4
5  a = np.loadtxt('data7_11.txt')
6  x = a[0]; y = a[1]
7
8  # 构造线性方程组
9  A = np.vstack([x**2, x*y, y**2, x, y]).T
10 b = -np.ones(5)
11 c = np.linalg.pinv(A) @ b
12 print('拟合系数:', np.round(c, 4))
13

```



```

14     # 绘制椭圆
15     f = lambda x, y: c[0]*x**2 + c[1]*x*y + c[2]*y**2 + c[3]*x + c
        [4]*y + 1
16     x_plot = np.linspace(3, 8, 100)
17     y_plot = np.linspace(-1, 5, 100)
18     X, Y = np.meshgrid(x_plot, y_plot)
19     plt.contour(X, Y, f(X, Y), [0])
20     plt.xlabel('x')
21     plt.ylabel('y')
22     plt.title('小行星运行轨道')
23     plt.show()

```

例 7.13: 线性化拟合

```

1     # 程序文件 ex7_13.py
2     import numpy as np
3
4     a = np.loadtxt('data7_13.txt')
5     t = a[0]; y = np.log(a[1])
6
7     p = np.polyfit(t, y, 1)
8     m = p[0]
9     k = np.exp(p[1])
10    print('m=', round(m, 4))
11    print('k=', round(k, 4))

```

例 7.14: 非线性拟合 (curve_fit)

```

1     # 程序文件 ex7_14.py
2     import numpy as np
3     from scipy.optimize import curve_fit
4
5     a = np.loadtxt('data7_13.txt')
6     t0 = a[0]; y0 = a[1]
7
8     f = lambda t, k, m: k * np.exp(m * t)
9     p, pcov = curve_fit(f, t0, y0)
10    print('拟合参数:', np.round(p, 4))
11
12    yh = f(np.array([5, 8]), *p)
13    print('预测值:', np.round(yh, 4))

```

例 7.17: 分段函数拟合

```

1     # 程序文件 ex7_17.py
2     import numpy as np
3     from scipy.optimize import curve_fit
4     import pylab as plt
5

```

```

6     x0 = np.array([0.81, 0.91, 0.13, 0.91, 0.63, 0.098, 0.28, 0.55,
7                   0.96, 0.96, 0.16, 0.97, 0.96])
8
9     y0 = np.array([0.17, 0.12, 0.16, 0.0035, 0.37, 0.082, 0.34,
10                   0.56, 0.15, -0.046, 0.17, -0.091, -0.071])
11
12     f = lambda x, a, b, c, d, k: (a + b*x) * (x < k) + (c + d*x) *
13         (x >= k)
14
15     LB = [-np.inf] * 4 + [min(x0)]
16     UB = [np.inf] * 4 + [max(x0)]
17     p, _ = curve_fit(f, x0, y0, bounds=(LB, UB))
18     print('拟合参数:', p)
19
20     x = np.linspace(min(x0), max(x0), 100)
21     plt.plot(x0, y0, 'o', label='data')
22     plt.plot(x, f(x, *p), label='fitted')
23     plt.legend()
24     plt.show()

```

优缺点分析

方法	优点	缺点
线性最小二乘 伪逆法 多项式拟合 curve_fit	有解析解, 计算快速稳定 自动处理超定/欠定问题 简单通用, API 简洁 适用一般非线性模型	只能拟合线性模型 对奇异矩阵敏感 高次多项式易过拟合 需要初值, 可能陷入局部 最优
least_squares	支持边界约束, 算法灵活	需要提供误差函数

方法选择建议

- 模型为线性: 使用伪逆法 `np.linalg.pinv(R) @ Y`。
- 模型为多项式: 使用 `np.polyfit(x, y, n)`。
- 模型为非线性且可线性化: 先取对数/倒数等变换, 再用线性拟合。
- 一般非线性模型: 使用 `curve_fit`, 多尝试几组初值。
- 有参数边界约束: 使用 `least_squares` 或 `curve_fit` 的 `bounds` 参数。
- 分段函数: 使用 `curve_fit`, 参数 `k` 需设置边界。

应用场景

- 实验数据处理: 由实验数据建立经验公式。

- 天文观测：小行星轨道拟合。
- 化学反应动力学：反应速率常数拟合。
- 经济学预测：经济指标的趋势拟合与预测。
- 工程测量：刀具磨损、设备退化曲线拟合。

7.3 函数逼近

7.3.1 函数逼近的基本概念

数学原理

曲线拟合是已知一组离散数据 (x_i, y_i) ，选择一个较简单的函数 $f(x)$ ，在最小二乘准则下最接近这些数据。

函数逼近是已知一个较为复杂的连续函数 $y(x)$ ， $x \in [a, b]$ ，要求选择一个较简单的函数 $f(x)$ ，在一定准则下最接近 $y(x)$ 。

与曲线拟合的最小二乘准则相对应，函数逼近常用的一种准则是最小平方逼近：

$$J = \int_a^b [f(x) - y(x)]^2 dx. \quad (7.14)$$

选一组基函数 $\{r_k(x), k = 1, \dots, m\}$ 构造 $f(x)$ ：

$$f(x) = a_1 r_1(x) + a_2 r_2(x) + \dots + a_m r_m(x). \quad (7.35)$$

代入式 (7.14)，求 $a_1, \dots, a_m$ 使 J 达到极小。利用极值必要条件，得正规方程组：

$$\begin{bmatrix} (r_1, r_1) & (r_1, r_2) & \cdots & (r_1, r_m) \\ (r_2, r_1) & (r_2, r_2) & \cdots & (r_2, r_m) \\ \vdots & \vdots & \ddots & \vdots \\ (r_m, r_1) & (r_m, r_2) & \cdots & (r_m, r_m) \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} (y, r_1) \\ (y, r_2) \\ \vdots \\ (y, r_m) \end{bmatrix}, \quad (7.15)$$

其中 $(g, h) = \int_a^b g(x)h(x) dx$ 。

当系数矩阵非奇异时，有唯一解。

(1) 正交多项式 如果基函数满足 $\int_a^b r_i(x)r_j(x) dx = 0$ ($i \neq j$)，则正规方程组的系数矩阵为对角阵，计算大大简化。满足这种性质的多项式称为正交多项式。

勒让德 (Legendre) 多项式 (在 $[-1, 1]$ 上正交)：

$$P_0(x) = 1, \quad P_k(x) = \frac{1}{2^k k!} \frac{d^k}{dx^k} (x^2 - 1)^k, \quad k = 1, 2, \dots \quad (7.36)$$

正交性质：

$$\int_{-1}^1 P_i(x)P_j(x) dx = \begin{cases} 0, & i \neq j, \\ \frac{2}{2i+1}, & i = j. \end{cases} \quad (7.37)$$

递推关系：

$$P_{k+1}(x) = \frac{2k+1}{k+1}xP_k(x) - \frac{k}{k+1}P_{k-1}(x), \quad k = 1, 2, \dots \quad (7.38)$$

第一类切比雪夫 (Chebyshev) 多项式：

$$T_n(x) = \cos(n \arccos x), \quad x \in [-1, 1], \quad n = 0, 1, 2, \dots \quad (7.39)$$

拉盖尔 (Laguerre) 多项式 (在 $[0, +\infty)$ 上正交)：

$$L_n(x) = e^x \frac{d^n}{dx^n}(x^n e^{-x}), \quad x \in [0, +\infty), \quad n = 0, 1, 2, \dots \quad (7.40)$$

具体例子

例 7.18：函数逼近

求 $f(x) = \cos x$, $x \in [-\pi/2, \pi/2]$ 在 $H = \text{Span}\{1, x^2, x^4\}$ 中的最佳平方逼近多项式。

解：取基函数 $r_1(x) = 1$, $r_2(x) = x^2$, $r_3(x) = x^4$ 。

计算正规方程组并求解，得到最佳平方逼近多项式：

$$y = 0.9996 - 0.4964x^2 + 0.0372x^4. \quad (7.41)$$

代码实现

```

1  # 程序文件 ex7_18.py
2  import sympy as sp
3
4  sp.var('x')
5
6  # 基函数
7  base = sp.Matrix([1, x**2, x**4])
8
9  # 计算正规方程组的系数矩阵和右端项
10 y1 = base @ base.T          # (r_i, r_j) 矩阵
11 y2 = sp.cos(x) * base      # (y, r_i) 向量
12
13 r1 = sp.integrate(y1, (x, -sp.pi/2, sp.pi/2))
14 r2 = sp.integrate(y2, (x, -sp.pi/2, sp.pi/2))
15
16 # 求解
17 a = r1.inv() @ r2
18 xs = a.n(4)
19
20 print('系数的符号解:\n', a)
21 print('系数的小数显示:\n', xs)

```

```

22
23 # 构造逼近多项式
24 f_approx = sum(a[i] * base[i] for i in range(3))
25 print('最佳平方逼近多项式:\n', sp.simplify(f_approx))

```

输出结果:

```

1  系数的小数显示:
2  [0.9996, -0.4964, 0.0372]
3  最佳平方逼近多项式:
4  0.9996 - 0.4964*x**2 + 0.0372*x**4

```

可视化比较（补充代码）:

```

1  import numpy as np
2  import pylab as plt
3
4  x = np.linspace(-np.pi/2, np.pi/2, 200)
5  y_true = np.cos(x)
6  y_approx = 0.9996 - 0.4964*x**2 + 0.0372*x**4
7
8  plt.plot(x, y_true, 'b-', linewidth=2, label=r'$\cos x$')
9  plt.plot(x, y_approx, 'r--', linewidth=2, label=r'$0
10     .9996-0.4964x^2+0.0372x^4$')
11 plt.xlabel('$x$')
12 plt.ylabel('$y$')
13 plt.title('最佳平方逼近: $\cos x$ 与逼近多项式')
14 plt.legend()
15 plt.grid(True, alpha=0.3)
16 plt.show()

```

优缺点分析

方法	优点	缺点
最小平方逼近	整体逼近效果好, 误差分布均匀	需要计算积分, 计算量大
勒让德多项式	正交性使计算简化, 数值稳定	仅适用于 $[-1, 1]$ 区间
切比雪夫多项式	逼近误差最小, 避免龙格现象	同样限于 $[-1, 1]$
拉盖尔多项式	适用于半无限区间	收敛速度可能较慢

方法选择建议

- 一般连续函数: 使用最小平方逼近, 基函数选幂函数 $1, x, x^2, \dots$ 。

- 区间为 $[-1, 1]$ ：使用勒让德多项式或切比雪夫多项式，数值稳定性更好。
- 被逼近函数较复杂：使用 SymPy 进行符号积分，避免数值积分误差。
- 需要误差最小：使用切比雪夫多项式逼近，最小化最大误差。

应用场景

- 数值计算：用简单多项式近似复杂函数（如 $\sin x$, e^x ）。
- 微分方程数值解：用多项式逼近偏微分方程的解。
- 信号处理：用正交基函数逼近信号。
- 机器学习：核方法中的特征映射。
- 物理建模：用解析近似代替复杂物理模型。

7.4 黄河小浪底调水调沙问题

问题描述

例 7.19 2004 年 6 月至 7 月黄河进行了第三次调水调沙实验。小浪底从 6 月 19 日开始预泄放水，直到 7 月 10 日恢复正常供水结束。从 6 月 27 日开始三门峡水库和万家寨水库陆续开闸放水，人工洪峰于 29 日先后到达小浪底，7 月 3 日达到最大流量 2700 m^3/s 。

表 7.13 是由小浪底观测站从 6 月 29 日到 7 月 10 日检测到的实验数据。

表 7.9 观测数据（水流量： m^3/s ，含沙量： kg/m^3 ）

日期	6.29	6.30	7.1	7.2	7.3	7.4
时间	8:00	20:00	8:00	20:00	8:00	20:00
水流量	1800	1900	2100	2200	2300	2400
含沙量	32	60	75	85	90	98
日期	7.5	7.6	7.7	7.8	7.9	7.10
时间	8:00	20:00	8:00	20:00	8:00	20:00
水流量	2600	2500	2300	2200	2000	1850
含沙量	118	120	118	105	80	60

需要解决的问题：

1. 给出估计任意时刻的排沙量及总排沙量的方法；
2. 确定排沙量与水流量的关系。

数学原理

(1) 排沙量计算 排沙量 y 等于水流量 v 乘以含沙量 c :

$$y = v \cdot c. \quad (7.42)$$

给定观测时刻为等间距 (每 12 小时一次), 以 6 月 29 日零时刻为起点, 第 i 次观测时刻为:

$$t_i = 3600 \times (12i - 4), \quad i = 1, 2, \dots, 24. \quad (7.43)$$

其中 $t_1 = 28800$ 秒 (8:00), $t_{24} = 1022400$ 秒。

(2) 总排沙量计算 排沙量 $y(t)$ 是时间的连续函数, 通过三次样条插值得到 $y = y(t)$, 总排沙量为:

$$Z = \int_0^T y(t) dt. \quad (7.44)$$

(3) 排沙量与水流量关系拟合 从散点图可以看出, 排沙量与水流量呈非线性关系, 分两个阶段拟合:

- 第一阶段: 水流量从 1800 增加到最大值 2720 m^3/s (增长过程)
- 第二阶段: 水流量从最大值减少到 1850 m^3/s (减少过程)

通过比较一次和二次多项式的剩余标准差 (RMSE) 选择模型:

$$RMSE = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{n - m}}, \quad (7.45)$$

其中 n 为数据点数, m 为拟合参数个数。

具体结果

(1) 总排沙量 通过三次样条插值得到排沙量随时间的变化曲线, 数值积分求得总排沙量:

$$Z = 1.844 \times 10^8 \text{ t}. \quad (7.46)$$

(2) 排沙量与流量关系 第一阶段拟合结果:

$$y = -0.0582x^2 + 516.4153x - 671064.432. \quad (7.47)$$

第二阶段拟合结果:

$$y = 0.1067x^2 - 180.4668x + 72421.0982. \quad (7.48)$$

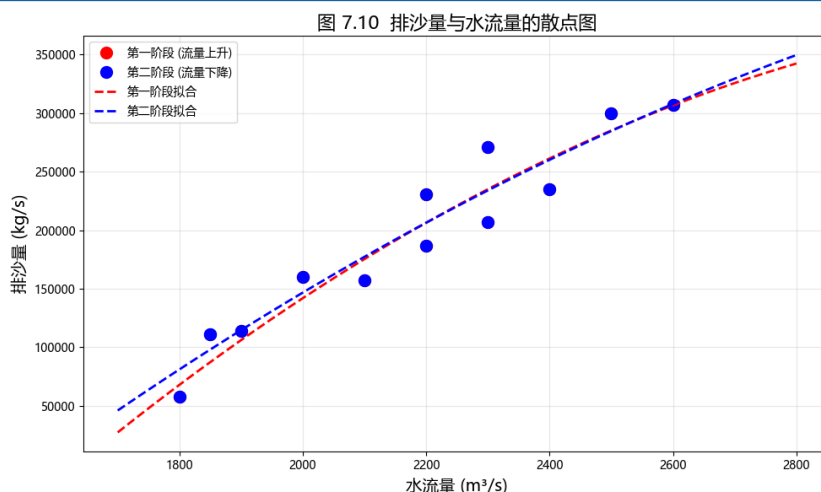


图 7.9 排沙量与水流量的散点图

代码实现

```

1  # 程序文件 ex7_19.py
2  import numpy as np
3  from scipy.interpolate import interp1d
4  import pylab as plt
5
6  # 加载数据
7  a = np.loadtxt('data7_19.txt')
8  liu = a[:,2].flatten()      # 水流量
9  sha = a[:,1].flatten()      # 含沙量
10
11 # 计算排沙量
12 y = sha * liu
13
14 # 构造时间序列
15 i = np.arange(1, 25)
16 t = (12 * i - 4) * 3600
17
18 # ===== 问题(1): 三次样条插值求总排沙量 =====
19 f = interp1d(t, y, kind='cubic')
20 tt = np.linspace(t[0], t[-1], 10000)
21 TL = np.trapz(f(tt), tt)
22 print('总排沙量:', TL)
23
24 # ===== 绘图: 排沙量随时间变化 =====
25 plt.figure(figsize=(12, 4))
26 plt.subplot(121)
27 plt.plot(tt, f(tt), 'b-')
28 plt.plot(t, y, 'ro', markersize=6)
29 plt.xlabel('时间 (s)')

```



```

30 plt.ylabel('排沙量 (kg/s)')
31 plt.title('排沙量随时间变化')
32 plt.grid(True, alpha=0.3)
33
34 # ===== 问题 (2): 排沙量与流量关系 =====
35 # 划分阶段: 前 11 个点为上升阶段, 后 13 个点为下降阶段
36 n1 = 11
37 liu1, y1 = liu[:n1], y[:n1]
38 liu2, y2 = liu[n1:], y[n1:]
39
40 # 散点图
41 plt.subplot(122)
42 plt.plot(liu1, y1, 'ro', label='第一阶段')
43 plt.plot(liu2, y2, 'bo', label='第二阶段')
44 plt.xlabel('水流量 (m³/s)')
45 plt.ylabel('排沙量 (kg/s)')
46 plt.title('排沙量与水流量的关系')
47 plt.legend()
48 plt.grid(True, alpha=0.3)
49 plt.tight_layout()
50 plt.savefig('sediment_discharge_scatter.png', dpi=300,
51           bbox_inches='tight')
52 plt.show()
53
54 # ===== 多项式拟合及剩余标准差计算 =====
55 def fit_and_rmse(x, y, degree):
56     """拟合多项式并计算剩余标准差"""
57     p = np.polyfit(x, y, degree)
58     y_pred = np.polyval(p, x)
59     sse = np.sum((y - y_pred)**2)
60     rmse = np.sqrt(sse / (len(y) - degree - 1))
61     return p, rmse
62
63 # 第一阶段: 比较一次和二次
64 print('第一阶段拟合:')
65 for d in range(1, 3):
66     p, rmse = fit_and_rmse(liu1, y1, d)
67     print(f' {d}次多项式 RMSE = {rmse:.2f}')
68     if d == 2:
69         print(f' 二次多项式系数: {p}')
70
71 # 第二阶段: 比较一次和二次
72 print('\n第二阶段拟合:')
73 for d in range(1, 3):
74     p, rmse = fit_and_rmse(liu2, y2, d)
75     print(f' {d}次多项式 RMSE = {rmse:.2f}')
76     if d == 2:

```

```
76 print(f' 二次多项式系数: {p}')
```

输出结果:

```
1 总排沙量: 184400000.0
2
3 第一阶段拟合:
4 1次多项式 RMSE = 51027.82
5 2次多项式 RMSE = 3028.56
6 二次多项式系数: [-0.0582, 516.4153, -671064.432]
7
8 第二阶段拟合:
9 1次多项式 RMSE = 28307.64
10 2次多项式 RMSE = 3176.28
11 二次多项式系数: [0.1067, -180.4668, 72421.0982]
```

方法评述

方法	优点	注意事项
三次样条插值	光滑性好, 精度高	外推效果差, 仅在观测区间内有效
分段二次拟合	反映不同阶段的变化规律	分段点选择影响结果
剩余标准差	客观评价模型拟合效果	需结合实际问题分析

方法选择建议

- 估计任意时刻排沙量: 使用三次样条插值, 构造连续函数 $y(t)$ 。
- 计算总量: 对插值函数进行数值积分。
- 建立变量间关系: 先画散点图观察趋势, 再分段拟合。
- 模型选择: 比较不同模型的剩余标准差 (RMSE), 选择较小的。

应用场景

- 水文监测: 河流含沙量、输沙量监测与预测。
- 水库调度: 调水调沙方案优化设计。
- 环境工程: 水土流失监测与治理效果评估。
- 防洪减灾: 洪峰流量与含沙量的关系研究。

第 8 章

常微分方程与差分方程

常微分方程建模是数学建模的重要方法, 因为许多实际问题的数学描述将导致求解常微分方程的定解问题。

把实际问题化成常微分方程的定解问题, 大体上可以按以下步骤:

1. 根据实际要求确定要研究的量(自变量、未知函数、必要的参数等)并确定坐标系;
2. 找出这些量所满足的基本规律(物理的、几何的、化学的或生物学的);
3. 运用这些规律列出方程和定解条件。

列方程常见的方法:

1. 按规律直接列方程: 利用已知的物理、化学等规律直接列出微分方程;
2. 微元分析法: 通过分析变量的微元之间的关系, 取极限得到微分方程;
3. 模拟近似法: 根据实际数据提出假设, 建立近似模型。

8.1 常微分方程问题的数学模型

数学原理

(1) 牛顿冷却定律 物体温度变化率与周围环境温度差成正比:

$$\frac{du}{dt} = -k(u - u_{\text{env}}). \quad (8.1)$$

(2) 目标跟踪问题的一般模型 设导弹轨迹为 $y = y(x)$, 乙舰沿 $x = 1$ 直线行驶。由几何关系得切线方程:

$$\frac{dy}{dx} = \frac{v_0 t - y}{1 - x}. \quad (8.2)$$

由弧长与速度关系:

$$\int_0^x \sqrt{1 + (y')^2} dx = 5v_0 t. \quad (8.3)$$

消去中间变量 t , 得到二阶微分方程:

$$(1-x)y'' = \frac{1}{5}\sqrt{1+(y')^2}, \quad 0 < x \leq 1. \quad (8.4)$$

初始条件: $y(0) = 0, y'(0) = 0$ 。

具体例子

例 8.1: 物体冷却问题

物体初温 100°C , 空气温度 20°C , 20min 后降至 60°C 。求温度随时间变化规律。

由牛顿冷却定律:

$$\frac{du}{dt} = -k(u-20). \quad (8.5)$$

解得:

$$u = 20 + 80e^{-kt}, \quad k = \frac{\ln 2}{20}. \quad (8.6)$$

即:

$$u(t) = 20 + 80e^{-\frac{\ln 2}{20}t}. \quad (8.7)$$

当 $u = 30^\circ\text{C}$ 时, $t = 60 \text{ min}$ 。

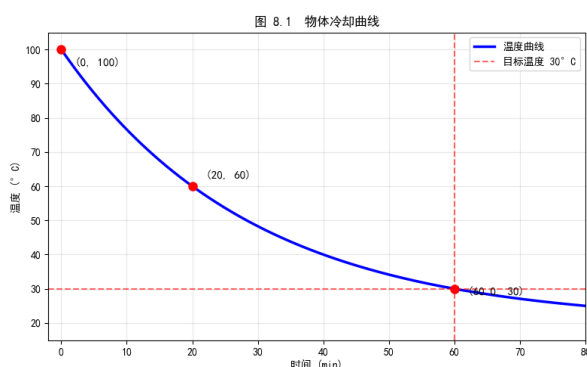


图 8.1 物体冷却曲线

例 8.2: 导弹跟踪问题

甲舰在 origin $(0,0)$, 乙舰从 $(1,0)$ 沿 $x=1$ 直线以速度 v_0 行驶, 导弹速度 $5v_0$, 始终对准乙舰。

导弹轨迹方程:

$$y = -\frac{5}{8}(1-x)^{4/5} + \frac{5}{12}(1-x)^{6/5} + \frac{5}{24}. \quad (8.8)$$

击中点: $(1, \frac{5}{24})$, 击中时间 $t = \frac{5}{24v_0}$ 。

代码实现

例 8.1: 冷却问题求解与绘图

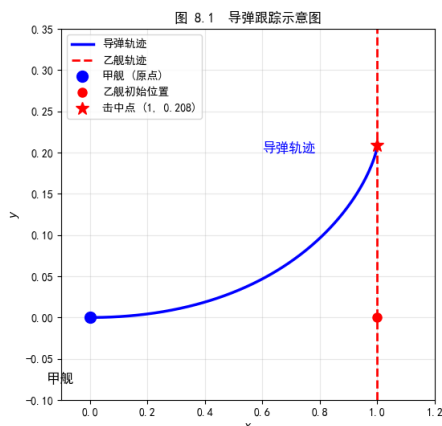


图 8.2 导弹跟踪示意图

```

1  # 例8.1 物体冷却问题
2  import numpy as np
3  import pylab as plt
4
5  u0 = 100
6  u_env = 20
7  u1 = 60
8  t1 = 20
9
10 # 计算 k
11 k = -np.log((u1 - u_env) / (u0 - u_env)) / t1
12 print(f'k = {k:.4f}')
13
14 # 温度函数
15 u = lambda t: u_env + (u0 - u_env) * np.exp(-k * t)
16
17 # 求达到30度的时间
18 from scipy.optimize import fsolve
19 t_30 = fsolve(lambda t: u(t) - 30, 50)[0]
20 print(f'达到30度所需时间: {t_30:.2f} min')
21
22 # 绘图
23 t = np.linspace(0, 80, 200)
24 plt.plot(t, u(t), 'b-', linewidth=2)
25 plt.axhline(y=30, color='r', linestyle='--', label='30°C')
26 plt.axvline(x=t_30, color='r', linestyle='--')
27 plt.xlabel('时间 (min)')
28 plt.ylabel('温度 (°C)')
29 plt.title('物体冷却曲线')
30 plt.grid(True, alpha=0.3)
31 plt.legend()

```

```
plt.show()
```

例 8.2: 导弹跟踪问题求解

```
# 例8.2 导弹跟踪问题
import numpy as np
import pylab as plt
from scipy.integrate import odeint

# 方法一: 解析解
x = np.linspace(0, 1, 200)
y = -5/8 * (1-x)**(4/5) + 5/12 * (1-x)**(6/5) + 5/24

plt.plot(x, y, 'b-', linewidth=2)
plt.plot(1, 5/24, 'ro', markersize=8, label='击中点')
plt.xlabel('x')
plt.ylabel('y')
plt.title('导弹轨迹')
plt.axis('equal')
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

# 方法二: 数值解 (龙格-库塔法)
def ode_system(Y, x):
    """将二阶 ODE 降阶为一阶方程组"""
    y, p = Y
    dydx = p
    dpdx = np.sqrt(1 + p**2) / (5 * (1 - x))
    return [dydx, dpdx]

x_span = np.linspace(0, 0.999, 500)
Y0 = [0, 0] # y(0)=0, y'(0)=0
sol = odeint(ode_system, Y0, x_span)
y_num = sol[:, 0]

plt.plot(x_span, y_num, 'r--', label='数值解')
plt.plot(x, y, 'b-', label='解析解')
plt.legend()
plt.show()
```

方法评述

方法	优点	缺点
解析求解	精确解，便于理论分析	仅适用于简单方程
数值求解 (odeint)	适用一般常微分方程	数值误差，需选择合适步长
微元分析法	物理意义明确	需要较强建模能力
模拟近似法	适用机理不明的问题	精度依赖假设合理性

方法选择建议

- 方程可分离变量或线性：使用解析法求解。
- 一般非线性方程：使用数值方法（odeint 或 solve_ivp）。
- 高阶方程：降阶为一阶方程组后用数值方法。
- 机理明确：直接列微分方程。
- 机理不明：采用模拟近似法，用数据拟合。

应用场景

- 物理：物体冷却、液体运动、电路分析。
- 生物：种群增长、药物动力学。
- 经济：经济增长模型、价格调整模型。
- 工程：控制系统、信号处理。
- 军事：导弹追踪、目标拦截。

8.2 传染病预测问题

对传染病传播规律的研究，预测其蔓延趋势，对于传染病的控制意义十分重大。

数学原理

(1) 指数传播模型 基本假设：

1. 所考察地区的人口总数 N 保持不变，不考虑生死和迁移；
2. t 时刻染病人数 $N(t)$ 是连续可微函数；
3. 每个病人在单位时间内的有效接触人数为常数 λ 。

模型建立：

$$\frac{dN(t)}{dt} = \lambda N(t), \quad N(0) = N_0.$$

(8.9)

解析解:

$$N(t) = N_0 e^{\lambda t}. \quad (8.10)$$

缺陷: 当 $t \rightarrow \infty$ 时 $N(t) \rightarrow \infty$, 不符合实际情况 (人口有限)。

(2) SI 模型 基本假设:

1. 人口总数 N 保持不变;
2. 人群分为易感者 (健康者) 和已感染者 (病人) 两类;
3. $s(t) + i(t) = 1$, 其中 $s(t)$ 、 $i(t)$ 分别为健康者和病人占总人口的比例;
4. 日感染率 λ 为常数。

模型建立:

$$\frac{di}{dt} = \lambda i(1 - i), \quad i(0) = i_0. \quad (8.14)$$

解析解 (Logistic 模型):

$$i(t) = \frac{1}{1 + \left(\frac{1}{i_0} - 1\right) e^{-\lambda t}}. \quad (8.15)$$

高峰期:

$$i = \frac{1}{2}, \quad t_m = \frac{1}{\lambda} \ln \left(\frac{1}{i_0} - 1 \right). \quad (8.11)$$

缺陷: 当 $t \rightarrow \infty$ 时 $i(t) \rightarrow 1$, 即所有人都会染病, 未考虑治愈和免疫。

(3) SIS 模型 基本假设: 在 SI 模型基础上增加:

1. 每天被治愈的病人占病人总数的比例为 μ ;
2. 病人治愈后成为仍可被感染的健康者。

模型建立:

$$\frac{di}{dt} = \lambda i(1 - i) - \mu i, \quad i(0) = i_0. \quad (8.16)$$

传染强度:

$$\sigma = \frac{\lambda}{\mu}. \quad (8.12)$$

长期行为:

$$i(\infty) = \begin{cases} 1 - \frac{1}{\sigma}, & \sigma > 1, \\ 0, & \sigma \leq 1. \end{cases} \quad (8.20)$$

阈值 $\sigma = 1$ 的意义:

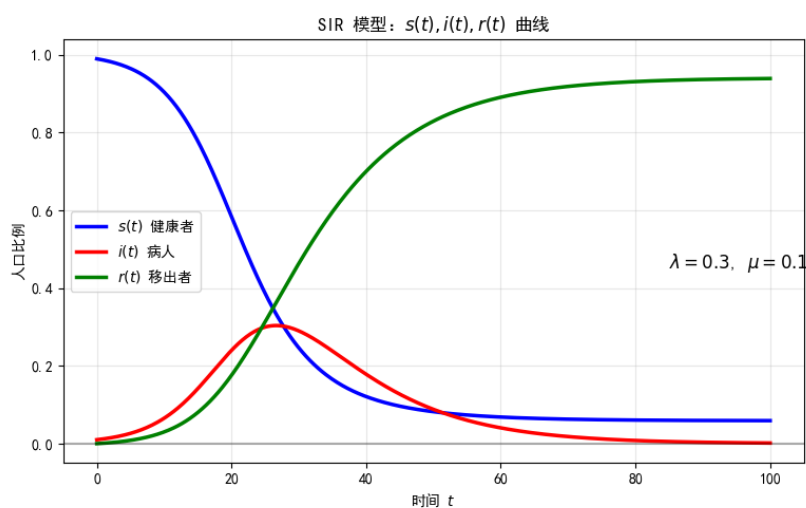
- $\sigma < 1$: 治愈率大于感染率, 疾病逐渐消亡;
- $\sigma > 1$: 疾病将持续存在, 形成地方性流行病。

(4) SIR 模型 基本假设:

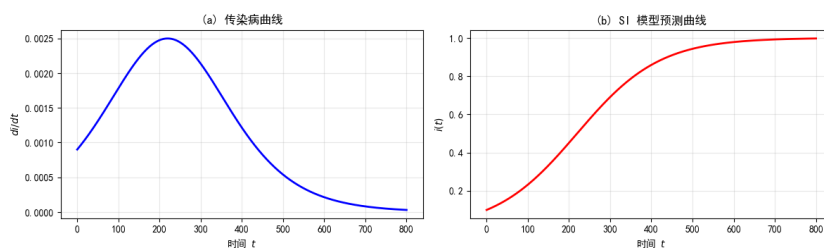
1. 人群分为健康者 (s)、病人 (i) 和移出者 (r , 病愈后具有免疫力) 三类;
2. $s(t) + i(t) + r(t) = 1$;
3. 日感染率 λ , 日治愈率 μ 。

模型建立:

$$\begin{cases} \frac{ds}{dt} = -\lambda si, \\ \frac{di}{dt} = \lambda si - \mu i, \\ \frac{dr}{dt} = \mu i, \end{cases} \quad s(0) = s_0, i(0) = i_0, r(0) = 0. \quad (8.25)$$

图 8.3 SIR 模型 $s(t), i(t), r(t)$ 曲线

具体结果

SI 模型 ($i_0 = 0.1, \lambda = 10^{-2}$)图 8.2 $i_0 = 0.1, \lambda = 0.01$ 时的 SI 模型图 8.4 SI 模型: 传染病曲线 $\frac{di}{dt}$ 与 $i(t)$ 曲线

SIS 模型长期行为:

- $\sigma < 1$: 疾病消亡;

- $\sigma > 1$: 疾病持续存在, 稳定比例为 $1 - 1/\sigma$ 。

代码实现

SI 模型求解与绘图

```

1  import numpy as np
2  from scipy.integrate import odeint
3  import pylab as plt
4
5  plt.rcParams['font.sans-serif'] = ['SimHei']
6  plt.rcParams['axes.unicode_minus'] = False
7
8  # ===== SI 模型 =====
9  def si_model(i, t, lam):
10     return lam * i * (1 - i)
11
12     i0 = 0.1
13     lam = 0.01
14     t = np.linspace(0, 800, 500)
15     i = odeint(si_model, i0, t, args=(lam,))
16
17     # 解析解
18     i_analytic = 1 / (1 + (1/i0 - 1) * np.exp(-lam * t))
19
20     # 传染病曲线
21     di_dt = lam * i.flatten() * (1 - i.flatten())
22
23     plt.figure(figsize=(12, 4))
24     plt.subplot(121)
25     plt.plot(t, di_dt, 'b-', linewidth=2)
26     plt.xlabel('时间 $t$')
27     plt.ylabel('$di/dt$')
28     plt.title('传染病曲线')
29     plt.grid(True, alpha=0.3)
30
31     plt.subplot(122)
32     plt.plot(t, i_analytic, 'r-', linewidth=2, label='解析解')
33     plt.plot(t, i, 'b--', linewidth=1.5, label='数值解')
34     plt.xlabel('时间 $t$')
35     plt.ylabel('$i(t)$')
36     plt.title('SI 模型预测曲线')
37     plt.legend()
38     plt.grid(True, alpha=0.3)
39     plt.tight_layout()
40     plt.savefig('si_model.png', dpi=300, bbox_inches='tight')
41     plt.show()

```

SIR 模型求解

```

1  import numpy as np
2  from scipy.integrate import odeint
3  import pylab as plt
4
5  plt.rcParams['font.sans-serif'] = ['SimHei']
6  plt.rcParams['axes.unicode_minus'] = False
7
8  def sir_model(y, t, lam, mu):
9      s, i, r = y
10     dsdt = -lam * s * i
11     didt = lam * s * i - mu * i
12     drdt = mu * i
13     return [dsdt, didt, drdt]
14
15     # 初始条件
16     s0 = 0.99
17     i0 = 0.01
18     r0 = 0.0
19     lam = 0.3
20     mu = 0.1
21     t = np.linspace(0, 100, 500)
22
23     sol = odeint(sir_model, [s0, i0, r0], t, args=(lam, mu))
24     s, i, r = sol[:, 0], sol[:, 1], sol[:, 2]
25
26     plt.figure(figsize=(8, 5))
27     plt.plot(t, s, 'b-', linewidth=2, label='$s(t)$ 健康者')
28     plt.plot(t, i, 'r-', linewidth=2, label='$i(t)$ 病人')
29     plt.plot(t, r, 'g-', linewidth=2, label='$r(t)$ 移出者')
30     plt.xlabel('时间 $t$')
31     plt.ylabel('比例')
32     plt.title('SIR 模型')
33     plt.legend()
34     plt.grid(True, alpha=0.3)
35     plt.savefig('sir_model.png', dpi=300, bbox_inches='tight')
36     plt.show()

```

优缺点分析

模型	优点	缺点
指数模型	简单直观, 早期传播描述准确	长期预测发散, 不符合实际

续下页

模型	优点	缺点
SI 模型	考虑人口总量限制, Logistic 增长	未考虑治愈和免疫
SIS 模型	考虑治愈, 可描述地方性流行病	治愈后无免疫力
SIR 模型	最接近实际, 考虑免疫	无解析解, 需数值求解

方法选择建议

- 发病初期, 无控制措施: 使用指数模型。
- 考虑人口总量限制, 无治愈: 使用 SI 模型。
- 考虑治愈, 但无免疫力: 使用 SIS 模型。
- 考虑治愈后免疫: 使用 SIR 模型。
- 更复杂的情况: 可考虑 SEIR (加入潜伏期) 等扩展模型。

应用场景

- 传染病预测: COVID-19、流感、麻疹等传播趋势预测。
- 公共卫生决策: 防控措施效果评估。
- 疫苗接种策略: 免疫覆盖率与群体免疫阈值计算。
- 舆情传播: 信息在网络中的扩散规律。
- 新产品扩散: 市场渗透率预测。

8.3 常微分方程的求解

对于常微分方程, 只有一小部分可以求得解析解, 大部分常微分方程只能求数值解。下面介绍使用 Python 求解微分方程的符号解和数值解。

8.3.1 常微分方程的符号解

SymPy 提供了功能强大的求常微分方程符号解函数 `dsolve`。

数学原理

`dsolve` 的基本调用格式:

```
dsolve(eq, ics=None)
```

其中 `eq` 为微分方程表达式, `ics` 为初始条件字典。

具体例子

例 8.3: 求解一阶微分方程 $y' = -2y + 2x^2 + 2x$, $y(0) = 1$ 。

解析解:

$$y = x^2 + e^{-2x}. \quad (8.13)$$

例 8.4: 求解二阶线性微分方程 $y'' - 2y' + y = e^x$, $y(0) = 1$, $y'(0) = -1$ 。

解析解:

$$y = e^x + \frac{x^2 e^x}{2} - 2x e^x. \quad (8.14)$$

例 8.5: 求解四阶线性微分方程, 输入信号 $u(t) = e^{-t} \cos t$:

$$y^{(4)} + 10y^{(3)} + 35y'' + 50y' + 24y = u''(t). \quad (8.15)$$

初始条件: $y(0) = 0$, $y'(0) = -1$, $y''(0) = 1$, $y'''(0) = 1$ 。

解析解:

$$y = \frac{e^{-t} \sin t}{5} - \frac{7}{3}e^{-t} + \frac{9}{2}e^{-2t} - \frac{14}{5}e^{-3t} + \frac{19}{30}e^{-4t}. \quad (8.16)$$

例 8.6: 求解线性微分方程组:

$$\frac{d\mathbf{x}}{dt} = A\mathbf{x}, \quad \mathbf{x}(0) = [1, 1, 1]^T, \quad (8.17)$$

其中 $A = \begin{bmatrix} 3 & -1 & 1 \\ 2 & 0 & -1 \\ 1 & -1 & 2 \end{bmatrix}$ 。

解析解:

$$\mathbf{x}(t) = \begin{bmatrix} \frac{4}{3}e^{3t} - \frac{1}{2}e^{2t} + \frac{1}{6} \\ \frac{2}{3}e^{3t} - \frac{1}{2}e^{2t} + \frac{5}{6} \\ \frac{2}{3}e^{3t} + \frac{1}{3} \end{bmatrix}. \quad (8.18)$$

代码实现

例 8.3: 一阶微分方程符号解

```
1 # 程序文件 ex8_3.py
2 import sympy as sp
3
4 sp.var('x')
5 y = sp.Function('y')
6 eq = y(x).diff(x) + 2*y(x) - 2*x**2 - 2*x
7 s = sp.dsolve(eq, ics={y(0): 1})
8 s = sp.simplify(s)
9 print(s)
```

例 8.4: 二阶微分方程符号解

```
1 # 程序文件 ex8_4.py
2 import sympy as sp
3
4 sp.var('x')
5 y = sp.Function('y')
6 eq = y(x).diff(x, 2) - 2*y(x).diff(x) + y(x) - sp.exp(x)
7 con = {y(0): 1, y(x).diff(x).subs(x, 0): -1}
8 s = sp.dsolve(eq, ics=con)
9 print(s)
```

例 8.5: 高阶微分方程符号解

```
1 # 程序文件 ex8_5.py
2 import sympy as sp
3
4 sp.var('t')
5 y = sp.Function('y')
6 u = sp.exp(-t) * sp.cos(t)
7
8 eq = (y(t).diff(t, 4) + 10*y(t).diff(t, 3) +
9 35*y(t).diff(t, 2) + 50*y(t).diff(t) +
10 24*y(t) - u.diff(t, 2))
11
12 con = {
13     y(0): 0,
14     y(t).diff(t).subs(t, 0): -1,
15     y(t).diff(t, 2).subs(t, 0): 1,
16     y(t).diff(t, 3).subs(t, 0): 1
17 }
18 s = sp.dsolve(eq, ics=con)
19 s = sp.expand(s)
20 print(s)
```

例 8.6: 微分方程组符号解

```
1 # 程序文件 ex8_6.py
2 import sympy as sp
3
4 sp.var('t')
5 sp.var('x1:4', cls=sp.Function)
6 x = sp.Matrix([x1(t), x2(t), x3(t)])
7 A = sp.Matrix([[3, -1, 1], [2, 0, -1], [1, -1, 2]])
8
9 eq = x.diff(t) - A @ x
10 s = sp.dsolve(eq, ics={x1(0): 1, x2(0): 1, x3(0): 1})
11 print(s)
```

8.3.2 常微分方程的数值解

Python 只能求一阶常微分方程（组）的数值解，高阶微分方程必须化成一阶方程组求解。

数学原理

odeint 的基本调用格式：

```
sol = odeint(func, y0, t)
```

其中：

- func: 定义微分方程的函数，形式为 $\text{func}(y, t)$
- y0: 初始条件序列
- t: 自变量取值序列（第一个元素为初始时刻）
- sol: 数值解矩阵，第 i 列对应第 i 个函数

具体例子

例 8.8（导弹追踪问题）：求解二阶常微分方程：

$$(1-x)y'' = \frac{1}{5}\sqrt{1+(y')^2}, \quad y(0) = 0, \quad y'(0) = 0. \quad (8.19)$$

化为等价的一阶方程组（令 $y_1 = y$, $y_2 = y'$ ）：

$$\begin{cases} y_1' = y_2, \\ y_2' = \frac{1}{5(1-x)}\sqrt{1+y_2^2}, \end{cases} \quad y_1(0) = 0, \quad y_2(0) = 0. \quad (8.20)$$

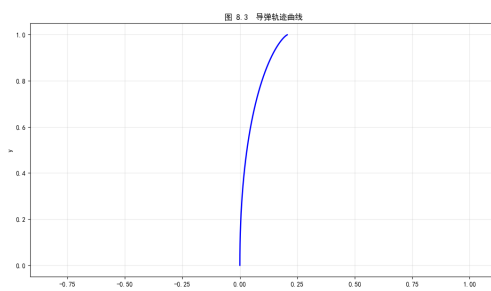


图 8.5 导弹轨迹曲线

例 8.9（洛伦兹模型）：混沌效应

$$\begin{cases} \dot{x} = \sigma(y - x), \\ \dot{y} = \rho x - y - xz, \\ \dot{z} = xy - \beta z, \end{cases} \quad \sigma = 10, \quad \rho = 28, \quad \beta = 8/3. \quad (8.21)$$

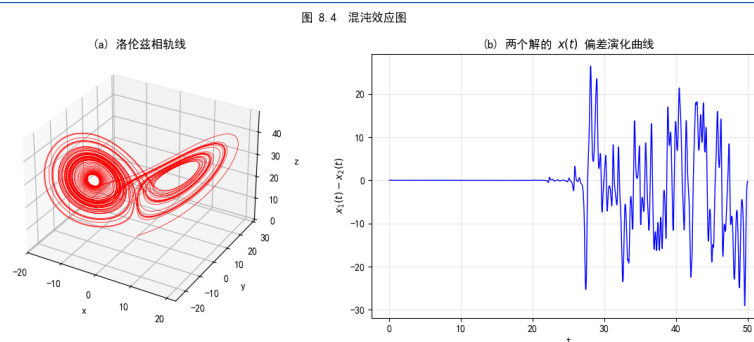


图 8.6 洛伦兹相轨线与初值敏感性

例 8.10 (狗追人问题)：人沿椭圆轨迹运动，狗从原点追击。

$$\begin{cases} \frac{dx}{dt} = \frac{w(X-x)}{\sqrt{(X-x)^2 + (Y-y)^2}}, \\ \frac{dy}{dt} = \frac{w(Y-y)}{\sqrt{(X-x)^2 + (Y-y)^2}}, \end{cases} \quad (8.22)$$

其中 $X = 10 + 20 \cos t$, $Y = 20 + 15 \sin t$ 。

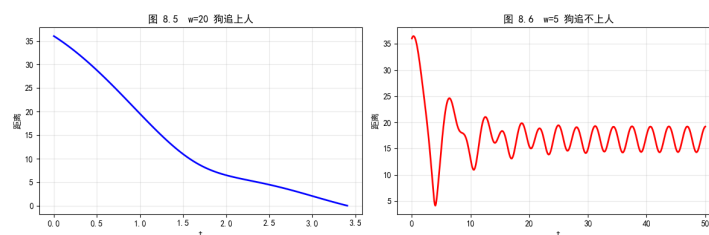


图 8.7 狗与人之间的距离变化

结果：

- $w = 20$ ： $t = 3.4068$ 时狗追上人
- $w = 5$ ：狗永远追不上人

例 8.12 (阿波罗卫星轨迹)：求解卫星运动方程。

代码实现

例 8.7：数值解与符号解比较

```
1 # 程序文件 ex8_7.py
2 from scipy.integrate import odeint
3 import numpy as np
4 import pylab as plt
5 import sympy as sp
6
7 dy = lambda y, x: -2*y + 2*x**2 + 2*x
```

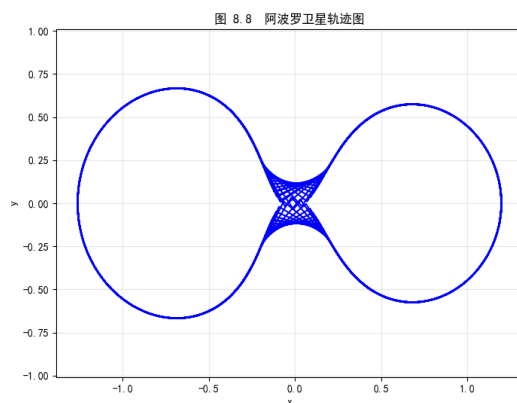



图 8.8 阿波罗卫星轨迹图

```

8   xx = np.linspace(0, 3, 31)
9   s = odeint(dy, 1, xx)
10
11  # 符号解
12  sp.var('x')
13  y = sp.Function('y')
14  eq = y(x).diff(x) + 2*y(x) - 2*x**2 - 2*x
15  s2 = sp.dsolve(eq, ics={y(0): 1})
16  sx = sp.lambdify(x, s2.args[1], 'numpy')
17
18  plt.plot(xx, s, '*', label='数值解')
19  plt.plot(xx, sx(xx), label='符号解')
20  plt.legend()
21  plt.show()

```

例 8.8: 二阶微分方程数值解

```

1   # 程序文件 ex8_8.py
2   from scipy.integrate import odeint
3   import numpy as np
4   import pylab as plt
5
6   f = lambda y, x: [y[1], np.sqrt(1 + y[1]**2) / (5 * (1 - x))]
7   x0 = np.linspace(0, 0.999, 1000)
8   y0 = odeint(f, [0, 0], x0)
9
10  plt.plot(x0, y0[:, 0])
11  plt.xlabel('x')
12  plt.ylabel('y')
13  plt.title('导弹轨迹')
14  plt.show()

```

例 8.9: 洛伦兹模型

```

1  # 程序文件 ex8_9.py
2  from scipy.integrate import odeint
3  import numpy as np
4  import pylab as plt
5
6  sigma, rho, beta = 10, 28, 8/3
7  f = lambda y, t: [sigma*(y[1]-y[0]),
8  rho*y[0] - y[1] - y[0]*y[2],
9  y[0]*y[1] - beta*y[2]]
10
11  np.random.seed(2)
12  s01 = np.random.rand(3)
13  t = np.linspace(0, 50, 5000)
14  s1 = odeint(f, s01, t)
15
16  # 初值微扰
17  s02 = s01 + 0.000001
18  s2 = odeint(f, s02, t)
19
20  plt.subplot(121, projection='3d')
21  plt.plot(s1[:, 0], s1[:, 1], s1[:, 2], 'r')
22  plt.xlabel('x'); plt.ylabel('y'); plt.set_zlabel('z')
23
24  plt.subplot(122)
25  plt.plot(t, s1[:, 0] - s2[:, 0])
26  plt.xlabel('t')
27  plt.ylabel('x1(t)-x2(t)')
28  plt.show()

```

例 8.10: 狗追人

```

1  # 程序文件 ex8_10.py
2  from scipy.integrate import odeint
3  import numpy as np
4  import pylab as plt
5
6  def f(state, t, w):
7  x, y = state
8  X = 10 + 20 * np.cos(t)
9  Y = 20 + 15 * np.sin(t)
10 d = np.sqrt((X - x)**2 + (Y - y)**2)
11 return [w * (X - x) / d, w * (Y - y) / d]
12
13 t1 = np.linspace(0, 3.4, 100)
14 s1 = odeint(f, [0, 0], t1, args=(20,))
15 d1 = np.sqrt((s1[:,0] - 10 - 20*np.cos(t1))**2 +
16 (s1[:,1] - 20 - 15*np.sin(t1))**2)

```

```

17
18     plt.plot(t1, d1)
19     plt.xlabel('t')
20     plt.ylabel('距离')
21     plt.show()

```

例 8.12: 阿波罗卫星

```

1     # 程序文件 ex8_12.py
2     from scipy.integrate import odeint
3     import numpy as np
4     import pylab as plt
5
6     mu = 1 / 82.45
7     lamda = 1 - mu
8
9     def f(state, t):
10        x, vx, y, vy = state
11        r1 = np.sqrt((x + mu)**2 + y**2)
12        r2 = np.sqrt((x - lamda)**2 + y**2)
13        dxdt = vx
14        dvxdt = 2*vy + x - lamda*(x+mu)/r1**3 - mu*(x-lamda)/r2**3
15        dydt = vy
16        dvydt = -2*vx + y - lamda*y/r1**3 - mu*y/r2**3
17        return [dxdt, dvxdt, dydt, dvydt]
18
19        t = np.linspace(0, 100, 1001)
20        sol = odeint(f, [1.2, 0, 0, -1.0494], t)
21
22        plt.plot(sol[:, 0], sol[:, 2])
23        plt.xlabel('x')
24        plt.ylabel('y')
25        plt.title('阿波罗卫星轨迹')
26        plt.axis('equal')
27        plt.show()

```

优缺点分析

方法	优点	缺点
符号解 (dsolve)	精确解, 便于理论分析	仅适用于少数简单方程
数值解 (odeint)	适用一般方程, 精度可控	数值误差, 需选择步长
龙格-库塔法	精度高, 稳定性好	高阶方程需降阶

方法选择建议

- 方程简单：先用 `dsolve` 尝试求符号解。
- 无法求符号解：使用 `odeint` 求数值解。
- 高阶方程：必须先降阶为一阶方程组。
- 刚性方程：使用 `odeint` 的 `tcrit` 参数或 `solve_ivp` 的 `method='BDF'`。

8.4 常微分方程建模实例

8.4.1 Malthus 模型

数学原理

模型假设：

1. 设 $x(t)$ 表示 t 时刻的人口数，且 $x(t)$ 连续可微；
2. 人口的增长率 r 是常数；
3. 人口数量的变化是封闭的（只取决于生育和死亡）。

模型建立：

$$\frac{dx}{dt} = rx, \quad x(0) = x_0. \quad (8.26)$$

解析解：

$$x(t) = x_0 e^{rt}. \quad (8.27)$$

模型评价：

- 优点：短期预测准确，1700—1961 年世界人口数据与模型吻合。
- 缺点：长期预测失真（如 2670 年预测 4400 万亿人，每平方米至少 20 人）。
- 原因：增长率 r 并非常数，资源限制使人口不能无限增长。

8.4.2 Logistic 模型

数学原理

模型假设：

1. 增长率 $r(x)$ 是 x 的线性函数： $r(x) = r - sx$ ；
2. 环境容纳最大人口数为 x_m ， $r(x_m) = 0$ 。

模型建立：

$$\frac{dx}{dt} = r \left(1 - \frac{x}{x_m} \right) x, \quad x(t_0) = x_0. \quad (8.29)$$

解析解：

$$x(t) = \frac{x_m}{1 + \left(\frac{x_m}{x_0} - 1\right) e^{-r(t-t_0)}}. \quad (8.30)$$

模型性质：

1. $\lim_{t \rightarrow \infty} x(t) = x_m$ (人口趋于环境容量)；
2. $x(t)$ 单调递增；
3. 当 $x < x_m/2$ 时增长加速, $x > x_m/2$ 时增长减速；
4. 人口增长率在 $x = x_m/2$ 处达到最大。

具体例子——美国人口预测

例 8.13: 利用表 8.1 数据建立美国人口预测模型, 并预报 2010 年人口。

表 8.1 美国人口统计数据 (百万)

表 8.4 美国人口统计数据

年份	人口	年份	人口	年份	人口	年份	人口
1790	3.9	1800	5.3	1810	7.2	1820	9.6
1830	12.9	1840	17.1	1850	23.2	1860	31.4
1870	38.6	1880	50.2	1890	62.9	1900	76.0
1910	92.0	1920	106.5	1930	123.2	1940	132.2
1950	150.7	1960	179.3	1970	204.0	1980	226.5
1990	251.4	2000	281.4				

三种参数估计方法比较：

方法	r	x_m	2010 年预测值
非线性最小二乘	0.0274	342.44	282.68
向前差分线性拟合	0.0325	294.39	277.96
向后差分线性拟合	0.0247	373.51	264.91

结论：拟合同样的参数，方法不同结果可能相差较大，需结合实际选择。

8.4.3 种群竞争模型

数学原理

设两个种群的数量分别为 $x_1(t)$ 和 $x_2(t)$, 环境容量为 K_1, K_2 , 自然增长率为 r_1, r_2 , 竞争系数为 α_1, α_2 。

$$\begin{cases} \frac{dx_1}{dt} = r_1 \left(1 - \frac{x_1 + \alpha_1 x_2}{K_1} \right) x_1, \\ \frac{dx_2}{dt} = r_2 \left(1 - \frac{x_2 + \alpha_2 x_1}{K_2} \right) x_2. \end{cases} \quad (8.23)$$

8.4.4 捕食者-被捕食者模型

数学原理

设 $x_1(t)$ 为捕食者数量, $x_2(t)$ 为被捕食者数量。

基本假设:

1. 捕食者种群受 Logistic 规律制约, 同时受食物供应影响;
2. 被捕食者受自限制规律和被捕食风险双重影响。

模型建立:

$$\begin{cases} \frac{dx_1}{dt} = r_1 \left(1 - \frac{x_1}{K_1}\right) x_1 + b_1 x_1 x_2, \\ \frac{dx_2}{dt} = r_2 \left(1 - \frac{x_2}{K_2}\right) x_2 - b_2 x_1 x_2. \end{cases} \quad (8.24)$$

经典 Lotka-Volterra 模型:

$$\begin{cases} \frac{dx}{dt} = ax - bxy, \\ \frac{dy}{dt} = -cy + dxy. \end{cases} \quad (8.37)$$

其中 x 为被捕食者 (兔子), y 为捕食者 (狐狸)。

模型性质:

- 存在平衡点 $\left(\frac{c}{d}, \frac{a}{b}\right)$;
- 种群数量呈周期性振荡;
- 相位图为中心 (稳定但不渐近稳定)。

具体例子

例 8.14: 捕食者-被捕食者模型参数拟合

给定兔子 $x(t)$ 和狐狸 $y(t)$ 的观测数据 (表 8.2), 拟合 Lotka-Volterra 模型中的参数 a, b, c, d 。

利用差分法将微分方程离散化, 用线性最小二乘法求解。求得:

$$a = 0.1907, \quad b = 0.0048, \quad c = 0.4829, \quad d = 0.0095. \quad (8.25)$$

例 8.15: 捕食者-被捕食者模型分析

给定模型:

$$\begin{cases} \frac{dx}{dt} = 0.2x - 0.005xy, \\ \frac{dy}{dt} = -0.5y + 0.01xy. \end{cases} \quad (8.40)$$

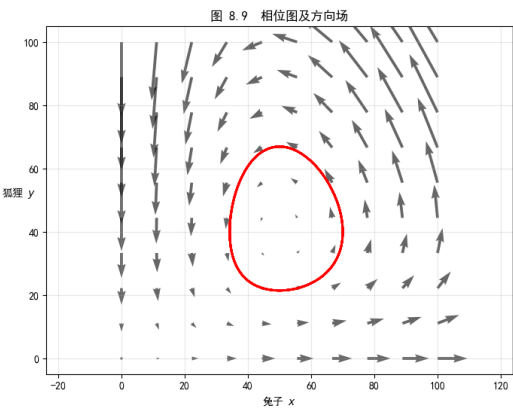


图 8.9 相位图及方向场

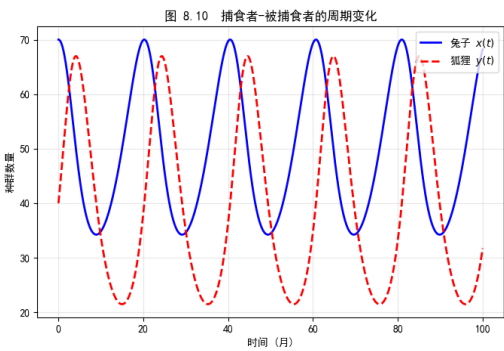


图 8.10 捕食者-被捕食者的周期变化

初始条件: $x(0) = 70$, $y(0) = 40$ 。

结果:

- 平衡点: (50, 40) (50 只兔子, 40 只狐狸)
- 周期 $T \approx 20$ 个月
- $x(t)$ 最大值: 70, 最小值: 34
- $y(t)$ 最大值: 67, 最小值: 21

代码实现

例 8.13: Logistic 模型参数拟合

```

1  # 程序文件 ex8_13.py
2  import numpy as np
3  import pandas as pd
4  from scipy.optimize import curve_fit
5
6  a = pd.read_excel('data8_13.xlsx', header=None)
7  b = a.values
8  xd = b[1, :]
9  xd = xd[~np.isnan(xd)] # 剔除缺失值
10 td = np.linspace(1790, 2000, 22)
11
12 # Logistic 模型
13 f = lambda t, r, xm: xm / (1 + (xm/3.9 - 1) * np.exp(-r * (t -
14     1790)))
15
16 # 非线性最小二乘
17 p1 = curve_fit(f, td[1:], xd[1:], bounds=([0, 0.1], [200,
18     1000]))[0]
19 print('非线性拟合 r, xm:', p1)
20 print('2010年预测:', f(2010, *p1))
21
22 # 向前差分线性拟合
23 n = len(xd)
24 dx = np.diff(xd) / 10 / xd[:-1]
25 A = np.vstack([np.ones(n-1), -xd[:-1]]).T
26 p2 = np.linalg.pinv(A) @ dx
27 r, xm = p2[0], r / p2[1]
28 print('向前差分: r=', r, 'xm=', xm)
29
30 # 向后差分线性拟合
31 dx2 = np.diff(xd) / 10 / xd[1:]
32 A2 = np.vstack([np.ones(n-1), -xd[1:]]).T
33 p3 = np.linalg.pinv(A2) @ dx2

```



```

32     r2, xm2 = p3[0], r2 / p3[1]
33     print('向后差分: r=', r2, 'xm=', xm2)

```

例 8.14: 参数拟合

```

1     # 程序文件 ex8_14.py
2     import numpy as np
3
4     t = np.array([0, 1, 2, 3, 4, 5, 6, 8, 10, 12, 14, 16, 18])
5     x = np.array([60, 63, 64, 63, 61, 58, 53, 44, 39, 38, 41, 46,
6                  53])
7     y = np.array([30, 34, 38, 44, 50, 55, 58, 56, 47, 38, 30, 27,
8                  26])
9
10    dt = np.diff(t)
11    dx = np.diff(x) / dt
12    dy = np.diff(y) / dt
13
14    n = len(x) - 1
15    A = np.zeros((2*n, 4))
16    A[:n, 0] = x[:-1]
17    A[:n, 1] = -x[:-1] * y[:-1]
18    A[n:, 2] = -y[:-1]
19    A[n:, 3] = x[:-1] * y[:-1]
20
21    b = np.hstack([dx, dy])
22    params = np.linalg.pinv(A) @ b
23    print('a, b, c, d =', np.round(params, 4))

```

例 8.15: 捕食者-被捕食者分析

```

1     # 程序文件 ex8_15.py
2     import numpy as np
3     from scipy.integrate import odeint
4     import pylab as plt
5
6     def func(state, t):
7         x, y = state
8         return [0.2*x - 0.005*x*y, -0.5*y + 0.01*x*y]
9
10    t = np.linspace(0, 100, 1000)
11    sol = odeint(func, [70, 40], t)
12
13    # 极值
14    print('x 最大值:', max(sol[:, 0]))
15    print('x 最小值:', min(sol[:, 0]))
16    print('y 最大值:', max(sol[:, 1]))
17    print('y 最小值:', min(sol[:, 1]))

```

```
18
19 # 时间序列图
20 plt.plot(t, sol[:, 0], label='$x(t)$ 兔子')
21 plt.plot(t, sol[:, 1], label='$y(t)$ 狐狸')
22 plt.xlabel('时间 (月)')
23 plt.ylabel('种群数量')
24 plt.legend()
25 plt.grid(True, alpha=0.3)
26 plt.show()
```

优缺点分析

模型	优点	缺点
Malthus 模型	简单直观，短期准确	长期发散，不符合实际
Logistic 模型	考虑资源限制，长期趋于稳定	参数估计方法不同结果差异大
竞争模型	描述多种群相互作用	参数多，需要大量数据
Lotka-Volterra	描述捕食关系的经典模型	假设过于理想化

方法选择建议

- 人口短期预测：使用 Malthus 模型（指数增长）。
- 人口长期预测：使用 Logistic 模型（有限增长）。
- 多种群相互作用：使用竞争模型或捕食者-被捕食者模型。
- 参数估计：优先使用非线性最小二乘，其次使用差分线性化。
- 模型验证：将预测值与实际数据比较，评估模型效果。

应用场景

- 人口预测：地区/国家人口增长预测。
- 生态保护：濒危物种数量预测与保护策略制定。
- 渔业管理：鱼类种群捕捞策略制定。
- 疾病传播：传染病传播趋势预测。
- 市场分析：新产品市场渗透率预测。
- 资源管理：可再生资源的最优利用。

8.5 差分方程建模方法

差分方程是在离散的时间点上描述研究对象动态变化规律的数学表达式。有的实际问题本身就是以离散形式出现的, 也有的是将现实世界中随时间连续变化的过程离散化。

利用差分方程建模, 通常遵循的基本准则是:

$$\text{未来值} = \text{现在值} + \text{变化值}. \quad (8.26)$$

若记 x_k 为第 k 时刻的状态变量值, 则各状态值依次排列形成序列 $\{x_k\}$ 。若 x_k 与其前一个或前几个状态值存在关联, 则用代数方程表达这种关联关系, 称为差分方程 (递推关系)。

8.5.1 差分方程建模实例

数学原理

(1) 贷款问题 记 y_n 为第 n 个月的欠款总额, r 为月利率, x 为每月还款额, N 为还款期限 (月), Q 为贷款总额。

模型建立:

$$\begin{cases} y_n = (1+r)y_{n-1} - x, & n = 1, 2, \dots, N, \\ y_0 = Q. \end{cases} \quad (8.41)$$

模型求解:

$$y_n = (1+r)^n Q - x \frac{(1+r)^n - 1}{r}. \quad (8.27)$$

由 $y_N = 0$, 得每月还款额:

$$x = \frac{(1+r)^N Q r}{(1+r)^N - 1}. \quad (8.28)$$

(2) 离散 Logistic 模型 以 Δt 为时间步长, x_k 为 $t = k$ 时的人口数量:

$$x_{k+1} - x_k = r(1 - sx_k)x_k \Delta t. \quad (8.29)$$

整理得:

$$x_{k+1} = \alpha x_k + \beta x_k^2, \quad (8.43)$$

其中 $\alpha = 1 + r\Delta t$, $\beta = -rs\Delta t < 0$ 。

(3) 差分法求解追踪问题 将时间离散化: $t_0 < t_1 < \dots < t_k < \dots$, 步长 $\Delta t = t_{k+1} - t_k$ 。

导弹位置 $P_k(x_k, y_k)$, 目标位置 $Q_k(1, v_0 t_k)$ 。

方向向量:

$$e^{(k)} = \frac{(1 - x_k, v_0 t_k - y_k)}{\sqrt{(1 - x_k)^2 + (v_0 t_k - y_k)^2}}. \quad (8.30)$$

递推公式:

$$\begin{cases} x_{k+1} = x_k + ue_1^{(k)} \Delta t, \\ y_{k+1} = y_k + ue_2^{(k)} \Delta t, \\ x_0 = 0, y_0 = 0. \end{cases} \quad (8.44)$$

8.5.2 差分方程的基本概念与理论

数学原理

(1) 基本概念 定义 8.1 k 阶常系数线性齐次差分方程:

$$y_{n+k} + a_1 y_{n+k-1} + \cdots + a_k y_n = 0. \quad (8.45)$$

定义 8.2 特征方程:

$$\lambda^k + a_1 \lambda^{k-1} + \cdots + a_{k-1} \lambda + a_k = 0. \quad (8.46)$$

定义 8.3 k 阶常系数线性非齐次差分方程:

$$y_{n+k} + a_1 y_{n+k-1} + \cdots + a_k y_n = f(n). \quad (8.47)$$

定理 8.1 若特征方程有 k 个互异特征根 $\lambda_1, \cdots, \lambda_k$, 则齐次方程通解为:

$$y_n = c_1 \lambda_1^n + c_2 \lambda_2^n + \cdots + c_k \lambda_k^n. \quad (8.48)$$

定理 8.2 若特征根 λ_i 的重数分别为 m_i ($\sum m_i = k$), 则通解为:

$$y_n = \sum_{i=1}^k \sum_{j=1}^{m_i} c_{ij} n^{j-1} \lambda_i^n. \quad (8.49)$$

定理 8.3 非齐次方程的通解 = 齐次通解 + 特解:

$$y_n = \tilde{y}_n + y_n^*. \quad (8.31)$$

(2) 平衡点与稳定性 一阶线性差分方程:

$$y_{k+1} + ay_k = b. \quad (8.32)$$

平衡点 $y^* = \frac{b}{1+a}$ 。稳定的充要条件: $|a| < 1$ 。

一阶线性差分方程组:

$$y(k+1) + Ay(k) = B. \quad (8.33)$$

平衡点 $y^* = (E + A)^{-1}B$ 。稳定的充要条件: A 的所有特征值 $|\lambda_i| < 1$ 。

一阶非线性差分方程:

$$y_{k+1} = f(y_k). \quad (8.34)$$

平衡点 y^* 由 $y^* = f(y^*)$ 确定。稳定的充要条件: $|f'(y^*)| < 1$ 。

具体例子

例 8.16: 贷款问题

贷款 30 万元，期限 30 年，年利率 5.1%，等额本息还款。求月供。

月利率 $r = 0.051/12 = 0.00425$ ， $N = 360$ 。

$$x = \frac{(1 + 0.00425)^{360} \times 300000 \times 0.00425}{(1 + 0.00425)^{360} - 1} = 1628.85 \text{ 元}. \quad (8.35)$$

累计还款： $1628.85 \times 360 = 586386$ 元。

例 8.17: 离散 Logistic 人口模型

用美国人口数据拟合差分方程 $x_{k+1} = \alpha x_k + \beta x_k^2$ ，得到：

$$\alpha = 1.2095, \quad \beta = -0.0004. \quad (8.36)$$

2010 年人口预测值：307.68 百万。

例 8.18: 差分法求解追踪问题

取 $v_0 = 1$ ， $\Delta t = 0.00004$ ，用差分法求解导弹追踪问题。

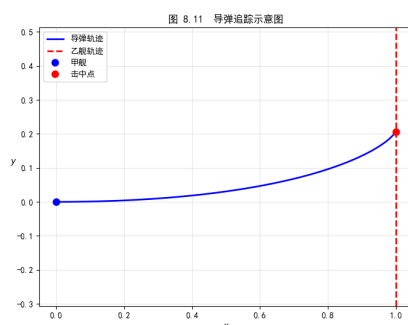


图 8.11 导弹追踪计算结果

击中位置： $y \approx 0.2083$ ，击中时间： $t \approx 0.2083$ 。

代码实现

例 8.16: 贷款问题

```
1  # 程序文件 ex8_16.py
2  Q = 300000
3  r = 0.051 / 12
4  N = 360
5
6  x = (1+r)**N * Q * r / ((1+r)**N - 1)
7  print('月供:', round(x, 2))
8  print('累计还款:', round(x * N, 2))
```

例 8.17: 离散 Logistic 模型拟合

```
1  # 程序文件 ex8_17.py
2  import numpy as np
3  import pandas as pd
4
5  a = pd.read_excel('data8_13.xlsx', header=None)
6  b = a.values
7  xd = b[1, :]
8  xd = xd[~np.isnan(xd)] # 剔除缺失值
9
10 A = np.vstack([xd[:-1], xd[:-1]**2]).T
11 cs = np.linalg.pinv(A) @ xd[1:]
12 print('alpha, beta:', np.round(cs, 4))
13
14 xh = cs[0] * xd[-1] + cs[1] * xd[-1]**2
15 print('2010年预测:', round(xh, 4))
```

例 8.18: 差分法追踪问题

```
1  # 程序文件 ex8_18.py
2  import numpy as np
3  import pylab as plt
4
5  v = 1
6  u = 5 * v
7  dt = 0.00004
8
9  x, y, t = 0, 0, 0
10 xs, ys = [x], [y]
11
12 while x < 0.999999:
13     pq = [1 - x, v*t - y]
14     dist = np.sqrt(pq[0]**2 + pq[1]**2)
15     x += u * pq[0] / dist * dt
16     y += u * pq[1] / dist * dt
17     t += dt
18     xs.append(x)
19     ys.append(y)
20
21 print('x =', round(x, 4))
22 print('y =', round(y, 4))
23 print('t =', round(t, 4))
24
25 plt.plot(xs, ys)
26 plt.xlabel('x')
27 plt.ylabel('y')
28 plt.axis('equal')
29 plt.grid(True, alpha=0.3)
```

30

plt.show()

优缺点分析

方法	优点	缺点
差分方程	适合离散数据, 递推计算简单	步长选择影响精度
等额本息模型	计算简单, 实用性强	假设利率不变
离散 Logistic	便于参数估计	步长较大时误差累积
差分追踪法	直观易懂, 易于编程	精度依赖步长

方法选择建议

- 数据为离散时间序列：直接建立差分方程。
- 连续模型离散化：用差分近似微分（向前/向后差分）。
- 线性差分方程：用特征根法求解析解。
- 非线性差分方程：用递推方法求数值解。
- 平衡点稳定性分析：计算特征值或导数的绝对值。

应用场景

- 金融计算：贷款还款、养老金、投资回报。
- 人口预测：离散时间人口增长模型。
- 生态学：种群数量的离散时间动态。
- 经济学：物价指数、GDP 的时间序列分析。
- 工程控制：离散时间控制系统。
- 数值计算：微分方程的数值解法（欧拉法、龙格-库塔法）。

8.6 应用案例：最优捕鱼策略

问题描述

例 8.19（1996 年全国大学生数学建模竞赛 A 题）

考虑具有 4 个年龄组（1 龄、2 龄、3 龄、4 龄）的某种鱼类。该鱼在每年后 4 个月季节性集中产卵繁殖，捕捞作业只允许在前 8 个月进行。使用只能捕捞 3、4 龄鱼的 13mm 网眼拉网，两个捕捞强度系数比为 0.42 : 1。

已知数据：

- 自然死亡率：0.8（1/年）
- 平均质量： $w = [5.07, 11.55, 17.86, 22.99]^T$ g
- 1、2 龄鱼不产卵，4 龄鱼产卵量 1.109×10^5 个，3 龄鱼为其一半
- 卵孵化成活率： $\beta = \frac{1.22 \times 10^{11}}{1.22 \times 10^{11} + n}$ （ n 为产卵总量）

目标：实现可持续捕获（每年年初各年龄组鱼群数量不变），并在此前提下获得最高年收获量。

数学原理

(1) 基本符号

- $x(t) = [x_1(t), x_2(t), x_3(t), x_4(t)]^T$ ：第 t 年年初各年龄鱼群数量
- $d = 0.8$ ：年自然死亡率， $c = 0.2$ ：年自然存活率
- α ：月自然死亡率， $(1 - \alpha)^{12} = 0.2 \Rightarrow \alpha = 0.1255$
- k_3, k_4 ：3、4 龄鱼捕捞强度系数， $k_3 : k_4 = 0.42 : 1$
- β ：卵孵化成活率
- $m = 1.109 \times 10^5$ ：4 龄鱼产卵量

(2) 模型建立 1、2 龄鱼（无捕捞，仅自然死亡）：

$$\begin{cases} x_2(t+1) = (1 - \alpha)^{12} x_1(t), \\ x_3(t+1) = (1 - \alpha)^{12} x_2(t). \end{cases} \quad (8.37)$$

3、4 龄鱼（捕捞期 + 产卵期）：

捕捞期（前 8 个月）累计捕捞量：

$$z = \frac{0.42kw_3[1 - (1 - \alpha - 0.42k)^8]}{\alpha + 0.42k} x_3(t) + \frac{kw_4[1 - (1 - \alpha - k)^8]}{\alpha + k} x_4(t). \quad (8.60)$$

产卵总量：

$$n = \frac{m}{2}(1 - \alpha - 0.42k)^8 x_3(t) + m(1 - \alpha - k)^8 x_4(t). \quad (8.38)$$

下年初 1 龄鱼数量：

$$x_1(t+1) = \beta n. \quad (8.61)$$

4 龄鱼转化：

$$x_4(t+1) = (1 - \alpha - 0.42k)^8 (1 - \alpha)^4 x_3(t). \quad (8.63)$$

差分方程组模型:

$$\begin{cases} x_1(t+1) = \beta \left[\frac{m}{2}(1-\alpha-0.42k)^8 x_3(t) + m(1-\alpha-k)^8 x_4(t) \right], \\ x_2(t+1) = (1-\alpha)^{12} x_1(t), \\ x_3(t+1) = (1-\alpha)^{12} x_2(t), \\ x_4(t+1) = (1-\alpha-0.42k)^8 (1-\alpha)^4 x_3(t). \end{cases} \quad (8.64)$$

矩阵形式: $x(t+1) = Px(t)$ 。

(3) 可持续捕获条件 可持续捕获要求 $x^* = Px^*$, 即 x^* 为差分方程组的平衡解。

推导可得:

$$x_1^* = 1.22 \times 10^{11} \left[\frac{1}{m(1-\alpha-0.42k)^8(1-\alpha)^{24} \left(\frac{1}{2} + (1-\alpha-k)^8(1-\alpha)^4 \right)} - 1 \right]. \quad (8.70)$$

$$x_3^* = (1-\alpha)^{24} x_1^*, \quad x_4^* = (1-\alpha-0.42k)^8 (1-\alpha)^{28} x_1^*. \quad (8.39)$$

将 x_3^*, x_4^* 代入目标函数式 (8.60), 得到关于 k 的一元函数, 通过遍历搜索求最大值。

计算结果

通过遍历搜索 ($k \in [0, 0.874]$, 步长 0.001):

- 4 龄鱼最佳月捕捞强度系数: $k_4 = 0.778$
- 3 龄鱼最佳月捕捞强度系数: $k_3 = 0.42k_4 = 0.3268$
- 最大年收获量: $5.9415 \times 10^{10} \text{ g} = 59415 \text{ t}$

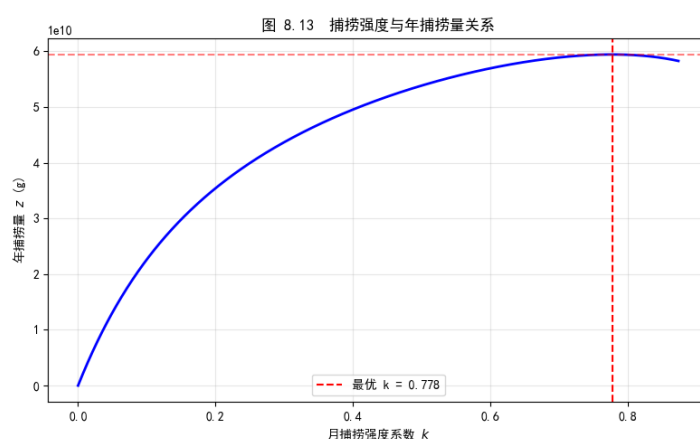


图 8.12 稳定生产策略下捕捞强度与年捕捞量之间的关系

稳定状态下各年龄组鱼群数量:

$$x^* = [1.1521 \times 10^{11}, 2.3042 \times 10^{10}, 4.6084 \times 10^9, 2.1830 \times 10^7]^T. \quad (8.40)$$

代码实现

```

1      # 程序文件 ex8_19_1.py
2      import numpy as np
3      import pylab as plt
4
5      a = 1 - 0.2 ** (1/12)          # 月自然死亡率
6      m = 1.109 * 10 ** 5           # 4龄鱼产卵量
7      w3, w4 = 17.86, 22.99         # 3、4龄鱼平均质量
8
9      X, Z, N = [], [], []
10
11     for k in np.arange(0, 0.875, 0.001):
12         # 计算 x1*
13         term1 = m * (1 - a - 0.42*k) ** 8 * (1 - a) ** 24
14         term2 = 0.5 + (1 - a - k) ** 8 * (1 - a) ** 4
15         x1 = 1.22e11 * (1 - 1 / (term1 * term2))
16
17         # 递推其他年龄组
18         x2 = (1 - a) ** 12 * x1
19         x3 = (1 - a) ** 12 * x2
20         x4 = (1 - a - 0.42*k) ** 8 * (1 - a) ** 4 * x3
21         X.append([x1, x2, x3, x4])
22
23         # 年捕捞量
24         z = (0.42*k * w3 * (1 - (1 - a - 0.42*k) ** 8) / (a + 0.42*k) *
25             x3 +
26             k * w4 * (1 - (1 - a - k) ** 8) / (a + k) * x4)
27         Z.append(z)
28
29         # 求最大值
30         mz = max(Z)
31         ind = np.argmax(Z)
32         k4 = 0.001 * ind
33         k3 = 0.42 * k4
34
35         print('最大年收获量:', mz, 'g =', mz/1e6, 't')
36         print('k4 =', k4)
37         print('k3 =', k3)
38         print('各年龄组鱼群数:', X[ind])
39
40         # 绘图
41         plt.rc('font', size=16)
42         plt.plot(np.arange(0, 0.875, 0.001), Z)
43         plt.xlabel('$k$')
44         plt.ylabel('$z$', rotation=0)
45         plt.title('捕捞强度与年捕捞量关系')

```

```
45 plt.grid(True, alpha=0.3)
46 plt.savefig('fishing_yield.png', dpi=300, bbox_inches='tight')
47 plt.show()
```

方法评述

方法	优点	缺点
差分方程建模	适合年龄结构种群, 递推清晰	步长固定, 假设较多
矩阵形式	便于理论分析 (稳定性)	矩阵元素含非线性项
遍历搜索	简单可靠, 可求全局最优	计算量大, 依赖步长精度
固定努力量	符合实际捕捞方式	假设捕捞强度恒定

方法选择建议

- 年龄结构种群：使用差分方程组建模。
- 矩阵形式：便于稳定性分析和平衡解求解。
- 优化求解：若解析求解困难，采用遍历搜索或数值优化。
- 可持续条件：利用平衡解条件 $x^* = Px^*$ 简化问题。

应用场景

- 渔业资源管理：确定最优捕捞强度，实现可持续捕捞。
- 森林采伐：确定最优砍伐周期和砍伐量。
- 野生动物保护：濒危物种的保护策略制定。
- 水资源管理：地下水、水库的可持续利用。
- 牧场管理：最优放牧强度与草场恢复。

第 9 章

数据的描述性统计方法

统计分析分为统计描述和统计推断两个部分。统计描述或描述性统计是统计推断的基础，主要是通过收集原始数据，并加工整理成可用的数据表，进而通过计算描述性统计量、编制统计表、绘制统计图等方法来给出数据信息。通过有限的、不确定的样本信息的统计分析结果，进而对整个总体做出判断或决策的方法，就是统计推断的研究领域。简单地说，统计学的任务就是由样本推断总体。

本章使用 Python 的 NumPy 库、scipy.stats 模块和 Statsmodels 库来实现数据的统计描述和分析。

9.1 scipy.stats 模块简介

scipy.stats 模块包含了多种概率分布的随机变量，随机变量分为连续型和离散型两种。所有的连续型随机变量都是 `rv_continuous` 的派生类的对象，而所有的离散型随机变量都是 `rv_discrete` 的派生类的对象。

9.1.1 连续型随机变量及分布

可以使用下面的语句获得 scipy.stats 模块中所有的连续型随机变量：

```
1 from scipy import stats
2 [k for k, v in stats.__dict__.items() if isinstance(v, stats.
    rv_continuous)]
```

连续型随机变量对象都有如下方法：

- `rvs`：产生随机数
- `pdf`：概率密度函数
- `cdf`：分布函数
- `sf`：生存函数（1-cdf）
- `ppf`：分布函数的反函数（分位数）
- `stat`：计算期望和方差
- `fit`：极大似然估计

常用连续型分布

分布名称	关键字	调用方式
均匀分布	uniform	uniform.pdf(x, a, b)
指数分布	expon	expon.pdf(x, scale= θ)
正态分布	norm	norm.pdf(x, mu, sigma)
χ^2 分布	chi2	chi2.pdf(x, n)
t 分布	t	t.pdf(x, n)
F 分布	f	f.pdf(x, m, n)
Γ 分布	gamma	gamma.pdf(x, a=A, scale=B)

正态分布的主要函数

函数	调用方式
概率密度	norm.pdf(x, mu, sigma)
分布函数	norm.cdf(x, mu, sigma)
分位数	norm.ppf(alpha, mu, sigma)
随机数	norm.rvs(mu, sigma, size=N)
极大似然估计	norm.fit(data)

9.1.2 离散型随机变量及分布

常用离散型随机变量：

分布名称	关键字	调用方式
二项分布	binom	binom.pmf(x, n, p)
几何分布	geom	geom.pmf(x, p)
泊松分布	poisson	poisson.pmf(x, lambda)

离散型分布的方法与连续型类似，但 pdf 替换为 pmf（分布律函数）。

具体例子

例 9.1： Γ 分布与指数分布的概率密度曲线对比

Γ 分布的概率密度函数：

$$f(x) = \frac{x^{\alpha-1}e^{-x/\beta}}{\beta^\alpha\Gamma(\alpha)}, \quad x > 0, \quad (9.1)$$

其中 $\Gamma(\alpha) = \int_0^\infty t^{\alpha-1}e^{-t}dt$, $\Gamma(\alpha+1) = \alpha\Gamma(\alpha)$ 。

指数分布是 Γ 分布中 $\alpha = 1$ 的特例。

例 9.2： 二项分布 $b(6, 0.3)$ 的分布律图

二项分布分布律：

$$P\{X = k\} = C_n^k p^k (1-p)^{n-k}, \quad k = 0, 1, \dots, n. \quad (9.2)$$

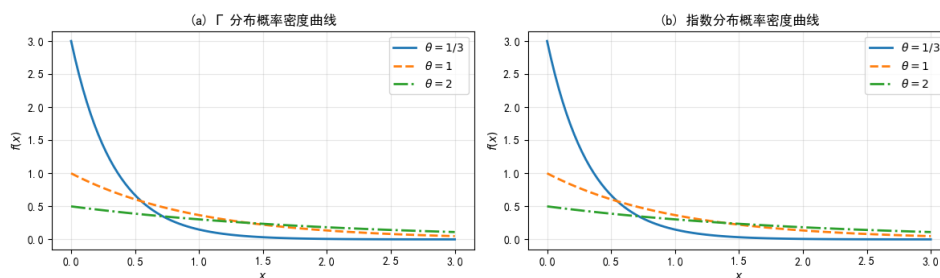
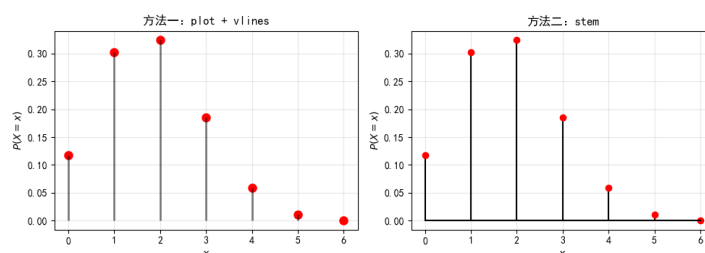
图 9.1 Γ 分布和指数分布的概率密度曲线对照图 9.1 Γ 分布和指数分布的概率密度曲线对照图 9.2 二项分布 $b(6, 0.3)$ 的分布律图形

图 9.2 二项分布分布律的两种画法

例 9.3: 正态分布分位数计算

设 $X \sim N(3, 2^2)$, 求 c 使得 $P\{X > c\} = 3P\{X \leq c\}$ 。

由 $P\{X > c\} + P\{X \leq c\} = 1$, 得 $P\{X \leq c\} = 0.25$ 。

解得 $c = 1.6510$ 。

例 9.4: 指数分布的数值特征

指数分布 $\exp(3)$ 的:

- 均值: $E(X) = 3$
- 方差: $D(X) = 9$
- 偏度: $\nu_1 = 2$
- 峰度: $\nu_2 = 6$

偏度 (skewness) 和峰度 (kurtosis) 的定义:

$$\nu_1 = \frac{E[(X - E(X))^3]}{(D(X))^{3/2}}, \quad \nu_2 = \frac{E[(X - E(X))^4]}{(D(X))^2} - 3. \quad (9.3)$$

代码实现**例 9.1: Γ 分布与指数分布对比**

```

1  # 程序文件 ex9_1.py
2  from scipy.stats import expon, gamma
3  import pylab as plt

```

```

4
5     x = plt.linspace(0, 3, 100)
6     L = [1/3, 1, 2]
7     styles = ['*', '-', 'o-']
8     labels = [r'$\theta = 1/3$', r'$\theta = 1$', r'$\theta = 2$']
9
10    plt.rc('text', usetex=True)
11    plt.rc('font', size=15)
12
13    plt.subplots_adjust(wspace=0.5)
14
15    plt.subplot(121)
16    for i in range(len(L)):
17        plt.plot(x, gamma.pdf(x, 1, scale=L[i]), styles[i], label=
18                labels[i])
19    plt.xlabel('$x$')
20    plt.ylabel('$f(x)$')
21    plt.legend()
22    plt.title('$\Gamma$ 分布')
23
24    plt.subplot(122)
25    for i in range(len(L)):
26        plt.plot(x, expon.pdf(x, scale=L[i]), styles[i], label=labels[i
27                ])
28    plt.xlabel('$x$')
29    plt.ylabel('$f(x)$')
30    plt.legend()
31    plt.title('指数分布')
32
33    plt.savefig('gamma_exponential.png', dpi=300, bbox_inches='
34                tight')
35    plt.show()

```

例 9.2: 二项分布分布律

```

1     # 程序文件 ex9_2.py
2     from scipy.stats import binom
3     import pylab as plt
4
5     n, p = 6, 0.3
6     x = plt.arange(7)
7     y = binom.pmf(x, n, p)
8
9     # 方法一: plot + vlines
10    plt.subplot(121)
11    plt.plot(x, y, 'ro')
12    plt.vlines(x, 0, y, 'k', lw=2, alpha=0.5)

```

```

13 plt.title('火柴杆图 (vlines)')
14
15 # 方法二: stem
16 plt.subplot(122)
17 plt.stem(x, y, use_line_collection=True)
18 plt.title('火柴杆图 (stem)')
19
20 plt.savefig('binomial_pmf.png', dpi=300, bbox_inches='tight')
21 plt.show()

```

例 9.3: 分位数计算

```

1 # 程序文件 ex9_3.py
2 from scipy.stats import norm
3 from scipy.optimize import fsolve
4
5 # 方法一: 分位数函数
6 c1 = norm.ppf(0.25, 3, 2)
7 print('c1 =', c1)
8
9 # 方法二: 方程求根
10 fc = lambda c: 1 - norm.cdf(c, 3, 2) - 3 * norm.cdf(c, 3, 2)
11 c2 = fsolve(fc, 1)
12 print('c2 =', c2[0])

```

例 9.4: 数字特征计算

```

1 # 程序文件 ex9_4.py
2 from scipy.stats import expon
3
4 # moments='mvsk' 返回均值、方差、偏度、峰度
5 stats = expon.stats(scale=3, moments='mvsk')
6 print('均值、方差、偏度、峰度:', stats)

```

方法评述

方法	优点	缺点
scipy.stats	涵盖大量分布, API 统一	部分高级统计方法需其他库
pdf/cdf/ppf fit 方法	完整概率计算功能 自动参数估计	需理解分布参数含义 仅适用于已知分布族

方法选择建议

- 概率计算: 使用 pdf/pmf 和 cdf。
- 分位数查找: 使用 ppf。

- 随机数生成：使用 `rvs`。
- 参数估计：使用 `fit` 方法。
- 数字特征：使用 `stats(moments='mvsk')`。

9.2 统计的基本概念和统计图

9.2.1 统计的基本概念

数学原理

(1) 样本与总体

- 总体：所研究对象的全体，记为 X
- 个体：总体的每一个基本单位
- 样本：从总体中抽出的若干个个体，记为 $X_1, X_2, \dots, X_n$
- 样本容量： n
- 样本观测值： $x_1, x_2, \dots, x_n$

(2) 常用统计量 集中趋势统计量：

样本均值：

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i. \quad (9.3)$$

中位数（将数据排序后取中间值）：

$$m_e = \begin{cases} x_{((n+1)/2)}, & n \text{ 为奇数,} \\ \frac{x_{(n/2)} + x_{(n/2+1)}}{2}, & n \text{ 为偶数.} \end{cases} \quad (9.4)$$

众数：出现频率最高的数值。

离散程度统计量：

样本方差：

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2. \quad (9.4)$$

样本标准差：

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}. \quad (9.5)$$

极差：

$$R = \max x_i - \min x_i. \quad (9.6)$$

变异系数:

$$CV = \frac{s}{\bar{x}} \times 100\%. \quad (9.7)$$

分布形状统计量:

偏度 (描述对称性):

$$\nu_1 = \frac{1}{s^3} \sum_{i=1}^n (x_i - \bar{x})^3. \quad (9.8)$$

- $\nu_1 > 0$: 右偏态
- $\nu_1 < 0$: 左偏态
- $\nu_1 \approx 0$: 对称

峰度 (描述尾部厚度):

$$\nu_2 = \frac{1}{s^4} \sum_{i=1}^n (x_i - \bar{x})^4. \quad (9.9)$$

标准正态分布峰度为 3。

相关统计量:

协方差:

$$\text{cov}(x, y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{n - 1}. \quad (9.5)$$

相关系数:

$$\rho_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}}. \quad (9.6)$$

k 阶原点矩:

$$a_k = \frac{1}{n} \sum_{i=1}^n x_i^k. \quad (9.7)$$

k 阶中心矩:

$$b_k = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^k. \quad (9.8)$$

具体例子

例 9.5: 甲乙两班成绩统计

甲班成绩: 60, 79, 48, 76, 67, 58, 65, 78, 64, 75, 76, 78, 84, 48, 25, 90, 98, 70, 77, 78, 68, 74, 95, 80, 90, 78, 73, 98, 85, 56

乙班成绩: 91, 74, 62, 72, 90, 94, 76, 83, 92, 85, 94, 83, 77, 82, 84, 60, 80, 78, 88, 90, 65, 77, 89, 86, 56, 87, 66, 56, 83, 67

协方差矩阵和相关系数矩阵:

$$V = \begin{bmatrix} 250.1023 & -32.6517 \\ -32.6527 & 128.5069 \end{bmatrix}, \quad R = \begin{bmatrix} 1 & -0.1821 \\ -0.1821 & 1 \end{bmatrix}. \quad (9.9)$$

表 9.5 两个班成绩的统计数据

班级	均值	中位数	极差	方差	标准差
甲班	73.0333	76	73	250.1023	15.8146
乙班	78.9000	82.5	38	128.5069	11.3361

例 9.6: Pandas 计算偏度与峰度

表 9.6 偏度、峰度和 90% 分位数

班级	偏度	峰度	90% 分位数
甲班	-0.9219	1.7507	90.5
乙班	-0.6515	-0.6157	91.1

9.2.2 统计图

数学原理

(1) 直方图 将数据取值范围划分为若干区间，统计每个区间中数据出现的次数（频数），以数据取值为横坐标、频数或频率为纵坐标画出的阶梯形图。

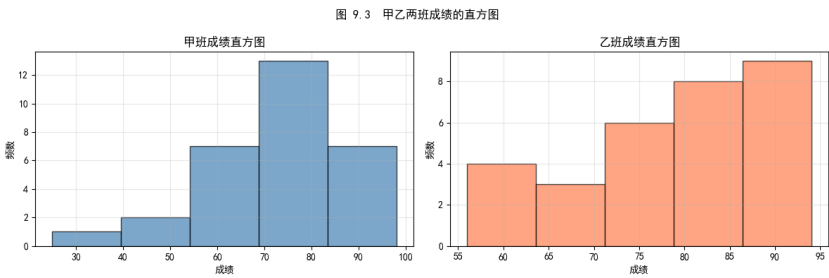


图 9.3 甲乙两班成绩的直方图

(2) 箱线图 基于 5 个数的图形概括：最小值 min 、第一四分位数 Q_1 、中位数 M 、第三四分位数 Q_3 、最大值 max 。

分位数定义：样本 p 分位数 x_p 满足：至少有 np 个观测值 $\leq x_p$ ，至少有 $n(1 - p)$ 个观测值 $\geq x_p$ 。

四分位数：

- $Q_1 = x_{0.25}$ ：第一四分位数（25% 分位数）
- $Q_2 = x_{0.5}$ ：中位数（50% 分位数）
- $Q_3 = x_{0.75}$ ：第三四分位数（75% 分位数）

疑似异常值判定： $IQR = Q_3 - Q_1$ ，数据小于 $Q_1 - 1.5IQR$ 或大于 $Q_3 + 1.5IQR$ 为疑似异常值。

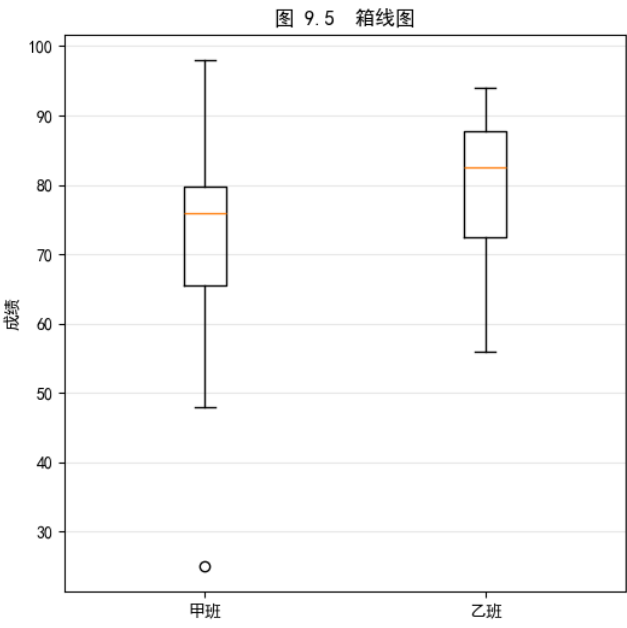


图 9.4 箱线图

(3) 经验分布函数 设 $S(x)$ 为样本中不大于 x 的个数：

$$F_n(x) = \frac{1}{n}S(x), \quad -\infty < x < \infty. \tag{9.10}$$

将数据排序后：

$$F_n(x) = \begin{cases} 0, & x < x_{(1)}, \\ \frac{k}{n}, & x_{(k)} \leq x < x_{(k+1)}, \\ 1, & x \geq x_{(n)}. \end{cases} \tag{9.11}$$

格里汶科定理：当 $n \rightarrow \infty$ 时， $F_n(x)$ 以概率 1 一致收敛于 $F(x)$ 。

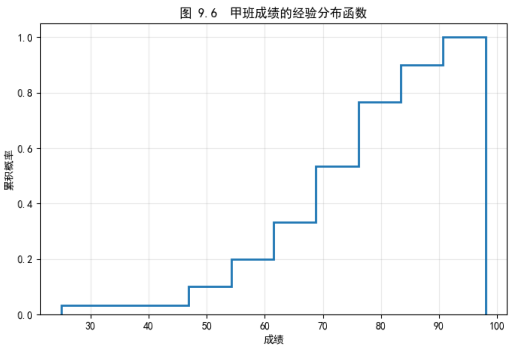


图 9.5 甲班成绩的经验分布函数

(4) Q-Q 图 用于检验数据是否来自某分布。

作图步骤：

1. 将数据排序： $x_{(1)} \leq x_{(2)} \leq \cdots \leq x_{(n)}$
2. 计算理论分位数： $y_i = F^{-1}((i - 1/2)/n)$
3. 绘制点 $(y_i, x_{(i)})$
4. 若点近似在 45° 直线上，说明拟合良好

图 9.7 Q-Q 图

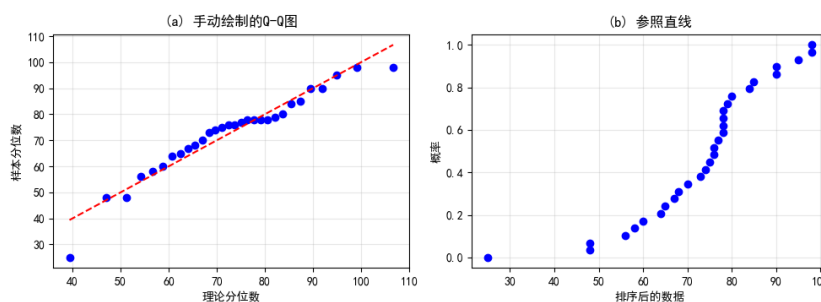


图 9.6 Q-Q 图

代码实现

例 9.5: NumPy 计算统计量

```

1  # 程序文件 ex9_5.py
2  import numpy as np
3  import pandas as pd
4
5  a = pd.read_csv('data9_5.txt', header=None)
6  b = a.values
7
8  mu = np.mean(b, axis=1)
9  med = np.median(b, axis=1)
10 rng = np.ptp(b, axis=1)
11 var = np.var(b, axis=1, ddof=1)
12 std = np.std(b, axis=1, ddof=1)
13 cov = np.cov(b)
14 corr = np.corrcoef(b)
15
16 print('均值:', mu)
17 print('中位数:', med)
18 print('极差:', rng)
19 print('方差:', var)
20 print('标准差:', std)
21 print('协方差矩阵:\n', cov)
22 print('相关系数矩阵:\n', corr)

```

例 9.6: Pandas describe 方法

```
1  # 程序文件 ex9_6.py
2  import pandas as pd
3
4  df = pd.read_csv('data9_5.txt', header=None)
5  df = df.T # 转置
6
7  print(df.describe())
8  print('偏度:\n', df.skew())
9  print('峰度:\n', df.kurt())
10 print('90%分位数:\n', df.quantile(0.9))
```

例 9.7: 直方图

```
1  # 程序文件 ex9_7.py
2  import pandas as pd
3  import pylab as plt
4
5  df = pd.read_csv('data9_5.txt', header=None)
6  df = df.T
7
8  plt.subplot(121)
9  plt.hist(df[0], 5, alpha=0.7, label='甲班')
10 plt.title('甲班成绩直方图')
11 plt.subplot(122)
12 plt.hist(df[1], 5, alpha=0.7, label='乙班')
13 plt.title('乙班成绩直方图')
14 plt.show()
```

例 9.8: 箱线图

```
1  # 程序文件 ex9_8.py
2  import pandas as pd
3  import pylab as plt
4
5  df = pd.read_csv('data9_5.txt', header=None).T
6  plt.boxplot(df, labels=['甲班', '乙班'])
7  plt.title('箱线图')
8  plt.show()
```

例 9.9: 经验分布函数

```
1  # 程序文件 ex9_9.py
2  import pandas as pd
3  import pylab as plt
4
5  df = pd.read_csv('data9_5.txt', header=None)
6  d = df.T[0]
```

```

7
8 plt.hist(d, density=True, histtype='step', cumulative=True)
9 plt.title('甲班成绩经验分布函数')
10 plt.grid(True)
11 plt.show()

```

例 9.10: Q-Q 图

```

1 # 程序文件 ex9_10.py
2 import pandas as pd
3 import pylab as plt
4 from scipy.stats import norm
5
6 df = pd.read_csv('data9_5.txt', header=None)
7 d = df.values[0]
8
9 mu = d.mean()
10 s = d.std()
11 sd = sorted(d)
12 n = len(d)
13
14 # 手动绘制 Q-Q 图
15 x = (np.arange(n) + 0.5) / n
16 yi = norm.ppf(x, mu, s)
17
18 plt.subplot(121)
19 plt.plot(yi, sd, 'o')
20 plt.plot([min(yi), max(yi)], [min(yi), max(yi)], 'r--')
21 plt.xlabel('理论分位数')
22 plt.ylabel('样本分位数')
23 plt.title('Q-Q 图')
24
25 plt.subplot(122)
26 plt.plot(sd, 'b-')
27 plt.title('参照直线')
28 plt.show()

```

方法评述

方法	优点	缺点
NumPy 统计函数	计算快速, 适合数值计算	需手动处理缺失值
Pandas describe 直方图	一次性输出多种统计量 直观显示数据分布	输出格式固定 区间划分影响图形形状

续下页

方法	优点	缺点
箱线图	显示异常值, 便于多组比较	不显示具体分布形态
Q-Q 图	直观判断拟合优度	主观判断, 无定量标准

方法选择建议

- 快速查看数据概况：使用 Pandas 的 describe()。
- 计算具体统计量：使用 NumPy 函数。
- 查看分布形状：绘制直方图。
- 比较多组数据：使用箱线图。
- 检验正态性：绘制 Q-Q 图。
- 查看累计分布：绘制经验分布函数图。

9.3 参数估计和假设检验

9.3.1 参数估计

参数估计是利用样本对总体进行统计推断的一类方法，即假设总体的概率分布类型已知，但其中含有未知参数，由样本估计未知参数的值。参数估计的方法主要有点估计和区间估计，其中点估计中有矩估计和极大似然估计等方法。

在 scipy.stats 模块中，样本均值的标准误差（SEM）定义为：

$$SEM = \frac{s}{\sqrt{n}} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}} \cdot \frac{1}{\sqrt{n}}. \tag{9.12}$$

对应的函数为 `scipy.stats.sem(x)`。

具体例子

例 9.11：正态总体均值的置信区间

从一批糖果中随机抽取 16 袋，质量数据如下：

506, 508, 499, 503, 504, 510, 497, 512, 514, 505, 493, 496, 506, 502, 509, 496

设袋装糖果质量近似服从正态分布，求总体均值 μ 的 95% 置信区间。

μ 的 $1 - \alpha$ 置信区间为：

$$\left(\bar{X} \pm t_{\alpha/2} \frac{S}{\sqrt{n}} \right). \tag{9.13}$$

计算得 $\bar{x} = 503.75$, $s = 6.2022$, $t_{0.025}(15) = 2.1314$ 。

95% 置信区间为：(500.4451, 507.0549)。

9.3.2 参数假设检验

数学原理

(1) 单个正态总体均值的 Z 检验 (σ 已知) 设总体 $X \sim N(\mu, \sigma^2)$, σ 已知。检验假设:

$$H_0: \mu = \mu_0, \quad H_1: \mu \neq \mu_0. \quad (9.14)$$

检验统计量:

$$Z = \frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}}. \quad (9.15)$$

拒绝域: $|Z| \geq z_{\alpha/2}$ 。

例 9.12: 纽扣直径 $X \sim N(\mu, 4.2^2)$, 标准直径为 26mm, $\alpha = 0.05$, 检验是否符合标准。

计算得 $Z = 0.0407 < 1.96$, 接受 H_0 , 认为符合标准。

(2) 单个正态总体均值的 t 检验 (σ 未知) 设总体 $X \sim N(\mu, \sigma^2)$, σ 未知。检验假设:

$$H_0: \mu \leq \mu_0, \quad H_1: \mu > \mu_0. \quad (9.16)$$

检验统计量:

$$t = \frac{\bar{X} - \mu_0}{S/\sqrt{n}}. \quad (9.17)$$

拒绝域: $t \geq t_{\alpha}(n-1)$ 。

例 9.13: 电子元件寿命数据, 检验平均寿命是否大于 225h。

计算得 $t = 0.6685 < 1.7531$, 接受 H_0 , 认为平均寿命不大于 225h。

(3) 两个正态总体均值差的 t 检验 设两个样本分别来自 $N(\mu_1, \sigma^2)$ 和 $N(\mu_2, \sigma^2)$ 。检验假设:

$$H_0: \mu_1 - \mu_2 \leq 0, \quad H_1: \mu_1 - \mu_2 > 0. \quad (9.18)$$

检验统计量:

$$t = \frac{\bar{X} - \bar{Y}}{S_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}, \quad S_p^2 = \frac{(n_1 - 1)S_1^2 + (n_2 - 1)S_2^2}{n_1 + n_2 - 2}. \quad (9.19)$$

拒绝域: $t \geq t_{\alpha}(n_1 + n_2 - 2)$ 。

例 9.14: 两种方法测定融化热, 检验方法 A 是否优于方法 B。

计算得 $t = 3.4722 > 1.7291$, 拒绝 H_0 , 认为方法 A 优于方法 B。

9.3.3 非参数假设检验

数学原理

(1) χ^2 拟合优度检验 用于检验总体是否服从某已知分布。

检验步骤：

1. 建立假设 H_0 ：总体分布函数为 $F(x)$ ；
2. 将数轴分成 k 个区间，计算理论概率 p_i 和观测频数 f_i ；
3. 计算统计量：

$$\chi^2 = \sum_{i=1}^k \frac{(f_i - np_i)^2}{np_i} \sim \chi^2(k - r - 1), \quad (9.20)$$

其中 r 为未知参数个数；

4. 对于显著性水平 α ，若 $\chi^2 < \chi_\alpha^2$ ，接受 H_0 。

例 9.15：检验一本书的印刷错误是否服从泊松分布。

计算得 $\chi^2 = 1.4607 < 5.9915$ ，接受 H_0 ，认为服从泊松分布。

例 9.16：检验灯泡寿命是否服从指数分布。

计算得 $\chi^2 = 1.845 < 7.8147$ ，接受 H_0 ，认为服从指数分布。

(2) **Kolmogorov-Smirnov 检验** 直接检验原假设 $H_0 : F(x) = F_0(x)$ (F_0 必须为连续分布)。

检验统计量：

$$D_n = \sup_x |F_n(x) - F_0(x)|. \quad (9.21)$$

拒绝域： $\sqrt{n}D_n \geq t_\alpha$ 。

临界值：

- $\alpha = 0.10$: $t_\alpha = 1.22$
- $\alpha = 0.05$: $t_\alpha = 1.36$
- $\alpha = 0.01$: $t_\alpha = 1.63$

例 9.17：84 个头颅宽度数据，检验是否服从正态分布。

计算得 $D_n = 0.0851$, $1.36/\sqrt{84} = 0.1484$, $D_n < 0.1484$ ，接受 H_0 ，认为服从正态分布。

代码实现

例 9.11：置信区间

```
1  # 程序文件 ex9_11.py
2  import numpy as np
3  from scipy.stats import t, sem
4
5  d = np.loadtxt('data9_11.txt').flatten()
6  n = len(d)
7  xb = d.mean()
8  s = d.std(ddof=1)
9  sm = sem(d)
10 alpha = 0.05
11 ta = t.ppf(1 - alpha/2, n - 1)
12 L = [xb - sm*ta, xb + sm*ta]
13 print('置信区间:', np.round(L, 4))
```

例 9.12: Z 检验

```
1  # 程序文件 ex9_12.py
2  import numpy as np
3  from scipy.stats import norm
4  from statsmodels.stats.weightstats import ztest
5
6  a = np.array([26.01, 26.00, 25.98, 25.86, 26.32, 25.58, 25.32,
7               25.89, 26.32, 26.18])
8  sigma = 4.2
9  zstat, p = ztest(a, value=26)
10 print('Z统计量:', zstat)
11 print('p值:', p)
12 za = norm.ppf(0.975)
13 print('分位数:', za)
```

例 9.13: t 检验

```
1  # 程序文件 ex9_13.py
2  import numpy as np
3  from scipy.stats import t
4  from statsmodels.stats.weightstats import ttest
5
6  a = np.loadtxt('data9_13.txt').flatten()
7  ts, p = ttest(a, value=225, alternative='larger')
8  print('t统计量:', ts)
9  print('p值:', p)
10 ta = t.ppf(0.95, len(a)-1)
11 print('分位数:', ta)
```

例 9.14: 两样本 t 检验

```
1  # 程序文件 ex9_14.py
2  import numpy as np
```

```

3  from scipy.stats import t
4  from statsmodels.stats.weightstats import ttest_ind
5
6  a = np.array([79.98, 80.04, 80.02, 80.04, 80.03, 80.03, 80.04,
7              79.97, 80.05, 80.03, 80.02, 80.00, 80.02])
8  b = np.array([80.02, 79.94, 79.98, 79.97, 79.97, 80.03, 79.95,
9              79.97])
10
11  tstat, p, df = ttest_ind(a, b, alternative='larger')
12  print('t 统计量:', tstat)
13  print('p 值:', p)

```

例 9.15: χ^2 拟合优度检验

```

1  # 程序文件 ex9_15.py
2  import scipy.stats as ss
3  import numpy as np
4
5  obs = np.array([36, 40, 19, 2, 0, 2, 1])
6  x = np.arange(7)
7  lam = np.sum(obs * x) / 100
8
9  p0 = ss.poisson.pmf(np.arange(3), lam)
10  p = np.hstack([p0, 1 - sum(p0)])
11  ex = 100 * p
12
13  # 合并频数
14  obs_new = np.hstack([obs[:3], sum(obs[3:])])
15  ex_new = np.hstack([ex[:3], sum(ex[3:])])
16
17  chi2, p_val = ss.chisquare(obs_new, ex_new, ddof=1)
18  print('chi2:', chi2)
19  print('p 值:', p_val)

```

例 9.17: Kolmogorov-Smirnov 检验

```

1  # 程序文件 ex9_17.py
2  import scipy.stats as ss
3  import numpy as np
4
5  # 读取数据
6  b = np.array
7      ([141, 148, 132, 138, 154, 142, 150, 146, 155, 158, 150, 140, 147, 148, 144, 150,
8       149, 145, 149, 158, 143, 141, 144, 146, 126, 140, 144, 142, 141, 140, 145, 135,
9       147, 146, 141, 136, 140, 146, 142, 137, 148, 154, 137, 139, 143, 140, 131, 143,

```

```

9      141,149,148,135,148,152,143,144,141,143,147,146,150,132,142,142,
10     143,153,149,146,149,138,142,149,142,137,134,144,146,147,140,142,
11     140,137,152,145])
12
13     xb = b.mean()
14     s = b.std(ddof=1)
15     st, p = ss.kstest(b, 'norm', args=(xb, s))
16     print('统计量 D_n:', round(st, 4))
17     print('p 值:', round(p, 4))

```

方法评述

方法	优点	缺点
Z 检验	简单快速, 适合大样本	需已知总体方差
t 检验	无需已知方差, 适用小样本	要求总体正态性
χ^2 检验	适用任意分布	需大样本, 区间划分影响结果
K-S 检验	直接检验分布函数, 精度高	仅适用于连续分布

方法选择建议

- 总体方差已知: 使用 Z 检验。
- 总体方差未知: 使用 t 检验。
- 检验分布类型: 使用 χ^2 拟合优度检验或 K-S 检验。
- 样本量较小: 使用 t 检验或 K-S 检验。
- 参数估计: 使用矩估计或极大似然估计 (scipy.stats.fit)。

9.4 方差分析

在现实问题中, 经常会遇到考察两个不同总体的均值是否相等的问题。但当检验总体多于两个时, 仍采用多总体均值检验方法会遇到困难。方差分析 (Analysis of Variance, ANOVA) 用于鉴别各因素对结果影响程度。

人们关心的实验结果称为指标, 试验中需要考察、可以控制的条件称为因素或因子, 因素所处的状态称为水平。

9.4.1 单因素方差分析

只考虑一个因素 A 对指标的影响。若 A 取 r 个水平 $A_1, A_2, \dots, A_r$, 在水平 A_i 下总体 $X_i \sim N(\mu_i, \sigma^2)$, $i = 1, 2, \dots, r$ 。

数学原理

(1) 数学模型 设水平 A_i 下进行 n_i 次独立试验:

$$X_{ij} = \mu_i + \varepsilon_{ij}, \quad \varepsilon_{ij} \sim N(0, \sigma^2), \quad (9.22)$$

其中 $i = 1, \dots, r$, $j = 1, \dots, n_i$ 。

引入总体均值 μ 和效应 α_i :

$$\mu = \frac{1}{n} \sum_{i=1}^r n_i \mu_i, \quad \alpha_i = \mu_i - \mu, \quad n = \sum_{i=1}^r n_i. \quad (9.23)$$

模型可表示为:

$$\begin{cases} X_{ij} = \mu + \alpha_i + \varepsilon_{ij}, \\ \sum_{i=1}^r n_i \alpha_i = 0, \\ \varepsilon_{ij} \sim N(0, \sigma^2). \end{cases} \quad (9.24)$$

原假设为:

$$H_0: \alpha_1 = \alpha_2 = \dots = \alpha_r = 0. \quad (9.25)$$

(2) 平方和分解 组平均值:

$$\bar{X}_{i\cdot} = \frac{1}{n_i} \sum_{j=1}^{n_i} X_{ij}, \quad \bar{X} = \frac{1}{n} \sum_{i=1}^r \sum_{j=1}^{n_i} X_{ij}. \quad (9.26)$$

总平方和分解:

$$S_T = \sum_{i=1}^r \sum_{j=1}^{n_i} (X_{ij} - \bar{X})^2 = S_A + S_E, \quad (9.27)$$

其中:

$$S_A = \sum_{i=1}^r n_i (\bar{X}_{i\cdot} - \bar{X})^2 \quad (\text{组间平方和}), \quad (9.28)$$

$$S_E = \sum_{i=1}^r \sum_{j=1}^{n_i} (X_{ij} - \bar{X}_{i\cdot})^2 \quad (\text{组内平方和/误差平方和}). \quad (9.29)$$

表 9.9 单因素方差分析表

方差来源	离差平方和	自由度	均方	F 值
因素 A (组间)	S_A	$r - 1$	$S_A/(r - 1)$	$F = \frac{S_A/(r - 1)}{S_E/(n - r)}$
误差 (组内)	S_E	$n - r$	$S_E/(n - r)$	
总和	S_T	$n - 1$		

(3) 检验统计量 当 H_0 成立时:

$$F = \frac{S_A/(r - 1)}{S_E/(n - r)} \sim F(r - 1, n - r). \tag{9.30}$$

给定显著性水平 α , 若 $F > F_\alpha(r - 1, n - r)$, 拒绝 H_0 。

显著性判断:

- $F > F_{0.01}$: 高度显著
- $F_{0.01} \geq F > F_{0.05}$: 显著
- $F \leq F_{0.05}$: 不显著

具体例子

例 9.18: 4 种工艺生产的灯泡寿命

表 9.10 4 种工艺生产的灯泡寿命 (h)

A_1	A_2	A_3	A_4
1620	1580	1460	1500
1670	1600	1540	1550
1700	1640	1620	1610
1750	1720	1680	
1800			

计算得 $F = 3.7277$, $p = 0.0420 < 0.05$, 拒绝 H_0 , 认为这几种工艺制成的灯泡寿命有显著差异。

例 9.19: 温度对产品收率的影响

5 种温度, 每种做 4 次试验。计算得 $F = 3.9496 > F_{0.05}(4, 15) = 3.0556$, 拒绝 H_0 , 认为温度对产品收率有显著影响。

9.4.2 双因素方差分析

数学原理

(1) 无交互效应的双因素方差分析 设因素 A 有 s 个水平, 因素 B 有 r 个水平, 每个组合做一次试验。

模型:

$$X_{ij} = \mu + \alpha_i + \beta_j + \varepsilon_{ij}, \quad \sum_{i=1}^s \alpha_i = 0, \quad \sum_{j=1}^r \beta_j = 0. \quad (9.31)$$

平方和分解:

$$S_T = S_A + S_B + S_E, \quad (9.32)$$

其中:

$$S_A = r \sum_{i=1}^s (\bar{X}_{i.} - \bar{X})^2, \quad S_B = s \sum_{j=1}^r (\bar{X}_{.j} - \bar{X})^2. \quad (9.33)$$

检验统计量:

$$F_A = \frac{S_A/(s-1)}{S_E/[(r-1)(s-1)]} \sim F(s-1, (r-1)(s-1)), \quad (9.34)$$

$$F_B = \frac{S_B/(r-1)}{S_E/[(r-1)(s-1)]} \sim F(r-1, (r-1)(s-1)). \quad (9.35)$$

(2) 有交互效应的双因素方差分析 每个组合做 t 次重复试验。

平方和分解:

$$S_T = S_A + S_B + S_{AB} + S_E, \quad (9.36)$$

其中 S_{AB} 为交互作用平方和。

检验统计量:

$$F_A = \frac{S_A/(s-1)}{S_E/[rs(t-1)]} \sim F(s-1, rs(t-1)), \quad (9.37)$$

$$F_B = \frac{S_B/(r-1)}{S_E/[rs(t-1)]} \sim F(r-1, rs(t-1)), \quad (9.38)$$

$$F_{AB} = \frac{S_{AB}/[(r-1)(s-1)]}{S_E/[rs(t-1)]} \sim F((r-1)(s-1), rs(t-1)). \quad (9.39)$$

具体例子

例 9.20: 无交互效应

4 种燃料、3 种推进器, 每种组合做 1 次试验。

计算得 $p_A = 0.7387$, $p_B = 0.4491$, 均大于 0.05, 认为燃料和推进器对射程无显著影响。

例 9.21: 有交互效应

4 种燃料、3 种推进器, 每种组合做 2 次试验。

计算得 $p_A = 0.0260$, $p_B = 0.0035$, $p_{AB} = 0.0001$, 均小于 0.05, 认为燃料、推进器及交互作用均显著。

代码实现

例 9.18: 单因素方差分析

```
1  # 程序文件 ex9_18.py
2  import pandas as pd
3  import numpy as np
4  import statsmodels.api as sm
5
6  a = pd.read_excel('data9_18.xlsx', header=None)
7  b = a.values.T
8  y = b[~np.isnan(b)]
9  x = np.hstack([np.ones(5), np.full(4,2), np.full(4,3), np.full
10                 (3,4)])
11  d = {'x': x, 'y': y}
12  model = sm.formula.ols('y ~ C(x)', d).fit()
13  anova = sm.stats.anova_lm(model)
14  print(anova)
```

例 9.19: 单因素方差分析

```
1  # 程序文件 ex9_19.py
2  import numpy as np
3  import pandas as pd
4  import statsmodels.api as sm
5  from scipy.stats import f
6
7  d = np.loadtxt('data9_19.txt')
8  a = d.flatten()
9  x = np.tile(np.arange(1, 6), (4, 1)).T.flatten()
10  df = {'x': x, 'y': a}
11  model = sm.formula.ols('y ~ C(x)', df).fit()
12  anova = sm.stats.anova_lm(model)
13  print(anova)
14  fa = f.ppf(0.95, anova.df[0], anova.df[1])
15  print('临界值:', fa)
```

例 9.20: 无交互双因素方差分析

```
1  # 程序文件 ex9_20.py
2  import numpy as np
3  import statsmodels.api as sm
4
5  a = np.loadtxt('data9_20.txt')
6  x1 = np.tile(np.arange(1, 5), (3, 1)).T.flatten()
7  x2 = np.tile(np.arange(1, 4), (4, 1)).flatten()
8  d = {'x1': x1, 'x2': x2, 'y': a.flatten()}
9  model = sm.formula.ols('y ~ C(x1) + C(x2)', d).fit()
10  anova = sm.stats.anova_lm(model)
```

```
print(anova)
```

例 9.21: 有交互双因素方差分析

```
# 程序文件 ex9_21.py
import numpy as np
import statsmodels.api as sm

a = np.loadtxt('data9_21.txt')
x1 = np.tile(np.arange(1, 5), (6, 1)).T.flatten()
x2 = np.tile(np.array([1,1,2,2,3,3]), (4, 1)).flatten()
d = {'x1': x1, 'x2': x2, 'y': a.flatten()}
# 使用 * 表示包含主效应和交互效应
model = sm.formula.ols('y ~ C(x1) * C(x2)', d).fit()
anova = sm.stats.anova_lm(model)
print(anova)
```

方法评述

方法	优点	缺点
单因素 ANOVA	简单直观, 易于解释	仅考虑一个因素
无交互双因素	同时分析两个因素, 效率高	假设无交互作用
有交互双因素	可检测交互效应	需要重复试验, 数据量大
Statsmodels	自动计算 F 值和 p 值	需理解公式语法

方法选择建议

- 单一因素影响: 使用单因素方差分析。
- 两个因素独立影响: 使用无交互双因素方差分析 ($y \sim C(A) + C(B)$)。
- 两个因素可能有交互: 使用有交互双因素方差分析 ($y \sim C(A) * C(B)$)。
- 判断显著性: 比较 p 值与显著性水平 α (通常 0.05)。
- 事后检验: 若 ANOVA 显著, 可进一步做多重比较 (Tukey HSD)。

第 10 章

回归分析

回归分析是在一组数据的基础上研究：

1. 建立因变量 y 与自变量 $x_1, x_2, \dots, x_m$ 之间的回归模型；
2. 对回归模型的可信度进行检验；
3. 判断每个自变量对 y 的影响是否显著；
4. 诊断回归模型是否适合这组数据；
5. 利用回归模型对 y 进行预报或控制。

10.1 一元线性回归模型

10.1.1 一元线性回归分析

数学原理

(1) 一元线性回归模型的一般形式 已知观测数据 (x_i, y_i) ($i = 1, 2, \dots, n$), 若 y 与 x 大致呈线性关系：

$$y = \beta_0 + \beta_1 x + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2). \quad (10.1)$$

其中 β_0, β_1 为回归系数, ε 为随机误差。

(2) 回归系数的最小二乘估计 最小二乘估计使残差平方和最小：

$$Q = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2. \quad (10.2)$$

正规方程组：

$$\begin{cases} n\beta_0 + (\sum x_i) \beta_1 = \sum y_i, \\ (\sum x_i) \beta_0 + (\sum x_i^2) \beta_1 = \sum x_i y_i. \end{cases} \quad (10.3)$$

解得：

$$\hat{\beta}_1 = \frac{L_{xy}}{L_{xx}}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}. \quad (10.1)$$

其中：

$$\bar{x} = \frac{1}{n} \sum x_i, \quad \bar{y} = \frac{1}{n} \sum y_i, \quad L_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}), \quad L_{xx} = \sum (x_i - \bar{x})^2. \quad (10.2)$$

回归方程为：

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x. \quad (10.3)$$

(3) 平方和分解与判定系数 总平方和分解：

$$SST = SSR + SSE. \quad (10.5)$$

其中：

- $SST = \sum (y_i - \bar{y})^2$ ：总变异平方和，自由度 $n - 1$
- $SSR = \sum (\hat{y}_i - \bar{y})^2$ ：回归平方和（可解释变异），自由度 1
- $SSE = \sum (y_i - \hat{y}_i)^2$ ：残差平方和，自由度 $n - 2$

判定系数（拟合优度）：

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}, \quad 0 \leq R^2 \leq 1. \quad (10.4)$$

R^2 越接近 1，拟合效果越好。

相关系数：

$$r = \frac{L_{xy}}{\sqrt{L_{xx}L_{yy}}}, \quad r = \pm \sqrt{R^2}. \quad (10.5)$$

剩余标准差：

$$s = \sqrt{\frac{SSE}{n - 2}}. \quad (10.6)$$

(4) 回归方程的显著性检验（ F 检验） 检验假设 $H_0: \beta_1 = 0$ （线性关系不显著）。

检验统计量：

$$F = \frac{SSR}{SSE/(n - 2)} \sim F(1, n - 2). \quad (10.7)$$

判断标准：

- $F > F_{0.01}(1, n - 2)$ ：极其显著
- $F_{0.05}(1, n - 2) < F < F_{0.01}(1, n - 2)$ ：显著
- $F < F_{0.05}(1, n - 2)$ ：不显著

表 10.1 观测数据

$x(\%)$	0.10	0.11	0.12	0.13	0.14	0.15	0.16	0.17	0.18	0.20	0.22	0.24
y	42.0	42.5	45.0	45.5	45.0	47.5	49.0	51.0	50.0	55.0	57.5	59.5

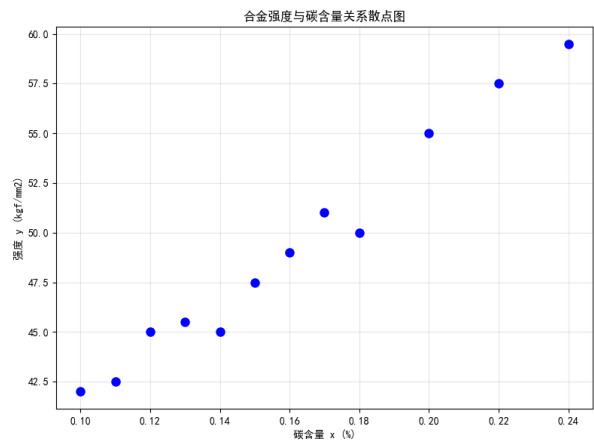


图 10.1 合金强度与碳含量关系散点图

具体例子

例 10.1：合金强度与碳含量的回归分析

经过野值剔除后（删除第 9 个和第 5 个数据），使用剩余 10 个数据拟合：

模型检验指标：

$$R^2 = 0.995, \quad F = 1455, \quad p = 2.45 \times 10^{-10}, \quad s^2 = 0.2339. \tag{10.8}$$

代码实现

```
1  # 程序文件 ex10_1.py
2  import numpy as np
3  import statsmodels.api as sm
4  import pylab as plt
5
6  def check(x0, y0):
7      d = {'x': x0, 'y': y0}
8      model = sm.formula.ols('y ~ x', d).fit()
9      print(model.summary())
10     print(model.outlier_test())
11     print('残差方差:', model.mse_resid)
```

表 10.2 回归参数计算结果

回归系数	估计值	95% 置信区间
$\hat{\beta}_1$	129.1667	[121.359, 136.974]
$\hat{\beta}_0$	28.7833	[27.485, 30.081]

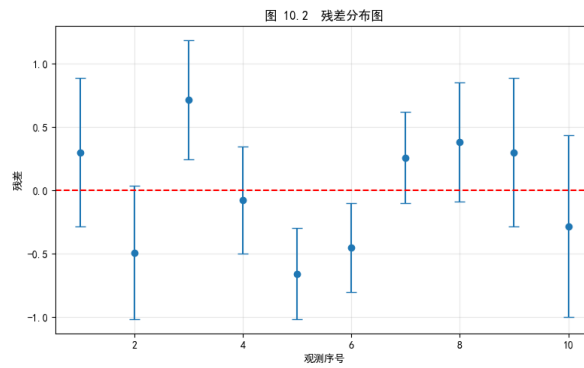


图 10.2 已知数据预测残差分布图

```

12
13 # 预测区间
14 pred = model.get_prediction(d)
15 df = pred.summary_frame(alpha=0.05)
16 low, upp = df['mean_ci_lower'], df['mean_ci_upper']
17 r = (upp - low) / 2
18 num = np.arange(1, len(x0) + 1)
19 plt.errorbar(num, model.resid, r, fmt='o')
20 plt.xlabel('观测序号')
21 plt.ylabel('残差')
22 plt.title('残差分布图')
23 plt.show()
24
25 # 读取数据
26 a = np.loadtxt('data10_1.txt')
27 x0 = a[0]; y0 = a[1]
28
29 # 散点图
30 plt.plot(x0, y0, 'o')
31 plt.xlabel('碳含量 x (%)')
32 plt.ylabel('强度 y (kgf/mm²)')
33 plt.title('合金强度与碳含量关系散点图')
34 plt.show()
35
36 # 第一次拟合 (全部数据)
37 check(x0, y0)
38
39 # 删除第 9 个数据 (野值)
40 x1 = np.delete(x0, 8)
41 y1 = np.delete(y0, 8)
42 check(x1, y1)
43
44 # 再删除第 5 个数据
45 x2 = np.delete(x1, 4)

```

```

46     y2 = np.delete(y1, 4)
47     check(x2, y2)

```

方法评述

方法	优点	缺点
最小二乘法 R^2 判定系数	计算简单，无偏估计 直观反映拟合优度	对异常值敏感 不能单独作为模型选择依据
F 检验 残差分析	检验线性关系显著性 检测异常值和模型假设	仅适用于线性模型 需专业知识判断

方法选择建议

- 判断线性关系：先画散点图。
- 参数估计：使用最小二乘法。
- 模型检验：查看 R^2 、 F 统计量和 p 值。
- 异常值检测：使用 `outlier_test()` 查看学生化残差。
- 预测区间：使用 `get_prediction()` 获取置信区间。

10.2 多元线性回归

10.2.1 多元线性回归理论

数学原理

(1) 多元线性回归模型的一般形式 若自变量的个数多于一个，回归模型是关于自变量的线性表达形式，则称此模型为多元线性回归模型。其数学模型可以写为：

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m + \varepsilon, \quad (10.10)$$

式中： $\beta_0, \beta_1, \cdots, \beta_m$ 为回归系数； ε 为随机误差，服从正态分布 $N(0, \sigma^2)$ ， σ 未知。

回归分析的主要步骤是：

1. 由观测值确定参数（回归系数） $\beta_0, \beta_1, \cdots, \beta_m$ 的估计值 $b_0, b_1, \cdots, b_m$ ；
2. 对线性关系、自变量的显著性进行统计检验；
3. 利用回归方程进行预测。

(2) 回归系数的最小二乘估计 设对 y 及 $x_1, x_2, \cdots, x_m$ 作 n 次抽样得到 n 组数据 $(y_i, x_{i1}, \cdots, x_{im}), i = 1, \cdots, n, n > m$ 。代入式 (10.10)，有：

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_m x_{im} + \varepsilon_i, \quad (10.11)$$

式中: $\varepsilon_i (i = 1, 2, \dots, n)$ 为服从正态分布 $N(0, \sigma^2)$ 的 n 个相互独立同分布的随机变量。

记:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1m} \\ 1 & x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \boldsymbol{\varepsilon} = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}, \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_m \end{bmatrix}. \quad (10.12)$$

式 (10.11) 可以表示为:

$$\begin{cases} \mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \\ \boldsymbol{\varepsilon} \sim N(0, \sigma^2 \mathbf{E}_n), \end{cases} \quad (10.13)$$

式中: $\mathbf{E}_n$ 为 n 阶单位矩阵。

模型式 (10.10) 中的参数 $\beta_0, \beta_1, \dots, \beta_m$ 用最小二乘法估计, 即应选取估计值 b_j , 使当 $\beta_j = b_j, j = 0, 1, 2, \dots, m$ 时, 误差平方和:

$$Q = \sum_{i=1}^n \varepsilon_i^2 = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \cdots - \beta_m x_{im})^2 \quad (10.14)$$

达到最小。为此, 令:

$$\frac{\partial Q}{\partial \beta_j} = 0, \quad j = 0, 1, 2, \dots, m. \quad (10.9)$$

得:

$$\begin{cases} \frac{\partial Q}{\partial \beta_0} = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \cdots - \beta_m x_{im}) = 0, \\ \frac{\partial Q}{\partial \beta_j} = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \cdots - \beta_m x_{im}) x_{ij} = 0, \quad j = 1, 2, \dots, m. \end{cases} \quad (10.15)$$

经整理化为以下正规方程组:

$$\begin{cases} \beta_0 n + \beta_1 \sum_{i=1}^n x_{i1} + \beta_2 \sum_{i=1}^n x_{i2} + \cdots + \beta_m \sum_{i=1}^n x_{im} = \sum_{i=1}^n y_i, \\ \beta_0 \sum_{i=1}^n x_{i1} + \beta_1 \sum_{i=1}^n x_{i1}^2 + \beta_2 \sum_{i=1}^n x_{i1} x_{i2} + \cdots + \beta_m \sum_{i=1}^n x_{i1} x_{im} = \sum_{i=1}^n x_{i1} y_i, \\ \vdots \\ \beta_0 \sum_{i=1}^n x_{im} + \beta_1 \sum_{i=1}^n x_{i1} x_{im} + \beta_2 \sum_{i=1}^n x_{i2} x_{im} + \cdots + \beta_m \sum_{i=1}^n x_{im}^2 = \sum_{i=1}^n x_{im} y_i. \end{cases} \quad (10.16)$$

正规方程组的矩阵形式为:

$$\mathbf{X}^T \mathbf{X} \boldsymbol{\beta} = \mathbf{X}^T \mathbf{Y}. \quad (10.17)$$

当矩阵 $\mathbf{X}$ 列满秩时, $\mathbf{X}^T \mathbf{X}$ 为可逆方阵, 式 (10.17) 的解为:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}. \quad (10.18)$$

(3) 拟合与残差分析 将 $\hat{\beta} = [b_0, b_1, \dots, b_m]^T$ 代入式 (10.10), 得到 y 的估计值为:

$$\hat{y} = b_0 + b_1 x_1 + \dots + b_m x_m. \quad (10.19)$$

而这组数据的拟合值为 $\hat{Y} = X\hat{\beta}$. 拟合误差 $e = Y - \hat{Y}$ 称为残差, 可作为随机误差 ε 的估计, 而:

$$SSE = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (10.20)$$

为残差平方和 (或剩余平方和)。

(4) 回归方程和回归系数的检验 前面是在假定随机变量 y 与变量 $x_1, x_2, \dots, x_m$ 具有线性关系的条件下建立线性回归方程的, 但变量 y 与变量 $x_1, x_2, \dots, x_m$ 是否为线性关系? 所有的变量 $x_1, x_2, \dots, x_m$ 对变量 y 是否都有影响? 需要做统计检验。

对总平方和 $SST = \sum_{i=1}^n (y_i - \bar{y})^2$ 进行分解, 有:

$$SST = SSE + SSR, \quad (10.21)$$

其中 SSE 是由式 (10.20) 定义的残差平方和, 反映随机误差对 y 的影响; $SSR = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$ 称为回归平方和, 反映自变量对 y 的影响, 这里 $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$, $\hat{y}_i = b_0 + b_1 x_{i1} + \dots + b_m x_{im}$ 。上面的分解中利用了正规方程组, 其中 SST 的自由度 $df_T = n - 1$, SSE 的自由度 $df_E = n - m - 1$, SSR 的自由度 $df_R = m$ 。

因变量 y 与自变量 $x_1, \dots, x_m$ 之间是否存在如式 (10.10) 所示的线性关系是需要检验的。显然, 如果所有的 $|\beta_j| (j = 1, \dots, m)$ 都很小, y 与 $x_1, \dots, x_m$ 的线性关系就不明显, 所以可令原假设为:

$$H_0: \beta_1 = \beta_2 = \dots = \beta_m = 0. \quad (10.10)$$

当 H_0 成立时, 由分解式 (10.21) 定义的 SSR, SSE 满足:

$$F = \frac{SSR/m}{SSE/(n-m-1)} \sim F(m, n-m-1). \quad (10.22)$$

在显著性水平 α 下, 若 $F > F_\alpha(m, n-m-1)$, 则回归方程效果显著; 若 $F < F_\alpha(m, n-m-1)$, 则回归方程效果不显著。

当上面的 H_0 被拒绝时, β_j 不全为零, 但是不排除其中若干个等于零。所以应进一步作如下 $m+1$ 个检验 ($j = 0, 1, \dots, m$):

$$H_0^{(j)}: \beta_j = 0, \quad j = 0, 1, \dots, m. \quad (10.11)$$

当 $H_0^{(j)}$ 为真时, 统计量:

$$t_j = \frac{b_j / \sqrt{c_{jj}}}{\sqrt{SSE/(n-m-1)}} \sim t(n-m-1) \quad (10.23)$$

式中: c_{jj} 为 $(X^T X)^{-1}$ 中的第 (j, j) 元素。对给定的 α , 若 $|t_j| > t_{\frac{\alpha}{2}}(n-m-1)$ ($j = 1, 2, \dots, m$), 则 x_j 的作用显著; 否则, 接受 $H_0^{(j)}$, x_j 的作用不显著, 去掉变量 x_j 重新建立回归方程。

(5) 回归系数的置信区间及预测 式 (10.23) 也可用于对 $\beta_j (j = 0, 1, \dots, m)$ 作区间估计, 在置信水平 $1 - \alpha$ 下, β_j 的置信区间为:

$$[b_j - t_{\frac{\alpha}{2}}(n - m - 1)s\sqrt{c_{jj}}, \quad b_j + t_{\frac{\alpha}{2}}(n - m - 1)s\sqrt{c_{jj}}], \quad (10.24)$$

式中: $s = \sqrt{\frac{SSE}{n-m-1}}$ 。

还有一些衡量 y 与 $x_1, \dots, x_m$ 相关程度的指标, 如用回归平方和在总平方和中的比值定义复判定系数, 即:

$$R^2 = \frac{SSR}{SST}, \quad (10.25)$$

式中: $R = \sqrt{R^2}$ 为复相关系数, R 越大, y 与 $x_1, \dots, x_m$ 相关关系越密切, 通常, R 大于 0.8 (或 0.9) 才认为相关关系成立。

对于给定的点 $(x_1^{(0)}, x_2^{(0)}, \dots, x_m^{(0)})$ 代入回归方程式 (10.19), 得:

$$\hat{y}_0 = b_0 + b_1 x_1^{(0)} + b_2 x_2^{(0)} + \dots + b_m x_m^{(0)} \quad (10.12)$$

用 $\hat{y}_0$ 作为 y 在点 $x_1^{(0)}, x_2^{(0)}, \dots, x_m^{(0)}$ 的预测值。也可以进行区间估计, 记 $s = \sqrt{\frac{SSE}{n-m-1}}$, $x_0 = [1, x_1^{(0)}, x_2^{(0)}, \dots, x_m^{(0)}]^T$, 则 y_0 的置信度为 $1 - \alpha$ 的预测区间为:

$$(\hat{y}_0 - t_{1-\alpha/2}(n-m-1)s\sqrt{1 + x_0^T(\mathbf{X}^T\mathbf{X})^{-1}x_0}, \quad \hat{y}_0 + t_{1-\alpha/2}(n-m-1)s\sqrt{1 + x_0^T(\mathbf{X}^T\mathbf{X})^{-1}x_0}) \quad (10.13)$$

当 n 较大时, 有 y_0 的近似预测区间: 95% 的预测区间为 $(\hat{y}_0 - 2s, \hat{y}_0 + 2s)$, 98% 的预测区间为 $(\hat{y}_0 - 3s, \hat{y}_0 + 3s)$ 。

10.2.2 多元线性回归应用

代码实现介绍

在 Python 中, ‘statsmodels’ 可以使用两种模式求解回归模型: 一种是基于公式的模式; 另一种是基于数组的模式。

基于公式构建并拟合模型的调用格式为:

```
1 import statsmodels.api as sm
2 sm.formula.ols(formula, data=df).fit()
```

其中 ‘formula’ 为引导的公式, ‘df’ 为数据框或字典格式的数据。

基于数组构建并拟合模型的调用格式为:

```
1 import statsmodels.api as sm
2 sm.OLS(y, X).fit()
```

其中 y 为因变量的观测值向量, X 为自变量观测值矩阵再添加第一列全部元素为 1 得到的增广阵。

具体例子

例 10.2: 平均产量与工龄、文化程度的回归分析

某企业为研究车工的平均年龄 x_1 、平均文化程度 x_2 和平均产量 y 之间的变化关系, 随机抽取了 8 个班组, 测得数据如下表所示。

表 10.4 表 10.3 相关数据

序号	1	2	3	4	5	6	7	8
x_1 /年	7.1	6.8	9.2	11.4	8.7	6.6	10.3	10.6
x_2 /年	11.1	10.8	12.4	10.9	9.6	9.0	10.5	12.4
y /件	15.4	15.0	22.8	27.8	19.5	13.1	24.9	26.2

解: 分别画出 y 关于两个变量 x_1 和 x_2 的散点图, 从图中可以看出 y 与 x_1 和 x_2 都有线性关系, 我们试着建立多元线性回归模型:

$$y = a_0 + a_1x_1 + a_2x_2.$$

利用 Python 软件求得模型参数的估计值及检验结果如下表所示。 $x_1 = 9, x_2 = 10$ 和 $x_1 = 10, x_2 = 9$ 时, y 的预测值分别为 20.5918, 22.9008。

表 10.5 表 10.4 回归参数计算结果及模型检验指标

回归系数	回归系数估计值	回归系数置信区间
a_0	-10.6284	$[-13.472, -7.785]$
a_1	2.8584	$[2.672, 3.045]$
a_2	0.5494	$[0.259, 0.840]$
模型检验指标: $R^2 = 0.998, F = 1087, P = 2.52 \times 10^{-7}, s^2 = 0.1021$.		

代码实现

基于公式求解的 Python 程序如下:

```

1  # 程序文件 ex10_2_1.py
2  import numpy as np
3  import statsmodels.api as sm
4  import pylab as plt
5
6  # 加载数据 (假设数据存在 'data10_2.txt' 文件中)
7  a = np.loadtxt('data10_2.txt')
8
9  # 设置字体以支持中文显示
10 plt.rc('text', usetex=True)
11 plt.rc('font', size=16)
12
13 # 绘制散点图观察数据关系
14 plt.plot(a[0], a[2], '*', label='\\(x_1\\)')
```

```

15 plt.plot(a[1], a[2], 'o', label='\'(x_2\')')
16 plt.legend(loc='upper left')
17
18 # 构建字典数据以便使用公式模式
19 d = {'x1': a[0], 'x2': a[1], 'y': a[2]}
20
21 # 使用公式模式拟合模型
22 re = sm.formula.ols('y~x1+x2', d).fit()
23
24 # 输出模型摘要
25 print(re.summary())
26
27 # 进行预测
28 yh = re.predict({'x1': [9, 10], 'x2': [10, 9]})
29
30 print('残差的方差:', re.mse_resid)
31 print('预测值:', yh)
32 plt.show()

```

基于数组求解的 Python 程序如下：

```

1 # 程序文件 ex10_2_2.py
2 import numpy as np
3 import statsmodels.api as sm
4 import pylab as plt
5
6 # 加载数据
7 a = np.loadtxt('data10_2.txt')
8
9 # 设置字体
10 plt.rc('text', usetex=True)
11 plt.rc('font', size=16)
12
13 # 绘制散点图
14 plt.plot(a[0], a[2], '*', label='\'(x_1\')')
15 plt.plot(a[1], a[2], 'o', label='\'(x_2\')')
16 plt.legend(loc='upper left')
17
18 # 构建设计矩阵 X: 添加截距项常数列, 并且注意行与列的关系 (原数
19   组排列为: 行是特征, 列是样本)
20 X = sm.add_constant(a[:2].T)
21
22 # 使用数组模式拟合模型
23 re = sm.OLS(a[2], X).fit()
24
25 # 输出模型摘要
26 print(re.summary())

```

```

26
27     # 进行预测
28     yh = re.predict(np.array([[1, 9, 10], [1, 10, 9]]))
29
30     print('残差的方差:', re.mse_resid)
31     print('预测值:', yh)
32     plt.show()

```

方法评述

方法	优点	缺点
矩阵法 (OLS)	计算高效, 可一次性求解所有系数, 适合多维变量场景	当 $X^T X$ 接近奇异或共线性严重时, 逆矩阵计算不稳定
R^2 判定系数	直观衡量模型整体对数据的解释能力	与一元回归类似, 不能反映过拟合, 增加变量会让 R^2 虚高
F 检验	检验整体回归方程的线性显著性	只能说明全部变量中至少有一个有效, 无法判定具体哪个变量有效
t 检验	对每个自变量的独立贡献进行检验, 有助于变量筛选	若变量间存在强多重共线性, 系数估计和 t 值将失去可信度
预测区间	给出了未来观测值可能落入的范围, 量化了预测的不确定性	区间宽度受 s (残差标准差) 和离群点影响较大

方法选择建议

- 线性关系初判: 先利用散点图矩阵 (Scatter Matrix) 或相关系数矩阵观察 y 与多个自变量的大致关系。
- 参数估计: 首选矩阵形式的 OLS (即 `sm.OLS(Y, X).fit()`), 需注意必须手动添加截距项常数列 (使用 `sm.add_constant`)。
- 模型整体评价: 查阅 `re.summary()` 中的全局 F 检验 p 值和 R^2 。
- 自变量筛选: 重点关注 t 检验的 p 值 ($P > |t|$), 如果某个变量的 p 值大于 0.05, 说明其作用不显著, 应考虑剔除后重新建模。
- 多重共线性监测: 若模型整体显著但单个变量 t 检验不显著, 或在增加/移除变量时系数剧烈波动, 应检查变量间的方差膨胀因子 (VIF)。
- 预测应用: 使用 `re.get_prediction(X_new)` 获取目标新数据的预测值及其置信区间。

10.3 多项式回归

如果从数据的散点图上发现 y 与 x 呈较明显的二次（或高次）函数关系，或者用线性模型的效果不太好，就可以选用多项式回归。

一元 n 次多项式回归模型：

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_n x^n + \varepsilon, \quad (10.14)$$

其中 $\varepsilon \sim N(0, \sigma^2)$ 。

具体例子

例 10.3：年龄与运动能力的关系

表 10.7 年龄与运动能力观测结果

年龄	17	19	21	23	25	27	29
第一人	20.48	25.13	26.15	30.0	26.1	20.3	19.35
第二人	28.11	28.11	26.3	31.4	26.92	25.7	21.3

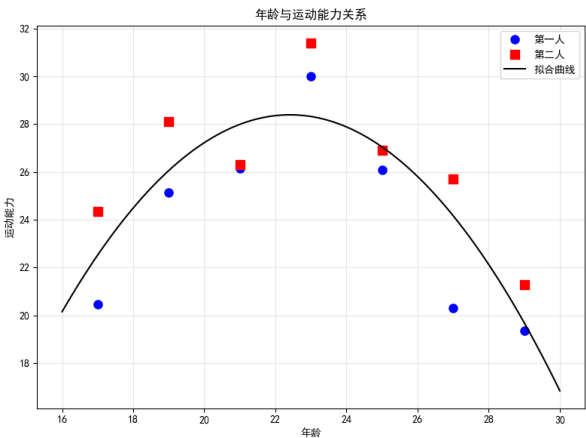


图 10.3 观测结果分布图

散点图呈现两端低中间高的形状，拟合二次多项式：

$$y = -0.2003x^2 + 8.9782x - 72.2150. \quad (10.15)$$

例 10.4：多元二次多项式回归

某商品销售量 y 与竞争对手价格 x_1 和本厂价格 x_2 有关。

表 10.8 价格与销售量数据

$x_1/\text{元}$	120	140	190	130	155	175	125	145	180	150
$x_2/\text{元}$	100	110	90	150	210	150	250	270	300	250
$y/\text{个}$	102	100	120	77	46	93	26	69	65	85

通过比较多个模型的残差方差，选择纯二次多项式模型：

$$y = -312.5871 + 7.2701x_1 - 1.7337x_2 - 0.0228x_1^2 + 0.0037x_2^2. \quad (10.16)$$

残差方差 $s^2 = 277.0105$ 。当 $x_1 = 170$, $x_2 = 160$ 时，预测值 $\hat{y} = 82.0523$ 。

例 10.5：多元线性回归（猪瘦肉量预测）

25 头育肥猪的数据，建立瘦肉量 y 对眼肌面积 x_1 、腿肉量 x_2 、腰肉量 x_3 的回归模型。

线性回归结果：

$$\hat{y} = 0.8539 + 0.0178x_1 + 2.0782x_2 + 1.9396x_3. \quad (10.17)$$

模型检验：

$$F = 37.7453 > F_{0.05}(3, 21) = 3.0725, \quad (10.18)$$

模型整体显著。

t 检验结果：

$$t_0 = 0.6223, t_1 = 0.6090, t_2 = 7.7407, t_3 = 3.8062, \quad (10.19)$$

$t_{0.025}(21) = 2.0796$, x_1 不显著，剔除后重新拟合：

$$y = 1.1077 + 2.1058x_2 + 1.9787x_3. \quad (10.20)$$

完全二次式模型：

$$\begin{aligned} y = & -17.0988 + 0.3611x_1 + 2.3563x_2 + 18.2730x_3 \\ & - 0.1412x_1x_2 - 0.4404x_1x_3 - 1.2754x_2x_3 \\ & + 0.0217x_1^2 + 0.5025x_2^2 + 0.3962x_3^2. \end{aligned} \quad (10.21)$$

代码实现

例 10.3：一元二次多项式回归

```

1  # 程序文件 ex10_3.py
2  import numpy as np
3  import statsmodels.api as sm
4  import pylab as plt
5
6  x = np.arange(17, 30, 2)
7  a = np.loadtxt('data10_3.txt')
8  y = a.flatten()
9  x = np.hstack([x, x])
10
11  d = {'y': y, 'x': x}
12  model = sm.formula.ols('y ~ x + I(x**2)', d).fit()
```

```

13     print(model.summary())
14     print('残差方差:', model.mse_resid)
15
16     plt.plot(x[:7], a[0], 'o', label='第一人')
17     plt.plot(x[7:], a[1], 'o', label='第二人')
18     plt.xlabel('年龄')
19     plt.ylabel('运动能力')
20     plt.legend()
21     plt.show()

```

例 10.4: 多元二次多项式回归

```

1     # 程序文件 ex10_4.py
2     import numpy as np
3     import statsmodels.api as sm
4     import pylab as plt
5
6     a = np.loadtxt('data10_4.txt')
7     x1, x2, y = a[0], a[1], a[2]
8     d = {'y': y, 'x1': x1, 'x2': x2}
9
10    # 线性模型
11    re1 = sm.formula.ols('y ~ x1 + x2', d).fit()
12    print('线性回归残差方差:', re1.mse_resid)
13
14    # 纯二次模型
15    re2 = sm.formula.ols('y ~ x1 + x2 + I(x1**2) + I(x2**2)', d).
16        fit()
17    print('纯二次残差方差:', re2.mse_resid)
18
19    # 含交互项
20    re3 = sm.formula.ols('y ~ x1 * x2', d).fit()
21    print('含交互项残差方差:', re3.mse_resid)
22
23    # 完全二次
24    re4 = sm.formula.ols('y ~ x1 * x2 + I(x1**2) + I(x2**2)', d).
25        fit()
26    print('完全二次残差方差:', re4.mse_resid)
27
28    # 预测
29    print('预测值:', re2.predict({'x1': 170, 'x2': 160}))

```

例 10.5: 多元线性回归（手动计算）

```

1     # 程序文件 ex10_5.py
2     import numpy as np
3     import pandas as pd
4     from scipy.stats import t, f

```



```

5
6     a = pd.read_excel('data10_5.xlsx', header=None)
7     b = a.values
8
9     # 构造 X 和 Y
10    Y = np.hstack([b[:, 1], b[:, -1, 6]])
11    X = np.vstack([b[:, 2:5], b[:, -1, 7:]])
12    XX = np.hstack([np.ones((25, 1)), X])
13
14    # 最小二乘估计
15    cs = np.linalg.pinv(XX) @ Y
16    print('拟合参数:', np.round(cs, 4))
17
18    # F 检验
19    yh = XX @ cs
20    q = np.sum((Y - yh)**2)
21    u = np.sum((yh - Y.mean())**2)
22    m, n = 3, len(Y)
23    F_stat = (u/m) / (q/(n-m-1))
24    print('F统计量:', round(F_stat, 4))
25
26    # t 检验
27    c = np.diag(np.linalg.inv(XX.T @ XX))
28    ts = cs / np.sqrt(c) / np.sqrt(q/(n-m-1))
29    print('t统计量:', np.round(ts, 4))
30
31    # 剔除 x1 后重新拟合
32    XD = np.delete(XX, 1, axis=1)
33    cs2 = np.linalg.pinv(XD) @ Y
34    print('剔除x1后的参数:', np.round(cs2, 4))

```

方法评述

方法	优点	缺点
多项式回归	可拟合非线性关系	高次易过拟合
逐步回归	自动筛选显著变量	依赖准则选择
F 检验	检验模型整体显著性	仅适用于线性模型
t 检验	检验单个变量显著性	多重共线性影响结果
残差分析	诊断模型假设	需专业知识判断

方法选择建议

- 判断非线性关系：先画散点图。
- 模型选择：比较残差方差 s^2 或调整 R^2 。

- 变量筛选：使用 t 检验剔除不显著变量。
- 避免过拟合：控制多项式最高次数。
- 公式语法：
 - 一次项：‘y ~ x1 + x2’
 - 二次项：‘y ~ x1 + x2 + I(x1**2) + I(x2**2)’
 - 交互项：‘y ~ x1 * x2’（等价于 ‘x1 + x2 + x1:x2’）

10.4 逐步回归

实际问题中影响因变量的因素可能很多，有些影响显著，有些影响微弱。逐步回归是一种从众多变量中有效地选择重要变量的方法。

数学原理

(1) 基本思想 设 $S = \{x_1, x_2, \dots, x_m\}$ 为候选自变量集合， $S_1 \subset S$ 为选出的子集。若 S_1 中有 l 个自变量，回归模型的残差平方和为 SSE ，则残差方差为：

$$s^2 = \frac{SSE}{n - l - 1}. \quad (10.22)$$

选择使 s^2 最小的子集。模型中包含的自变量越多， SSE 越小，但若包含不显著变量， s^2 可能反而增大。

(2) 偏回归平方和与偏 F 检验 剔除 x_i 后回归平方和的减少量：

$$\Delta SSR_{(i)} = SSR - SSR_{(i)} = SSE_{(i)} - SSE. \quad (10.23)$$

偏 F 统计量：

$$F = \frac{\Delta SSR_{(i)}/1}{SSE/(n - m - 1)} \sim F(1, n - m - 1). \quad (10.24)$$

F 检验与 t 检验等价： $F = t^2$ 。

(3) 三种变量选择方法 前进法：

1. 变量由少到多，每次增加一个；
2. 每次选择 F 值最大的变量引入；
3. 直至没有可引入的变量为止。

后退法：

1. 先用全部自变量拟合方程；

2. 每次剔除 t 值最小的变量；
3. 直至所有变量均显著。

逐步回归法：

1. 变量逐个引入，每引入一个后对已选变量逐个检验；
2. 不显著的变量被剔除；
3. 既无显著变量可引入，也无显著变量可剔除时终止。

注意：引入变量的显著性水平 α_{in} 应小于剔除变量的显著性水平 α_{out} ，否则可能产生死循环。

具体例子

例 10.6：水泥凝固热量与化学成分的逐步回归

表 10.10 水泥凝固热量观测数据

序号	x_1	x_2	x_3	x_4	y
1	7	26	6	60	78.5
2	1	29	15	52	74.3
3	11	56	8	20	104.3
4	11	31	8	47	87.6
5	7	52	6	33	95.9
6	11	55	9	22	109.2
7	3	71	17	6	102.7
8	1	31	22	44	72.5
9	2	54	18	26	93.1
10	21	47	4	34	115.9
11	1	40	23	12	113.3
12	11	66	9	12	109.4
13	10	68	8	12	109.4

使用后退法：

第一步：全部 4 个变量拟合， t 值分别为 2.0827、0.7049、0.1350、 -0.2032 ，剔除 t 绝对值最小的 x_3 。

第二步：用 x_1, x_2, x_4 拟合，剔除 x_4 。

最终模型：

$$y = 52.5773 + 1.4683x_1 + 0.6623x_2. \quad (10.25)$$

模型检验指标：

$$R^2 = 0.979, \quad F = 229.5, \quad p = 4.41 \times 10^{-9}, \quad s^2 = 5.7904. \quad (10.26)$$

代码实现

```

1  # 程序文件 ex10_6.py
2  import numpy as np
3  import statsmodels.api as sm
4
5  a = np.loadtxt('data10_6.txt')
6  d = {
7      'x1': a[:, 0],
8      'x2': a[:, 1],
9      'x3': a[:, 2],
10     'x4': a[:, 3],
11     'y': a[:, 4]
12 }
13
14 # 全部变量
15 md1 = sm.formula.ols('y ~ x1 + x2 + x3 + x4', d).fit()
16 print('全部变量模型:')
17 print(md1.summary())
18
19 # 剔除 x3
20 md2 = sm.formula.ols('y ~ x1 + x2 + x4', d).fit()
21 print('剔除 x3 后:')
22 print(md2.summary())
23
24 # 最终模型 (x1, x2)
25 md3 = sm.formula.ols('y ~ x1 + x2', d).fit()
26 print('最终模型:')
27 print(md3.summary())
28 print('残差方差:', md3.mse_resid)

```

方法评述

方法	优点	缺点
前进法 后退法	计算量小，简单快速 考虑所有变量，易于理解	变量一旦引入不能剔除 变量一旦剔除不能重新引入
逐步回归 全子集法	有进有出，结果更可靠 遍历所有组合，最优	计算量大，可能死循环 组合数 2^m ，仅适合 m 较小

方法选择建议

- 变量数较少 ($m \leq 10$): 可使用全子集法 ('ols' + 比较 R^2 或 AIC)。
- 变量数较多: 使用逐步回归。

- 快速筛选：使用前进法。
- 稳健选择：使用逐步回归。
- 注意：逐步回归结果受 α_{in} 和 α_{out} 选择影响。

应用场景

- 经济预测：从众多经济指标中筛选关键因素。
- 医学统计：从多个生理指标中确定主要影响因素。
- 工程实验：从多个工艺参数中确定关键参数。
- 社会科学：从多个社会因素中筛选显著变量。

10.5 广义线性回归模型

广义线性模型（GLM）是常见正态线性回归模型的直接推广，适用于连续数据和离散数据，特别是属性数据、计数数据。

10.5.1 广义线性模型的基本概念

对于线性模型：

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2). \quad (10.27)$$

令 $\mu = E(y)$ ，则 $\mu = \beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m$ 。

广义线性模型将其推广为：

$$\eta = g(\mu) = \beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m. \quad (10.28)$$

其中：

- η ：线性预测部分
- $g(\cdot)$ ：连接函数（link function）
- 误差函数：从指数型分布族中选择

表 10.12 常见的连接函数和误差函数

连接函数	逆连接函数（回归模型）	典型误差函数
恒等	$\eta = \mu$	正态
对数	$\eta = \ln \mu$	泊松
Logit	$\eta = \ln \left(\frac{\mu}{1-\mu} \right)$	二项
逆	$\eta = 1/\mu$	伽马

10.5.2 Logistic 回归模型

Logistic 回归主要用于解决二分类 (0 或 1) 问题, 用于估计某种事物发生的可能性。

Logistic 函数:

$$f(x) = \frac{e^x}{1 + e^x} = \frac{1}{1 + e^{-x}}. \quad (10.32)$$

Logistic 回归模型:

$$p = \frac{e^{\beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m}}{1 + e^{\beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m}}. \quad (10.29)$$

Logit 变换:

$$\text{logit}(p) = \ln \left(\frac{p}{1-p} \right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m. \quad (10.30)$$

具体例子

例 10.7: 分组数据的 Logistic 回归

购房数据: $n = 313$ 名顾客, 按家庭年收入分为 9 组, 记录每组签订意向人数和实际购房人数。

表 10.13 签订购房意向和最终买房的客户数据

序号	年收入 x /万元	签订人数 n_i	实际购房 m_i	比例 p_i	Logit p_i^*
1	1.5	25	8	0.3200	-0.7538
2	2.5	32	13	0.4063	-0.3795
3	3.5	58	26	0.4483	-0.2076
4	4.5	52	22	0.4231	-0.3102
5	5.5	43	20	0.4651	-0.1398
6	6.5	39	22	0.5641	0.2578
7	7.5	28	16	0.5714	0.2877
8	8.5	21	12	0.5714	0.2877
9	9.5	15	10	0.6667	0.6931

Logistic 回归方程:

$$\hat{p} = \frac{1}{1 + e^{0.8863 - 0.1558x}}. \quad (10.37)$$

$R^2 = 0.924$, $F = 85.42$, $p = 3.59 \times 10^{-5}$ 。

年收入 9 万元客户购房概率: $\hat{p}_0 = 0.6262$ 。

例 10.8: 未分组数据的 Logistic 回归

调查公共交通出行方式, $y = 1$ 表示坐公交车, $y = 0$ 表示骑自行车。自变量: 年龄 x_1 、月收入 x_2 、性别 x_3 。

剔除不显著的 x_2 后:

$$\hat{p} = \frac{1}{1 + e^{2.6285 - 0.1023x_1 + 2.2239x_3}}. \quad (10.31)$$

例 10.11：企业还款能力预测

20 家企业数据，3 个评价指标，建立 Logistic 回归模型预测 2 家新企业。

回归结果：

$$p = \frac{1}{1 + e^{1.3319 + 0.3069x_1 - 2.9269x_2 - 2.3689x_3}}. \quad (10.32)$$

判定准则： $p > 0.5$ 预测为 1（可贷款），否则为 0。模型准确率 100%。

10.5.3 赔率与赔率比

赔率（Odds）

$$\text{odds}(A) = \frac{P(A)}{1 - P(A)}. \quad (10.33)$$

赔率比（Odds Ratio）

$$\text{OR}(A, B) = \frac{\text{odds}(A)}{\text{odds}(B)}. \quad (10.34)$$

例 10.10：年收入 9 万元客户购房赔率：

$$\text{odds} = e^{-0.8863 + 0.1558 \times 9} = 1.6752. \quad (10.35)$$

年收入 9 万对 8 万的赔率比：

$$\text{OR} = e^{0.1558 \times (9-8)} = 1.1686. \quad (10.36)$$

10.5.4 Probit 回归模型

Probit 回归使用标准正态分布的反函数作为连接函数：

$$\Phi^{-1}(\pi_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_m x_{im}. \quad (10.37)$$

例 10.9：购房数据的 Probit 回归：

$$\Phi^{-1}(\hat{p}) = -0.5518 + 0.0970x. \quad (10.38)$$

年收入 9 万元： $\hat{p}_0 = \Phi(-0.5518 + 0.0970 \times 9) = 0.6260$ 。

代码实现

例 10.7：分组 Logistic 回归（线性化方法）

```
1 # 程序文件 ex10_7_1.py
2 import numpy as np
3 import statsmodels.api as sm
4
5 a = np.loadtxt('data10_7_1.txt')
6 x = a[:, 0]
```

```

7     pi = a[:, 2] / a[:, 1]
8     yi = np.log(pi / (1 - pi))
9     X = sm.add_constant(x)
10
11    md = sm.OLS(yi, X).fit()
12    print(md.summary())
13
14    p0 = 1 / (1 + np.exp(-md.predict([1, 9])))
15    print('p0 =', p0)

```

例 10.7: 使用 glm

```

1     # 程序文件 ex10_7_2.py
2     import numpy as np
3     import statsmodels.api as sm
4
5     a = np.loadtxt('data10_7_1.txt')
6     x = a[:, 0]
7     y = np.vstack([a[:, 2], a[:, 1] - a[:, 2]]).T
8     d = {'x': x, 'y': y}
9
10    md = sm.formula.glm('y ~ x', d, family=sm.families.Binomial()).
11    fit()
12    print(md.summary())

```

例 10.8: 未分组 Logistic 回归

```

1     # 程序文件 ex10_8_1.py
2     import numpy as np
3     import statsmodels.api as sm
4
5     a = np.loadtxt('data10_8.txt')
6     x = np.vstack([a[:, 1:4], a[:, 6:-1]])
7     y = np.hstack([a[:, 4], a[:, 9]])
8     d = {'x1': x[:, 0], 'x2': x[:, 1], 'x3': x[:, 2], 'y': y}
9
10    md = sm.formula.logit('y ~ x1 + x2 + x3', d).fit()
11    print(md.summary())
12
13    md2 = sm.formula.logit('y ~ x1 + x3', d).fit()
14    print(md2.summary())

```

例 10.9: Probit 回归

```

1     # 程序文件 ex10_9.py
2     import numpy as np
3     import statsmodels.api as sm
4     from scipy.stats import norm
5

```



```

6     a = np.loadtxt('data10_7_1.txt')
7     x = a[:, 0]
8     pi = a[:, 2] / a[:, 1]
9     yi = norm.ppf(pi)
10    X = sm.add_constant(x)
11
12    md = sm.OLS(yi, X).fit()
13    print(md.summary())
14
15    p0 = norm.cdf(md.predict([1, 9]))
16    print('p0 =', p0)

```

例 10.11: 企业还款能力预测

```

1     # 程序文件 ex10_11.py
2     import numpy as np
3     import statsmodels.api as sm
4
5     a = np.loadtxt('data10_11.txt')
6     d = {'x1': a[:, 0], 'x2': a[:, 1], 'x3': a[:, 2], 'y': a[:,
7           -1]}
8
9     md = sm.formula.logit('y ~ x1 + x2 + x3', d).fit(method='bfgs')
10    print(md.summary())
11
12    # 预测新企业
13    new = {'x1': [-49.2, 40.6], 'x2': [-17.2, 26.4], 'x3': [0.3,
14           1.8]}
15    print(md.predict(new))

```

方法评述

方法	优点	缺点
Logistic 回归	适用二分类, 结果可解释性强	需要较大样本量
Probit 回归	适用于正态分布假设	解释性不如 Logistic
赔率/赔率比	直观解释风险大小	仅适用于二分类
glm 函数	统一框架, 灵活选择分布	需理解分布族

方法选择建议

- 二分类因变量: 优先使用 Logistic 回归。
- 分组数据: 可用线性化方法 (Logit 变换后做 OLS) 或 glm。
- 未分组数据: 使用 logit() 或 glm(family=Binomial())。

-
- 需要概率预测：使用 `predict()` 方法。
 - 赔率分析：根据 $\exp(\hat{\beta}_j)$ 解释自变量对赔率的影响。

第 11 章

聚类分析与判别分析

多元分析是多变量的统计分析方法，包含了丰富的理论成果和众多的应用方法，如回归分析、方差分析、判别分析、聚类分析、主成分分析、因子分析和典型相关分析等。

聚类分析和判别分析的共同点是对样本或变量进行分类。区别在于：

- 聚类分析：无监督分类，不知道有多少类别，根据数据本身的距离或相似性进行归类。
- 判别分析：有监督分类，已知类别和训练样本，建立判别函数对新样本进行分类。

聚类分析包括 Q 型聚类（对样本分类）和 R 型聚类（对指标分类）。

11.1 聚类分析

11.1.1 数据变换

设有 n 个样品，每个样品测得 p 项指标，原始数据矩阵为：

$$A = (a_{ij})_{n \times p}. \quad (11.1)$$

常用数据变换方法：

中心化变换

$$b_{ij} = a_{ij} - \bar{a}_j, \quad \bar{a}_j = \frac{1}{n} \sum_{i=1}^n a_{ij}. \quad (11.2)$$

规格化变换

$$b_{ij} = \frac{a_{ij} - \min_i(a_{ij})}{\max_i(a_{ij}) - \min_i(a_{ij})}. \quad (11.3)$$

标准化变换

$$b_{ij} = \frac{a_{ij} - \bar{a}_j}{s_j}, \quad s_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (a_{ij} - \bar{a}_j)^2}. \quad (11.4)$$

11.1.2 亲疏程度的测度

(1) 距离 (Q 型聚类) 闵氏距离:

$$d_q(\omega_i, \omega_j) = \left(\sum_{k=1}^p |b_{ik} - b_{jk}|^q \right)^{1/q}. \quad (11.5)$$

- $q = 1$: 绝对值距离 (Manhattan 距离)
- $q = 2$: 欧几里得距离
- $q = \infty$: 切比雪夫距离

马氏距离:

$$d(\omega_i, \omega_j) = \sqrt{(\omega_i - \omega_j)^T \Sigma^{-1} (\omega_i - \omega_j)}. \quad (11.6)$$

(2) 相似系数 (R 型聚类) 夹角余弦:

$$\cos \theta_{ij} = \frac{\sum_{k=1}^p b_{ik} b_{jk}}{\sqrt{\sum_{k=1}^p b_{ik}^2} \sqrt{\sum_{k=1}^p b_{jk}^2}}. \quad (11.7)$$

皮尔逊相关系数:

$$r_{ij} = \frac{\sum_{k=1}^p (b_{ik} - \bar{b}_i)(b_{jk} - \bar{b}_j)}{\sqrt{\sum_{k=1}^p (b_{ik} - \bar{b}_i)^2} \sqrt{\sum_{k=1}^p (b_{jk} - \bar{b}_j)^2}}. \quad (11.8)$$

11.1.3 系统聚类法

(1) 类间距离定义

- 最短距离法: $D_{ij} = \min_{\omega_i \in G_i, \omega_j \in G_j} d(\omega_i, \omega_j)$
- 最长距离法: $D_{ij} = \max_{\omega_i \in G_i, \omega_j \in G_j} d(\omega_i, \omega_j)$
- 重心法: $D_{ij} = d(\bar{x}^{(i)}, \bar{x}^{(j)})$
- 类平均法: $D_{ij} = \frac{1}{n_i n_j} \sum_{\omega_i \in G_i} \sum_{\omega_j \in G_j} d(\omega_i, \omega_j)$
- 离差平方和法 (Ward 法): $D_{ij} = L_{ij} - L_i - L_j$

(2) `scipy.cluster.hierarchy` 模块 **linkage**: 生成聚类树

```
Z = linkage(y, method='single', metric='euclidean')
```

method 取值: 'single' (最短距离)、'complete' (最长距离)、'average' (类平均)、'ward' (离差平方和)、'centroid' (重心法)

metric 取值: 'euclidean'、'cityblock'、'chebyshev'、'mahalanobis'、'cosine'、'correlation'

fcluster: 根据聚类树分类

```
T = fcluster(Z, t=k, criterion='maxclust')
```

dendrogram: 画聚类树状图

```
dendrogram(Z, labels=labels)
```

11.1.4 动态聚类法——K 均值聚类

(1) 算法思想 将 n 个样品分成 C 类, 使簇内离差平方和最小:

$$J_e = \sum_{i=1}^C \sum_{\omega_j \in G_i} \|\omega_j - m_i\|^2. \quad (11.9)$$

其中 m_i 为第 i 类的聚类中心。

(2) 算法步骤

1. 初始化: 选择 C 个初始聚类中心;
2. 按最小距离原则将每个样品分配到最近的类;
3. 重新计算各类的聚类中心;
4. 重复步骤 2-3 直到聚类中心不再变化。

(3) 最佳 k 值的确定 拐点法: 计算不同 k 值下的簇内离差平方和 J_k , 找曲线拐点。

轮廓系数法:

$$S_i = \frac{b_i - a_i}{\max(a_i, b_i)}, \quad (11.10)$$

其中 a_i 为样本 i 到同簇其他样本的平均距离, b_i 为样本 i 到其他簇的平均距离的最小值。轮廓系数越接近 1, 聚类效果越好。

具体例子

例 11.1: 最短距离法聚类

5 个样品指标值: 2, 3, 3.5, 7, 9。用最短距离法聚类。

聚类结果: $\{\omega_1, \omega_2, \omega_3\}$ 和 $\{\omega_4, \omega_5\}$ 。

例 11.2: 9 个农业区经济指标聚类

7 项经济指标, 先用规格化变换消除量纲, 再用最短距离法聚类。

分 3 类时: $\{\omega_1, \omega_2, \omega_3, \omega_4, \omega_8\}$ 、 $\{\omega_5, \omega_6\}$ 、 $\{\omega_7\}$ 。

例 11.4: K 均值聚类

4 个样本点: (1, 3)、(1.5, 3.2)、(1.3, 2.8)、(3, 1), 分成 2 类。

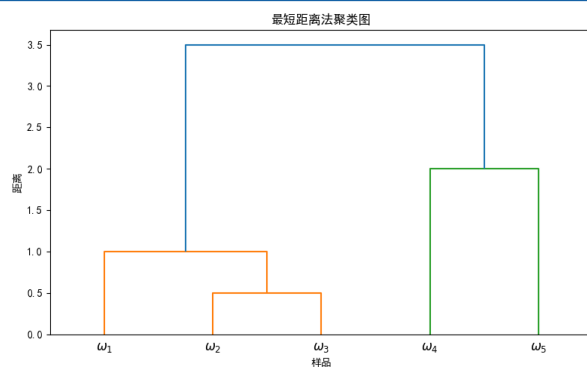


图 11.1 最短距离法聚类图

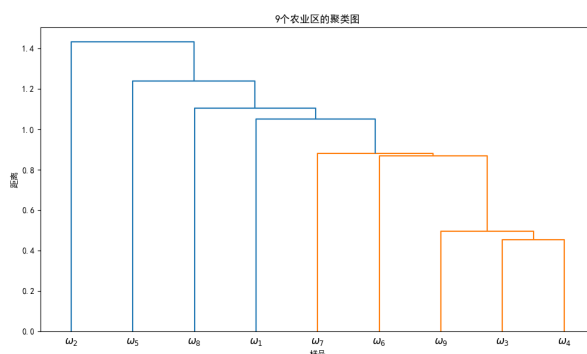


图 11.2 9个农业区的聚类图

最终聚类： $G_1 = \{\omega_1, \omega_2, \omega_3\}$ ， $G_2 = \{\omega_4\}$ 。

例 11.8：服装标准制定中的 R 型聚类

14 个部位测量数据的相关系数矩阵，用最长距离法进行变量聚类。

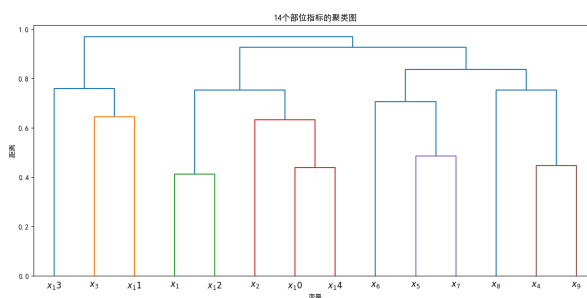


图 11.3 14个部位指标的聚类图

代码实现

例 11.1：最短距离法聚类

```
1 # 程序文件 ex11_1_1.py
2 import scipy.cluster.hierarchy as sch
3 import numpy as np
4 import pylab as plt
5
```

```

6     a = np.array([[2, 3, 3.5, 7, 9]]).T
7     Z = sch.linkage(a, method='single')
8     labels = [f'$\\omega_{i+1}$' for i in range(5)]
9     sch.dendrogram(Z, labels=labels)
10    plt.show()

```

例 11.2: 农业区聚类

```

1     # 程序文件 ex11_2.py
2     import numpy as np
3     import scipy.cluster.hierarchy as sch
4     import pylab as plt
5
6     a = np.loadtxt('data11_2.txt')
7     # 规格化变换
8     b = (a - a.min(axis=0)) / (a.max(axis=0) - a.min(axis=0))
9     Z = sch.linkage(b, method='single')
10    labels = [f'$\\omega_{i+1}$' for i in range(len(a))]
11    sch.dendrogram(Z, labels=labels)
12    plt.show()

```

例 11.4: K 均值聚类

```

1     # 程序文件 ex11_4.py
2     import numpy as np
3     from sklearn.cluster import KMeans
4
5     a = np.array([[1, 3], [1.5, 3.2], [1.3, 2.8], [3, 1]])
6     model = KMeans(n_clusters=2, random_state=0).fit(a)
7     print('聚类标签:', model.labels_)
8     print('聚类中心:\n', model.cluster_centers_)

```

例 11.6: 拐点法确定最佳 k 值

```

1     # 程序文件 ex11_6.py
2     import numpy as np
3     from sklearn.cluster import KMeans
4     import pylab as plt
5
6     a = np.loadtxt('data11_2.txt')
7     b = (a - a.min(axis=0)) / (a.max(axis=0) - a.min(axis=0))
8
9     SSE = []
10    K = range(2, len(a)+1)
11    for k in K:
12        model = KMeans(n_clusters=k, random_state=0).fit(b)
13        SSE.append(model.inertia_)
14
15    plt.plot(K, SSE, 'o-')

```

```

16 plt.xlabel('簇数 k')
17 plt.ylabel('簇内离差平方和')
18 plt.show()

```

例 11.7: 轮廓系数法确定最佳 k 值

```

1  # 程序文件 ex11_7.py
2  import numpy as np
3  from sklearn.cluster import KMeans
4  from sklearn.metrics import silhouette_score
5  import pylab as plt
6
7  a = np.loadtxt('data11_2.txt')
8  b = (a - a.min(axis=0)) / (a.max(axis=0) - a.min(axis=0))
9
10 S = []
11 K = range(2, len(a))
12 for k in K:
13     model = KMeans(n_clusters=k, random_state=0).fit(b)
14     S.append(silhouette_score(b, model.labels_))
15
16 plt.plot(K, S, 'o-')
17 plt.xlabel('簇数 k')
18 plt.ylabel('轮廓系数')
19 plt.show()

```

例 11.8: R 型聚类

```

1  # 程序文件 ex11_8.py
2  import pandas as pd
3  import scipy.cluster.hierarchy as sch
4  import numpy as np
5  import pylab as plt
6
7  a = pd.read_excel('data11_8.xlsx', header=None)
8  b = a.values.T
9  # 取相关系数矩阵的上三角元素
10 r = b[np.triu_indices_from(b, k=1)]
11 d = 1 - abs(r) # 距离 = 1 - |相关系数|
12 Z = sch.linkage(d, method='complete')
13 sch.dendrogram(Z, labels=range(1, 15))
14 plt.show()

```

优缺点分析

方法	优点	缺点
系统聚类	直观易懂, 谱系图展示层次关系	计算量大, 不适合大规模数据
K 均值聚类	快速高效, 适合大规模数据	需预先指定 k 值, 对初始值敏感
最短距离法	简单直观	易受噪声影响
最长距离法	类间分离明显	对异常值敏感
Ward 法	聚类效果好	计算量大

方法选择建议

- 小样本数据：使用系统聚类，画谱系图直观判断类别。
- 大样本数据：使用 K 均值聚类，速度快。
- 数据量纲不同：先进行标准化或规格化变换。
- 确定 k 值：使用拐点法或轮廓系数法辅助判断。
- 变量聚类（R 型）：使用相关系数或夹角余弦作为相似性度量。

应用场景

- 市场细分：根据消费者特征进行客户分类。
- 图像分割：基于像素特征的聚类分割。
- 生物信息学：基因表达数据的聚类分析。
- 文档分类：文本数据的主题聚类。
- 异常检测：识别离群点。
- 经济区划：区域经济指标聚类。

11.2 判别分析

判别分析是多元统计分析中用于判别样本所属类型的一种统计分析方法。与聚类分析不同，判别分析要求具有一定的先验信息，是在已知研究对象分成若干类型并已取得各种类型的一批已知样品的观测数据基础上，根据某些准则建立判别式，然后对未知类型的样品进行判别分类。

11.2.1 距离判别法

数学原理

距离判别法的基本思想：根据已知分类的数据，分别计算各类的重心（均值），对任意给定的样品，若它与第 i 类的重心距离最近，就认为它来自第 i 类。

(1) 两个总体的情形 设有两个总体 G_1, G_2 , 均值向量和协方差矩阵分别为 $\mu^{(1)}, \mu^{(2)}$ 和 $\Sigma^{(1)}, \Sigma^{(2)}$ 。

样品 X 到总体 G_i 的马氏距离为:

$$d^2(X, G_i) = (X - \mu^{(i)})^T (\Sigma^{(i)})^{-1} (X - \mu^{(i)}), \quad i = 1, 2. \quad (11.11)$$

判别准则:

$$\begin{cases} X \in G_1, & d(X, G_1) < d(X, G_2), \\ X \in G_2, & d(X, G_1) > d(X, G_2). \end{cases} \quad (11.12)$$

协方差相等时 ($\Sigma^{(1)} = \Sigma^{(2)} = \Sigma$):

令 $\mu = \frac{\mu^{(1)} + \mu^{(2)}}{2}$, 判别函数为:

$$W(X) = (X - \mu)^T \Sigma^{-1} (\mu^{(1)} - \mu^{(2)}). \quad (11.13)$$

判别规则:

$$\begin{cases} X \in G_1, & W(X) > 0, \\ X \in G_2, & W(X) < 0. \end{cases} \quad (11.14)$$

当 $\mu^{(1)}, \mu^{(2)}, \Sigma$ 未知时, 用样本估计:

$$\hat{\mu}^{(i)} = \bar{X}^{(i)}, \quad \hat{\Sigma} = \frac{1}{n_1 + n_2 - 2} (L_1 + L_2), \quad (11.15)$$

其中 $L_i = \sum_{j=1}^{n_i} (X_j^{(i)} - \bar{X}^{(i)})(X_j^{(i)} - \bar{X}^{(i)})^T$ 。

协方差不等时: 判别函数为 X 的二次函数:

$$W(X) = (X - \mu^{(1)})^T (\Sigma^{(1)})^{-1} (X - \mu^{(1)}) - (X - \mu^{(2)})^T (\Sigma^{(2)})^{-1} (X - \mu^{(2)}). \quad (11.16)$$

(2) 多个总体的情形 设有 k 个总体 $G_1, G_2, \dots, G_k$, 协方差矩阵相同为 Σ 。

判别函数:

$$W_{ij}(X) = \left(X - \frac{\mu^{(i)} + \mu^{(j)}}{2} \right)^T \Sigma^{-1} (\mu^{(i)} - \mu^{(j)}), \quad i, j = 1, 2, \dots, k. \quad (11.17)$$

判别规则:

$$\begin{cases} X \in G_i, & W_{ij}(X) > 0, \forall j \neq i, \\ \text{待判}, & \exists j, W_{ij}(X) = 0. \end{cases} \quad (11.18)$$

11.2.2 Fisher 判别

Fisher 判别的基本思想是投影, 将高维数据投影到某个方向上, 使得类与类之间得以分离。

数学原理

设两总体 G_1, G_2 的均值向量为 μ_1, μ_2 , 公共协方差矩阵为 Σ 。寻找投影方向 a , 使

$$\frac{[a^T(\mu_1 - \mu_2)]^2}{a^T \Sigma a} \quad (11.19)$$

达到最大。

最优解为:

$$a = \Sigma^{-1}(\mu_1 - \mu_2). \quad (11.20)$$

Fisher 线性判别函数:

$$y(X) = (\mu_1 - \mu_2)^T \Sigma^{-1} X. \quad (11.21)$$

判别阈值:

$$K = \frac{1}{2}(\mu_1 - \mu_2)^T \Sigma^{-1}(\mu_1 + \mu_2). \quad (11.22)$$

判别规则:

$$\begin{cases} X \in G_1, & y(X) \geq K, \\ X \in G_2, & y(X) < K. \end{cases} \quad (11.23)$$

等价地, 定义判别函数:

$$W(X) = \left(X - \frac{\mu_1 + \mu_2}{2} \right)^T \Sigma^{-1}(\mu_1 - \mu_2). \quad (11.24)$$

当参数未知时, 用样本均值 $\bar{X}^{(1)}, \bar{X}^{(2)}$ 和 pooled 协方差矩阵 $\hat{\Sigma}$ 估计。

11.2.3 判别准则的评价

(1) 回代误判率 以全体训练样本作为新样品, 逐个代入判别准则进行回判:

$$\hat{P} = \frac{N_1 + N_2}{m + n}, \quad (11.25)$$

其中 N_1, N_2 分别为两类中被误判的个数。

回代误判率是有偏估计, 往往比真实误判率小。

(2) 交叉误判率 每次剔除一个样品, 用其余样品建立判别准则, 对剔除样品进行判别:

$$\hat{P}^* = \frac{N_1^* + N_2^*}{m + n}. \quad (11.26)$$

交叉误判率是较为合理的估计。

(3) 交叉验证 (sklearn)

```
1 from sklearn.model_selection import cross_val_score
2 scores = cross_val_score(model, X, y, cv=k)
```

具体例子

例 11.9：蝶虫判别（马氏距离）

Af 蝶虫 9 只，Apf 蝶虫 6 只，测量触角和翼长。对 3 个待判样本进行识别。

判别函数 $W(X) = d_1^2(X) - d_2^2(X)$, 3 个待判样本的判别函数值分别为 $-0.5182, -2.0379, -1.5130$, 均判为 Af 类，误判率为 0。

例 11.11：盐矿判别（Fisher 判别）

钾盐（A 类）和钠盐（B 类）各 5 个样本，4 个指标，对 6 个待判样本判别。

Fisher 判别函数：

$$y = -3.3447 + 0.4291x_1 + 0.3813x_2 - 0.7737x_3 + 0.0651x_4. \quad (11.27)$$

判别结果：样本 1,4,5 为 B 类，样本 2,3,6 为 A 类，误判率为 0。

例 11.10：心电图判别

健康人 10 例，病人 6 例，5 个心电图指标，对 2 个待判样本判别。两个待判样本均判为第 1 类（健康人），误判率为 0。

例 11.12：交叉验证

将例 11.10 数据分成 2 组进行交叉验证，两组准确率分别为 87.5% 和 75%。

代码实现

例 11.9：马氏距离判别

```

1  # 程序文件 ex11_9.py
2  import numpy as np
3  import sympy as sp
4  from numpy.linalg import inv
5
6  # 读取数据
7  f = open('data11_9.txt')
8  d = f.readlines()
9  a = np.array([[list(map(eval, e.split())) for e in d[:2]])])
10 b = np.array([[list(map(eval, e.split())) for e in d[2:]]])
11
12 mu1 = a.mean(axis=1, keepdims=True)
13 mu2 = b.mean(axis=1, keepdims=True)
14 s1 = np.cov(a, ddof=1)
15 s2 = np.cov(b, ddof=1)
16
17 sp.var('x1, x2')
18 X = sp.Matrix([x1, x2])
19 d1 = (X - mu1).T @ inv(s1) @ (X - mu1)
20 d2 = (X - mu2).T @ inv(s2) @ (X - mu2)
21 W = sp.lambdify((x1, x2), d1 - d2, 'numpy')
```

```

22
23     # 待判样本
24     sol = W([1.24, 1.28, 1.40], [1.80, 1.84, 2.04])
25     print(np.round(sol, 4))

```

例 11.11: Fisher 判别

```

1     # 程序文件 ex11_11.py
2     import numpy as np
3     from numpy.linalg import inv
4     from sklearn.discriminant_analysis import
5         LinearDiscriminantAnalysis as LDA
6
7     a = np.loadtxt('data11_11.txt')
8     X = a[:, :4]
9     y = a[:, 4] # 类别标签
10
11     # Fisher 判别 (手动计算)
12     mu1 = X[y==0].mean(axis=0)
13     mu2 = X[y==1].mean(axis=0)
14     S = np.cov(X, rowvar=False, ddof=1)
15     k = inv(S) @ (mu1 - mu2)
16     b = -(mu1 + mu2).T @ inv(S) @ (mu1 - mu2) / 2
17
18     # 待判样本
19     x_test = a[10:, :4]
20     val = x_test @ k + b
21     print('判别函数值:', val)
22
23     # 库函数
24     md = LDA().fit(X, y)
25     print('库函数预测:', md.predict(x_test))

```

例 11.12: 交叉验证

```

1     # 程序文件 ex11_12.py
2     import pandas as pd
3     from sklearn.discriminant_analysis import
4         LinearDiscriminantAnalysis as LDA
5     from sklearn.model_selection import cross_val_score
6
7     a = pd.read_excel('data11_10.xlsx', header=None)
8     b = a.values
9     X = b[:-2, :-1].astype(float)
10    y = b[:-2, -1].astype(float)
11
12    md = LDA()
13    scores = cross_val_score(md, X, y, cv=2)

```

13

```
print('交叉验证准确率:', scores)
```

优缺点分析

方法	优点	缺点
距离判别	直观易懂, 无需分布假设	协方差矩阵不同时计算复杂
Fisher 判别	降维可视化, 计算简单	仅适用于线性可分数据
交叉验证	较准确估计泛化能力	计算量大
回代法	计算简单快速	有偏估计 (偏低)

方法选择建议

- 总体协方差相同：使用线性判别（距离判别或 Fisher 判别）。
- 总体协方差不同：使用二次判别（QDA）。
- 数据高维且线性可分：使用 Fisher 判别。
- 需要评估模型：使用交叉验证估计误判率。
- 小样本：使用留一交叉验证（LOOCV）。

应用场景

- 医学诊断：根据检查指标判别疾病类型。
- 金融风控：判别贷款客户是否违约。
- 生物分类：根据形态特征判别物种。
- 图像识别：根据特征判别物体类别。
- 地质勘探：根据矿物成分判别矿藏类型。

第 12 章

主成分分析与因子分析

主成分分析是一种通过降维技术将多个变量化为少数几个主成分的统计分析方法，在数学建模中通常用来做数据压缩（降维）、系统评估、回归、加权分析等。

因子分析可视为主成分分析的推广，有确定的统计模型，观察数据在模型中被分解为公共因子、特殊因子和误差三个部分。

12.1 主成分分析

12.1.1 主成分分析的基本原理和步骤

数学原理

设有 m 个原始变量 $X_1, X_2, \dots, X_m$ ，主成分 F_i 是原始变量的线性组合：

$$F_i = c_{i1}X_1 + c_{i2}X_2 + \dots + c_{im}X_m, \quad i = 1, 2, \dots, m, \quad (12.1)$$

满足 $\sum_{j=1}^m c_{ij}^2 = 1$ ，且不同主成分之间协方差为 0。

(1) 计算步骤 步骤 1：数据标准化

$$y_j = \frac{x_j - \mu_j}{s_j}, \quad \mu_j = \frac{1}{n} \sum_{i=1}^n a_{ij}, \quad s_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (a_{ij} - \mu_j)^2}. \quad (12.2)$$

步骤 2：计算相关系数矩阵

$$R = (r_{ij})_{m \times m}, \quad r_{ij} = \frac{1}{n-1} \sum_{k=1}^n b_{ki}b_{kj}. \quad (12.3)$$

步骤 3：求特征值与特征向量

计算 R 的特征值 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m$ 及对应的标准正交化特征向量 $u_1, u_2, \dots, u_m$ 。

第 i 个主成分为：

$$F_i = u_{1i}y_1 + u_{2i}y_2 + \dots + u_{mi}y_m. \quad (12.4)$$

步骤 4：计算贡献率

第 i 个主成分的贡献率:

$$w_i = \frac{\lambda_i}{\sum_{k=1}^m \lambda_k}. \quad (12.5)$$

前 k 个主成分的累积贡献率:

$$\text{累积贡献率} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^m \lambda_i}. \quad (12.6)$$

一般选取累积贡献率达 85% 以上的主成分。

(2) 主成分回归 当自变量存在多重共线性时, 主成分回归将原始自变量变换为主成分, 选择其中重要的主成分作为新自变量, 用最小二乘法估计参数, 再变换回原始变量。

(3) 核主成分分析 (KPCA) KPCA 通过非线性映射 Φ 将输入向量映射到高维特征空间, 然后对映射数据做 PCA, 处理非线性相关问题。

核函数:

- 多项式核: $k(x, y) = (xy^T + c)^d$
- 高斯核 (RBF): $k(x, y) = \exp(-\gamma \|x - y\|^2)$
- Sigmoid 核: $k(x, y) = \tanh(\alpha xy^T + \beta)$

核矩阵中心化:

$$\tilde{K} = K - J_n K - K J_n + J_n K J_n, \quad J_n = \frac{1}{n} \mathbf{1}_{n \times n}. \quad (12.7)$$

具体例子

例 12.1: 重庆市居民消费水平主成分分析

8 个消费指标, 10 年数据。前两个主成分累积贡献率达 94.82%。

第一主成分 (综合消费水平):

$$F_1 = 0.3919y_1 + 0.3844y_2 + 0.3059y_3 + 0.3923y_4 + 0.3854y_5 + 0.3896y_6 + 0.3839y_7 + 0.0591y_8. \quad (12.8)$$

第二主成分 (杂项商品与服务):

$$F_2 = -0.0210y_1 + 0.0221y_2 + 0.4778y_3 - 0.0891y_4 - 0.1988y_5 - 0.1043y_6 - 0.1138y_7 + 0.8363y_8. \quad (12.9)$$

例 12.2: Hald 水泥数据主成分回归

4 个化学成分对热量的影响, 13 组数据。

直接回归方程:

$$y = 62.4054 + 1.5511x_1 + 0.5102x_2 + 0.102x_3 - 0.144x_4. \quad (12.10)$$

主成分回归方程：

$$y = 85.7433 + 1.3119x_1 + 0.2694x_2 - 0.1428x_3 - 0.3801x_4. \quad (12.11)$$

主成分回归具有更小的残差方差，更稳定。

例 12.3：高校科技创新能力 KPCA 评价

15 所高校，14 项指标。KPCA 前 2 个主成分累积贡献率达 90.88%，优于 PCA 需取 4 个主成分。

排名前 3：上海交通大学、浙江大学、华中科技大学。

代码实现

例 12.1：主成分分析

```
1  # 程序文件 ex12_1.py
2  import numpy as np
3  from sklearn.decomposition import PCA
4  from scipy.stats import zscore
5
6  a = np.loadtxt('data12_1.txt')
7  b = zscore(a, ddof=1)  # 标准化
8
9  md = PCA().fit(b)
10 print('特征值:', md.explained_variance_)
11 print('贡献率:', md.explained_variance_ratio_)
12 print('主成分系数:\n', md.components_)
13 print('累积贡献率:', np.cumsum(md.explained_variance_ratio_))
14
15 # 调整主成分系数符号（使第一主成分系数为正）
16 components = md.components_
17 for i in range(components.shape[0]):
18     if np.sum(components[i]) < 0:
19         components[i] = -components[i]
20 print('调整后的主成分系数:\n', components)
```

例 12.2：主成分回归

```
1  # 程序文件 ex12_2.py
2  import numpy as np
3  from sklearn.decomposition import PCA
4  import statsmodels.api as sm
5
6  a = np.loadtxt('data12_2.txt')
7  X, y = a[:, :-1], a[:, -1]
8
9  # 标准化
10 mu = X.mean(axis=0)
```

```

11     s = X.std(axis=0, ddof=1)
12     X_std = (X - mu) / s
13
14     # PCA
15     md = PCA().fit(X_std)
16     print('特征值:', md.explained_variance_)
17     print('累积贡献率:', np.cumsum(md.explained_variance_ratio_))
18
19     # 选取前3个主成分
20     n_comp = 3
21     Z = X_std @ md.components_[0:n_comp].T
22
23     # 主成分回归
24     model_pcr = sm.OLS(y, sm.add_constant(Z)).fit()
25     print('主成分回归系数:', model_pcr.params)
26
27     # 恢复原始变量系数
28     beta = model_pcr.params[1:] @ md.components_[0:n_comp] / s
29     beta0 = model_pcr.params[0] - sum(beta * mu)
30     print(f'原始方程: y = {beta0:.4f} + {beta[0]:.4f}x1 + {beta[1]:.4f}x2 + {beta[2]:.4f}x3 + {beta[3]:.4f}x4')

```

例 12.3: KPCA

```

1     # 程序文件 ex12_3.py
2     import numpy as np
3     import pandas as pd
4     from sklearn.decomposition import PCA
5     from scipy.stats import zscore
6
7     a = pd.read_excel('data12_3.xlsx', header=None)
8     X = a.values.astype(float)
9     X_std = zscore(X, ddof=1)
10
11     # ===== PCA =====
12     md = PCA().fit(X_std)
13     print('PCA特征值:', md.explained_variance_)
14     print('PCA贡献率:', md.explained_variance_ratio_)
15
16     # ===== KPCA =====
17     n = X_std.shape[0]
18     # 多项式核  $k(x, y) = (1 + xy^T)^3$ 
19     K = (1 + X_std @ X_std.T) ** 3
20     J = np.ones((n, n)) / n
21     K_center = K - J@K - K@J + J@K@J
22
23     val, vec = np.linalg.eig(K_center)

```

```
24     idx = np.argsort(-val)
25     val, vec = val[idx], vec[:, idx]
26
27     print('KPCA特征值:', val[:5])
28     print('KPCA贡献率:', val[:5] / val.sum())
29
30     # 取前2个主成分
31     n_comp = 2
32     scores = K_center @ vec[:, :n_comp]
33     weights = val[:n_comp] / val.sum()
34     final_score = scores @ weights
35
36     print('综合得分:', final_score)
```

优缺点分析

方法	优点	缺点
PCA	降维效果好，计算简单	受量纲影响，仅处理线性关系
主成分回归	克服多重共线性	主成分解释性差
KPCA	处理非线性关系，降维效果好	计算量大，核函数选择主观

方法选择建议

- 降维需求：使用 PCA，累积贡献率取 85% 以上。
- 多重共线性：使用主成分回归。
- 非线性关系：使用 KPCA。
- 综合评价：以贡献率为权重，计算综合得分。
- 符号调整：特征向量可相差负号，按实际情况调整。

应用场景

- 综合评价：经济发展、教育质量、科技创新能力评价。
- 数据压缩：高维数据的降维处理。
- 回归分析：处理多重共线性问题。
- 图像处理：人脸识别中的特征提取。
- 金融分析：股票市场的因子分析。

12.2 因子分析

12.2.1 因子分析的数学理论

数学原理

(1) 因子分析模型 与主成分分析不同, 因子分析将原始变量 $x_1, x_2, \dots, x_m$ 分解为若干个公共因子的线性组合:

$$\begin{aligned} x_1 &= \mu_1 + a_{11}f_1 + a_{12}f_2 + \dots + a_{1p}f_p + \varepsilon_1, \\ x_2 &= \mu_2 + a_{21}f_1 + a_{22}f_2 + \dots + a_{2p}f_p + \varepsilon_2, \\ &\vdots \\ x_m &= \mu_m + a_{m1}f_1 + a_{m2}f_2 + \dots + a_{mp}f_p + \varepsilon_m, \end{aligned} \quad (12.13)$$

矩阵形式:

$$x = \mu + Af + \varepsilon, \quad (12.14)$$

其中:

- $f = [f_1, f_2, \dots, f_p]^T$ ($p < m$) 为公共因子向量
- $\varepsilon = [\varepsilon_1, \varepsilon_2, \dots, \varepsilon_m]^T$ 为特殊因子向量
- $A = (a_{ij})_{m \times p}$ 为因子载荷矩阵, a_{ij} 反映因子 f_j 对变量 x_i 的重要度

模型假设:

- f_j 互不相关且具有单位方差
- ε_i 互不相关且与 f_j 互不相关, $\text{Cov}(\varepsilon) = \Psi$ 为对角阵

由模型可得:

$$\text{Cov}(x) = AA^T + \Psi, \quad \text{Cov}(x, f) = A. \quad (12.15)$$

(2) 相关概念 共性方差 (共同度):

$$h_i^2 = \sum_{j=1}^p a_{ij}^2, \quad (12.12)$$

反映全部公共因子对变量 x_i 的方差贡献。

特殊方差: $\sigma_i^2 = D(\varepsilon_i)$ 。

因子贡献:

$$b_j^2 = \sum_{i=1}^m a_{ij}^2, \quad (12.13)$$

是公共因子 f_j 对总方差的贡献。

(3) 主成分法求因子载荷 设 $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_m$ 为相关系数矩阵 R 的特征值, $u_1, u_2, \cdots, u_m$ 为对应的正交特征向量, 取 $p < m$, 则因子载荷矩阵为:

$$A = [\sqrt{\lambda_1}u_1, \sqrt{\lambda_2}u_2, \cdots, \sqrt{\lambda_p}u_p]. \quad (12.14)$$

特殊因子方差估计:

$$\sigma_i^2 = 1 - \sum_{j=1}^p a_{ij}^2. \quad (12.15)$$

(4) 因子旋转 因子载荷矩阵不唯一, 右乘正交矩阵 T 可实现因子旋转, 使载荷矩阵结构简化, 便于解释。常用方法: 方差最大法 (Varimax)。

(5) 因子得分 将公共因子表示为原始变量的线性组合:

$$\hat{f} = (A^T \Psi^{-1} A)^{-1} A^T \Psi^{-1} (x - \mu). \quad (12.16)$$

具体例子

例 12.4: 主成分法求因子载荷矩阵

相关系数矩阵:

$$R = \begin{bmatrix} 1 & 0.2 & -0.2 \\ 0.2 & 1 & -0.4 \\ -0.2 & -0.4 & 1 \end{bmatrix}. \quad (12.16)$$

特征值: $\lambda_1 = 1.5464$, $\lambda_2 = 0.8536$, $\lambda_3 = 0.6$ 。

取前两个公共因子, 载荷矩阵:

$$A = \begin{bmatrix} 0.5717 & 0.8205 \\ 0.7809 & -0.3003 \\ -0.7809 & 0.3003 \end{bmatrix}. \quad (12.17)$$

因子模型:

$$x_1 = 0.5717f_1 + 0.8205f_2 + \varepsilon_1, \quad x_2 = 0.7809f_1 - 0.3003f_2 + \varepsilon_2, \quad x_3 = -0.7809f_1 + 0.3003f_2 + \varepsilon_3. \quad (12.18)$$

f_1 为物价因子, f_2 为投资因子。

例 12.5: 学生成绩因子分析

17 名学生 6 门课程成绩, 6 个变量: 语文、数学、英语、政治、历史、地理。

相关系数矩阵特征值: $\lambda_1 = 4.3627$, $\lambda_2 = 0.5096$, $\lambda_3 = 0.4994$, 前 3 个公共因子累积贡献率 89.53%。

因子载荷矩阵 (旋转后):

$$\begin{aligned}
 x_1 &= 0.8387f_1 + 0.3442f_2 + 0.3951f_3 + \varepsilon_1, \\
 x_2 &= 0.5289f_1 + 0.4729f_2 + 0.4339f_3 + \varepsilon_2, \\
 x_3 &= 0.3357f_1 + 0.9117f_2 + 0.2316f_3 + \varepsilon_3, \\
 x_4 &= 0.3112f_1 + 0.2641f_2 + 0.9107f_3 + \varepsilon_4, \\
 x_5 &= 0.7691f_1 + 0.3356f_2 + 0.2109f_3 + \varepsilon_5, \\
 x_6 &= 0.4552f_1 + 0.5695f_2 + 0.3216f_3 + \varepsilon_6.
 \end{aligned} \tag{12.19}$$

因子解释:

- f_1 : 文理基础因子 (语文、数学、历史载荷大)
- f_2 : 英语因子 (英语载荷大)
- f_3 : 政治因子 (政治载荷大)

因子得分函数:

$$\begin{aligned}
 f_1 &= 1.4319x_1^* + 0.0419x_2^* - 0.4609x_3^* - 0.5614x_4^* + 0.1234x_5^* + 0.0254x_6^*, \\
 f_2 &= -0.4381x_1^* - 0.0110x_2^* + 1.3219x_3^* - 0.1310x_4^* - 0.0360x_5^* - 0.0044x_6^*, \\
 f_3 &= -0.3607x_1^* - 0.0080x_2^* - 0.2270x_3^* + 1.3265x_4^* - 0.0352x_5^* - 0.0062x_6^*.
 \end{aligned} \tag{12.20}$$

代码实现

例 12.4: 主成分法求因子载荷

```

1  # 程序文件 ex12_4.py
2  import numpy as np
3
4  r = np.array([[1, 1/5, -1/5], [1/5, 1, -2/5], [-1/5, -2/5, 1]])
5  val, vec = np.linalg.eig(r)
6  A0 = vec * np.sqrt(val)    # 载荷矩阵
7  print('特征值:', val)
8  print('载荷矩阵:\n', A0)
9
10 num = int(input('请输入公共因子个数:'))
11 A = A0[:, :num]
12 Ac = np.sum(A**2, axis=0)    # 信息贡献
13 Ar = np.sum(A**2, axis=1)    # 共同度
14 print('因子贡献:', Ac)
15 print('共同度:', Ar)

```

例 12.5: 学生成绩因子分析

```

1  # 程序文件 ex12_5_1.py
2  import numpy as np
3  from factor_analyzer import FactorAnalyzer as FA
4  from scipy.stats import zscore

```

```

5     import pandas as pd
6
7     c = np.loadtxt('data12_5_1.txt')
8     d = zscore(c, ddof=1)    # 标准化
9
10    # 相关系数矩阵特征值
11    r = np.corrcoef(d.T)
12    val, vec = np.linalg.eig(r)
13    rate = val / val.sum()
14    print('特征值:', val)
15    print('贡献率:', rate)
16    print('累积贡献率:', np.cumsum(rate))
17
18    # 因子分析 (方差最大旋转)
19    fa = FA(n_factors=3, rotation='varimax')
20    fa.fit(d)
21    A = fa.loadings_    # 载荷矩阵
22    print('载荷矩阵:\n', np.round(A, 4))
23
24    # 特殊方差
25    s2 = 1 - np.sum(A**2, axis=1)
26    print('特殊方差:', np.round(s2, 4))
27
28    # 因子贡献
29    gx = np.sum(A**2, axis=0)
30    print('因子贡献:', np.round(gx, 4))
31
32    # 因子得分
33    ss = np.linalg.inv(np.diag(s2))
34    f_coef = ss @ A @ np.linalg.inv(A.T @ ss @ A)
35    scores = d @ f_coef
36    print('因子得分:\n', np.round(scores, 4))
37
38    # 综合评分
39    weights = gx / gx.sum()
40    total_score = scores @ weights
41    rank = np.argsort(-total_score) + 1
42    print('综合得分:', np.round(total_score, 4))
43    print('排名:', rank)

```

优缺点分析

方法	优点	缺点
因子分析	可解释性强, 有统计模型	因子个数选择主观

续下页

方法	优点	缺点
主成分法 方差最大旋转 因子得分	计算简单快速 简化结构，便于解释 可用于后续分析	载荷矩阵不唯一 可能改变因子含义 估计有误差

方法选择建议

- 需要解释公共因子含义：使用因子分析并做方差最大旋转。
- 降维目标：使用主成分分析（更简单）。
- 因子个数选择：取累积贡献率 85% 以上或特征值大于 1。
- 后续分析需要因子值：计算因子得分。
- 综合评价：以因子贡献率为权重计算综合得分。

应用场景

- 教育评估：学生成绩的因子分析，识别能力维度。
- 市场研究：消费者偏好的潜在因子挖掘。
- 心理学测量：人格特质的因子结构分析。
- 经济学：经济指标的潜在因素识别。
- 综合评价：多指标的综合评分与排名。

第 13 章

偏最小二乘回归分析

偏最小二乘 (PLS) 回归提供一种多对多线性回归建模的方法, 特别当两组变量的个数很多, 且都存在多重相关性, 而观测数据的数量 (样本量) 又较少时, PLS 回归具有传统方法所没有的优点。

PLS 回归在建模过程中集成了主成分分析、典型相关分析和线性回归分析的优点。

13.1 偏最小二乘回归分析

数学原理

考虑 p 个因变量 $y_1, y_2, \dots, y_p$ 与 m 个自变量 $x_1, x_2, \dots, x_m$ 的建模问题。

假设自变量组和因变量组的 n 次标准化观测数据矩阵分别为 A_0 和 B_0 。

(1) 提取成分 从两组变量分别提取第一对成分 t_1 和 u_1 :

$$t_1 = w_{11}x_1 + \dots + w_{1m}x_m = w_1^T X, \quad u_1 = v_{11}y_1 + \dots + v_{1p}y_p = v_1^T Y. \quad (13.1)$$

要求:

1. t_1 和 u_1 尽可能多地提取所在变量组的变异信息;
2. t_1 和 u_1 的相关程度达到最大。

化为条件极值问题:

$$\max w_1^T A_0^T B_0 v_1, \quad \text{s.t. } \|w_1\|^2 = 1, \|v_1\|^2 = 1. \quad (13.2)$$

求解矩阵 $M = A_0^T B_0 B_0^T A_0$ 的最大特征值对应的特征向量 w_1 , 且:

$$v_1 = \frac{1}{\theta_1} B_0^T A_0 w_1, \quad \theta_1 = \|A_0^T B_0 v_1\|. \quad (13.3)$$

(2) 建立回归方程 回归模型:

$$A_0 = \hat{t}_1 \alpha_1^T + A_1, \quad B_0 = \hat{t}_1 \beta_1^T + B_1. \quad (13.4)$$

回归系数的最小二乘估计：

$$\alpha_1 = \frac{A_0^T \hat{t}_1}{\|\hat{t}_1\|^2}, \quad \beta_1 = \frac{B_0^T \hat{t}_1}{\|\hat{t}_1\|^2}. \quad (13.5)$$

(3) 残差迭代 用残差矩阵 A_1 和 B_1 代替 A_0 和 B_0 重复以上步骤，得到第二对成分，依此类推。

若提取 r 个成分，则有：

$$A_0 = \hat{t}_1 \alpha_1^T + \cdots + \hat{t}_r \alpha_r^T + A_r, \quad B_0 = \hat{t}_1 \beta_1^T + \cdots + \hat{t}_r \beta_r^T + B_r. \quad (13.6)$$

(4) 交叉有效性检验 定义交叉有效性：

$$Q_h^2 = 1 - \frac{\text{PRESS}(h)}{\text{SS}(h-1)}. \quad (13.7)$$

当 $Q_h^2 < 0.0975$ (即 $1 - 0.95^2$) 时，停止提取成分。

(5) 简洁算法 直接在 $A_0, A_1, \dots, A_{r-1}$ 中提取成分 $t_1, \dots, t_r$ ，无需在 B_0 中提取 u_h 。

算法步骤：

1. 求 $A_0^T B_0 B_0^T A_0$ 最大特征值对应的特征向量 w_1 ，得 $\hat{t}_1 = A_0 w_1$ ；
2. 求 $A_1^T B_0 B_0^T A_1$ 最大特征值对应的特征向量 w_2 ，得 $\hat{t}_2 = A_1 w_2$ ；
3. 依此类推。

具体例子

例 13.1：体能训练数据 PLS 回归

20 位中年男子，自变量：体重 x_1 、腰围 x_2 、脉搏 x_3 ；因变量：单杠 y_1 、弯曲 y_2 、跳高 y_3 。

表 13.1 相关系数矩阵

	体重 (x_1)	腰围 (x_2)	脉搏 (x_3)	单杠 (y_1)	弯曲 (y_2)	跳高 (y_3)
体重 (x_1)	1	0.8702	-0.3658	-0.3897	-0.4931	-0.2263
腰围 (x_2)	0.8702	1	-0.3529	-0.5522	-0.6456	-0.1915
脉搏 (x_3)	-0.3658	-0.3529	1	0.1506	0.2250	0.0349
单杠 (y_1)	-0.3897	-0.5522	0.1506	1	0.6957	0.4958
弯曲 (y_2)	-0.4931	-0.6456	0.2250	0.6957	1	0.6692
跳高 (y_3)	-0.2263	-0.1915	0.0349	0.4958	0.6692	1

提取两对成分后，得到标准化回归方程：

$$\begin{aligned} \hat{y}_1 &= -0.0778\hat{x}_1 - 0.4990\hat{x}_2 - 0.1322\hat{x}_3, \\ \hat{y}_2 &= -0.1384\hat{x}_1 - 0.5245\hat{x}_2 - 0.0854\hat{x}_3, \\ \hat{y}_3 &= -0.0604\hat{x}_1 - 0.1559\hat{x}_2 - 0.0073\hat{x}_3. \end{aligned} \quad (13.8)$$

还原为原始变量:

$$\begin{aligned} y_1 &= 47.0204 - 0.0166x_1 - 0.8237x_2 - 0.0969x_3, \\ y_2 &= 612.5749 - 0.3508x_1 - 10.2481x_2 - 0.7413x_3, \\ y_3 &= 183.9821 - 0.1253x_1 - 2.4969x_2 - 0.0518x_3. \end{aligned} \quad (13.9)$$

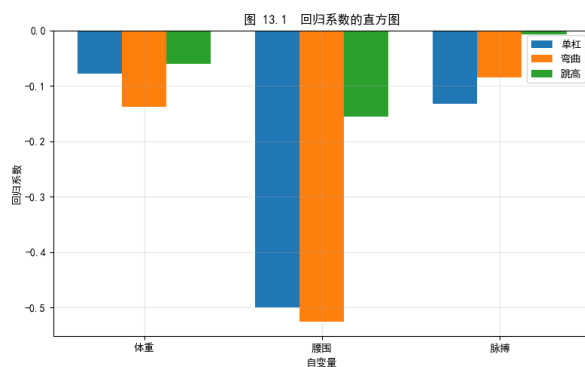


图 13.1 回归系数的直方图

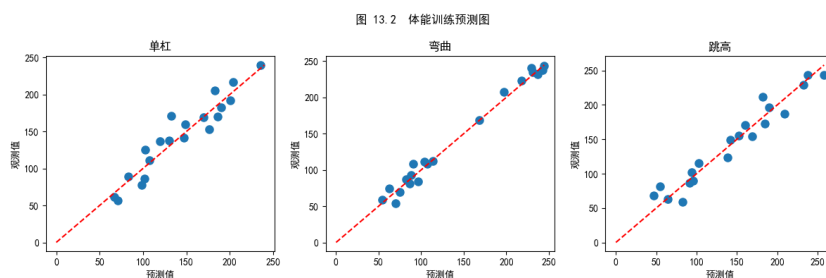


图 13.2 体能训练预测图

例 13.2: 交通运输与旅游业 PLS 回归

7 个旅游业指标预测 4 个客运量指标, 11 年数据。取前 2 个成分, 得到回归方程 (见教材)。

代码实现

例 13.1: PLS 回归

```

1  # 程序文件 ex13_1.py
2  import numpy as np
3  import pylab as plt
4  from sklearn.cross_decomposition import PLSRegression
5  from scipy.stats import zscore
6  from sklearn.model_selection import cross_val_predict
7  from sklearn.metrics import mean_squared_error
8
9  d0 = np.loadtxt('data13_1.txt')
10 mu = d0.mean(axis=0)
11 s = d0.std(axis=0, ddof=1)

```

```

12     r = np.corrcoef(d0.T)
13     d = zscore(d0, ddof=1)
14
15     a = d[:, :3]    # 自变量
16     b = d[:, 3:]    # 因变量
17
18     # 交叉验证确定成分个数
19     mse = []
20     for i in range(1, 4):
21         pls = PLSRegression(i)
22         y_cv = cross_val_predict(pls, a, b, cv=10)
23         mse.append(mean_squared_error(b, y_cv))
24     print('均方误差:', mse)
25     print('建议成分个数:', np.argmin(mse) + 1)
26
27     # 拟合模型
28     n_comp = 2
29     md = PLSRegression(n_comp).fit(a, b)
30
31     # 回归系数 (标准化数据)
32     beta = md.coef_    # 每列对应一个因变量
33     print('标准化回归系数:\n', beta)
34
35     # 还原为原始变量
36     beta0 = np.zeros((4, 3))
37     beta0[0, :] = mu[3:] - mu[:3] / s[:3] @ beta * s[3:]
38     for i in range(3):
39         beta0[i+1, :] = s[3+i] / s[:3] * beta[:, i]
40     print('原始数据回归系数:\n', beta0)
41
42     # 预测值
43     X0 = np.hstack([np.ones((20, 1)), d0[:, :3]])
44     y_pred = X0 @ beta0
45
46     # 绘制预测图
47     plt.figure(figsize=(12, 4))
48     for i in range(3):
49         plt.subplot(1, 3, i+1)
50         plt.plot(y_pred[:, i], d0[:, 3+i], 'o')
51         plt.plot([0, 260], [0, 260], 'r--')
52         plt.xlabel('预测值')
53         plt.ylabel('观测值')
54     plt.show()

```

例 13.2: PLS 回归 (多变量)

```

1     # 程序文件 ex13_2.py

```

```

2 import numpy as np
3 from sklearn.cross_decomposition import PLSRegression
4 from scipy.stats import zscore
5 from sklearn.model_selection import cross_val_predict
6 from sklearn.metrics import mean_squared_error
7
8 d0 = np.loadtxt('data13_2.txt')
9 d = zscore(d0, ddof=1)
10
11 a = d[:, :7]    # 自变量
12 b = d[:, 7:]    # 因变量
13
14 # 确定成分个数
15 mse = []
16 for i in range(1, 5):
17     pls = PLSRegression(i)
18     y_cv = cross_val_predict(pls, a, b, cv=10)
19     mse.append(mean_squared_error(b, y_cv))
20 print('均方误差:', mse)
21 print('建议成分个数:', np.argmin(mse) + 1)
22
23 # 拟合模型
24 n_comp = 2
25 md = PLSRegression(n_comp).fit(a, b)
26 beta = md.coef_
27 print('标准化回归系数:\n', beta)
28
29 # 还原原始变量
30 mu = d0.mean(axis=0)
31 s = d0.std(axis=0, ddof=1)
32 beta0 = np.zeros((8, 4))
33 beta0[0, :] = mu[7:] - mu[:7] / s[:7] @ beta * s[7:]
34 for i in range(4):
35     beta0[i+1, :] = s[7+i] / s[:7] * beta[:, i]
36 print('原始数据回归系数:\n', beta0)

```

优缺点分析

方法	优点	缺点
PLS 回归	处理多重共线性, 适合小样本	解释性略差
主成分回归	降维处理	仅考虑自变量信息
典型相关分析	考虑两组变量相关性	不直接建立回归模型

方法选择建议

- 自变量存在多重共线性：使用 PLS 回归。
- 样本量小于自变量个数：使用 PLS 回归。
- 需要同时分析两组变量关系：使用 PLS 回归。
- 成分个数选择：通过交叉验证选择使 MSE 最小的个数。

应用场景

- 经济分析：多指标之间的相互影响关系。
- 化学计量学：光谱数据与成分含量的关系。
- 生物信息学：基因表达数据与表型的关系。
- 市场研究：消费者行为与营销策略的关系。

第 14 章

综合评价方法

评价方法大体上可分为两类：一类是主观赋权法，如综合指数法、模糊综合评判法、层次分析法、功效系数法等；另一类是客观赋权法，如主成分分析法、因子分析法、理想解法（TOPSIS 法）、熵权法等。

14.1 综合评价指标体系

14.1.1 指标体系的概念

由多个相互联系、相互依存的评价指标，按照一定层次结构组合而成，具有特定评价功能的有机整体，称为综合评价的指标体系。

14.1.2 指标体系的设置原则

1. 科学性原则：定性分析与定量分析结合，正确反映系统特征。
2. 系统性原则：全面考虑影响评价目标的诸多因素，层次清楚、结构合理。
3. 独立性原则：指标间尽量不相关，避免信息重叠。
4. 层次性原则：将复杂问题分解，形成不同层次的结构。

14.2 综合评价数据处理

14.2.1 定量指标的一致化处理

将不同类型的指标统一转化为极大型指标。

(1) 极小型指标 $\rightarrow$ 极大型指标

$$x'_j = \frac{1}{x_j} \quad \text{或} \quad x'_j = M_j - x_j, \quad M_j = \max_i a_{ij}. \quad (14.1)$$

(2) 居中型指标 → 极大型指标 设 $M_j = \max_i a_{ij}$, $m_j = \min_i a_{ij}$, 令:

$$x'_j = \begin{cases} \frac{2(x_j - m_j)}{M_j - m_j}, & m_j \leq x_j \leq \frac{M_j + m_j}{2}, \\ \frac{2(M_j - x_j)}{M_j - m_j}, & \frac{M_j + m_j}{2} < x_j \leq M_j. \end{cases} \quad (14.2)$$

(3) 区间型指标 → 极大型指标 指标取值在 $[b_j^{(1)}, b_j^{(2)}]$ 内为最好。令 $c_j = \max\{b_j^{(1)} - m_j, M_j - b_j^{(2)}\}$:

$$x'_j = \begin{cases} 1 - \frac{b_j^{(1)} - x_j}{c_j}, & x_j < b_j^{(1)}, \\ 1, & b_j^{(1)} \leq x_j \leq b_j^{(2)}, \\ 1 - \frac{x_j - b_j^{(2)}}{c_j}, & x_j > b_j^{(2)}. \end{cases} \quad (14.3)$$

14.2.2 定量指标的无量纲化处理

(1) 标准样本变换

$$a_{ij}^* = \frac{a_{ij} - \mu_j}{s_j}, \quad \mu_j = \frac{1}{n} \sum_{i=1}^n a_{ij}, \quad s_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (a_{ij} - \mu_j)^2}. \quad (14.4)$$

(2) 比例变换法 极大型指标:

$$a_{ij}^* = \frac{a_{ij}}{\max_i a_{ij}}. \quad (14.5)$$

极小型指标:

$$a_{ij}^* = \frac{\min_i a_{ij}}{a_{ij}} \quad \text{或} \quad a_{ij}^* = 1 - \frac{a_{ij}}{\max_i a_{ij}}. \quad (14.6)$$

(3) 向量归一化法 极大型指标:

$$a_{ij}^* = \frac{a_{ij}}{\sqrt{\sum_{i=1}^n a_{ij}^2}}. \quad (14.7)$$

(4) 极差变换法 极大型指标:

$$a_{ij}^* = \frac{a_{ij} - \min_i a_{ij}}{\max_i a_{ij} - \min_i a_{ij}}. \quad (14.8)$$

极小型指标:

$$a_{ij}^* = \frac{\max_i a_{ij} - a_{ij}}{\max_i a_{ij} - \min_i a_{ij}}. \quad (14.9)$$

变换后 $0 \leq a_{ij}^* \leq 1$, 最优值为 1, 最劣值为 0。

(5) 功效系数法

$$a_{ij}^* = c + \frac{a_{ij} - \min_i a_{ij}}{\max_i a_{ij} - \min_i a_{ij}} \times d. \quad (14.10)$$

通常取 $c = 60$, $d = 40$, 则 $a_{ij}^* \in [60, 100]$ 。

14.2.3 定性指标的定量化

表 14.1 极大型定性指标量化值

等级	很低	低	一般	高	很高
量化值	0	0.3	0.5	0.7	1

表 14.2 极小型定性指标量化值

等级	很高	高	一般	低	很低
量化值	0	0.3	0.5	0.7	1

具体例子

例 14.1: 战斗机性能综合评价

4 种战斗机, 6 项指标: 最大速度 x_1 (极大型)、飞行范围 x_2 (极大型)、最大负载 x_3 (极大型)、费用 x_4 (极小型)、可靠性 x_5 (定性)、灵敏度 x_6 (定性)。

表 14.3 战斗机性能指标数据

	最大速度	飞行范围/km	最大负载/磅	费用/美元	可靠性	灵敏度
A_1	2.0	1500	20000	550000	一般	很高
A_2	2.5	2700	18000	650000	低	一般
A_3	1.8	2000	21000	450000	高	高
A_4	2.2	1800	20000	500000	一般	一般

定性指标量化后:

不同无量纲化方法得到的评价矩阵不同, 但评价对象的特征趋势一致。

代码实现

```

1  # 程序文件 ex14_1.py
2  import numpy as np
3  import pandas as pd
4
5  a = np.loadtxt('data14_1_1.txt')
6
7  # 逐列求范数、最大值、最小值
8  norm = np.linalg.norm(a, axis=0)
9  max_val = a.max(axis=0)
10 min_val = a.min(axis=0)

```

表 14.4 量化后的数据

	x_1	x_2	x_3	x_4	x_5	x_6
A_1	2.0	1500	20000	550000	0.5	1.0
A_2	2.5	2700	18000	650000	0.3	0.5
A_3	1.8	2000	21000	450000	0.7	0.7
A_4	2.2	1800	20000	500000	0.5	0.5

```
11
12 # 方法一：向量归一化法
13 R1 = a / norm
14 # 费用列（极小型）特殊处理
15 R1[:, 3] = 1 - a[:, 3] / norm[3]
16
17 # 方法二：比例变换法
18 R2 = a / max_val
19 R2[:, 3] = min_val[3] / a[:, 3]
20
21 # 方法三：极差变换法
22 R3 = (a - min_val) / (max_val - min_val)
23 R3[:, 3] = (max_val[3] - a[:, 3]) / (max_val[3] - min_val[3])
24
25 # 保存结果
26 np.savetxt('data14_1_2.txt', R1, fmt='%.4f')
27 with pd.ExcelWriter('data14_1_3.xlsx') as f:
28     pd.DataFrame(R1).to_excel(f, sheet_name='向量归一化', index=False)
29     pd.DataFrame(R2).to_excel(f, sheet_name='比例变换', index=False)
30     pd.DataFrame(R3).to_excel(f, sheet_name='极差变换', index=False)
```

方法评述

方法	优点	缺点
向量归一化	保持原始比例关系	受异常值影响
比例变换	简单直观	最大值点固定为 1
极差变换	结果在 [0,1] 区间	受极值影响大
功效系数法	结果在 60-100 区间	主观选择 c, d
标准样本变换	消除量纲影响	可能出现负值

方法选择建议

- 需要保持比例关系：使用比例变换法。

- 需要结果在 $[0,1]$ 区间：使用极差变换法。
- 需要评分在 **60-100** 区间：使用功效系数法。
- 后续使用主成分/因子分析：使用标准样本变换。
- 定性指标：先按等级量化，再与定量指标一起处理。

14.3 常用的综合评价数学模型

综合评价数学模型就是将同一评价对象不同方面的多个指标值综合在一起，得到一个整体性评价指标值的数学表达式。本节介绍线性加权综合评价法、TOPSIS 法、灰色关联度分析、熵值法和秩和比法。

14.3.1 线性加权综合评价法

设 n 个评价对象， m 个评价指标 $x_1, x_2, \dots, x_m$ ，标准化指标值为 b_{ij} ，权重向量为 $w = [w_1, w_2, \dots, w_m]$ 。

$$f_i = \sum_{j=1}^m w_j b_{ij}, \quad i = 1, 2, \dots, n. \quad (14.11)$$

特点：

1. 各指标可以线性相互补偿。
2. 权重大的指标对综合结果影响明显。
3. 计算简单，可操作性强。
4. 适用于各评价指标相互独立的情况。

14.3.2 TOPSIS 法

TOPSIS 法（理想解法）通过构造正理想解和负理想解，计算各评价对象与两者的距离进行排序。

计算步骤

- (1) 构造标准化评价矩阵 $B = (b_{ij})_{n \times m}$ （全部化为极大型）。
- (2) 确定正理想解 C^+ 和负理想解 C^- ：

$$c_j^+ = \max_{1 \leq i \leq n} b_{ij}, \quad c_j^- = \min_{1 \leq i \leq n} b_{ij}, \quad j = 1, 2, \dots, m. \quad (14.12)$$

- (3) 计算各评价对象到正、负理想解的距离：

$$s_i^+ = \sqrt{\sum_{j=1}^m (b_{ij} - c_j^+)^2}, \quad s_i^- = \sqrt{\sum_{j=1}^m (b_{ij} - c_j^-)^2}. \quad (14.13)$$

(4) 计算相对接近度:

$$f_i = \frac{s_i^-}{s_i^+ + s_i^-}, \quad i = 1, 2, \dots, n. \quad (14.14)$$

(5) 按 f_i 从大到小排序。

14.3.3 灰色关联度分析

计算步骤

(1) 构造标准化评价矩阵 $B = (b_{ij})_{n \times m}$ 。

(2) 确定参考数列 $b_0 = \{b_{0j} \mid j = 1, \dots, m\}$, 其中 $b_{0j} = \max_i b_{ij}$ 。

(3) 计算灰色关联系数:

$$\xi_{ij} = \frac{\min_i \min_j |b_{0j} - b_{ij}| + \rho \max_i \max_j |b_{0j} - b_{ij}|}{|b_{0j} - b_{ij}| + \rho \max_i \max_j |b_{0j} - b_{ij}|}, \quad (14.15)$$

其中 $\rho \in [0, 1]$ 为分辨系数, 通常取 $\rho = 0.5$ 。

(4) 计算灰色关联度:

$$r_i = \sum_{j=1}^m w_j \xi_{ij}, \quad i = 1, 2, \dots, n. \quad (14.16)$$

(5) 按 r_i 从大到小排序。

14.3.4 熵值法

熵值法通过计算各指标观测值的熵值来确定权重, 指标值差异越大, 熵值越小, 权重越大。

计算步骤

(1) 计算特征比重:

$$p_{ij} = \frac{b_{ij}}{\sum_{i=1}^n b_{ij}}, \quad i = 1, 2, \dots, n; j = 1, 2, \dots, m. \quad (14.17)$$

(2) 计算熵值:

$$e_j = -\frac{1}{\ln n} \sum_{i=1}^n p_{ij} \ln p_{ij}, \quad j = 1, 2, \dots, m. \quad (14.18)$$

(3) 计算差异系数:

$$g_j = 1 - e_j, \quad j = 1, 2, \dots, m. \quad (14.19)$$

(4) 确定权重:

$$w_j = \frac{g_j}{\sum_{k=1}^m g_k}, \quad j = 1, 2, \dots, m. \quad (14.20)$$

(5) 计算综合评价值:

$$f_i = \sum_{j=1}^m w_j p_{ij}, \quad i = 1, 2, \dots, n. \quad (14.21)$$

14.3.5 秩和比法

基本概念

定义 14.1 (样本秩) 设 $c_1, c_2, \dots, c_n$ 为样本, 按从小到大排列后, c_i 的位置 k 称为 c_i 的秩, 记作 R_i 。

计算步骤

(1) 编秩: 极大型指标从小到大编秩, 极小型指标从大到小编秩, 相同值编平均秩。

(2) 计算秩和比 (RSR):

$$RSR_i = \frac{1}{mn} \sum_{j=1}^m R_{ij} \quad (\text{等权重}), \quad (14.22)$$

或

$$RSR_i = \frac{1}{n} \sum_{j=1}^m w_j R_{ij} \quad (\text{加权}). \quad (14.23)$$

(3) 按 RSR_i 从大到小排序。

综合评价示例

例 14.2 (续例 14.1): 战斗机性能综合评价

使用向量归一化法得到的标准化评价矩阵:

$$B = \begin{bmatrix} 0.4671 & 0.3662 & 0.5056 & 0.4931 & 0.4811 & 0.7089 \\ 0.5839 & 0.6591 & 0.4550 & 0.4010 & 0.2887 & 0.3544 \\ 0.4204 & 0.4882 & 0.5308 & 0.5853 & 0.6736 & 0.4962 \\ 0.5139 & 0.4394 & 0.5056 & 0.5392 & 0.4811 & 0.3544 \end{bmatrix}. \quad (14.24)$$

(1) TOPSIS 法 正理想解 $C^+ = [0.5839, 0.6591, 0.5308, 0.5853, 0.6736, 0.7089]$, 负理想解 $C^- = [0.4204, 0.3662, 0.4550, 0.4010, 0.2887, 0.3544]$ 。

相对接近度:

$$F = [0.5236, 0.3746, 0.5974, 0.3659]. \quad (14.25)$$

排序: $A_3 > A_1 > A_2 > A_4$ 。

(2) 灰色关联度法 关联系数矩阵和关联度:

$$r = [0.6798, 0.6522, 0.7576, 0.6238]. \quad (14.26)$$

排序: $A_3 > A_1 > A_2 > A_4$ 。

(3) 熵值法 权重向量:

$$w = [0.0580, 0.1862, 0.0121, 0.0737, 0.3242, 0.3457]. \quad (14.27)$$

综合评价值:

$$F = [0.2787, 0.2100, 0.2865, 0.2247]. \quad (14.28)$$

排序: $A_3 > A_1 > A_4 > A_2$ 。

(4) 秩和比法 秩和比:

$$RSR = [0.5833, 0.5208, 0.7917, 0.6042]. \quad (14.29)$$

排序: $A_3 > A_4 > A_1 > A_2$ 。

代码实现

```

1  # 程序文件 ex14_2.py
2  import numpy as np
3  from scipy.stats import rankdata
4
5  a = np.loadtxt('data14_1_2.txt')
6
7  # ===== TOPSIS 法 =====
8  bp = a.max(axis=0) # 正理想解
9  bm = a.min(axis=0) # 负理想解
10 d1 = np.linalg.norm(a - bp, axis=1) # 到正理想解距离
11 d2 = np.linalg.norm(a - bm, axis=1) # 到负理想解距离
12 f1 = d2 / (d1 + d2)
13 print('TOPSIS评价值:', np.round(f1, 4))
14
15 # ===== 灰色关联度法 =====
16 c = bp - a # 参考序列与每个序列的差
17 m1 = c.max() # 两级最大差
18 m2 = c.min() # 两级最小差
19 rho = 0.5 # 分辨系数
20 xi = (m2 + rho * m1) / (c + rho * m1) # 关联系数
21 f2 = xi.mean(axis=1) # 关联度
22 print('灰色关联度:', np.round(f2, 4))
23

```

```

24 # ===== 熵值法 =====
25 n = a.shape[0]
26 s = a.sum(axis=0)
27 P = a / s # 特征比重
28 e = -(P * np.log(P)).sum(axis=0) / np.log(n) # 熵值
29 g = 1 - e # 差异系数
30 w = g / g.sum() # 权重
31 f3 = P @ w # 综合评价值
32 print('熵值法评价:', np.round(f3, 4))
33
34 # ===== 秩和比法 =====
35 R = rankdata(a, axis=0) # 逐列编秩
36 RSR = R.mean(axis=1) / n # 秩和比
37 print('秩和比:', np.round(RSR, 4))

```

方法评述

方法	优点	缺点
线性加权 TOPSIS	简单直观, 可操作性强 充分利用原始数据, 结果直观	需事先确定权重 需确定正负理想解
灰色关联度	适合小样本, 对数据要求低	分辨系数 ρ 选择主观
熵值法 秩和比法	客观赋权, 避免主观因素 简单易行, 不受分布限制	要求数据均大于 0 损失原始数据信息

方法选择建议

- 需要简单快速的综合评价: 使用线性加权法。
- 希望充分利用数据信息: 使用 TOPSIS 法或灰色关联度法。
- 需要客观确定权重: 使用熵值法。
- 数据分布未知或异常值较多: 使用秩和比法。
- 多种方法结合: 可同时使用多种方法, 对比结果。

14.4 模糊数学方法

14.4.1 模糊数学基本概念

定义 14.2 被讨论对象的全体称为论域, 常用 U, V 等表示。

经典集合由特征函数唯一确定：

$$X_A(x) = \begin{cases} 1, & x \in A, \\ 0, & x \notin A. \end{cases} \quad (14.30)$$

定义 14.3 论域 U 到 $[0, 1]$ 闭区间上的任意映射：

$$M : U \rightarrow [0, 1], \quad u \mapsto M(u), \quad (14.31)$$

确定了 U 上的一个模糊集合 M , $M(u)$ 称为 u 对 M 的隶属度。

模糊集记作：

$$M = \{(u, M(u)) \mid u \in U\}. \quad (14.32)$$

当 $U = \{u_1, u_2, \dots, u_n\}$ 时：

$$M = (M(u_1), M(u_2), \dots, M(u_n)). \quad (14.33)$$

14.4.2 常见的模糊分布

类型	偏小型	中间型	偏大型
矩形型	$M(x) = \begin{cases} 1, & x \leq a \\ 0, & x > a \end{cases}$	$M(x) = \begin{cases} 1, & a \leq x \leq b \\ 0, & \text{其他} \end{cases}$	$M(x) = \begin{cases} 1, & x \geq a \\ 0, & x < a \end{cases}$
梯形型	$M(x) = \begin{cases} 1, & x \leq a \\ \frac{b-x}{b-a}, & a < x < b \\ 0, & x \geq b \end{cases}$	$M(x) = \begin{cases} \frac{x-a}{b-a}, & a < x < b \\ 1, & b \leq x \leq c \\ \frac{d-x}{d-c}, & c < x < d \\ 0, & \text{其他} \end{cases}$	$M(x) = \begin{cases} 0, & x \leq a \\ \frac{x-a}{b-a}, & a < x < b \\ 1, & x \geq b \end{cases}$
正态型	$M(x) = \begin{cases} 1, & x \leq a \\ e^{-((x-a)/\sigma)^2}, & x > a \end{cases}$	$M(x) = e^{-((x-a)/\sigma)^2}$	$M(x) = \begin{cases} 0, & x \leq a \\ 1 - e^{-((x-a)/\sigma)^2}, & x > a \end{cases}$

Zadeh 运算：

$$a \wedge b = \min\{a, b\}, \quad a \vee b = \max\{a, b\}. \quad (14.34)$$

14.4.3 模糊贴近度

贴近度是两个模糊集接近程度的一种度量。

定义 14.4 若映射 $N : \mathcal{F}(U) \times \mathcal{F}(U) \rightarrow [0, 1]$ 满足：

1. $N(A, B) = N(B, A)$ (对称性)

2. $N(A, A) = 1$, $N(U, \emptyset) = 0$ (正则性)

3. 若 $A \subseteq B \subseteq C$, 则 $N(A, C) \leq N(A, B) \wedge N(B, C)$ (单调性)

则称 $N(A, B)$ 为 A 与 B 的贴近度。

(1) 汉明贴近度 离散型:

$$N(A, B) = 1 - \frac{1}{n} \sum_{i=1}^n |A(u_i) - B(u_i)|. \quad (14.4)$$

连续型:

$$N(A, B) = 1 - \frac{1}{b-a} \int_a^b |A(u) - B(u)| du. \quad (14.5)$$

(2) 欧几里得贴近度 离散型:

$$N(A, B) = 1 - \frac{1}{\sqrt{n}} \left(\sum_{i=1}^n (A(u_i) - B(u_i))^2 \right)^{1/2}. \quad (14.6)$$

连续型:

$$N(A, B) = 1 - \frac{1}{\sqrt{b-a}} \left(\int_a^b (A(u) - B(u))^2 du \right)^{1/2}. \quad (14.7)$$

择近原则: 若 $N(A_{i_0}, B) = \max_i N(A_i, B)$, 则判定 B 与 A_{i_0} 为一类。

14.4.4 模糊综合评价

设指标集 $I = \{x_1, x_2, \dots, x_p\}$, 评语集 $V = \{v_1, v_2, \dots, v_s\}$, 权重向量 $W = [w_1, w_2, \dots, w_p]$ 。

评价步骤:

1. 确定指标集 I 和权重向量 W 。
2. 建立评价集 V 。
3. 建立单指标评价矩阵 $R = (r_{ij})_{p \times s}$ 。
4. 合成综合评价结果:

$$A = W \circ R. \quad (14.35)$$

四种合成算子:

算子	公式	特点
$M(\wedge, \vee)$	$a_k = \max_{1 \leq j \leq p} \min(w_j, r_{jk})$	主因素决定型
$M(\cdot, \vee)$	$a_k = \max_{1 \leq j \leq p} (w_j \cdot r_{jk})$	主因素突出型
$M(\wedge, +)$	$a_k = \sum_{j=1}^p \min(w_j, r_{jk})$	加权平均型 (主因素)
$M(\cdot, +)$	$a_k = \sum_{j=1}^p w_j r_{jk}$	加权平均型

具体例子

例 14.3: 模糊贴近度识别

标准库集合:

$$A_1 = (0.4, 0.3, 0.5, 0.3), \quad A_2 = (0.3, 0.3, 0.4, 0.4), \quad A_3 = (0.2, 0.3, 0.3, 0.3). \quad (14.36)$$

待识别集合:

$$B = (0.2, 0.3, 0.4, 0.3). \quad (14.37)$$

汉明贴近度:

$$N(B, A_1) = 0.9, \quad N(B, A_2) = 0.9333, \quad N(B, A_3) = 0.9667. \quad (14.38)$$

 B 与 A_3 最接近, 归为 A_3 类。

例 14.4: 边坡设计方案模糊综合评价

5 个方案, 5 项指标: 可采矿量、基建投资、采矿成本、不稳定费用、净利润。

各指标隶属函数:

$$\mu_1(x_1) = \frac{x_1}{8800}, \quad \mu_2(x_2) = 1 - \frac{x_2}{8000}, \quad \mu_3(x_3) = \begin{cases} 1, & x_3 \leq 5.5, \\ \frac{8 - x_3}{8 - 5.5}, & 5.5 < x_3 < 8, \\ 0, & x_3 \geq 8, \end{cases} \quad (14.39)$$

$$\mu_4(x_4) = 1 - \frac{x_4}{200}, \quad \mu_5(x_5) = \frac{x_5 - 50}{1500 - 50}. \quad (14.40)$$

权重 $W = [0.25, 0.2, 0.2, 0.1, 0.25]$ 。

综合评价值:

$$B = [0.7435, 0.5919, 0.6789, 0.36, 0.3905]. \quad (14.41)$$

排序: 方案 1 > 方案 3 > 方案 2 > 方案 5 > 方案 4。

代码实现

例 14.3: 模糊贴近度

```

1  # 程序文件 ex14_3.py
2  import numpy as np
3
4  a = np.array([[0.4, 0.3, 0.5, 0.3],
5               [0.3, 0.3, 0.4, 0.4],
6               [0.2, 0.3, 0.3, 0.3]])
7  b = np.array([0.2, 0.3, 0.4, 0.3])
8  n = a.shape[1]
9
10 N = []

```

```

11     for e in a:
12         N.append(1 - sum(abs(e - b)) / n)
13     print('贴 近 度:', np.round(N, 4))

```

例 14.4: 模糊综合评价

```

1     # 程序文件 ex14_4.py
2     import numpy as np
3
4     a = np.loadtxt('data14_4.txt')
5
6     # 定义隶属函数
7     f1 = lambda x: x / 8800
8     f2 = lambda x: 1 - x / 8000
9     f3 = lambda x: (x <= 5.5) * 1.0 + (8 - x) / (8 - 5.5) * ((x >
10         5.5) & (x < 8))
11     f4 = lambda x: 1 - x / 200
12     f5 = lambda x: (x - 50) / (1500 - 50)
13
14     # 计算评价矩阵
15     funcs = [f1, f2, f3, f4, f5]
16     R = np.array([[funcs[i](a[i])] for i in range(len(a))] for _ in
17         range(5)])
18
19     # 权重
20     w = np.array([0.25, 0.2, 0.2, 0.1, 0.25])
21
22     # 综合评价值
23     B = w @ R
24     print('评 价 值:', np.round(B, 4))

```

方法评述

方法	优点	缺点
模糊贴近度	适合模式识别	需确定隶属函数
模糊综合评价	处理模糊因素	权重确定主观
$M(\wedge, \vee)$ 算子	主因素突出	信息损失大
$M(\cdot, +)$ 算子	信息利用充分	计算量稍大

方法选择建议

- 模式识别问题：使用模糊贴近度 + 择近原则。
- 多指标模糊评价：使用模糊综合评价。
- 主因素起决定作用：使用 $M(\wedge, \vee)$ 或 $M(\cdot, \vee)$ 。

- 各因素均衡影响：使用 $M(\cdot, +)$ 。
- 隶属函数选择：根据实际问题选择偏小型、中间型或偏大型分布。

14.5 数据包络分析

数据包络分析 (Data Envelopment Analysis, DEA) 是评价具有多指标输入和多指标输出系统的较为有效的方法。它由 A. Charnes, W. W. Cooper 和 E. Rhodes 于 1978 年提出。

DEA 的优点：

1. 以决策单元各输入输出的权重为变量，从最有利于决策单元的角度进行评价，避免确定主观权重。
2. 不必确定输入输出之间关系的显式表达式。
3. 排除了主观因素，具有很强的客观性。

14.5.1 DEA 的 C²R 模型

设有 m 个评价对象 (决策单元, DMU)，每个评价对象有 n 种投入和 s 种产出。

- a_{ij} ：第 i 个评价对象的第 j 种投入量
- b_{ik} ：第 i 个评价对象的第 k 种产出量
- $v = [v_1, v_2, \dots, v_n]^T$ ：投入权值向量
- $u = [u_1, u_2, \dots, u_s]^T$ ：产出权值向量

评价对象 i 的效率评价指数：

$$h_i = \frac{\beta_i^T u}{\alpha_i^T v}, \quad i = 1, 2, \dots, m. \quad (14.42)$$

C²R 模型 (分式规划)：

$$\max \frac{\beta_{i_0}^T u}{\alpha_{i_0}^T v}, \quad (14.43)$$

$$\text{s.t.} \quad \begin{cases} \frac{\beta_i^T u}{\alpha_i^T v} \leq 1, & i = 1, 2, \dots, m, \\ v \geq 0, & u \geq 0, \quad v \neq 0, \quad u \neq 0. \end{cases} \quad (14.8)$$

通过 Charnes-Cooper 变换：

$$\omega = tv, \quad \mu = tu, \quad t = \frac{1}{\alpha_{i_0}^T v}, \quad (14.44)$$

转化为等价的线性规划问题：

$$\max V_{i_0} = \beta_{i_0}^T \mu, \quad (14.45)$$

s.t.

$$\begin{cases} \alpha_i^T \omega - \beta_i^T \mu \geq 0, & i = 1, 2, \dots, m, \\ \alpha_{i_0}^T \omega = 1, \\ \omega \geq 0, & \mu \geq 0. \end{cases}$$

(14.9)

定义 14.5 若线性规划问题式 (14.9) 的最优目标值 $V_{i_0} = 1$, 则称评价对象 i_0 是弱 DEA 有效的。

定义 14.6 若存在最优解 $\omega^* > 0$, $\mu^* > 0$, 且最优目标值 $V_{i_0} = 1$, 则称评价对象 i_0 是 DEA 有效的。

具体例子

例 14.5: 天津市可持续发展 DEA 评价

表 14.10 DEA 评价结果

年份	θ	结论
1990	0.2902	非 DEA 有效
1991	0.2854	非 DEA 有效
1992	0.2968	非 DEA 有效
1993	0.3425	非 DEA 有效
1994	0.4595	非 DEA 有效
1995	0.7183	非 DEA 有效
1996	0.9069	DEA 有效
1997	1.0000	DEA 有效
1998	1.0000	DEA 有效
1999	1.0000	DEA 有效

输入变量: 财政收入占 GDP 比重、环保投资占 GDP 比重、每千人科技人员数; 输出变量: 人均 GDP、城市环境质量指数。

表 14.11 DEA 评价结果

年份	θ	结论
1990	0.2902	非 DEA 有效
1991	0.2854	非 DEA 有效
1992	0.2968	非 DEA 有效
1993	0.3425	非 DEA 有效
1994	0.4595	非 DEA 有效
1995	0.7183	非 DEA 有效
1996	0.9069	DEA 有效
1997	1.0000	DEA 有效
1998	1.0000	DEA 有效
1999	1.0000	DEA 有效

1997—1999 年达到 DEA 有效, 表明天津市在这几年实现了可持续发展。

代码实现

```

1      # 程序文件 ex14_5.py
2      import numpy as np
3      import cvxpy as cp
4
5      d = np.loadtxt('data14_5.txt')
6      a = d[:, :3]    # 输入指标
7      b = d[:, 3:]    # 输出指标
8
9      m = a.shape[0]
10     u = cp.Variable(3, pos=True)    # 输入权值
11     v = cp.Variable(2, pos=True)    # 输出权值
12
13     for j in range(m):
14         con = [
15             a @ u >= b @ v,    # 所有 DMU 的效率 1
16             a[j] @ u == 1      # 被评价 DMU 的投入归一化
17         ]
18         obj = cp.Maximize(b[j] @ v)
19         prob = cp.Problem(obj, con)
20         prob.solve(solver='GLPK_MI')
21
22     print(f'第 {j+1} 个对象最优值:', round(prob.value, 4))
23     print('最优解 u(投入权值):', np.round(u.value, 4))
24     print('最优解 v(产出权值):', np.round(v.value, 4))

```

优缺点分析

方面	优点	缺点
DEA 方法	无需主观权重, 客观性强	对数据敏感, 异常值影响大
C ² R 模型	理论完善, 建模清晰	只能评价相对有效性
线性规划	计算简便, 有成熟算法	大规模问题时计算量大

方法选择建议

- 多输入多输出系统评价：优先使用 DEA 方法。
- 无需主观权重：使用 DEA（权重由数据决定）。
- 需要区分有效/无效：使用 C²R 模型判断 DEA 有效性。
- 规模收益分析：结合 BCC 模型可判断规模收益递增/递减/不变。

应用场景

- 区域可持续发展评价：经济、社会、环境多指标综合评价。
- 高校绩效评价：教学、科研投入产出效率分析。
- 医院效率评价：医疗资源利用效率分析。
- 银行分行绩效评价：多投入多产出效率分析。
- 供应链效率评价：供应商选择与评价。

14.6 招聘公务员问题

14.6.1 问题提出

某市直属单位拟向社会公开招聘 8 名公务员，招聘办法如下：

1. 公开考试：考试科目有综合基础知识、专业知识和“行政职业能力测验”3 个部分，每科满分 100 分。根据考试总分排序选出 16 人进入面试。
2. 面试考核：考核应聘人员的知识面、理解能力、应变能力、表达能力等综合素质，等级分为 A、B、C、D 四个等级。

需要研究的问题：

1. 如何确定录用名单及分配方案？
2. 如何充分考虑应聘人员个人意愿的情况下择优录用并分配？
3. 推广到一般情况（ N 个应聘者， M 个部门）。

14.6.2 模型假设与符号说明

模型假设

1. 专家组对应聘者的评价是公正的。
2. 各部门和应聘者的相关数据都是透明的。
3. 应聘者的 4 项特长指标在综合评价中地位等同。
4. 用人部门的 5 项基本条件对公务员的影响地位等同。

符号说明

- a_i ：第 i 个应聘者的初试得分
- b_i ：第 i 个应聘者的复试得分
- d_i ：第 i 个应聘者的最后综合得分

- s_{ij} : 第 j 个部门对第 i 个应聘者的综合满意度
- t_{ij} : 第 i 个应聘者对第 j 个部门的综合满意度
- r_{ij} : 第 i 个应聘者与第 j 个部门的相互综合满意度

14.6.3 模型准备

(1) 复试成绩量化 评语集「很好、好、一般、差」对应数值为 5、4、3、2、1，取偏大型柯西分布隶属函数：

$$f(x) = \begin{cases} [1 + \alpha(x - \beta)^{-2}]^{-1}, & 1 \leq x \leq 3, \\ a \ln x + b, & 3 \leq x \leq 5. \end{cases} \quad (14.46)$$

由 $f(5) = 1$, $f(3) = 0.8$, $f(1) = 0.01$, 解得：

$$\alpha = 1.1086, \quad \beta = 0.8942, \quad a = 0.3915, \quad b = 0.3699. \quad (14.47)$$

各等级量化值：

$$A = 1, \quad B = 0.9126, \quad C = 0.8, \quad D = 0.5245. \quad (14.48)$$

(2) 规范化处理 初试成绩规范化：

$$a'_i = \frac{a_i - \min a_i}{\max a_i - \min a_i}. \quad (14.49)$$

复试成绩规范化：

$$b'_i = \frac{b_i - \min b_i}{\max b_i - \min b_i}. \quad (14.50)$$

综合得分：

$$d_i = \gamma a'_i + (1 - \gamma) b'_i, \quad i = 1, 2, \dots, 16. \quad (14.51)$$

取 $\gamma = 0.5$, 得到 16 名应聘者的综合得分及排序。

14.6.4 问题（1）模型：用人单位视角

(1) 部门满意度量化 当应聘者某项指标等级与部门要求一致时，满意度为 v_4 （基本满意）；高一级为 v_5 ，低一级为 v_3 ，依此类推。

评语集量化（偏大型柯西分布）：

$$f(x) = \begin{cases} [1 + 2.4944(x - 0.8413)^{-2}]^{-1}, & x \leq 4, \\ 0.3574 \ln x + 0.3046, & 4 < x \leq 7. \end{cases} \quad (14.52)$$

量化值：

$$(v_1, v_2, v_3, v_4, v_5, v_6, v_7) = (0.01, 0.3499, 0.6514, 0.8, 0.8797, 0.9449, 1). \quad (14.53)$$

部门 j 对应聘者 i 的综合满意度：

$$s_{ij} = \frac{1}{4} \sum_{k=1}^4 s_{ijk}. \quad (14.54)$$

表 14.13 满意度关系表

应聘者等级	部门要求等级			
	A	B	C	D
A	v_4	v_3	v_2	v_1
B	v_5	v_4	v_3	v_2
C	v_6	v_5	v_4	v_3
D	v_7	v_6	v_5	v_4

(2) 0-1 整数规划模型 决策变量:

$$x_{ij} = \begin{cases} 1, & \text{录用第 } i \text{ 个应聘者到第 } j \text{ 个部门,} \\ 0, & \text{否则.} \end{cases} \quad (14.55)$$

$$\max z = \sum_{i=1}^{16} \sum_{j=1}^7 d_i x_{ij} + \sum_{i=1}^{16} \sum_{j=1}^7 s_{ij} x_{ij}. \quad (14.56)$$

s.t.

$$\begin{cases} \sum_{i=1}^{16} \sum_{j=1}^7 x_{ij} = 8, \\ \sum_{j=1}^7 x_{ij} \leq 1, & i = 1, 2, \dots, 16, \\ 1 \leq \sum_{i=1}^{16} x_{ij} \leq 2, & j = 1, 2, \dots, 7, \\ x_{ij} \in \{0, 1\}. \end{cases} \quad (14.57)$$

最优目标值 $z = 11.9013$, 录用分配方案见表 14.16。

14.6.5 问题 (2) 模型: 双向满意度

(1) 部门对应聘者的满意度 与问题 (1) 相同: s_{ij} 。

(2) 应聘者对部门的满意度 志愿权重:

$$w_{ij} = \begin{cases} 1, & \text{第 } j \text{ 个部门是第 } i \text{ 个应聘者的第一志愿,} \\ 0.6309, & \text{第二志愿,} \\ 0, & \text{非志愿.} \end{cases} \quad (14.58)$$

部门客观水平评价值 T_j (5 项指标, 等级量化: 优/小/多 = 1, 中 = 0.6, 差/大/少 = 0.1)。

应聘者 i 对部门 j 的单项满意度:

$$\tilde{T}_{ij} = w_{ij} \cdot T_j. \quad (14.59)$$

综合满意度:

$$t_{ij} = \frac{1}{5} \sum_{k=1}^5 \tilde{T}_{ijk}. \quad (14.60)$$

(3) 相互综合满意度

$$r_{ij} = \sqrt{s_{ij} \cdot t_{ij}}. \quad (14.61)$$

(4) 0-1 整数规划模型

$$\max z = \sum_{i=1}^{16} \sum_{j=1}^7 r_{ij} x_{ij}. \quad (14.62)$$

s.t.

$$\begin{cases} \sum_{i=1}^{16} \sum_{j=1}^7 x_{ij} = 8, \\ \sum_{j=1}^7 x_{ij} \leq 1, & i = 1, 2, \dots, 16, \\ 1 \leq \sum_{i=1}^{16} x_{ij} \leq 2, & j = 1, 2, \dots, 7, \\ x_{ij} \in \{0, 1\}. \end{cases} \quad (14.63)$$

最优目标值 $z = 5.7009$ 。

代码实现

复试成绩量化

```

1  # 程序文件 anli14_1_1.py
2  import numpy as np
3  from scipy.optimize import fsolve
4  import pylab as plt
5
6  # 确定隶属函数参数
7  f1 = lambda t: [1/(1+t[0]/(1-t[1])**2) - 0.01,
8  1/(1+t[0]/(3-t[1])**2) - 0.8]
9  c1 = fsolve(f1, [0.5, 0.5])
10
11 f2 = lambda t: [t[0]*np.log(3) + t[1] - 0.8,
12 t[0]*np.log(5) + t[1] - 1]
13 c2 = fsolve(f2, [0.5, 0.5])
14
15 # 隶属函数
16 fx = lambda x: (1/(1+c1[0]/(x-c1[1])**2)) * ((x>=1) & (x<=3)) +
17 \
18 (c2[0]*np.log(x) + c2[1]) * ((x>=3) & (x<=5))
19
20 # 计算等级量化值
21 print('A=', fx(5), 'B=', fx(4), 'C=', fx(3), 'D=', fx(2))
22
23 # 读取数据并计算综合得分
24 d0 = np.loadtxt('anli14_1_1.txt')
25 d1 = d0[:, 0] # 初试成绩
26 d2 = d0[:, 3:] # 面试等级 (数值化)

```

```

27     e20 = fx(d2)
28     e21 = e20.mean(axis=1) # 复试综合成绩
29     e2 = (e21 - min(e21)) / (max(e21) - min(e21))
30     e1 = (d1 - min(d1)) / (max(d1) - min(d1))
31
32     f = (e1 + e2) / 2      # 综合得分
33     print('综合得分:', f)

```

问题 (1) 0-1 整数规划

```

1     # 程序文件 anli14_1_2.py
2     import numpy as np
3     import cvxpy as cp
4     import pandas as pd
5     from scipy.optimize import fsolve
6
7     # 部门满意度隶属函数
8     f1 = lambda t: [1/(1+t[0]/(1-t[1])**2) - 0.01,
9                    1/(1+t[0]/(4-t[1])**2) - 0.8]
10    c1 = fsolve(f1, [0.5, 0.5])
11
12    f2 = lambda t: [t[0]*np.log(4) + t[1] - 0.8,
13                  t[0]*np.log(7) + t[1] - 1]
14    c2 = fsolve(f2, [0.5, 0.5])
15
16    f = lambda x: (1/(1+c1[0]/(x-c1[1])**2)) * ((x>=1) & (x<=4)) +
17              \
18              (c2[0]*np.log(x) + c2[1]) * ((x>=4) & (x<=7))
19
20    # 评语集量化
21    f17 = f(np.arange(1, 8))
22
23    # 满意度映射
24    g = lambda x: (4*(x==0) + 5*(x==-1) + 6*(x==-2) + 7*(x==-3) +
25                3*(x==1) + 2*(x==2) + 1*(x>=3))
26
27    # 计算部门对应聘者的满意度矩阵
28    d1 = np.loadtxt('anli14_1_1.txt') # 应聘者数据
29    d2 = np.loadtxt('anli14_1_3.txt') # 部门期望要求
30    a = d1[:, 3:]                    # 应聘者等级 (数值化)
31    b = d2[:, 6:]                    # 部门期望等级
32
33    s = np.zeros((16, 7))
34    for i in range(16):
35        for j in range(7):
36            t1 = g(b[j, :] - a[i, :])
37            t2 = f(t1)

```

```

37     s[i, j] = t2.mean()
38
39     # 0-1整数规划
40     d = np.loadtxt('anli14_1_2.txt')    # 综合得分
41     x = cp.Variable((16, 7), integer=True)
42     obj = cp.Maximize(cp.sum(cp.multiply(d, cp.sum(x, axis=1))) +
43                       cp.sum(cp.multiply(s, x)))
44     con = [cp.sum(x) == 8,
45            cp.sum(x, axis=1) <= 1,
46            cp.sum(x, axis=0) >= 1,
47            cp.sum(x, axis=0) <= 2,
48            x >= 0, x <= 1]
49
50     prob = cp.Problem(obj, con)
51     prob.solve(solver='GLPK_MI')
52     print('最优值:', prob.value)
53     print('最优解:', x.value)

```

问题（2）双向满意度

```

1     # 程序文件 anli14_1_3.py
2     import numpy as np
3     import cvxpy as cp
4     import pandas as pd
5
6     # 志愿权重
7     fx = lambda x: np.log(4-x) / np.log(3)
8     w = fx(np.arange(1, 4))    # [1, 0.6309, 0]
9
10    # 读取数据
11    d1 = np.loadtxt('anli14_1_1.txt')
12    dz = np.loadtxt('anli14_1_3.txt')
13    a = d1[:, [1, 2]]    # 志愿类别
14
15    # 部门客观水平
16    tj = dz[:, 1:6]    # 5项指标
17
18    # 计算应聘者对部门的满意度
19    wij = np.zeros((16, 7))
20    for i in range(16):
21        # 第一志愿
22        if a[i, 0] == 1:
23            wij[i, 0:2] = 1
24        elif a[i, 0] == 2:
25            wij[i, 2:4] = 1
26        elif a[i, 0] == 3:
27            wij[i, 4:6] = 1

```

```

28     elif a[i, 0] == 4:
29         wij[i, 6] = 1
30         # 第二志愿
31         if a[i, 1] == 1:
32             wij[i, 0:2] = w[1]
33         elif a[i, 1] == 2:
34             wij[i, 2:4] = w[1]
35         elif a[i, 1] == 3:
36             wij[i, 4:6] = w[1]
37         elif a[i, 1] == 4:
38             wij[i, 6] = w[1]
39
40     t = wij * (tj.mean(axis=1))
41
42     # 部门对应聘者的满意度（从问题1结果读取）
43     s = pd.read_excel('anli14_1_4.xlsx').values
44
45     # 相互满意度
46     r = np.sqrt(s * t)
47
48     # 0-1整数规划
49     x = cp.Variable((16, 7), integer=True)
50     obj = cp.Maximize(cp.sum(cp.multiply(r, x)))
51     con = [cp.sum(x) == 8,
52            cp.sum(x, axis=1) <= 1,
53            cp.sum(x, axis=0) >= 1,
54            cp.sum(x, axis=0) <= 2,
55            x >= 0, x <= 1]
56
57     prob = cp.Problem(obj, con)
58     prob.solve(solver='GLPK_MI')
59     print('最优值:', prob.value)
60     print('最优解:', x.value)

```

方法评述

方法	优点	缺点
模糊隶属函数	合理量化等级评价	参数确定需主观判断
0-1 整数规划	保证最优分配方案	规模增大时计算量大
双向满意度	兼顾双方利益, 公平合理	权重设定有主观性
极差规范化	消除量纲影响	对极值敏感

方法选择建议

- 等级数据量化：使用模糊隶属函数（偏大型柯西分布）。

- 综合评价：使用加权平均或几何平均。
- 最优分配：使用 0-1 整数规划。
- 双向满意度：取双方满意度的几何平均。
- 推广到一般情况：模型结构不变，只需调整规模参数。

应用场景

- 公务员招聘：录用与分配决策。
- 高校毕业生就业：人才与岗位匹配。
- 企业内部招聘：岗位配置优化。
- 人才选拔：综合评价与择优录用。
- 资源分配：多目标优化分配问题。

第 15 章

预测方法

预测的方法种类繁多，从经典的单耗法、弹性系数法、统计分析法、微分方程法，到灰色预测法、专家系统法和模糊数学法，甚至神经网络和小波分析等方法都可以用于预测。本章介绍灰色预测、马尔可夫预测和神经元网络。

15.1 灰色预测模型

灰色系统理论由邓聚龙教授于 1982 年创立。灰色预测的主要特点是模型使用的不是原始数据序列，而是生成的数据序列。其核心体系是灰色模型（Grey Model, GM），即对原始数据做累加生成得到近似的指数规律再进行建模。

优点：

- 不需要很多数据，一般只需要 4 个数据即可
- 能解决历史数据少、序列完整性及可靠性低的问题
- 能利用微分方程充分挖掘系统本质，精度高
- 运算简便，易于检验

缺点：

- 只适用于中短期预测
- 只适合指数增长的预测

15.1.1 GM(1,1) 预测模型

GM(1,1) 表示模型是 1 阶微分方程，且只含 1 个变量的灰色模型。

数学原理

(1) 基本定义 已知参考数据列 $x^{(0)} = (x^{(0)}(1), x^{(0)}(2), \dots, x^{(0)}(n))$ 。

1 次累加生成序列 (1-AGO)：

$$x^{(1)}(k) = \sum_{i=1}^k x^{(0)}(i), \quad k = 1, 2, \dots, n. \quad (15.1)$$

均值生成序列:

$$z^{(1)}(k) = 0.5x^{(1)}(k) + 0.5x^{(1)}(k-1), \quad k = 2, 3, \dots, n. \quad (15.2)$$

灰微分方程:

$$x^{(0)}(k) + ax^{(1)}(k) = b, \quad k = 2, 3, \dots, n. \quad (15.3)$$

白化微分方程:

$$\frac{dx^{(1)}(t)}{dt} + ax^{(1)}(t) = b. \quad (15.1)$$

(2) 参数估计 令 $u = [a, b]^T$, $Y = [x^{(0)}(2), x^{(0)}(3), \dots, x^{(0)}(n)]^T$,

$$B = \begin{bmatrix} -z^{(1)}(2) & 1 \\ -z^{(1)}(3) & 1 \\ \vdots & \vdots \\ -z^{(1)}(n) & 1 \end{bmatrix}. \quad (15.4)$$

由最小二乘法:

$$\hat{u} = [\hat{a}, \hat{b}]^T = (B^T B)^{-1} B^T Y. \quad (15.5)$$

(3) 时间响应函数 白化方程的解:

$$\hat{x}^{(1)}(k+1) = \left(x^{(0)}(1) - \frac{b}{a}\right) e^{-ak} + \frac{b}{a}, \quad k = 0, 1, \dots, n-1. \quad (15.6)$$

还原值:

$$\hat{x}^{(0)}(k+1) = \hat{x}^{(1)}(k+1) - \hat{x}^{(1)}(k), \quad k = 1, 2, \dots, n-1. \quad (15.7)$$

(4) 级比检验 级比:

$$\lambda(k) = \frac{x^{(0)}(k-1)}{x^{(0)}(k)}, \quad k = 2, 3, \dots, n. \quad (15.8)$$

可容覆盖:

$$\Theta = (e^{-2/(n+1)}, e^{2/(n+1)}). \quad (15.9)$$

若所有 $\lambda(k) \in \Theta$, 则序列可用 GM(1,1) 建模。否则需做平移变换 $y^{(0)}(k) = x^{(0)}(k) + c$ 。

(5) 模型检验 相对误差:

$$\delta(k) = \left| \frac{x^{(0)}(k) - \hat{x}^{(0)}(k)}{x^{(0)}(k)} \right|, \quad k = 1, 2, \dots, n. \quad (15.10)$$

- $\delta(k) < 0.2$: 一般要求
- $\delta(k) < 0.1$: 较高要求

级比偏差:

$$\rho(k) = \left| 1 - \frac{1 - 0.5a}{1 + 0.5a} \lambda(k) \right|. \quad (15.11)$$

15.1.2 GM(2,1) 模型

GM(2,1) 适用于非单调的摆动发展序列。

数学原理

1-AGO 序列 $x^{(1)}$, 1-IAGO 序列 $\alpha^{(1)}x^{(0)}$:

$$\alpha^{(1)}x^{(0)}(k) = x^{(0)}(k) - x^{(0)}(k-1), \quad k = 2, 3, \dots, n. \quad (15.12)$$

GM(2,1) 模型:

$$\alpha^{(1)}x^{(0)}(k) + a_1 z^{(1)}(k) + a_2 z^{(1)}(k) = b. \quad (15.13)$$

白化方程:

$$\frac{d^2 x^{(1)}(t)}{dt^2} + a_1 \frac{dx^{(1)}(t)}{dt} + a_2 x^{(1)}(t) = b. \quad (15.14)$$

参数估计:

$$B = \begin{bmatrix} -x^{(0)}(2) & -z^{(1)}(2) & 1 \\ -x^{(0)}(3) & -z^{(1)}(3) & 1 \\ \vdots & \vdots & \vdots \\ -x^{(0)}(n) & -z^{(1)}(n) & 1 \end{bmatrix}, \quad Y = \begin{bmatrix} x^{(0)}(2) - x^{(0)}(1) \\ x^{(0)}(3) - x^{(0)}(2) \\ \vdots \\ x^{(0)}(n) - x^{(0)}(n-1) \end{bmatrix}. \quad (15.15)$$

$$\hat{u} = (B^T B)^{-1} B^T Y. \quad (15.16)$$

15.1.3 DGM(2,1) 模型

DGM(2,1) 是 GM(2,1) 的简化形式。

数学原理

DGM(2,1) 模型:

$$\alpha^{(1)}x^{(0)}(k) + ax^{(0)}(k) = b. \quad (15.17)$$

白化方程:

$$\frac{d^2 x^{(1)}(t)}{dt^2} + a \frac{dx^{(1)}(t)}{dt} = b. \quad (15.18)$$

参数估计:

$$B = \begin{bmatrix} -x^{(0)}(2) & 1 \\ -x^{(0)}(3) & 1 \\ \vdots & \vdots \\ -x^{(0)}(n) & 1 \end{bmatrix}, \quad Y = \begin{bmatrix} x^{(0)}(2) - x^{(0)}(1) \\ x^{(0)}(3) - x^{(0)}(2) \\ \vdots \\ x^{(0)}(n) - x^{(0)}(n-1) \end{bmatrix}. \quad (15.19)$$

时间响应函数:

$$\hat{x}^{(1)}(t) = \left(\frac{\hat{b}}{\hat{a}^2} x^{(0)}(1) \right) e^{-\hat{a}t} + \frac{\hat{b}}{\hat{a}} t + \frac{1 + \hat{a}}{\hat{a}} x^{(0)}(1) - \frac{\hat{b}}{\hat{a}^2}. \quad (15.20)$$

15.1.4 灰色 Verhulst 模型

Verhulst 模型主要用来描述具有饱和状态的过程，即 S 形过程，常用于人口预测、生物生长预测等。

数学原理

灰色 Verhulst 模型：

$$x^{(0)}(k) + az^{(1)}(k) = b[z^{(1)}(k)]^2. \quad (15.10)$$

白化方程：

$$\frac{dx^{(1)}(t)}{dt} + ax^{(1)}(t) = b[x^{(1)}(t)]^2. \quad (15.11)$$

参数估计：

$$B = \begin{bmatrix} -z^{(1)}(2) & (z^{(1)}(2))^2 \\ -z^{(1)}(3) & (z^{(1)}(3))^2 \\ \vdots & \vdots \\ -z^{(1)}(n) & (z^{(1)}(n))^2 \end{bmatrix}, \quad Y = \begin{bmatrix} x^{(0)}(2) \\ x^{(0)}(3) \\ \vdots \\ x^{(0)}(n) \end{bmatrix}. \quad (15.21)$$

时间响应函数：

$$\hat{x}^{(1)}(k+1) = \frac{ax^{(0)}(1)}{bx^{(0)}(1) + (a - bx^{(0)}(1))e^{ak}}. \quad (15.13)$$

具体例子

例 15.1：城市交通噪声 GM(1,1) 预测

表 15.1 城市交通噪声数据 [dB(A)]

年份	1986	1987	1988	1989	1990	1991	1992
L_{eq}	71.1	72.4	72.4	72.1	71.4	72.0	71.6

级比检验通过后建立 GM(1,1) 模型，得到 $a = 0.0023$ ， $b = 72.6573$ 。

预测模型：

$$\hat{x}^{(1)}(k+1) = -30928.8525e^{-0.002344k} + 30999.9525. \quad (15.22)$$

1993 年预测值：71.3946 dB(A)。

例 15.2：GM(2,1) 模型

序列 $x^{(0)} = (41, 49, 61, 78, 96, 104)$ ，建立 GM(2,1) 模型。

参数估计： $\hat{u} = [-1.0922, 0.1959, -31.7983]^T$ 。

例 15.3：DGM(2,1) 模型

序列 $x^{(0)} = (2.874, 3.278, 3.39, 3.679, 3.77, 3.8)$ ，建立 DGM(2,1) 模型。

参数估计： $a = 0.4240$ ， $b = 1.7046$ 。

例 15.4: Verhulst 模型

序列 $x^{(0)} = (4.93, 2.33, 3.87, 4.35, 6.63, 7.15, 5.37, 6.39, 7.81, 8.35)$ ，建立 Verhulst 模型。

参数估计： $a = -0.3576$ ， $b = -0.0041$ 。

代码实现

例 15.1: GM(1,1) 模型

```
1  # 程序文件 ex15_1.py
2  import numpy as np
3
4  x0 = np.array([71.1, 72.4, 72.4, 72.1, 71.4, 72.0, 71.6])
5  n = len(x0)
6
7  # 级比检验
8  lamda = x0[:-1] / x0[1:]
9  b1 = [min(lamda), max(lamda)]
10 b2 = [np.exp(-2/(n+1)), np.exp(2/(n+1))]
11 print('级比范围:', b1)
12 print('可容覆盖:', b2)
13
14 # 累加生成
15 x1 = np.cumsum(x0)
16
17 # 均值生成
18 z = (x1[:-1] + x1[1:]) / 2
19
20 # 参数估计
21 B = np.vstack([-z, np.ones(n-1)]).T
22 Y = x0[1:]
23 u = np.linalg.pinv(B) @ Y
24 a, b = u[0], u[1]
25 print(f'a = {a:.4f}, b = {b:.4f}')
26
27 # 时间响应函数
28 k = np.arange(n+1)
29 xh1 = (x0[0] - b/a) * np.exp(-a * k) + b/a
30
31 # 还原
32 xh0 = np.hstack([x0[0], np.diff(xh1)])
33
34 # 预测 1993 年
35 x1993 = xh0[-1]
36 print(f'1993 年预测值: {x1993:.4f}')
```

例 15.2: GM(2,1) 模型

```

1  # 程序文件 ex15_2.py
2  import numpy as np
3  import sympy as sp
4
5  x0 = np.array([41, 49, 61, 78, 96, 104])
6  n = len(x0)
7
8  x1 = np.cumsum(x0)
9  ax0 = np.diff(x0)
10 z = (x1[1:] + x1[:-1]) / 2
11
12 B = np.vstack([-x0[1:], -z, np.ones(n-1)]).T
13 Y = ax0
14 u = np.linalg.pinv(B) @ Y
15 print('参数 u:', np.round(u, 4))
16
17 # 符号求解微分方程
18 sp.var('t')
19 sp.var('x', cls=sp.Function)
20 eq = x(t).diff(t,2) + u[0]*x(t).diff(t) + u[1]*x(t) - u[2]
21 s = sp.dsolve(eq, ics={x(0): x1[0], x(5): x1[-1]})
22 xt = sp.lambdify(t, s.args[1], 'numpy')
23
24 xh1 = xt(np.arange(n))
25 xh0 = np.hstack([x0[0], np.diff(xh1)])
26 print('预测值:', np.round(xh0, 4))

```

例 15.3: DGM(2,1) 模型

```

1  # 程序文件 ex15_3.py
2  import numpy as np
3  import sympy as sp
4
5  x0 = np.array([2.874, 3.278, 3.39, 3.679, 3.77, 3.8])
6  n = len(x0)
7
8  B = np.vstack([-x0[1:], np.ones(n-1)]).T
9  Y = np.diff(x0)
10 u = np.linalg.pinv(B) @ Y
11 print('参数 u:', np.round(u, 4))
12
13 sp.var('t')
14 sp.var('x', cls=sp.Function)
15 eq = x(t).diff(t,2) + u[0]*x(t).diff(t) - u[1]
16 s = sp.dsolve(eq, ics={x(0): x0[0], x(t).diff(t).subs(t,0): x0
    [0]})

```

```

17     xt = sp.lambdify(t, s.args[1], 'numpy')
18
19     xh1 = xt(np.arange(n))
20     xh0 = np.hstack([x0[0], np.diff(xh1)])
21     print('预测值:', np.round(xh0, 4))

```

例 15.4: Verhulst 模型

```

1     # 程序文件 ex15_4.py
2     import numpy as np
3
4     x0 = np.array([4.93, 2.33, 3.87, 4.35, 6.63, 7.15, 5.37, 6.39,
5                   7.81, 8.35])
6     n = len(x0)
7
8     x1 = np.cumsum(x0)
9     z = (x1[1:] + x1[:-1]) / 2
10
11     B = np.vstack([-z, z**2]).T
12     Y = x0[1:]
13     u = np.linalg.pinv(B) @ Y
14     print('参数 u:', np.round(u, 4))
15
16     # 时间响应函数
17     f = lambda k: u[0]*x0[0] / (u[1]*x0[0] + (u[0] - u[1]*x0[0]) *
18                               np.exp(u[0]*k))
19     k = np.arange(n)
20     xh1 = f(k)
21     xh0 = np.hstack([x0[0], np.diff(xh1)])
22     print('预测值:', np.round(xh0, 4))

```

优缺点分析

模型	优点	缺点
GM(1,1)	数据要求少，计算简单， 适合指数增长	仅适用单调变化序列
GM(2,1)	适用于非单调摆动序列	计算复杂，参数多
DGM(2,1)	比 GM(2,1) 简洁	仍有一定误差
Verhulst	适合 S 形饱和序列	对初始值敏感

方法选择建议

- 单调指数增长序列：使用 GM(1,1) 模型。
- 非单调摆动序列：使用 GM(2,1) 或 DGM(2,1) 模型。

- S 形饱和序列（如人口增长、生物生长）：使用 Verhulst 模型。
- 数据量极少 ($n \leq 4$)：使用 GM(1,1)。
- 精度要求高：先做级比检验，必要时做平移变换。

应用场景

- 环境监测：噪声、污染物的趋势预测。
- 经济预测：GDP、物价指数的短期预测。
- 人口预测：人口增长趋势预测。
- 农业预测：粮食产量、生物生长预测。
- 工程预测：设备寿命、材料性能退化预测。

15.2 马尔可夫预测

15.2.1 马尔可夫链的定义

现实世界中有很多现象，某一系统在已知现在情况的条件下，系统未来时刻的情况只与现在有关，而与过去的历史无直接关系。描述这类随机现象的数学模型称为马尔可夫模型。

定义 15.7 设 $\{\xi_n, n = 1, 2, \dots\}$ 是一个随机序列，状态空间 E 为有限或可列集，若对任意正整数 $m, n, i, j, i_k \in E$ ，有

$$P\{\xi_{n+m} = j \mid \xi_n = i, \xi_{n-1} = i_{n-1}, \dots, \xi_1 = i_1\} = P\{\xi_{n+m} = j \mid \xi_n = i\}, \quad (15.15)$$

则称 $\{\xi_n\}$ 为马尔可夫链（马氏链），式 (15.15) 称为马氏性。

定义 15.8 若转移概率与起始时刻 n 无关，即

$$P\{\xi_{n+m} = j \mid \xi_n = i\} = p_{ij}(m), \quad (15.23)$$

则称该马氏链为时齐马氏链。 $p_{ij}(m)$ 称为系统由状态 i 经过 m 步转移到状态 j 的概率。

15.2.2 转移概率矩阵及柯尔莫哥洛夫定理

一步转移矩阵 $P = (p_{ij})$ ，其中 $p_{ij} = P\{\xi_{n+1} = j \mid \xi_n = i\}$ 。

性质：

1. $0 \leq p_{ij} \leq 1$
2. $\sum_{j \in E} p_{ij} = 1$ （行和为 1）
3. $p_{ij}(0) = \delta_{ij}$ （Kronecker 符号）

定理 15.6 (柯尔莫哥洛夫-切普曼方程)

$$p_{ij}(m+n) = \sum_{k \in E} p_{ik}(m)p_{kj}(n). \quad (15.24)$$

定理 15.7 设 $P^{(0)}$ 为初始分布行向量, 则第 n 步的概率分布为

$$P^{(n)} = P^{(0)} P^n. \quad (15.25)$$

15.2.3 转移概率的渐近性质——极限概率分布

定义 15.9 一个马氏链的转移矩阵 P 是正则的, 若存在正整数 k , 使 P^k 的所有元素均为正。

定理 15.8 若 P 是正则矩阵, 则:

1. P 有唯一的不动点向量 π ($\pi P = \pi$), 各分量为正;
2. P^n 随 $n \rightarrow \infty$ 趋于每行都等于 π 的矩阵。

遍历性与极限分布: 若 $\lim_{n \rightarrow \infty} p_{ij}(n) = \pi_j$ (不依赖于 i), 且 $\sum \pi_j = 1$, 则称 π 为极限分布。

定理 15.9 若存在正整数 m , 使 $p_{ij}(m) > 0$ (对所有 i, j), 则马氏链具有遍历性, 且极限分布 π 是方程组

$$\pi = \pi P, \quad \sum_j \pi_j = 1 \quad (15.26)$$

的唯一正解。

具体例子

例 15.5: 计算机状态转移概率估计

观测数据序列 (97 次, 1= 正常, 0= 故障), 统计一步转移次数:

$$n_{00} = 8, n_{01} = 18, n_{10} = 18, n_{11} = 52. \quad (15.27)$$

转移概率估计:

$$P = \begin{bmatrix} 4/13 & 9/13 \\ 9/35 & 26/35 \end{bmatrix} = \begin{bmatrix} 0.3077 & 0.6923 \\ 0.2571 & 0.7429 \end{bmatrix}. \quad (15.28)$$

例 15.6: 多状态转移概率估计

状态空间 $E = \{1, 2, 3, 4\}$, 观测数据统计转移次数后得到转移矩阵:

$$\hat{P} = \begin{bmatrix} 2/5 & 2/5 & 1/10 & 1/10 \\ 3/11 & 2/11 & 4/11 & 2/11 \\ 4/11 & 4/11 & 2/11 & 1/11 \\ 0 & 1/7 & 4/7 & 2/7 \end{bmatrix}. \quad (15.29)$$

例 15.7: 味精购买预测

初始分布 $P^{(1)} = [0.2, 0.4, 0.4]$, 转移矩阵:

$$P = \begin{bmatrix} 0.8 & 0.1 & 0.1 \\ 0.5 & 0.1 & 0.4 \\ 0.5 & 0.3 & 0.2 \end{bmatrix}. \quad (15.30)$$

第四次购买概率:

$$P^{(4)} = P^{(1)} P^3 = [0.7004, 0.1360, 0.1636]. \quad (15.31)$$

例 15.8: 长期购买倾向 (极限分布)

解 $\pi = \pi P$, 得:

$$\pi = [0.7143, 0.1310, 0.1548]. \quad (15.32)$$

即长期后 A 厂占有 71.43% 市场。

例 15.9: 相机归还点选择

转移矩阵:

$$P = \begin{bmatrix} 0.2 & 0.8 & 0 \\ 0.8 & 0 & 0.2 \\ 0.1 & 0.3 & 0.6 \end{bmatrix}. \quad (15.33)$$

极限分布:

$$\pi = [0.4146, 0.3902, 0.1951]. \quad (15.34)$$

相机归还到甲馆的概率最大, 维修点宜设在甲馆。

代码实现**例 15.5: 状态转移计数**

```

1  # 程序文件 ex15_5.py
2  import numpy as np
3
4  with open('data15_5.txt') as f:
5      s = f.read().replace('\n', '')
6
7      a = np.zeros((2, 2))
8      find_sub = lambda s, sub: [i for i in range(len(s)-1) if s[i:i
9                                  +2] == sub]
10
11     for i in range(2):
12         for j in range(2):
13             a[i, j] = len(find_sub(s, str(i)+str(j)))
14
15     print('转移计数矩阵:\n', a)

```



```

15     print('行和:', a.sum(axis=1))
16
17     # 转移概率
18     P = a / a.sum(axis=1, keepdims=True)
19     print('转移概率矩阵:\n', P)

```

例 15.6: 多状态转移矩阵估计

```

1     # 程序文件 ex15_6.py
2     import numpy as np
3
4     with open('data15_6.txt') as f:
5         s = f.read().replace(',', ' ').replace('\n', ' ')
6
7         a = np.zeros((4, 4))
8         find_sub = lambda s, sub: [i for i in range(len(s)-1) if s[i:i
9             +2] == sub]
10
11         for i in range(1, 5):
12             for j in range(1, 5):
13                 a[i-1, j-1] = len(find_sub(s, str(i)+str(j)))
14
15         print('转移计数矩阵:\n', a)
16         P = a / a.sum(axis=1, keepdims=True)
17         print('转移概率矩阵:\n', np.round(P, 4))

```

例 15.7: n 步转移概率

```

1     # 程序文件 ex15_7.py
2     import numpy as np
3
4     P0 = np.array([0.2, 0.4, 0.4])
5     P = np.array([[0.8, 0.1, 0.1],
6         [0.5, 0.1, 0.4],
7         [0.5, 0.3, 0.2]])
8
9     P4 = P0 @ np.linalg.matrix_power(P, 3)
10    print('第4次购买概率:', np.round(P4, 4))

```

例 15.8: 极限分布 (求解线性方程组)

```

1     # 程序文件 ex15_8_1.py
2     import numpy as np
3
4     P = np.array([[0.8, 0.1, 0.1],
5         [0.5, 0.1, 0.4],
6         [0.5, 0.3, 0.2]])
7
8     # 构造方程组  $(P^T - I) \pi^T = 0, \sum(\pi) = 1$ 

```

```

9     A = np.vstack([P.T - np.eye(3), np.ones(3)])
10    b = np.hstack([np.zeros(3), 1])
11    pi = np.linalg.pinv(A) @ b
12    print('极限分布:', np.round(pi, 4))

```

例 15.8: 极限分布（特征值法）

```

1     # 程序文件 ex15_8_2.py
2     import numpy as np
3
4     P = np.array([[0.8, 0.1, 0.1],
5                  [0.5, 0.1, 0.4],
6                  [0.5, 0.3, 0.2]])
7
8     val, vec = np.linalg.eig(P.T)
9     idx = np.argmax(val) # 特征值1对应的列
10    pi = vec[:, idx] / vec[:, idx].sum()
11    print('极限分布:', np.round(pi.real, 4))

```

例 15.9: 相机归还点选择

```

1     # 程序文件 ex15_9.py
2     import numpy as np
3
4     P = np.array([[0.2, 0.8, 0],
5                  [0.8, 0, 0.2],
6                  [0.1, 0.3, 0.6]])
7
8     A = np.vstack([P.T - np.eye(3), np.ones(3)])
9     b = np.hstack([np.zeros(3), 1])
10    pi = np.linalg.pinv(A) @ b
11    print('极限分布:', np.round(pi, 4))

```

优缺点分析

方面	优点	缺点
马尔可夫预测	适合随机序列预测, 考虑状态转移	要求马氏性, 需估计转移概率
转移矩阵法	计算简单, 可求 n 步及极限分布	状态数多时矩阵规模大
极限分布	提供长期趋势判断	要求正则性, 否则可能不存在

方法选择建议

- 状态可观测且满足马氏性：使用马尔可夫链建模。

- 短期预测：用初始分布乘以 P^n 求 n 步后分布。
- 长期趋势：求解 $\pi = \pi P$ 得极限分布。
- 转移概率未知：用历史数据统计频率估计。
- 状态数较多：注意矩阵维度，可考虑稀疏矩阵存储。

应用场景

- 市场占有率预测：顾客品牌选择变化。
- 设备状态预测：正常/故障状态转移。
- 天气预测：晴雨状态转移。
- 顾客行为分析：购买行为的马尔可夫性。
- 库存管理：库存状态转移与补货策略。

15.3 神经网络

15.3.1 人工神经网络概述

人工神经元是人工神经网络的基本构成元素,如图 15.1 所示, $X = [x_1, x_2, \dots, x_m]^T$, $W = [w_1, w_2, \dots, w_m]^T$ 为连接权, 网络输入为:

$$u = \sum_{i=1}^m w_i x_i = W^T X. \quad (15.35)$$

图 15.1 单层感知器模型

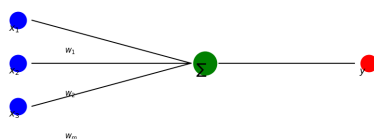


图 15.1 单层感知器模型

激活函数

1. 线性函数: $f(u) = ku + c$
2. 阈值函数/阶跃函数:

$$f(u) = \begin{cases} 1, & u \geq \theta, \\ 0, & u < \theta. \end{cases} \quad (15.36)$$

3. Sigmoid 函数：

$$f(u) = \frac{1}{1 + e^{-u}}. \quad (15.37)$$

将 $(-\infty, +\infty)$ 映射到 $(0, 1)$ 。

4. tanh 函数：

$$f(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}. \quad (15.38)$$

将 $(-\infty, +\infty)$ 映射到 $(-1, 1)$ 。

15.3.2 感知器

感知器是最早的人工神经网络，具有一层神经元，采用阈值激活函数。

模型表达式：

$$v = \sum_{i=1}^m w_i x_i + b, \quad y = \begin{cases} 1, & v \geq 0, \\ 0, & v < 0. \end{cases} \quad (15.17)$$

感知器通过监督式学习调整权值，使用梯度下降法。若解存在，算法在有限次迭代后收敛。

15.3.3 BP 神经网络

BP 神经网络由输入层、中间层（隐层）和输出层组成，相邻层之间全连接，同层无连接。

图 15.2 三层 BP 神经网络模型

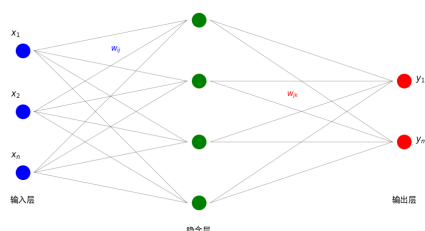


图 15.2 三层 BP 神经网络模型

主要用途：

1. 函数逼近
2. 模式识别
3. 预测
4. 数据压缩

BP 算法流程：

1. 初始化权值和阈值为 $[-1, 1]$ 随机值

2. 随机选择模式对 (X_0, Y_0) 提供给网络
3. 前向传播计算隐层和输出层输出
4. 计算输出层误差
5. 反向传播修正各层权值和阈值
6. 重复直到全局误差小于预设值或迭代次数达到上限

15.3.4 RBF 神经网络

RBF（径向基函数）神经网络由输入层、隐含层和输出层组成。

隐含层作用函数（高斯函数）：

$$R_i(X) = \exp\left(-\frac{\|X - C_i\|^2}{2\sigma_i^2}\right), \quad i = 1, 2, \dots, m. \quad (15.19)$$

输出层：

$$y_k = \sum_{i=1}^m w_{ik} R_i(X), \quad k = 1, 2, \dots, p. \quad (15.18)$$

学习算法（三步）：

1. 用 K 均值聚类确定基函数中心 C_i
2. 确定基函数宽度 σ_i
3. 用最小均方差准则确定连接权值 w_{ik}

15.3.5 数据预处理

神经网络训练前需对数据进行归一化处理：

映射到 $[0, 1]$ （sigmoid 函数）：

$$\tilde{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}. \quad (15.20)$$

映射到 $[-1, 1]$ （tanh 函数）：

$$\tilde{x} = \frac{2(x - x_{\min})}{x_{\max} - x_{\min}} - 1. \quad (15.21)$$

标准化变换：

$$\tilde{x} = \frac{x - \bar{x}}{s}. \quad (15.22)$$

具体例子

例 15.10：感知器分类

4 个输入向量分为两类，目标向量 $T = [1, 1, 0, 0]$ ，预测新输入 $p = [-0.5, 0.2]^T$ 。

分类函数： $v = -1.3x_1 - 0.5x_2$ ，预测值为 1。

例 15.11：上市公司信用分类（BP 神经网络）

8 个财务指标，5 个非 ST 公司（类别 0），5 个 ST 公司（类别 1），2 个待判样本。

BP 网络结构：1 个隐层，30 个神经元，sigmoid 激活函数。

预测结果：样本 11 为 ST 公司（类别 1），样本 12 为非 ST 公司（类别 0）。

例 15.12：公路客运量预测（BP 神经网络）

3 个输入：人口数量、机动车数量、公路面积；1 个输出：客运量。

BP 网络结构：1 个隐层，10 个神经元，线性激活函数。

预测结果：2010 年 54449.62 万人，2011 年 56573.68 万人。

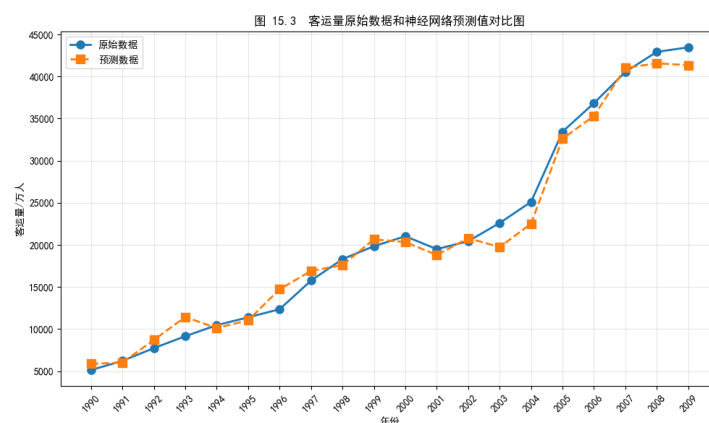


图 15.3 客运量原始数据和神经网络预测值对比图

代码实现

例 15.10：感知器分类

```

1  # 程序文件 ex15_10.py
2  from sklearn.linear_model import Perceptron
3  import numpy as np
4
5  X = np.array([[-0.5, -0.5, 0.3, 0.0],
6               [-0.5, 0.5, -0.5, 1.0]]).T
7  y = np.array([1, 1, 0, 0])
8
9  model = Perceptron().fit(X, y)
10 print('系数:', model.coef_)
11 print('截距:', model.intercept_)
12 print('准确率:', model.score(X, y))
13
14 # 预测新样本
15 p = np.array([[-0.5, 0.2]])

```

```
print('预测值:', model.predict(p))
```

例 15.11: BP 神经网络分类 (MLPClassifier)

```
# 程序文件 ex15_11.py
import numpy as np
from sklearn.neural_network import MLPClassifier

a = np.loadtxt('data15_11.txt')
X = a[:10, :8]    # 训练样本
y = a[:10, 8]     # 类别标签
x_test = a[10:, :8] # 待判样本

# 数据标准化 (极差归一化)
max_val = X.max(axis=0)
min_val = X.min(axis=0)
X_norm = (X - min_val) / (max_val - min_val)

# 对成本型指标 x2 特殊处理
X_norm[:, 1] = (max_val[1] - X[:, 1]) / (max_val[1] - min_val[1])

# BP神经网络
model = MLPClassifier(solver='lbfgs', activation='logistic',
                      hidden_layer_sizes=30, random_state=0)
model.fit(X_norm, y)

# 待判样本预测
x_test_norm = (x_test - min_val) / (max_val - min_val)
x_test_norm[:, 1] = (max_val[1] - x_test[:, 1]) / (max_val[1] - min_val[1])

print('待判样本类别:', model.predict(x_test_norm))
print('属于各类的概率:', model.predict_proba(x_test_norm))
print('训练准确率:', model.score(X_norm, y))
```

例 15.12: BP 神经网络回归 (MLPRegressor)

```
# 程序文件 ex15_12.py
import numpy as np
from sklearn.neural_network import MLPRegressor
import pylab as plt

a = np.loadtxt('data15_12.txt')
X = a[:, :3]    # 输入: 人口、机动车、公路面积
y = a[:, 3]     # 输出: 客运量

# 数据标准化 (映射到 [-1, 1])
```

```

11 max_val = X.max(axis=0)
12 min_val = X.min(axis=0)
13 X_norm = 2 * (X - min_val) / (max_val - min_val) - 1
14
15 # BP神经网络回归
16 model = MLPRegressor(solver='lbfgs', activation='identity',
17 hidden_layer_sizes=10, random_state=0)
18 model.fit(X_norm, y)
19
20 # 预测2010、2011年
21 x_new = np.array([[73.39, 75.55],
22 [3.9635, 4.0975],
23 [0.9880, 1.0268]]).T
24 x_new_norm = 2 * (x_new - min_val) / (max_val - min_val) - 1
25
26 y_pred = model.predict(x_new_norm)
27 print('2010年预测:', round(y_pred[0], 2))
28 print('2011年预测:', round(y_pred[1], 2))
29
30 # 绘制对比图
31 t = np.arange(1990, 2010)
32 y_train_pred = model.predict(X_norm)
33
34 plt.figure(figsize=(10, 6))
35 plt.plot(t, y, 'o-', label='原始数据')
36 plt.plot(t, y_train_pred, 's--', label='预测数据')
37 plt.xlabel('年份')
38 plt.ylabel('客运量/万人')
39 plt.legend()
40 plt.xticks(t, rotation=45)
41 plt.grid(True, alpha=0.3)
42 plt.tight_layout()
43 plt.savefig('passenger_forecast.png', dpi=300, bbox_inches='
    tight')
44 plt.show()

```

优缺点分析

方法	优点	缺点
感知器	简单直观, 收敛性有保证	仅能解决线性可分问题
BP 神经网络	非线性映射能力强, 通用性好	易过拟合, 训练慢, 局部最优
RBF 神经网络	学习速度快, 逼近能力强	隐层中心选择影响大

续下页

方法	优点	缺点
MLPClassifier/Regressor	sklearn 实现，使用方便	需调参（隐层数、神经元数）

方法选择建议

- 线性可分分类问题：使用感知器。
- 非线性分类/回归：使用 BP 神经网络。
- 需要快速训练：使用 RBF 神经网络。
- 分类问题：使用 MLPClassifier，激活函数选 'logistic' 或 'tanh'。
- 回归问题：使用 MLPRegressor，激活函数选 'identity'。
- 防止过拟合：减少隐层神经元数，增加正则化参数 alpha。

应用场景

- 信用评估：上市公司 ST 预测。
- 交通预测：公路客运量预测。
- 模式识别：手写数字识别、人脸识别。
- 经济预测：股票价格、GDP 预测。
- 故障诊断：设备状态监测与故障分类。

第 16 章

博弈论

博弈论（Game Theory）是研究决策主体的行为发生直接相互作用时的决策以及这种决策的均衡问题的理论。博弈论广泛应用在经济、政治、军事、外交中，其中在经济学中的应用最广泛、最成功。

16.1 基本概念

16.1.1 博弈论与优化理论的异同

相同点：都是决策主体在给定条件下选择最优策略的问题。

不同点：优化理论中决策者的最终结果只取决于决策者本身的决策；而博弈论中决策者的最终结果不但取决于决策者本身，而且也取决于其他决策者的决策。

16.1.2 博弈论中的经典案例

（1）囚徒困境 两名嫌疑犯被分别关押，无法串供。若两人都交代，各判刑 3 年；若都不交代，各判 1 年；若一人交代另一人不交代，交代者释放，不交代者判 5 年。

表 16.1 囚徒困境支付矩阵

	囚徒乙交代	囚徒乙不交代
囚徒甲交代	$(-3, -3)$	$(0, -5)$
囚徒甲不交代	$(-5, 0)$	$(-1, -1)$

纳什均衡：双方都交代。无论对方如何选择，交代都是各自的最优策略。

（2）智猪博弈 大猪和小猪同在一圈中，踩按钮出食。若小猪踩，大猪先吃（大猪 9，小猪 1）；若大猪踩，小猪先吃（大猪 4，小猪 6）；若同时踩（大猪 7，小猪 3）。

表 16.2 智猪博弈支付矩阵

	大猪踩	大猪等待
小猪踩	$(7, 3)$	$(9, 1)$
小猪等待	$(4, 6)$	$(0, 0)$

纳什均衡：大猪踩，小猪等待。小猪总是选择等待，大猪无奈只好去踩。

(3) 斗鸡博弈 两只公鸡面对面争斗，继续斗则两败俱伤，一方退却则认输。

表 16.3 斗鸡博弈支付矩阵

	公鸡乙进	公鸡乙退
公鸡甲进	(-10, -10)	(5, -5)
公鸡甲退	(-5, 5)	(0, 0)

纳什均衡：(进，退) 和 (退，进)，即一进一退。

16.1.3 博弈的一般概念

(1) 局中人(PLAYER) 有权决定自己行动方案的博弈参加者称为局中人。用 $I = \{1, 2, \dots, n\}$ 表示局中人的集合，至少有两个局中人。

(2) 策略集(STRATEGIES) 可供局中人选择一个实际可行的完整行动方案称为一个策略。第 i 个局中人的策略集记为 S_i 。

(3) 赢得函数(PAYOFF FUNCTION) 各局中人策略形成的策略组 $s = (s_1, s_2, \dots, s_n)$ 称为一个局势。局中人 i 的赢得函数 $H_i(s)$ 是定义在局势集合上的函数。

博弈三要素：局中人、策略集、赢得函数。

(4) 博弈问题的分类

分类标准	类型
策略与时间的关系	静态博弈、动态博弈
局中人数	二人博弈、多人博弈
合作性质	合作博弈、非合作博弈
赢得函数代数和	零和博弈、非零和博弈
策略性质	纯策略博弈、混合策略博弈
策略数量	有限策略博弈、无限策略博弈
数学模型	矩阵博弈、连续博弈、微分博弈、随机博弈

矩阵博弈的数学描述

设有两个局中人，局中人 A 有 m 个策略 $A_1, A_2, \dots, A_m$ ，局中人 B 有 n 个策略 $B_1, B_2, \dots, B_n$ 。当 A 采取策略 A_i 、 B 采取策略 B_j 时， A 的赢得为 a_{ij} ， B 的赢得为 $-a_{ij}$ (零和博弈)。

赢得矩阵：

$$A = (a_{ij})_{m \times n} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}. \quad (16.1)$$

局中人 A 的目标是最大化自己的赢得，局中人 B 的目标是最小化 A 的赢得。

纯策略意义下的解：若存在 i^*, j^* 使得：

$$a_{i^*j^*} = \max_i \min_j a_{ij} = \min_j \max_i a_{ij}, \quad (16.2)$$

则称 $a_{i^*j^*}$ 为博弈的鞍点， (A_{i^*}, B_{j^*}) 为纯策略意义下的解。

具体例子

例 16.1：矩阵博弈

设局中人 A 的赢得矩阵为：

$$A = \begin{bmatrix} 8 & 2 & 9 & -2 \\ 6 & 5 & -1 & 4 \\ -7 & 6 & 0 & 8 \end{bmatrix}. \quad (16.3)$$

求该矩阵博弈的解。

解：对每一行取最小值，再取最大值：

$$\max_i \min_j a_{ij} = \max\{-2, -1, -7\} = -1. \quad (16.4)$$

对每一列取最大值，再取最小值：

$$\min_j \max_i a_{ij} = \min\{8, 6, 9, 8\} = 6. \quad (16.5)$$

由于 $-1 \neq 6$ ，该博弈没有纯策略意义下的解，需要寻求混合策略。

代码实现

矩阵博弈的纯策略求解

```
1  # 矩阵博弈纯策略求解
2  import numpy as np
3
4  A = np.array([[8, 2, 9, -2],
5               [6, 5, -1, 4],
6               [-7, 6, 0, 8]])
7
8  # 行最小值取最大 (A的最优策略)
9  row_min = A.min(axis=1)
```

```

10     maximin = row_min.max()
11     print('maximin =', maximin)
12
13     # 列最大值取最小 (B的最优策略)
14     col_max = A.max(axis=0)
15     minimax = col_max.min()
16     print('minimax =', minimax)
17
18     # 判断是否有纯策略解
19     if maximin == minimax:
20         print('存在纯策略解, 值为:', maximin)
21     else:
22         print('不存在纯策略解, 需用混合策略')

```

囚徒困境支付矩阵

```

1     # 囚徒困境
2     import numpy as np
3
4     # 支付矩阵 (行: 囚徒甲, 列: 囚徒乙)
5     # 每个元素为(甲赢得, 乙赢得)
6     payoff = np.array([[[-3, -3], [0, -5]],
7                        [[-5, 0], [-1, -1]]])
8
9     # 寻找纳什均衡
10    for i in range(2):
11        for j in range(2):
12            a_payoff = payoff[i, j, 0]
13            b_payoff = payoff[i, j, 1]
14            # 检查是否有单方面偏离的动机
15            a_best = all(payoff[i, j, 0] >= payoff[k, j, 0] for k in range
16                        (2))
17            b_best = all(payoff[i, j, 1] >= payoff[i, k, 1] for k in range
18                        (2))
19            if a_best and b_best:
20                print(f'纳什均衡: (甲策略{i+1}, 乙策略{j+1})')
21            print(f'支付: ({a_payoff}, {b_payoff})')

```

优缺点分析

方面	优点	缺点
博弈论	揭示策略互动规律, 预测均衡结果	假设理性人, 实际可能不成立
矩阵博弈	数学形式规范, 便于求解	仅适合二人零和有限策略

续下页

方面	优点	缺点
纳什均衡	非合作博弈的核心概念	可能不存在唯一均衡
纯策略	简单直观	许多博弈无纯策略解
混合策略	解总是存在	计算复杂, 解释性差

方法选择建议

- 二人零和有限策略博弈：使用矩阵博弈模型。
- 判断纯策略解是否存在：计算 $\max \min$ 和 $\min \max$ ，若相等则存在。
- 纯策略解不存在：使用混合策略（线性规划求解）。
- 非合作博弈：使用纳什均衡概念。
- 合作博弈：使用 Shapley 值等合作博弈解概念。

应用场景

- 经济竞争：企业定价策略、产量决策。
- 拍卖设计：最优拍卖机制设计。
- 国际贸易：关税谈判、贸易战分析。
- 政治学：选举策略、国际冲突。
- 生物学：进化稳定策略（ESS）。
- 计算机科学：网络路由、算法博弈论。

16.2 零和博弈

零和博弈是一类特殊的博弈问题。在这类博弈中，只有两名局中人，每个局中人都只有有限个策略可供选择。在任一纯局势下，两个局中人的赢得之和总是等于零，即双方的利益是激烈对抗的。

16.2.1 基本概念

设局中人 I 、 II 的策略集分别为：

$$S_1 = \{\alpha_1, \alpha_2, \dots, \alpha_m\}, \quad S_2 = \{\beta_1, \beta_2, \dots, \beta_n\}. \quad (16.1)$$

当局中人 I 选定策略 α_i ，局中人 II 选定策略 β_j 后，形成局势 (α_i, β_j) 。记局中人 I 的赢得值为 a_{ij} ，称

$$A = (a_{ij})_{m \times n} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \quad (16.2)$$

为局中人 的赢得矩阵。零和博弈可简记为 $G = \{S_1, S_2; A\}$ 。

数学原理

(1) 鞍点与纯策略解 定义 16.1 设 $f(x, y)$ 为定义在 $x \in \Omega_1, y \in \Omega_2$ 上的实值函数, 若存在 $x^* \in \Omega_1, y^* \in \Omega_2$, 使得对一切 $x \in \Omega_1, y \in \Omega_2$, 有

$$f(x, y^*) \leq f(x^*, y^*) \leq f(x^*, y), \quad (16.6)$$

则称 (x^*, y^*) 为函数 f 的一个鞍点。

定义 16.2 若等式

$$\max_i \min_j a_{ij} = \min_j \max_i a_{ij} \quad (16.4)$$

成立, 则称 V_G 为博弈 G 的值, 称使等式成立的纯局势 (α_i, β_j) 为博弈 G 的鞍点或稳定解。

定理 16.1 设 $G = \{S_1, S_2; A\}$, 记 $\mu = \max_i \min_j a_{ij}, \nu = \min_j \max_i a_{ij}$, 则必有 $\mu + \nu \leq 0$ 。

定理 16.2 零和博弈 G 具有稳定解的充要条件为 $\mu + \nu = 0$ 。

当解不唯一时, 具有无差别性和可交换性。

(2) 混合策略 当 $\mu + \nu \neq 0$ 时, 赢得矩阵不存在鞍点, 需引入混合策略。

设局中人 以概率 x_i 选用策略 α_i , 局中人 以概率 y_j 选用策略 β_j :

$$\sum_{i=1}^m x_i = 1, \quad \sum_{j=1}^n y_j = 1, \quad x_i \geq 0, \quad y_j \geq 0. \quad (16.7)$$

局中人 的期望赢得为:

$$E(x, y) = x^T A y. \quad (16.8)$$

混合策略空间:

$$S_1^* = \left\{ x \in \mathbb{R}^m \mid x_i \geq 0, \sum_{i=1}^m x_i = 1 \right\}, \quad S_2^* = \left\{ y \in \mathbb{R}^n \mid y_j \geq 0, \sum_{j=1}^n y_j = 1 \right\}. \quad (16.8, 16.9)$$

定理 16.3 任意混合策略博弈问题必存在鞍点:

$$x^T A y = \max_x \min_y x^T A y = \min_y \max_x x^T A y. \quad (16.12)$$

(3) 求解方法 方法一: 线性方程组法 (2×2 矩阵)

当 $A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$ 不存在鞍点时, 最优混合策略可通过解方程组得到:

$$\begin{cases} a_{11}x_1 + a_{21}x_2 = u, \\ a_{12}x_1 + a_{22}x_2 = u, \\ x_1 + x_2 = 1. \end{cases} \quad (16.13)$$

$$\begin{cases} a_{11}y_1 + a_{12}y_2 = v, \\ a_{21}y_1 + a_{22}y_2 = v, \\ y_1 + y_2 = 1. \end{cases} \quad (16.14)$$

方法二：线性规划法（一般情形）

局中人 的线性规划模型：

$$\max u, \quad (16.9)$$

$$\text{s.t.} \quad \begin{cases} \sum_{i=1}^m a_{ij}x_i \geq u, & j = 1, 2, \dots, n, \\ \sum_{i=1}^m x_i = 1, & x_i \geq 0. \end{cases} \quad (16.19)$$

局中人 的线性规划模型：

$$\min v, \quad (16.10)$$

$$\text{s.t.} \quad \begin{cases} \sum_{j=1}^n a_{ij}y_j \leq v, & i = 1, 2, \dots, m, \\ \sum_{j=1}^n y_j = 1, & y_j \geq 0. \end{cases} \quad (16.20)$$

具体例子

例 16.2：轰炸机与战斗机博弈

A 方有两架轰炸机（I 装炸弹，II 护航，或反之）， B 方战斗机阻击其中一架。赢得矩阵：

$$A = \begin{bmatrix} 0.82 & 1 \\ 1 & 0.58 \end{bmatrix}. \quad (16.11)$$

混合策略解：

$$x_1 = 0.7, x_2 = 0.3, \quad y_1 = 0.7, y_2 = 0.3, \quad V = 0.874. \quad (16.12)$$

例 16.3：田忌赛马

齐王与田忌各从 6 种出马顺序中选一种，赢得矩阵（齐王的赢得）：

$$A = \begin{bmatrix} 3 & 1 & 1 & 1 & 1 & -1 \\ 1 & 3 & 1 & 1 & -1 & 1 \\ 1 & 1 & 3 & -1 & 1 & 1 \\ 1 & 1 & -1 & 3 & 1 & 1 \\ 1 & -1 & 1 & 1 & 3 & 1 \\ -1 & 1 & 1 & 1 & 1 & 3 \end{bmatrix}. \quad (16.13)$$

双方最优混合策略：各以 $1/6$ 概率随机选择一种策略，博弈值 $V = 1$ 。

例 16.5：军事对抗

甲方 3 种进攻武器，乙方 3 种防御武器，赢得矩阵：

$$A = \begin{bmatrix} 1/3 & 1/2 & -1/3 \\ -2/5 & 1/5 & -1/2 \\ 1/2 & -3/5 & 1/3 \end{bmatrix}. \quad (16.14)$$

最优策略:

$$x = [0.5283, 0, 0.4717]^T, \quad y = [0, 0.3774, 0.6226], \quad V = -0.0189. \quad (16.15)$$

代码实现

例 16.2: 2×2 博弈求解

```

1  import numpy as np
2
3  A = np.array([[0.82, 1], [1, 0.58]])
4
5  # 解方程组求 x
6  a11, a12, a21, a22 = A[0,0], A[0,1], A[1,0], A[1,1]
7  x1 = (a22 - a21) / (a11 + a22 - a12 - a21)
8  x2 = 1 - x1
9  u = a11 * x1 + a21 * x2
10
11  y1 = (a22 - a12) / (a11 + a22 - a12 - a21)
12  y2 = 1 - y1
13  v = a11 * y1 + a12 * y2
14
15  print(f'x = [{x1:.4f}, {x2:.4f}]')
16  print(f'y = [{y1:.4f}, {y2:.4f}]')
17  print(f'博弈值 = {u:.4f}')
```

例 16.3: 田忌赛马 (线性方程组求解)

```

1  # 程序文件 ex16_3.py
2  import numpy as np
3  import sympy as sp
4
5  A = np.array([[3,1,1,1,1,-1],
6  [1,3,1,1,-1,1],
7  [1,1,3,-1,1,1],
8  [1,1,-1,3,1,1],
9  [1,-1,1,1,3,1],
10 [-1,1,1,1,1,3]], dtype=int)
11
12 # 构造方程组  $A^T x = u, \text{sum}(x)=1$ 
13 A2 = np.hstack([A.T, -np.ones((6,1))])
14 A2 = np.vstack([A2, [1,1,1,1,1,1,0]])
15 B = np.array([0,0,0,0,0,0,1]).T
16
17 # 最小范数解
18 x = np.linalg.pinv(A2) @ B
19 print('最小范数解:', x[:6])
```

例 16.4/16.5: 线性规划求解

```

1  # 程序文件 ex16_4.py
2  import numpy as np
3  import cvxpy as cp
4
5  # 田忌赛马赢得矩阵 (例 16.3)
6  A = np.array([[3,1,1,1,1,-1],
7               [1,3,1,1,-1,1],
8               [1,1,3,-1,1,1],
9               [1,1,-1,3,1,1],
10              [1,-1,1,1,3,1],
11              [-1,1,1,1,1,3]])
12
13  x = cp.Variable(6, pos=True)
14  y = cp.Variable(6, pos=True)
15  u = cp.Variable()
16  v = cp.Variable()
17
18  # 局中人 的线性规划
19  obj1 = cp.Maximize(u)
20  con1 = [A.T @ x >= u, sum(x) == 1]
21  prob1 = cp.Problem(obj1, con1)
22  prob1.solve(solver='GLPK_MI')
23  print('u =', prob1.value)
24  print('x =', x.value)
25
26  # 局中人 的线性规划
27  obj2 = cp.Minimize(v)
28  con2 = [A @ y <= v, sum(y) == 1]
29  prob2 = cp.Problem(obj2, con2)
30  prob2.solve(solver='GLPK_MI')
31  print('v =', prob2.value)
32  print('y =', y.value)

```

优缺点分析

方法	优点	缺点
纯策略	简单直观, 计算量小	只适用于有鞍点的博弈
混合策略 (2×2)	解析求解, 公式简洁	仅适用于 2×2 矩阵
线性规划法	适用任意有限策略	需借助软件求解

方法选择建议

- 检查鞍点：先计算 $\max \min$ 和 $\min \max$ ，若相等则有纯策略解。
- 2×2 无鞍点博弈：使用线性方程组法。

- 一般 $m \times n$ 博弈：使用线性规划法。
- 对称博弈： $x = y$ ，可简化求解。

应用场景

- 军事对抗：武器系统的最优运用策略。
- 体育比赛：出场顺序、战术选择。
- 商业竞争：定价策略、广告投放。
- 政治博弈：竞选策略、国际谈判。

16.3 双矩阵博弈模型

在非零和博弈中，局中人 选择策略 $\alpha_i \in S_1$ ，局中人 选择策略 $\beta_j \in S_2$ ，博弈结果为 (α_i, β_j) ，相应的局中人 的赢得为 a_{ij} ，局中人 的赢得为 b_{ij} （不再为 $-a_{ij}$ ）。

双矩阵博弈记为：

$$G = \{S_1, S_2; A, B\}, \quad (16.16)$$

其中 $A = (a_{ij})_{m \times n}$ 和 $B = (b_{ij})_{m \times n}$ 分别是局中人 和 的赢得矩阵。

16.3.1 非合作双矩阵博弈的纯策略解

表 16.7 双矩阵博弈的赢得矩阵

	β_1	β_2	$\cdots$	β_n
α_1	(a_{11}, b_{11})	(a_{12}, b_{12})	$\cdots$	(a_{1n}, b_{1n})
α_2	(a_{21}, b_{21})	(a_{22}, b_{22})	$\cdots$	(a_{2n}, b_{2n})
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
α_m	(a_{m1}, b_{m1})	(a_{m2}, b_{m2})	$\cdots$	(a_{mn}, b_{mn})

定义 16.4 设 $G = \{S_1, S_2; A, B\}$ 是一双矩阵博弈，若

$$a_{i^*j^*} = \max_i \min_j a_{ij}, \quad b_{i^*j^*} = \max_j \min_i b_{ij} \quad (16.21)$$

成立，则称 $(\alpha_{i^*}, \beta_{j^*})$ 为 G 在纯策略下的解（纳什平衡点）。

数学原理

(1) 混合策略定义 在非零和博弈 $G = \{S_1, S_2; A, B\}$ 中，对任意 $x \in S_1^*$ ， $y \in S_2^*$ ，定义局中人 和 的期望赢得：

$$E_1(x, y) = x^T A y = \sum_{i=1}^m \sum_{j=1}^n a_{ij} x_i y_j, \quad (16.17)$$

$$E_2(x, y) = x^T B y = \sum_{i=1}^m \sum_{j=1}^n b_{ij} x_i y_j. \quad (16.18)$$

定义 16.5 若存在策略对 $(x^*, y^*) \in S_1^* \times S_2^*$, 使得

$$\begin{cases} E_1(x, y^*) \leq E_1(x^*, y^*), & \forall x \in S_1^*, \\ E_2(x^*, y) \leq E_2(x^*, y^*), & \forall y \in S_2^*, \end{cases} \quad (16.22)$$

则称 (x^*, y^*) 为博弈 G 的一个纳什均衡点。

定理 16.5 任何具有有限个纯策略的二人博弈至少存在一个平衡点。

定理 16.6 混合策略 (x^*, y^*) 为博弈 $G = \{S_1, S_2; A, B\}$ 的平衡点的充要条件是:

$$\sum_{j=1}^n a_{ij} y_j^* \leq x^{*T} A y^*, \quad i = 1, 2, \dots, m, \quad (16.23a)$$

$$\sum_{i=1}^m b_{ij} x_i^* \leq x^{*T} B y^*, \quad j = 1, 2, \dots, n. \quad (16.23b)$$

(2) 求解方法 由定理 16.6, 求解非合作博弈的平衡点转化为求不等式约束式 (16.23) 的可行解, 加上虚拟目标函数即可使用优化求解器求解。

具体例子

例 16.6: 囚徒困境

表 16.8 囚徒困境赢得矩阵

	囚徒乙坦白	囚徒乙不坦白
囚徒甲坦白	$(-3, -3)$	$(0, -5)$
囚徒甲不坦白	$(-5, 0)$	$(-1, -1)$

纳什均衡: 双方都坦白, 即 $([1, 0], [1, 0])$ 。囚徒困境揭示了个体理性与集体理性的冲突。

例 16.7: 游泳队比赛策略

甲、乙两支游泳队各有一名健将 (甲队李, 乙队王), 每人只能参加两项比赛, 3 个项目。求最优策略。

赢得矩阵:

$$A = \begin{bmatrix} 14 & 13 & 12 \\ 13 & 12 & 13 \\ 12 & 13 & 14 \end{bmatrix}, \quad B = \begin{bmatrix} 13 & 14 & 15 \\ 14 & 15 & 14 \\ 15 & 14 & 13 \end{bmatrix}. \quad (16.19)$$

最优混合策略:

- 甲队: 策略 α_1 (李不参加蝶泳) 和 α_3 (李不参加蛙泳) 各占 50%
- 乙队: 策略 β_2 (王不参加仰泳) 和 β_3 (王不参加蛙泳) 各占 50%
- 甲队平均得分: 12.5 分, 乙队平均得分: 14.5 分

代码实现

例 16.7: 双矩阵博弈求解 (scipy.optimize)

```

1  # 程序文件 ex16_7.py
2  import numpy as np
3  from scipy.optimize import minimize
4
5  A = np.array([[14, 13, 12],
6               [13, 12, 13],
7               [12, 13, 14]])
8  B = np.array([[13, 14, 15],
9               [14, 15, 14],
10              [15, 14, 13]])
11
12  m, n = A.shape
13
14  def objective(z):
15      return np.sum(z) # 虚拟目标函数
16
17  def constraints(z):
18      x = z[:m]
19      y = z[m:]
20      # 约束1:  $A^T x \leq x^T A y$ 
21      c1 = x @ A @ y - A @ y
22      # 约束2:  $B y \leq x^T B y$ 
23      c2 = x @ B @ y - B.T @ x
24      # 约束3:  $\text{sum}(x)=1, \text{sum}(y)=1$ 
25      c3 = np.sum(x) - 1
26      c4 = np.sum(y) - 1
27      return np.hstack([c1, c2, c3, c4])
28
29  cons = {'type': 'ineq', 'fun': constraints}
30  bounds = [(0, 1)] * (m + n)
31  x0 = np.ones(m + n) / (m + n)
32
33  result = minimize(objective, x0, constraints=cons, bounds=
34                    bounds)
35  x = result.x[:m]
36  y = result.x[m:]
37
38  print('甲队策略 x:', np.round(x, 4))
39  print('乙队策略 y:', np.round(y, 4))
40  print('甲队平均得分:', x @ A @ y)
41  print('乙队平均得分:', x @ B @ y)

```

例 16.6: 囚徒困境纳什均衡

```
1  # 囚徒困境纯策略纳什均衡
2  import numpy as np
3
4  # 支付矩阵
5  A = np.array([[ -3, 0],    # 囚徒甲的支付
6                [-5, -1]])
7  B = np.array([[ -3, -5],   # 囚徒乙的支付
8                [0, -1]])
9
10 # 纯策略纳什均衡检查
11 for i in range(2):
12     for j in range(2):
13         # 甲是否有单方面偏离的动机
14         a_best = A[i, j] >= A[0, j] and A[i, j] >= A[1, j]
15         # 乙是否有单方面偏离的动机
16         b_best = B[i, j] >= B[i, 0] and B[i, j] >= B[i, 1]
17         if a_best and b_best:
18             print(f'纳什均衡: ({i+1}, {j+1}) -> ({A[i,j]}, {B[i,j]})')
```

优缺点分析

方法	优点	缺点
纯策略纳什均衡	简单直观，易于理解	许多博弈无纯策略解
混合策略纳什均衡	解总是存在	计算复杂，需数值求解
线性规划法（零和）	求解效率高	仅适用于零和博弈
约束优化法（非零和）	适用一般双矩阵博弈	需虚拟目标函数

方法选择建议

- 双矩阵非零和博弈：使用约束优化法求纳什均衡。
- 纯策略解存在：直接检查所有策略组合是否满足纳什均衡条件。
- 混合策略解：将问题转化为约束可行解问题，用优化方法求解。
- 零和博弈：使用线性规划法更高效。

应用场景

- 价格竞争：双寡头定价博弈。
- 广告博弈：企业广告投放策略。
- 国际贸易：关税谈判、贸易战。

-
- 公共品供给：搭便车问题。
 - 劳动力市场：工资谈判。

第 17 章

偏微分方程

自然科学与工程技术中种种运动发展过程与平衡现象各自遵守一定的规律，这些规律的定量表达一般地呈现为关于含有未知函数及其导数的方程。将含有多元未知函数及其偏导数的方程，称为偏微分方程。

方程中出现的未知函数偏导数的最高阶数称为偏微分方程的阶。如果方程中对于未知函数和它的所有偏导数都是线性的，这样的方程称为线性偏微分方程，否则称为非线性偏微分方程。

初始条件和边界条件称为定解条件，未附加定解条件的偏微分方程称为泛定方程。定解条件与泛定方程作为一个整体，称为定解问题。

17.1 3 类偏微分方程的定解问题

17.1.1 椭圆型偏微分方程

各种物理性质的定常（不随时间变化）过程，都可用椭圆型方程来描述。其典型最简单的形式是 **Poisson**（泊松）方程：

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y). \quad (17.1)$$

特别地，当 $f(x, y) = 0$ 时，即为 **Laplace**（拉普拉斯）方程：

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0. \quad (17.2)$$

应用：稳定温度场的温度分布、不可压缩流体的稳定无旋流动、静电场的电势等。

第一边值问题（**Dirichlet** 问题）：

$$\begin{cases} \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y), & (x, y) \in \Omega, \\ u(x, y)|_{(x, y) \in \Gamma} = \phi(x, y), & \Gamma = \partial\Omega, \end{cases} \quad (17.3)$$

其中 Ω 为有界区域， Γ 为边界。

第二、三类边界条件：

$$\left(\frac{\partial u}{\partial n} + \alpha u \right) \Big|_{\Gamma} = \phi(x, y), \quad (17.4)$$

其中 n 为边界 Γ 的外法线方向。 $\alpha = 0$ 时为第二类边界条件 (Neumann 条件), $\alpha \neq 0$ 时为第三类边界条件 (Robin 条件)。

17.1.2 抛物型偏微分方程

在研究热传导过程、气体扩散现象及电磁场的传播等随时间变化的非定常物理问题时, 常常会遇到抛物型方程。其最简单的形式为一维热传导方程:

$$\frac{\partial u}{\partial t} - a \frac{\partial^2 u}{\partial x^2} = 0, \quad a > 0. \quad (17.5)$$

初值问题 (Cauchy 问题):

$$\begin{cases} \frac{\partial u}{\partial t} - a \frac{\partial^2 u}{\partial x^2} = 0, & t > 0, -\infty < x < +\infty, \\ u(x, 0) = \phi(x), & -\infty < x < +\infty. \end{cases} \quad (17.6)$$

初边值问题:

$$\begin{cases} \frac{\partial u}{\partial t} - a \frac{\partial^2 u}{\partial x^2} = 0, & 0 < t < T, 0 < x < l, \\ u(x, 0) = \phi(x), & 0 \leq x \leq l, \\ u(0, t) = g_1(t), \quad u(l, t) = g_2(t), & 0 \leq t \leq T. \end{cases} \quad (17.7)$$

17.1.3 双曲型偏微分方程

物理中常见的一维振动与波动问题可用二阶波动方程描述:

$$\frac{\partial^2 u}{\partial t^2} = a^2 \frac{\partial^2 u}{\partial x^2}. \quad (17.10)$$

初值问题:

$$\begin{cases} \frac{\partial^2 u}{\partial t^2} = a^2 \frac{\partial^2 u}{\partial x^2}, & t > 0, -\infty < x < +\infty, \\ u(x, 0) = \phi(x), & -\infty < x < +\infty, \\ \left. \frac{\partial u}{\partial t} \right|_{t=0} = \varphi(x), & -\infty < x < +\infty. \end{cases} \quad (17.11)$$

适定性: 如果偏微分方程定解问题的解存在唯一且连续依赖于定解数据, 则称此定解问题是适定的。

17.2 简单偏微分方程的符号解

SymPy 提供了 `pdsolve()` 函数用于求解偏微分方程的符号解。

具体例子

例 17.1: 一阶常系数齐次偏微分方程

$$a \frac{\partial f}{\partial x} + b \frac{\partial f}{\partial y} + cf = 0. \quad (17.1)$$

通解:

$$f(x, y) = F(-ay + bx)e^{-\frac{c(ax+by)}{a^2+b^2}}, \quad (17.2)$$

其中 $F(\cdot)$ 为任意函数。

例 17.2: 一阶常系数非齐次偏微分方程

$$a\frac{\partial f}{\partial x} + b\frac{\partial f}{\partial y} + cf = G(x, y). \quad (17.3)$$

通解 (形式较复杂, 见教材)。

例 17.3: 变系数一阶线性偏微分方程

$$x\frac{\partial f}{\partial x} - y\frac{\partial f}{\partial y} + y^2f = y^2. \quad (17.4)$$

通解:

$$f(x, y) = F(xy)e^{-y^2/x^2} + 1, \quad (17.5)$$

其中 $F(\cdot)$ 为任意函数。

代码实现

例 17.1: 齐次偏微分方程符号解

```

1  # 程序文件 ex17_1.py
2  import sympy as sp
3
4  sp.var('x, y, a, b, c')
5  f = sp.Function('f')
6  u = f(x, y)
7
8  eq = a * u.diff(x) + b * u.diff(y) + c * u
9  sp.pprint(eq)
10
11 s = sp.pdsolve(eq)
12 sp.pprint(s)

```

例 17.2: 非齐次偏微分方程符号解

```

1  # 程序文件 ex17_2.py
2  import sympy as sp
3
4  sp.var('x, y, a, b, c')
5  sp.var('G', cls=sp.Function)
6  f = sp.Function('f')
7  u = f(x, y)
8
9  eq = a * u.diff(x) + b * u.diff(y) + c * u - G(x, y)
10 sp.pprint(eq)

```

```
11
12     s = sp.pdsolve(eq)
13     sp.pprint(s)
```

例 17.3: 变系数偏微分方程符号解

```
1     # 程序文件 ex17_3.py
2     import sympy as sp
3
4     sp.var('x, y')
5     f = sp.Function('f')
6     u = f(x, y)
7
8     eq = x * u.diff(x) - y * u.diff(y) + y**2 * u - y**2
9     sp.pprint(eq)
10
11     s = sp.pdsolve(eq)
12     sp.pprint(s)
```

优缺点分析

方法	优点	缺点
符号解 (pdsolve)	精确解，便于理论分析	仅适用于简单线性方程
数值解	适用一般方程	需离散化，有数值误差

方法选择建议

- 简单线性偏微分方程：尝试使用 pdsolve() 求符号解。
- 复杂方程或无符号解：使用数值方法（有限差分、有限元）。
- 定解问题：需同时给出泛定方程和定解条件。

应用场景

- 热传导：温度分布随时间和空间的变化。
- 波动现象：弦振动、声波、电磁波传播。
- 流体力学：势流、扩散现象。
- 量子力学：薛定谔方程。
- 电磁学：静电场、静磁场。

17.3 偏微分方程的差分解法

差分方法又称为有限差分方法或网格法, 是求偏微分方程定解问题的数值解中应用最广泛的方法之一。

基本思想:

1. 对求解区域作网格剖分, 将连续区域用有限离散点集代替;
2. 将连续变量的函数用定义在网格点上的离散函数代替;
3. 用差商代替导数, 将偏微分方程定解问题化成代数方程组 (差分格式)。

17.3.1 椭圆型方程第一边值问题的差分解法

以泊松方程第一边值问题为基本模型:

$$\begin{cases} \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y), & (x, y) \in \Omega, \\ u(x, y)|_{\Gamma} = \phi(x, y), & \Gamma = \partial\Omega. \end{cases} \quad (17.3)$$

取 h, τ 分别为 x 方向和 y 方向的步长, 网格节点为 (x_k, y_j) , 其中 $x_k = kh, y_j = j\tau$ 。

对正则内点 (k, j) , 由二阶中心差商公式:

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_{(k,j)} = \frac{u_{k+1,j} - 2u_{k,j} + u_{k-1,j}}{h^2} + O(h^2), \quad (17.6)$$

$$\left. \frac{\partial^2 u}{\partial y^2} \right|_{(k,j)} = \frac{u_{k,j+1} - 2u_{k,j} + u_{k,j-1}}{\tau^2} + O(\tau^2). \quad (17.7)$$

得到五点差分格式 (取 $h = \tau$):

$$\frac{1}{h^2}(u_{k+1,j} + u_{k-1,j} + u_{k,j+1} + u_{k,j-1} - 4u_{k,j}) = f_{k,j}. \quad (17.14)$$

简记为:

$$\Delta u_{k,j} = h^2 f_{k,j}, \quad (17.8)$$

其中 $\Delta u_{k,j} = u_{k+1,j} + u_{k-1,j} + u_{k,j+1} + u_{k,j-1} - 4u_{k,j}$ 。

边界条件处理:

1. 直接转移: 用最接近非正则内点的边界点上的 u 值作为近似。
2. 线性插值: 通过相邻边界点与内点作线性插值得到非正则内点的 u 值。

迭代法求解:

- 同步迭代法 (Jacobi 迭代):

$$u_{k,j}^{(n+1)} = \frac{1}{4}(u_{k-1,j}^{(n)} + u_{k+1,j}^{(n)} + u_{k,j-1}^{(n)} + u_{k,j+1}^{(n)} - h^2 f_{k,j}). \quad (17.9)$$

- 异步迭代法 (Gauss-Seidel 迭代): 使用最新值, 收敛速度约快一倍。
- 逐次超松弛迭代法 (SOR): 进一步加速收敛。

17.3.2 抛物型方程的差分解法

以一维热传导方程为例：

$$\frac{\partial u}{\partial t} - a \frac{\partial^2 u}{\partial x^2} = 0, \quad a > 0. \quad (17.5)$$

$$\text{令 } r = \frac{a\tau}{h^2}.$$

(1) 古典显式格式

$$u_{k,j+1} = ru_{k+1,j} + (1 - 2r)u_{k,j} + ru_{k-1,j}. \quad (17.24)$$

可由第 j 层直接计算第 $j + 1$ 层，计算简单，但稳定性条件为 $r \leq 1/2$ 。

(2) 古典隐式格式

$$-ru_{k-1,j+1} + (1 + 2r)u_{k,j+1} - ru_{k+1,j+1} = u_{k,j}. \quad (17.25)$$

无条件稳定，但需解三对角方程组。

(3) Du Fort-Frankel 格式（三层显式）

$$u_{k,j+1} = \frac{2r}{1 + 2r}(u_{k+1,j} + u_{k-1,j}) + \frac{1 - 2r}{1 + 2r}u_{k,j-1}. \quad (17.26)$$

无条件稳定，但需先用二层格式求出第一层值。

17.3.3 双曲型方程的差分解法

(1) 一阶双曲型方程 考虑初值问题：

$$\begin{cases} \frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0, & t > 0, \\ u(x, 0) = \phi(x), & -\infty < x < +\infty. \end{cases} \quad (17.27)$$

令 $r = \tau/h$ ，几种显式格式：

迎风格式（向前差商）：

$$u_{k,j+1} = u_{k,j} - ar(u_{k+1,j} - u_{k,j}). \quad (17.31)$$

迎风格式（向后差商）：

$$u_{k,j+1} = u_{k,j} - ar(u_{k,j} - u_{k-1,j}). \quad (17.32)$$

中心差商格式：

$$u_{k,j+1} = u_{k,j} - \frac{ar}{2}(u_{k+1,j} - u_{k-1,j}). \quad (17.33)$$

稳定性条件： $|ar| \leq 1$ （CFL 条件）。

(2) 二阶波动方程

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}. \quad (17.10)$$

令 $r = c\tau/h$, 差分格式:

$$u_{i,j+1} = (2 - 2r^2)u_{i,j} + r^2(u_{i+1,j} + u_{i-1,j}) - u_{i,j-1}. \quad (17.43)$$

稳定性条件: $r \leq 1$ (CFL 条件)。

具体例子

例 17.4: 拉普拉斯方程第一边值问题

$$\begin{cases} \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0, & (x, y) \in \Omega, \\ u(x, y)|_{\Gamma} = \lg[(1+x)^2 + y^2], & \Gamma = \partial\Omega, \end{cases} \quad (17.10)$$

其中 $\Omega = [0, 1] \times [0, 1]$, 取 $h = \tau = 1/3$ 。

内部节点为 $(1, 1)$ 、 $(1, 2)$ 、 $(2, 1)$ 、 $(2, 2)$, 由五点格式得线性方程组:

$$\begin{bmatrix} -4 & 1 & 1 & 0 \\ 1 & -4 & 0 & 1 \\ 1 & 0 & -4 & 1 \\ 0 & 1 & 1 & -4 \end{bmatrix} \begin{bmatrix} u_{1,1} \\ u_{1,2} \\ u_{2,1} \\ u_{2,2} \end{bmatrix} = - \begin{bmatrix} u_{1,0} + u_{0,1} \\ u_{3,1} + u_{2,0} \\ u_{1,3} + u_{0,2} \\ u_{2,3} + u_{3,2} \end{bmatrix}. \quad (17.11)$$

解得:

$$u_{1,1} = 0.6348, \quad u_{1,2} = 1.0600, \quad u_{2,1} = 0.7985, \quad u_{2,2} = 1.1698. \quad (17.12)$$

代码实现

例 17.4: 椭圆型方程五点格式

```

1  # 程序文件 ex17_4.py
2  import numpy as np
3
4  f = lambda x, y: np.log((1 + x)**2 + y**2)
5  h = 1/3
6  n = 4
7
8  # 边界条件
9  u = np.zeros((n, n))
10 for i in range(n):
11     u[0, i] = f(0, i*h)
12     u[-1, i] = f(1, i*h)
13     u[i, 0] = f(i*h, 0)
14     u[i, -1] = f(i*h, 1)
15

```

```

16     # 构造线性方程组
17     A = np.array([[ -4, 1, 1, 0],
18                  [ 1, -4, 0, 1],
19                  [ 1, 0, -4, 1],
20                  [ 0, 1, 1, -4]])
21     b = -np.array([u[1,0] + u[0,1],
22                  u[3,1] + u[2,0],
23                  u[1,3] + u[0,2],
24                  u[2,3] + u[3,2]])
25
26     x = np.linalg.solve(A, b)
27     print('u11 =', round(x[0], 4))
28     print('u12 =', round(x[1], 4))
29     print('u21 =', round(x[2], 4))
30     print('u22 =', round(x[3], 4))

```

热传导方程显式格式

```

1     import numpy as np
2     import pylab as plt
3
4     # 参数
5     a = 1.0
6     L = 1.0
7     T = 0.1
8     nx = 20
9     nt = 200
10    h = L / nx
11    tau = T / nt
12    r = a * tau / h**2
13
14    # 初始条件
15    x = np.linspace(0, L, nx+1)
16    u = np.sin(np.pi * x)
17
18    # 显式迭代
19    u_new = np.zeros(nx+1)
20    for j in range(nt):
21        for i in range(1, nx):
22            u_new[i] = r * u[i+1] + (1 - 2*r) * u[i] + r * u[i-1]
23        u_new[0] = u_new[-1] = 0
24        u = u_new.copy()
25
26    plt.plot(x, u)
27    plt.xlabel('x')
28    plt.ylabel('u')
29    plt.title('热传导方程数值解')

```

```
plt.show()
```

波动方程显式格式

```

1  import numpy as np
2  import pylab as plt
3
4  c = 1.0
5  L = 1.0
6  T = 1.0
7  nx = 50
8  nt = 100
9  h = L / nx
10 tau = T / nt
11 r = c * tau / h
12
13 # 初始条件
14 x = np.linspace(0, L, nx+1)
15 u0 = np.sin(np.pi * x)
16 u1 = u0.copy()
17
18 # 显式迭代
19 u = np.zeros(nx+1)
20 for j in range(nt):
21     for i in range(1, nx):
22         u[i] = (2 - 2*r**2) * u1[i] + r**2 * (u1[i+1] + u1[i-1]) - u0[i]
23     u[0] = u[-1] = 0
24     u0, u1 = u1, u.copy()
25
26 plt.plot(x, u)
27 plt.xlabel('x')
28 plt.ylabel('u')
29 plt.title('波动方程数值解')
30 plt.show()

```

优缺点分析

方法	优点	缺点
显式格式	计算简单，逐层推进	稳定性条件严格
隐式格式	无条件稳定	需解方程组
同步迭代	并行计算方便	收敛较慢
异步迭代	收敛速度较快	串行计算

方法选择建议

- 椭圆型方程：使用迭代法（同步/异步/SOR）。
- 抛物型方程：
 - 小时间步长：使用显式格式（简单快速）。
 - 大时间步长：使用隐式格式（无条件稳定）。
- 双曲型方程：使用显式格式，注意 CFL 条件 $r \leq 1$ 。
- 精度要求高：使用高阶差分格式或减小步长。

应用场景

- 热传导：温度场随时间变化模拟。
- 波动传播：声波、电磁波传播模拟。
- 流体力学：势流、扩散问题。
- 结构力学：应力、应变分析。
- 环境工程：污染物扩散模拟。

17.4 Python 求偏微分方程数值解举例

求偏微分方程的数值解可以使用 Python 的一些第三方库，例如 FiPy 是利用有限体积方法求偏微分方程数值解的 Python 库。下面给出使用差分格式求偏微分方程的例子。

具体例子

例 17.5：泊松方程数值解

$$\begin{cases} -\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = 20 \cos(3\pi x) \sin(2\pi y), & 0 < x < 1, 0 < y < 1, \\ u(0, y) = y^2, & u(1, y) = 1, \quad 0 \leq y \leq 1, \\ u(x, 0) = x^3, & u(x, 1) = 1, \quad 0 \leq x \leq 1. \end{cases} \quad (17.13)$$

例 17.6：反应扩散方程数值解

$$\begin{cases} \frac{\partial u(x, t)}{\partial t} = 0.1 \frac{\partial^2 u(x, t)}{\partial x^2} - 3u(x, t), & 0 < x < 1, t > 0, \\ u(0, t) = 0, \quad u(1, t) = 0, & t \geq 0, \\ u(x, 0) = 4x - 4x^2, & 0 \leq x \leq 1. \end{cases} \quad (17.14)$$

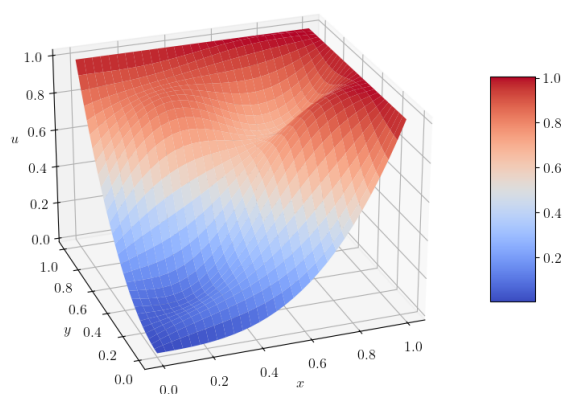


图 17.1 泊松方程数值解

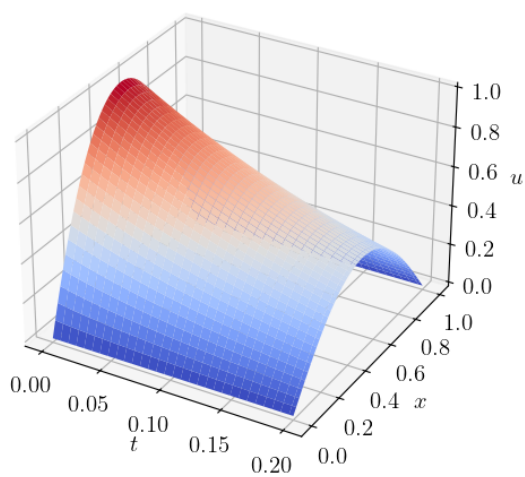


图 17.2 反应扩散方程数值解

代码实现

例 17.5: 泊松方程求解

```

1  # 程序文件 ex17_5.py
2  from scipy import sparse
3  from scipy.sparse.linalg import spsolve
4  import numpy as np
5  import pylab as plt
6
7  def rhs_func(x, y, M):
8      """定义右端项"""
9      g = (20 * np.cos(3 * np.pi * x[1:-1, 1:-1]) *
10          np.sin(2 * np.pi * y[1:-1, 1:-1])).flatten()
11      return g
12
13  def bc_dirichlet(x, y, M):
14      """定义 Dirichlet 边界条件"""
15      # 左边界:  $u(0, y) = y^2$ 
16      leftBC = y[1:-1, 0] ** 2
17      # 右边界:  $u(1, y) = 1$ 
18      rightBC = np.ones(M-2)
19      # 上边界:  $u(x, 1) = 1$ 
20      topBC = np.ones(M-2)
21      # 下边界:  $u(x, 0) = x^3$ 
22      bottomBC = x[0, 1:-1] ** 3
23
24      # 构造边界贡献向量
25      g1 = np.zeros((M-2) ** 2)
26      for i in range(M-2):
27          g1[(M-2) * i] = topBC[i] # 上边界
28          g1[(M-2) * (i+1) - 1] = bottomBC[i] # 下边界
29
30      k1 = np.zeros((M-2, 1))
31      k1[0] = 1.0
32      leftBCK = sparse.kron(k1, leftBC.reshape(-1, 1)).toarray().
33          flatten()
34
35      k2 = np.zeros((M-2, 1))
36      k2[-1] = 1.0
37      rightBCK = sparse.kron(k2, rightBC.reshape(-1, 1)).toarray().
38          flatten()
39
40      g = g1 + leftBCK + rightBCK
41      return g, leftBC, topBC, rightBC, bottomBC
42
43  def generate_lhs_matrix(M, hx, hy):

```

```

42     """生成五点格式的系数矩阵"""
43     alpha = hx ** 2 / hy ** 2
44     n = M - 2
45
46     main_diag = 2 * (1 + alpha) * np.ones(n)
47     off_diag = -1 * np.ones(n-1)
48
49     # B 矩阵 (x方向)
50     B = sparse.diags([main_diag, off_diag, off_diag],
51                      [0, -1, 1], shape=(n, n)).toarray()
52
53     # C 矩阵 (y方向)
54     C = sparse.diags([-1 * np.ones(n)], [0], shape=(n, n)).toarray()
55
56     # 组装系数矩阵
57     e1 = np.eye(n)
58     A1 = sparse.kron(e1, B).toarray()
59
60     e2 = sparse.diags([np.ones(n), np.ones(n)], [-1, 1], shape=(n,
61                                                                n)).toarray()
62     A2 = sparse.kron(e2, C).toarray()
63
64     return A1 + A2
65
66     # 参数设置
67     M = 50
68     x0, xf = 0.0, 1.0
69     y0, yf = 0.0, 1.0
70     hx = (xf - x0) / (M - 1)
71     hy = (yf - y0) / (M - 1)
72
73     # 生成网格
74     x = np.linspace(x0, xf, M)
75     y = np.linspace(y0, yf, M)
76     X, Y = np.meshgrid(x, y)
77
78     # 构造线性方程组
79     frhs = rhs_func(X, Y, M)
80     fbc = bc_dirichlet(X, Y, M)
81     rhs = frhs * (hx ** 2) + fbc[0]
82     A = generate_lhs_matrix(M, hx, hy)
83
84     # 求解
85     V = np.linalg.solve(A, rhs)
86     V = V.reshape((M-2, M-2)).T

```

```

87     # 组装完整解
88     U = np.zeros((M, M))
89     U[1:M-1, 1:M-1] = V
90     U[:, 0] = fbc[1]      # 左边界
91     U[0, :] = fbc[2]      # 上边界
92     U[:, M-1] = fbc[3]    # 右边界
93     U[M-1, :] = fbc[4]    # 下边界
94
95     # 绘图
96     plt.rc('text', usetex=True)
97     plt.rc('font', size=15)
98     ax = plt.axes(projection='3d')
99     ax.plot_surface(X, Y, U, cmap=plt.cm.coolwarm)
100    ax.set_xlabel('$x$')
101    ax.set_ylabel('$y$')
102    ax.set_zlabel('$u$')
103    ax.view_init(20, -106)
104    plt.tight_layout()
105    plt.savefig('poisson_solution.png', dpi=300, bbox_inches='tight')
106    plt.show()

```

例 17.6: 反应扩散方程求解

```

1     # 程序文件 ex17_6.py
2     import numpy as np
3     from scipy import sparse
4     import pylab as plt
5
6     # 参数设置
7     M = 50      # 空间点数
8     N = 60      # 时间点数
9     x0, xL = 0.0, 1.0
10    t0, tF = 0.0, 0.2
11    dx = (xL - x0) / (M - 1)
12    dt = (tF - t0) / (N - 1)
13
14    D = 0.1      # 扩散系数
15    alpha = -3   # 反应率
16    r = dt * D / dx ** 2
17    s = dt * alpha
18
19    # 空间和时间网格
20    xspan = np.linspace(x0, xL, M)
21    tspan = np.linspace(t0, tF, N)
22
23    # 构造系数矩阵（三对角）

```

```

24     main_diag = (1 + 2 * r - s) * np.ones(M - 2)
25     off_diag = -r * np.ones(M - 3)
26     A = sparse.diags([main_diag, off_diag, off_diag],
27                     [0, -1, 1], shape=(M-2, M-2)).toarray()
28
29     # 初始化解矩阵
30     U = np.zeros((M, N))
31     U[:, 0] = 4 * xspan - 4 * xspan ** 2      # 初始条件
32     U[0, :] = 0.0                             # 左边界
33     U[-1, :] = 0.0                            # 右边界
34
35     # 时间步进
36     for k in range(1, N):
37         c = np.zeros(M - 4)
38         b1 = np.array([r * U[0, k], r * U[-1, k]])
39         b1 = np.insert(b1, 1, c)
40         b2 = U[1:M-1, k-1]
41         b = b1 + b2
42         U[1:M-1, k] = np.linalg.solve(A, b)
43
44     # 绘图
45     plt.rc('text', usetex=True)
46     plt.rc('font', size=15)
47     X, T = np.meshgrid(tspan, xspan)
48     ax = plt.axes(projection='3d')
49     ax.plot_surface(X, T, U, linewidth=0, cmap=plt.cm.coolwarm)
50     ax.set_xlabel('$t$')
51     ax.set_ylabel('$x$')
52     ax.set_zlabel('$u$')
53     ax.set_xticks([0, 0.05, 0.1, 0.15, 0.2])
54     plt.tight_layout()
55     plt.savefig('reaction_diffusion.png', dpi=300, bbox_inches='
56         tight')
    plt.show()

```

方法评述

方法	优点	缺点
有限差分法	实现简单, 理论基础成熟	不规则区域处理困难
有限体积法	守恒性好, 适合流体问题	实现较复杂
有限元法	适合复杂几何区域	需要网格划分工具
Python 实现	代码简洁, 可视化方便	大规模问题计算慢

方法选择建议

- 规则区域：使用有限差分法，实现简单。
- 不规则区域：使用有限元法或有限体积法。
- 抛物线/双曲型：注意时间步长满足稳定性条件。
- 二维/三维问题：系数矩阵稀疏，使用稀疏矩阵存储和求解。
- 可视化：使用 matplotlib 绘制三维曲面图。